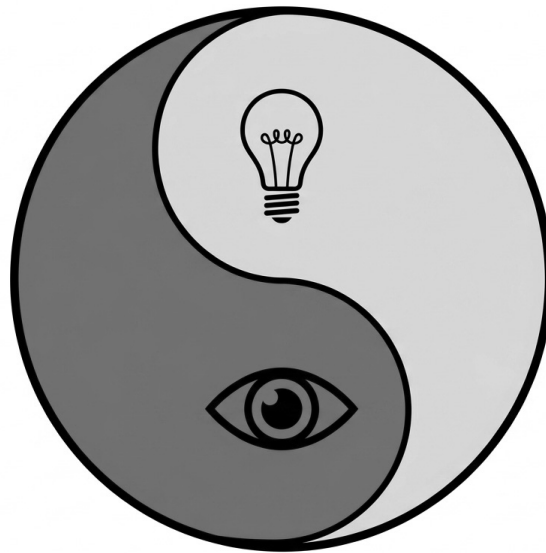


CT2890 Datorgrafik

Föreläsningar



Kjell Post

Mälardalens högskola

Innehåll

Föreläsning 1: Introduktion	4	Föreläsning 5: Geometrisk transformationer	82
Vad är datorgrafik?	5	Vektorer	84
Mål med kursen	6	Skalarprodukt	88
Angränsande områden	7	Kryssprodukt	90
Användningsområden	8	Matriser	91
Utvecklingen	15	Matrismultiplikation	92
Vektorgrafik	16	Transponering	94
Rastergrafik	17	Invers	94
Grafikstandards	20	2D-transformationer	95
Föreläsning 2: Introduktion till OpenGL	22	Homogena koordinater	99
Vad är OpenGL?	23	Sammansättning av 2D-transformationer	101
GL-paketet	24	3D-transformationer	104
Översikt över OpenGL:s API	25	Inverser till 2D och 3D-transformationer	107
Primitiva funktioner	26	Affina avbildningar	113
Den syntetiska kameran	29	Stelkroppsavbildningar	114
2D-grafik i OpenGL	31	3D-transformationer i OpenGL	115
Perspektiv- och parallellprojektion	32	Mellanspel: språket Postscript	121
OpenGL är tillståndsbaserat	33	Sammansatta transformationer i OpenGL	123
Textutskrifter	34		
Ett typiskt 2D-program i OpenGL	35	Föreläsning 6: 3D-Bilder	124
Föreläsning 3: 2D Rastergrafik	41	Den syntetiska kameran	125
Vad är svepkonvertering?	42	Renderingskedjan	127
Bildminnet	43	Från världs- till vykoordinater	128
Svepkonvertering av linjer	44	Projektion	129
En DDA-implementation	49	Taxonomi över projektioner	132
Bresenhamns algoritm	50	Specifikation av perspektivvy	136
Vad använder riktiga arbetsstationer?	54	Den stympade pyramiden	137
Varför $y = mx + b$ inte används inom datorgrafik	55	Parametrar till lab 2	138
Designbeslut för linjedragning	56	Perspektivtransformationen P	142
Svepkonvertering av polygoner	60	Härledning av P	144
Designbeslut för svepkonvertering av polygoner	66	Klippning	152
Klippning av polygoner	68	Transformation till skärmkoordinater	154
Anti-aliasing	74	The rendering pipeline	156
Area-sampling	75	Specifikation av vy i OpenGL	159
Supersampling	77	Lab 2 i OpenGL	166
OpenGL och svepkonvertering	79		

Föreläsning 11: Färg	170	Färg	279
Upplösning	171	Flera ljuskällor	280
Katodstråleröret (CRT)	173	Approximation av buktiga ytor	282
LCD-skärmar	176	Implementation	285
Monokromt ljus	180	Flat shading	286
Gamma-korrigerig	183	Mach-band	287
Färg	185	Gouraud shading	288
Lite om ögats fysiologi	186	Phong shading	290
Hur beskriva en färg?	189	Belysning i OpenGL	297
Hue, Saturation och Value	190	Lab 4 i OpenGL	314
Systematisering av färger	198		
Spektralfördelningar	200	Föreläsning 14B: Texturmappning	323
Terminologi från kolorimetri	202	2D texturmappning	324
RGB-systemet	204	Definitioner	325
CIE-standarden	207	Texturbeskrivningar	326
Färgmodeller	214	Från texels till pixels?	327
Att använda färg	218	Från pixels till texels	329
		Linjärinterpolering ger perspektivdistortion	331
Föreläsning 13: Skymda ytor	221	Designproblem	332
Problemformulering	222	Algoritm för texturmappning	335
Placering i renderingskedjan	223	Hur ska texturinformationen tolkas?	340
Eliminering av baksidor	224	3D texturmappning	341
Beräkning av normalen	226	Optimeringar för spelprogram	343
Eliminering av skymda ytor	228	Texturmappning i OpenGL	349
Z-buffertalgoritmen	230	Filtrering	353
Sveplinjealgoritmen	238	Lab 5 i OpenGL	358
BSP-träd	242		
Strålkastning	247	Föreläsning 14C: Globala belysningsmodeller	362
Beräkning av skärningspunkter	248	Globala belysningsmodeller	363
Optimeringar för skärningsberäkningar	256	Strålföljning	364
Andra algoritmer för skymda ytor	259	Strålföljningens huvudidé	366
Baksidor och skymda ytor i OpenGL	261	Pseudokod för strålföljning	370
Lab 3 i OpenGL	262	Resultatet	376
		Radiosity	379
Föreläsning 14A: Lokala belysningsmodeller	264	Vad är radiosity?	381
Belysning	265	Radiosity-ekvationen	383
Definition av lokal belysning	266	Formfaktorer	387
Ljuskällor	267	Hemikuben	389
Reflektion	268	Radiosity-bilder	390
Diffus reflektion	269	Progressiv radiosity-algoritm	391
Speglande reflektion	271	Jämförelse: strålföljning vs. radiosity	394
Bakgrundsbelysning	276	Ett par minimala strålföljare	395
Dämpningsfaktor	278		

Föreläsning 1: Introduktion

Vad är datorgrafik?
Mål med kursen
Angränsande områden
Användningsområden
Utvecklingen
Vektorgrafik
Rastergrafik
Grafikstandards

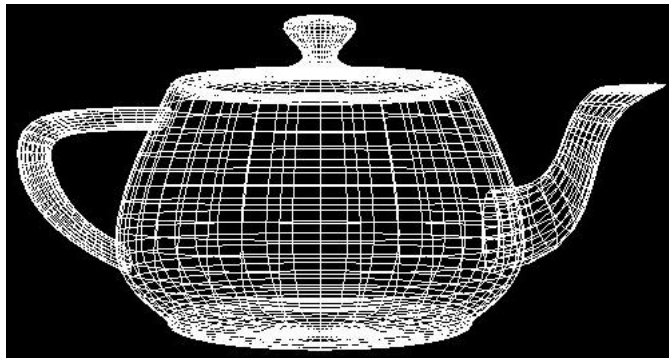
Vad är datorgrafik?

- **Modellering:**

Hur man bygger geometriska modeller av objekt.

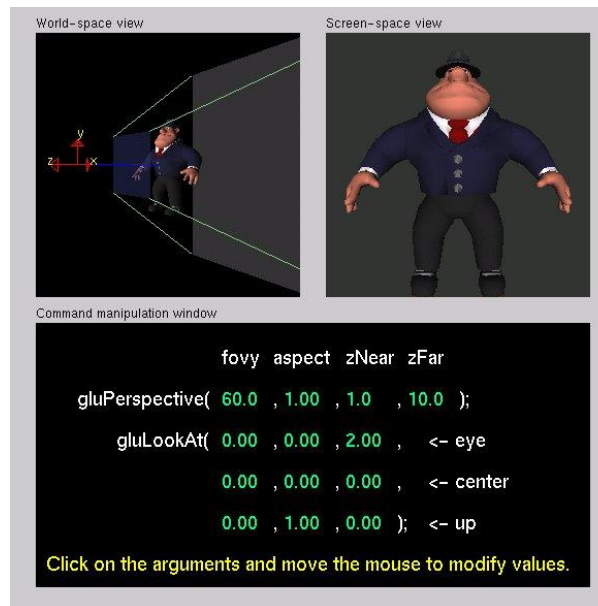
- **Framställning:**

Generera syntetiska bilder av modellerna.



Mål med kursen

- Ge förståelse för framställning av 2D och realistiska 3D bilder.
- Veta hur ett grafikbibliotek eller renderingsprogram fungerar internt och kunna identifiera dess begränsningar.



Datorgrafik: angränsande områden

- **Datavetenskap:** Algoritmer, datastrukturer, datorarkitektur.
- **Matematik:** Linjär algebra, analys, geometri, numeriska metoder.
- **Fysik:** Optik, termodynamik, radiometri.
- **Biologi:** Synsinnet
- **Psykologi:** Perception

Användningsområden

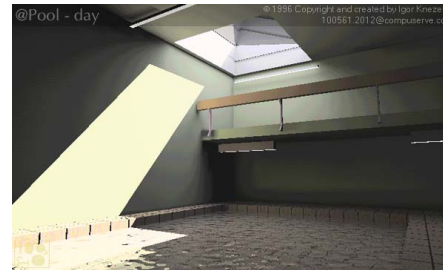
Datorgrafik används inom

- Vetenskap, medicin, etc.
- Ingenjörsvärlden
- Affärsvärlden
- Industri
- Konst, underhållning och reklam.
- Undervisning

Ingenjörsteknik: datorstödd design

Tillverkning av bilar, datorer, hus, etc.

- **Trådmodell:** tillåter interaktiva justeringar.
- **Ljussättning:** ger realism.
- **Tillverkning:** t ex kretskort, styrning av NC-maskin.



Underhållning

För film, spel, musikvideo, etc.

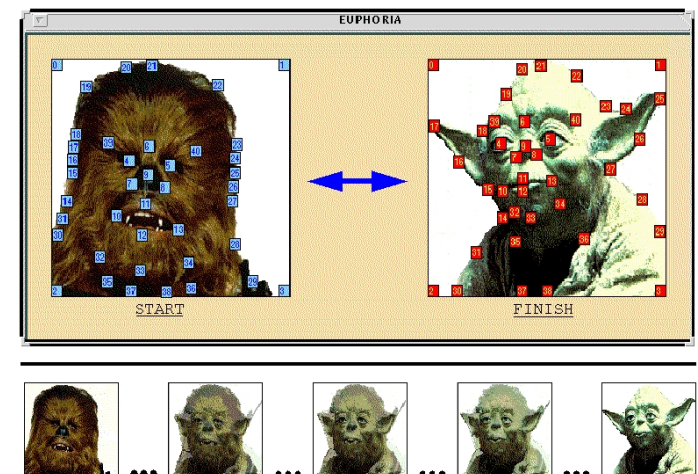
Filmen *Geri's Game* vann Oscar för bästa animerade kortfilm 1997.



(© Pixar Animation Studios.)

Datorgenererad konst

- Olika typer av ritprogram:
Photoshop, 3D Studio, etc.
- Animeringspaket: bilder genereras "frame-by-frame":
 - ▷ 24 st/s för film
 - ▷ 25 st/s för PAL-video
 - ▷ 30 st/s för NTSC-video
- Morphing:
Ett objekt transformeras gradvis till ett annat.



Utbildning

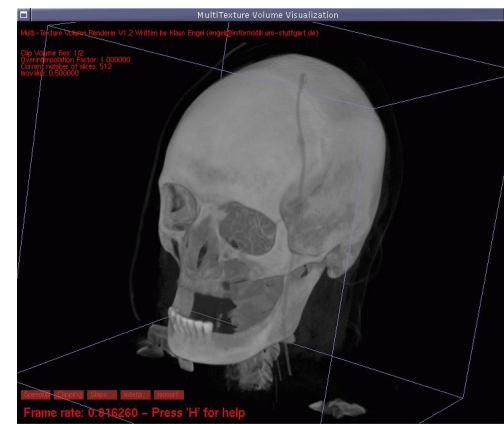
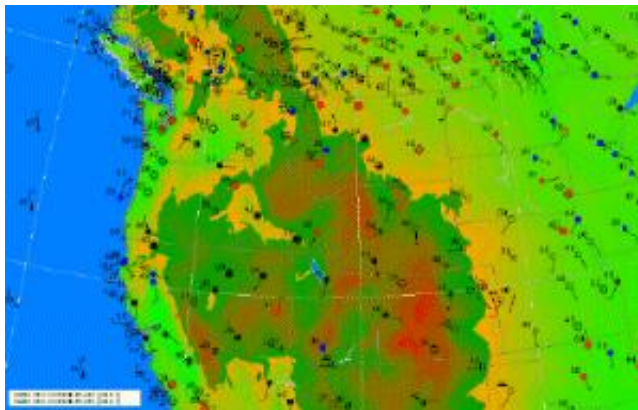
Simulatorer för flygplan, kärnkraftverk, etc.

- Säkert
- Billigt
- Kan studera olika miljöer



Visualisering

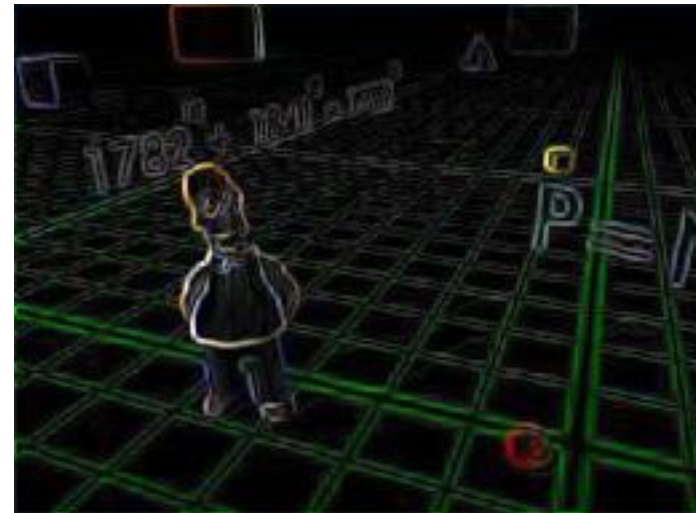
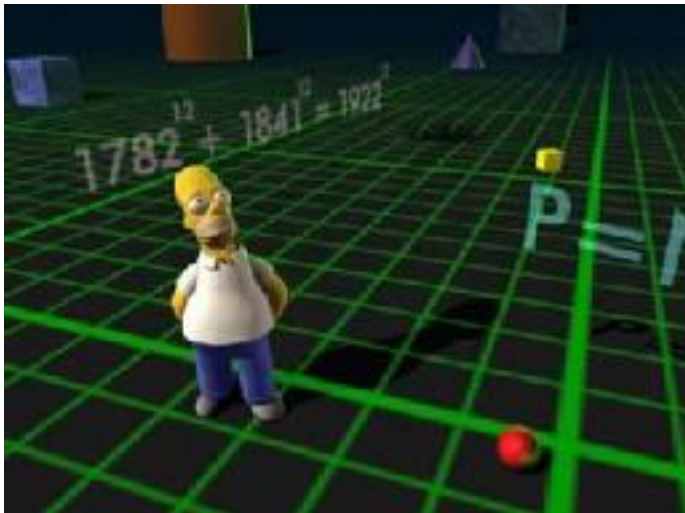
- Stora mängder av data konverteras till visuell form.
- Exempel: meteorologi, astronomi, biologi, etc.
- Ger möjlighet till insikt i komplicerade förlopp.



Bildbehandling

Att modifiera eller tolka en existerande bild.

- **Filter:** Tex göra bilden mindre suddig eller detektera kanter:

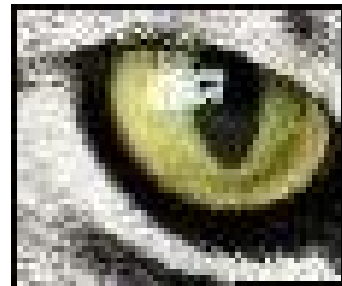
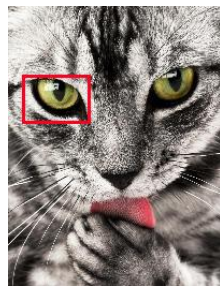


- **Förståelse:** Tex seende robotar.

Utvecklingen har styrts av hårdvaran

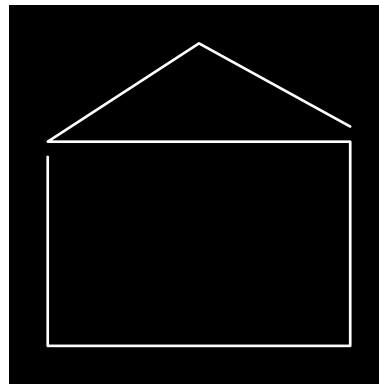
- **50-talet** CRT (cathode ray tube) introducerades.
- **60-talet** CAD-program, dyrbar utrustning.
- **80-talet** Persondatorn, "bitmap graphics":

En *bitmap* är en rektangulär matris av $s \times k$ *pixels*:



Vektorgrafik

- Figuren lagras som en sekvens av ändpunkter till linjer.
- Strålen går från ändpunkt till ändpunkt.
- Kräver ca 30 uppdateringar/s för att slippa blink.



Rastergrafik

- Lagrar bilden som pixels i en buffert (bildminnet).
- Minnet läses av en videokrets som visar bilden rad för rad.
- För varje pixel tänds eller släcks strålen.
- Monokromskärmar: en bit per pixel.

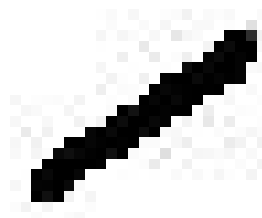
Rastergrafik med färg

- En färgskärm har tre strålar: röd, grön och blå.
- En pixel representeras vanligtvis av 8–24 bitar.
 - ▷ 24 bitar: 8 bitar röd, 8 bitar grön, 8 bitar blå.
Kräver en del bildminne.
 - ▷ 8 bitar: index till en färgtabell (palett).
256 rader i tabellen. Varje rad: 24 bitar RGB.

Vektor vs Raster

- En vektorskärm kan inte visa fyllda ytor eller färg.
- En rasterskärm kan bara visa diskreta punkter:
 - ▷ Allting måste beskrivas med pixels.
 - ▷ Lutande linjer, kurvor, cirklar blir kantiga.

Det är (grafik-)processorns uppgift att *svepkonvertera* geometriska beskrivningar till pixels.



Grafikstandards

Avsikten med standard:

- Göra det enkelt att flytta programvara.

Exempel:

- PostScript (2D)
- OpenGL (3D)
- VRML

OpenGL

- Ett API utvecklat av Silicon Graphics.
- Enkel, effektiv.
- Kan manipulera pixlar, kurvor och ytor.
- Portad till många plattformar.
- En industristandard.



Föreläsning 2: Introduktion till OpenGL

Vad är OpenGL?

GL-paketet

Översikt över OpenGL:s API

Primitiva funktioner

Den syntetiska kameran

2D-grafik i OpenGL

Perspektiv- och parallellprojektion

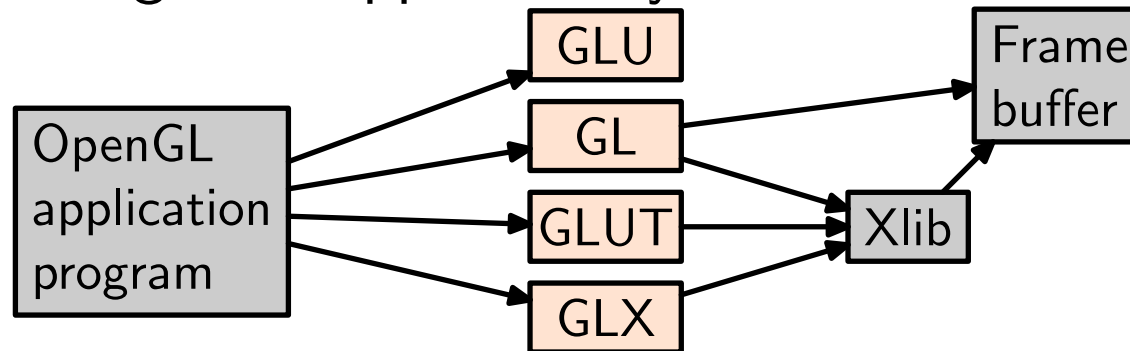
OpenGL är tillståndsbaserat

Textutskrifter

Ett typiskt 2D-program i OpenGL

Vad är OpenGL?

- Ett API för 3D-grafik, uppdelat i fyra bibliotek:



- GLU: Graphics Utility Library.

Hjälpfunktioner, använder bara GL-funktioner.

- GLUT: GL Utility Toolkit.

Huvudsakligen interface mot fönstersystemet.

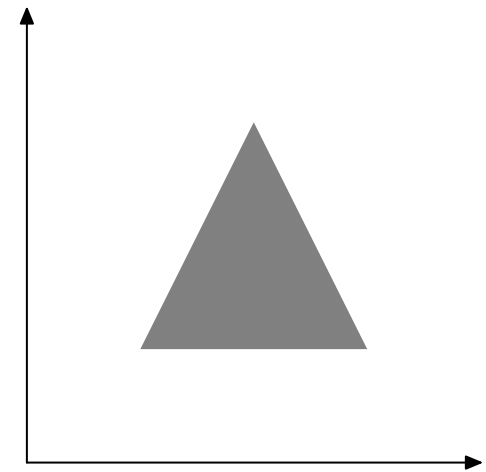
GL-paketet

- GL-paketet innehåller endast ett fåtal primitiver.

Grafikprocessorer brukar stödja dessa primitiver.

- Anrop till GL-funktioner börjar med `gl`.

```
glBegin(GL_POLYGON);  
    glVertex2f(300.0, 100.0);  
    glVertex2f(200.0, 300.0);  
    glVertex2f(100.0, 100.0);  
glEnd();
```



- Mer avancerade primitiver, t ex `gluSphere`, finns i GLU-paketet.
- Alla paket inkluderas vanligtvis med `#include <GL/glut.h>`.

Översikt över OpenGL:s API

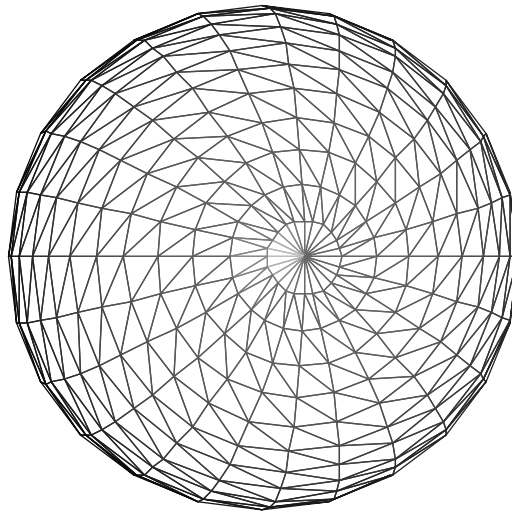
OpenGL innehåller flera hundra funktioner.

- **Primitiva funktioner:** För att beskriva punkter, linjer, polygoner, text, kurvor, etc.
- **Kamerafunktioner:** Position och orientering av kameran, samt typ av projektion.
- **Attributfunktioner:** För att ställa in färg, mönster, typsnitt, etc.
- **Transformationer:** Rotation, skalning, translation.
- **Hantering av händelser:** Tangentbord, mus, fönstersystemet. . .

Primitiva funktioner

- Alla modeller beskrivs med *hörnpunkter*:

```
glBegin(type);  
    glVertexnt(...);  
    ⋮  
    glVertexnt(...);  
glEnd();
```



- $type \in \{\text{GL_POINTS}, \text{GL_POLYGON}, \text{GL_QUADS}, \dots\}$
- n anger dimensionen, $n \in \{2, 3, 4\}$.
- t anger typen på argumenten:

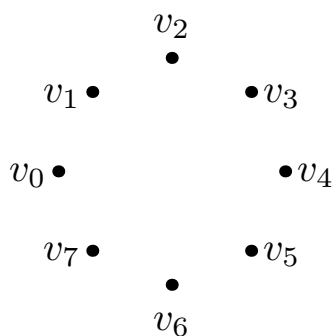
$t = i$ för `GLint`-typ

$t = f$ för `GLfloat`-typ

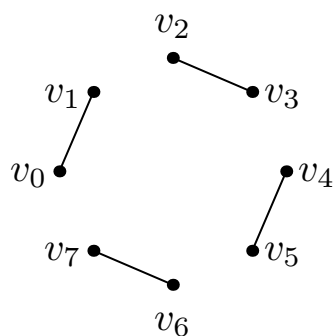
$t = d$ för `GLdouble`-typ

- Vi använder OpenGL:s `GLint` istf `int` osv.

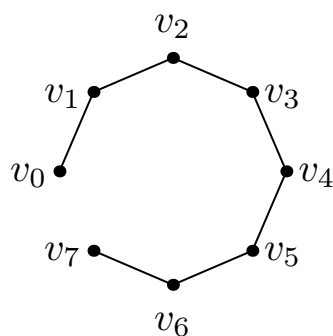
Exempel på primitiva funktioner



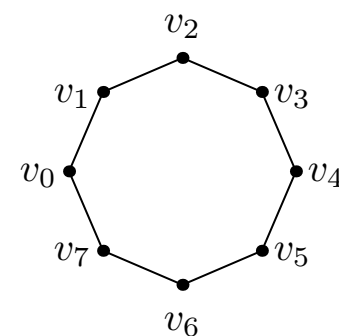
GL_POINTS



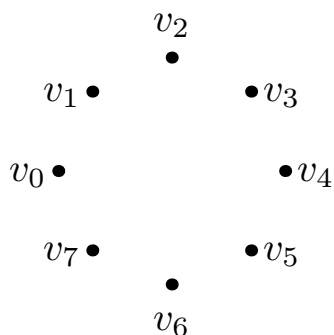
GL_LINES



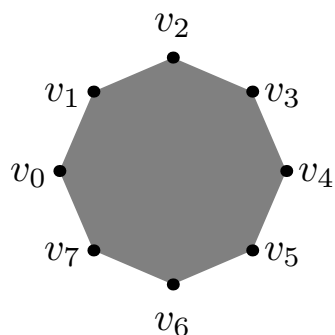
GL_LINE_STRIP



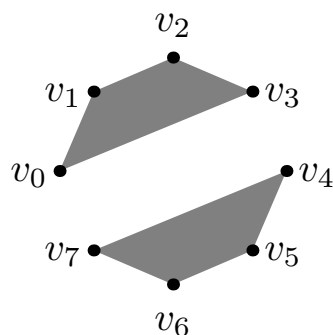
GL_LINE_LOOP



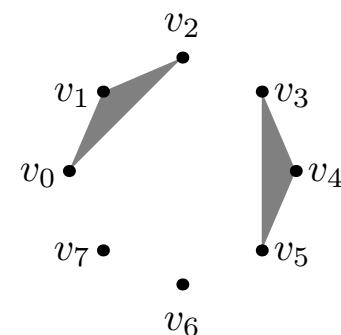
GL_POINTS



GL_POLYGON



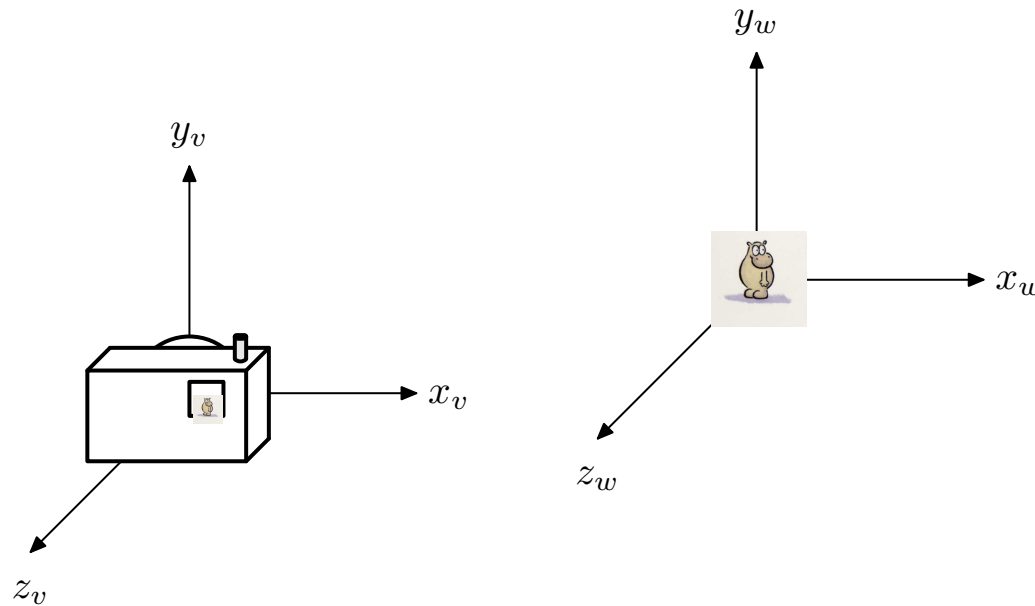
GL_QUADS



GL_TRIANGLES

Den syntetiska kameran

OpenGL, liksom många andra grafiksystem, använder sig av den s k *syntetiska kameramodellen*.



- Kamerans koordinatsystem kallas också *vykoordinatsystem*.
- Skärmen visar vad kameran ser.

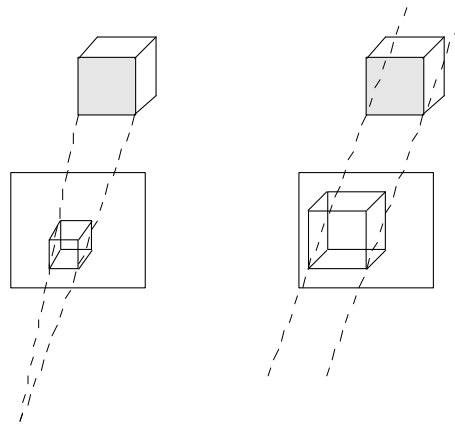
Den syntetiska kameran (forts)

- OpenGL har många inställningar för kameran:
 - ▷ Vad kameran "tittar" på.
 - ▷ Typ av projektion.
- Det mesta har att göra med 3D-grafik.
- Vi kommer att studera dessa detaljer senare.

2D-grafik i OpenGL

- OpenGL har egentligen bara 3D-grafik.
- I 2D-grafik är $z = 0$.
- I 2D-grafik låter vi kamerans vykoordinatsystem och världskoordinatsystemet sammanfalla.
- Vi använder s k *ortogonal projektion*, en form av *parallell-projektion*.

Perspektiv- och parallellprojektion



Perspektivprojektion *Parallellprojektion*

- Perspektivprojektion ger ett intryck av djup (3D).
- Parallellprojektion saknar djupinformation ($z = 0$).
- *Ortogonal projektion*: projektionsplanet $\perp$ mot strålarna.

OpenGL är tillståndsbaserat

OpenGL använder *tillstånd* för att komma ihåg bl a färg-, kamera- och belysningsinställningar.

- **24-bits färg:** `glColor3f(GLfloat r,GLfloat g,GLfloat b);`

Efterföljande primitiver ritas med en blandning av rött, grönt och blått, där $r, g, b = 0 \dots 1$.

- **Bakgrundsfärg:** `glClearColor(0.0, 0.0, 0.0, 0.0);`

Just detta anrop sätter bakgrundsfärgen till svart och genomskinlig (0.0).

Skärmen rensas sedan med `glClear(GL_COLOR_BUFFER_BIT);`

Textutskrifter: för debugging eller hjälptexter

- Använd följande procedur:

```
void printstring(void *font, char *s) {  
    while (*s)  
        glutBitmapCharacter(font, *s++);  
}
```

- Anropa med t ex

```
glRasterPos2i(x, y);  
printstring(GLUT_BITMAP_9_BY_15, "hej");
```

- Se `<GL/glut.h>` för andra typsnitt.

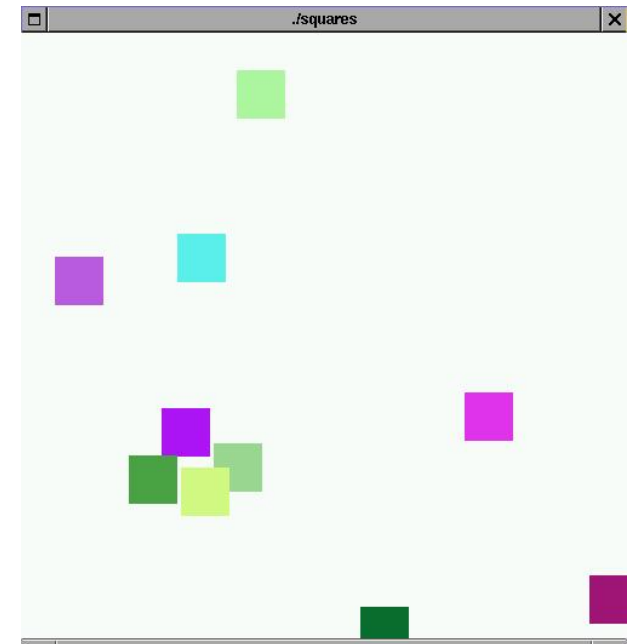
Ett typiskt 2D-program i OpenGL

Uppgift: rita 10 st kvadrater på skärmen i olika färger.

```
#include <stdio.h>
#include <stdlib.h>
#include <GL/glut.h>

#define XSIZE 500    /* window's width */
#define YSIZE 500    /* window's height */
#define SQSIZE 40    /* square size */
char *programe;

int main(int argc, char *argv[]) {
    programe = argv[0];
    /* let GLUT process command line arguments */
    glutInit(&argc, argv);
    /* initialize window, camera, etc */
    myinit();
    /* wait for an external event */
    glutMainLoop();
}
```



```
void myinit(void) {
    /* single buffering and true color */
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    /* create the window */
    glutInitWindowSize(XSIZE, YSIZE);
    glutCreateWindow(progname);

    /* call display() when the window needs to be drawn */
    glutDisplayFunc(display);

    /* call keys(key, x, y) when a key is pressed */
    glutKeyboardFunc(keys);

    /* white background color (we clear the window later) */
    glClearColor(1.0, 1.0, 1.0, 0.0);

    /* -- set up viewing, XSIZE x YSIZE with origin lower left -- */

    /* specify an orthogonal projection, window size XSIZE*YSIZE */
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, XSIZE, 0.0, YSIZE); /* left, right, bottom, top */

    /* switch back to the model view matrix (not really necessary here) */
    glMatrixMode(GL_MODELVIEW);
}
```

Callback-funktioner

Med anropet `glutDisplayFunc(display)` anger man att `display()` ska anropas när skärmen behöver ritas.

```
/* this function is called when the screen needs to be
   drawn, either when the program starts, or when the
   window is suddenly exposed. */
void display(void) {
    GLint i, x, y;
    GLfloat r, g, b;

    /* clear the window */
    glClear(GL_COLOR_BUFFER_BIT);

    for (i = 0; i < 10; i++) {    /* draw 10 squares */
        /* select a random location */
        x = rand() % XSIZE;
        y = rand() % YSIZE;
```

```
/* select a random color */
r = (GLfloat) rand() / RAND_MAX;
g = (GLfloat) rand() / RAND_MAX;
b = (GLfloat) rand() / RAND_MAX;
glColor3f(r, g, b);

/* create a filled square */
glBegin(GL_POLYGON);
glVertex2f(x+SQSIZE/2, y+SQSIZE/2);
glVertex2f(x-SQSIZE/2, y+SQSIZE/2);
glVertex2f(x-SQSIZE/2, y-SQSIZE/2);
glVertex2f(x+SQSIZE/2, y-SQSIZE/2);
glEnd();
}

/* ask OpenGL to draw the squares as soon
   as possible */
glFlush();
}
```

Med anropet `glutKeyboardFunc(keys)` anger man att `keys()` ska anropas när en tangent trycks ned.

```
/* key is the ASCII code;
   (x,y) is the cursor position */
void keys(unsigned char key, int x, int y) {
    switch(key) {
        case 'q':
        case 27:
            exit(0);
        case 'r':
            glutPostRedisplay(); /* redraw */
            break;
        default:
            /* fall thru */
    }
}
```

Med `glutPostRedisplay()` talar man om för OpenGL att skärmen behöver ritas om, vilket gör att `display()` anropas.

Mer OpenGL

Vi kommer att fördjupa oss mer i OpenGL.

- I samband med teoriavsnitten kommer motsvarande OpenGL-kod att visas.
- Varje laboration kommer att ges i en OpenGL-version.
- Fortsättningskursen använder ännu mer OpenGL.
- Följande böcker från Addison-Wesley är referensmaterial:
 - ▷ *OpenGL Programming Guide*
 - ▷ *OpenGL Reference Manual*

Föreläsning 3: 2D Rastergrafik

Vad är svepkonvertering?

Bildminnet

Svepkonvertering av linjer

- En DDA-implementation

- Bresenhamns algoritm

- Vad använder riktiga arbetsstationer?

- Varför $y = mx + b$ inte används inom datorgrafik

- Designbeslut för linjedragning

Svepkonvertering av polygoner

- Designbeslut för svepkonvertering av polygoner

- Klippning av polygoner

Anti-aliasing

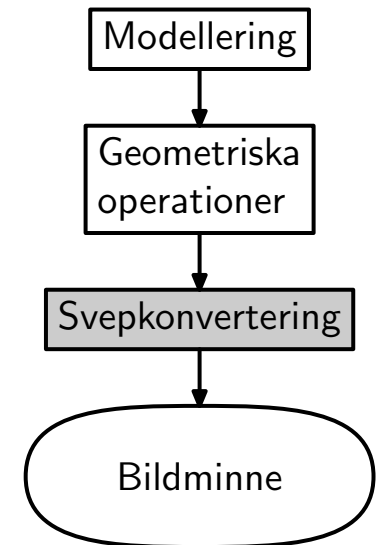
- Area-sampling

- Supersampling

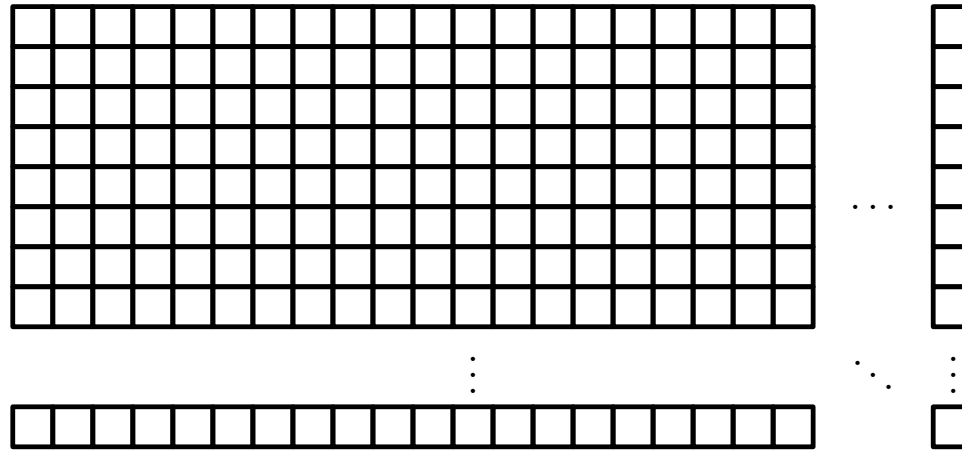
OpenGL och svepkonvertering

Vad är svepkonvertering?

- Att översätta en geometrisk beskrivning till pixels.
- Sista steget i ett renderingsverktyg.
- Varför studera det sista steget först?
 - ▷ Det är enklare (allting är 2D)
 - ▷ En del applikationer, t ex Postscript, använder det.
- Inte alla renderingsverktyg gör svepkonvertering, t ex strålföljning och radiosity.



Bildminnet



Antag en färgskärm där vi lagrar en pixel per byte.

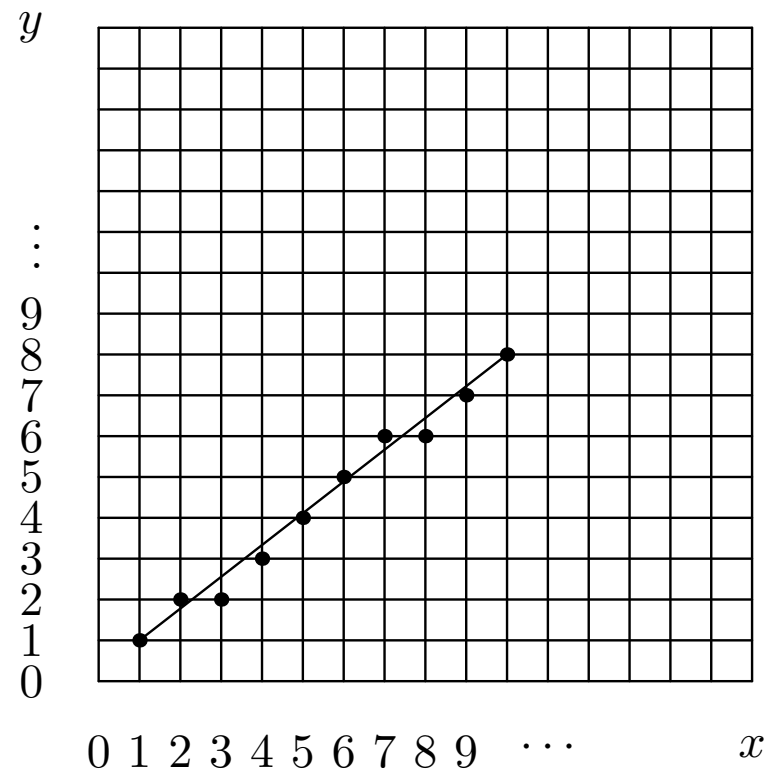
Bildminnet är typiskt en följd av bytes så att

$$addr(x, y) = addr(0, 0) + y \cdot XSIZE + x$$

där $XSIZE$ är antalet bytes per rad (sveplinje).

Svepkonvertering av linjer

Uppgift: Rita en linje från (x_0, y_0) till (x_1, y_1) .

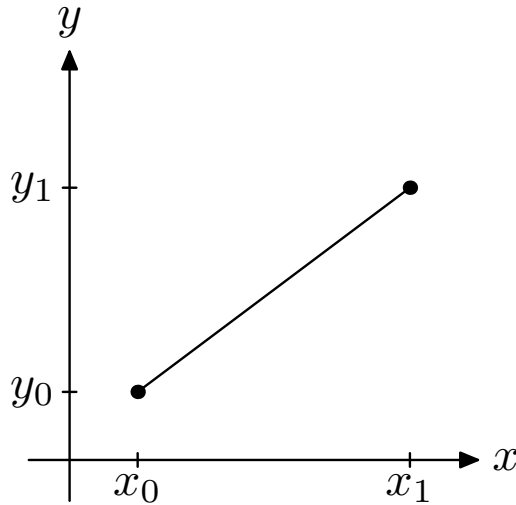


Problem: Vilka punkter ska passeras?

Antaganden

- Monokrom skärm. Vi studerar s k anti-aliasing senare.
- Ändpunkterna är heltalskoordinater.
- Lutningen på linjen $|m| \leq 1$.
- Om $|m| > 1$, byt plats på x och y för att undvika "glapp" mellan punkterna.
- Linjer med $|m| \in \{0, 1, \infty\}$ kan hanteras speciellt.
- Linjer är 1 pixels tjocka.

Linjens ekvation



$$y = mx + B$$

$$m = \frac{y_1 - y_0}{x_1 - x_0} = \frac{\Delta y}{\Delta x}$$

$$B = y_0 - m \cdot x_0$$

Pang-på-implementation

```
for  $x \in x_0 \dots x_1$   
     $y_i = m \cdot x_i + B$   
    write( $x_i, \lfloor y_i + 0.5 \rfloor$ )
```

Tidskrävande flyttalsmultiplikationer.

DDA-algoritmen

DDA = Differential Digital Analyzer.

Används för att lösa differkv inkrementellt genom att addera dx/dt till x och dy/dt till y för små dt .

Kan användas för att undvika multiplikation:

Antag att punkterna plottas från vänster till höger:

$$x_{i+1} = x_i + 1$$

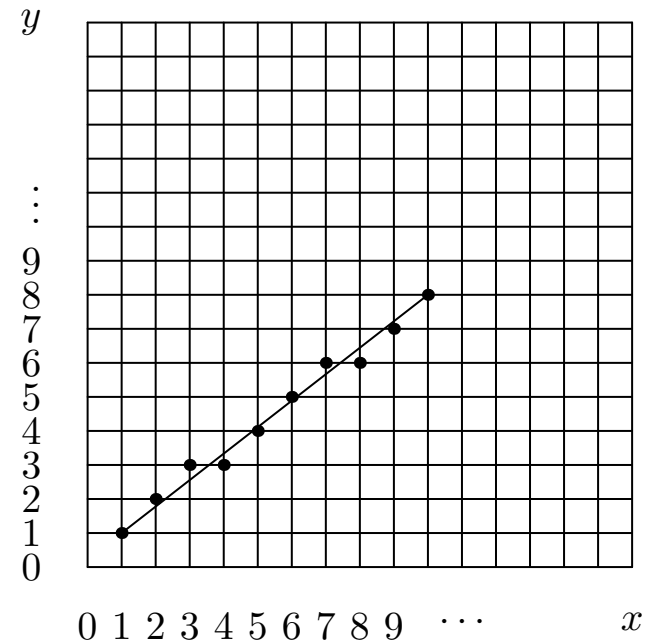
$$\begin{aligned} y_{i+1} &= m \cdot x_{i+1} + B \\ &= m(x_i + 1) + B \\ &= m \cdot x_i + B + m \\ &= y_i + m \end{aligned}$$

En DDA-implementation

```

for  $x \in x_0 \dots x_1$ 
    write( $x_i, \lfloor y_i + 0.5 \rfloor$ )
     $y_{i+1} := y_i + m$ 

```



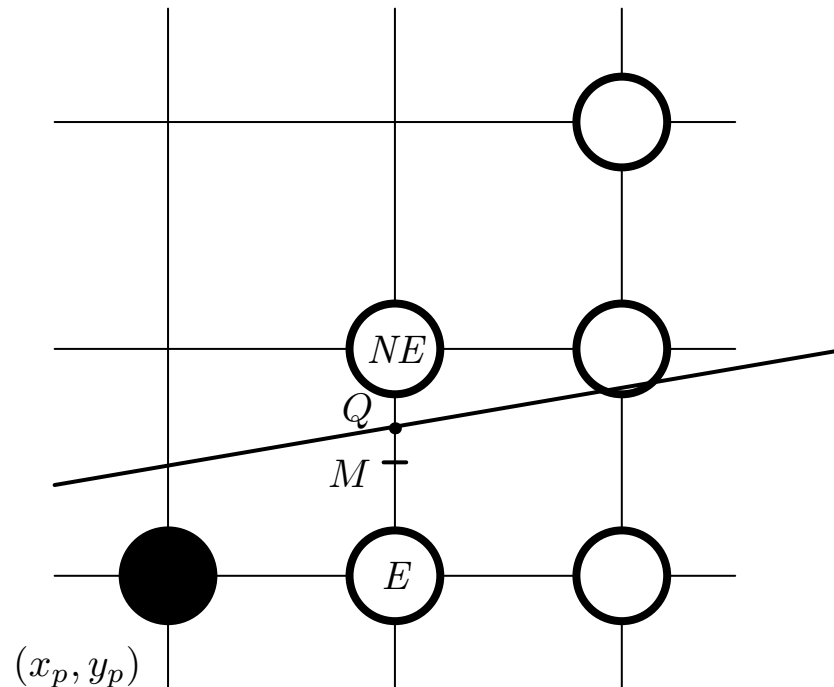
Nackdelar med DDA

- Fortfarande baserat på flyttal.
- Avrundningsfel kan ackumuleras.

Körexempel: $(1, 1) - (10, 8)$

Bresenham's algorithm (Midpoint-algoritmen)

Antag att $m \in [0, 1]$ och att vi just plottat (x_p, y_p) .

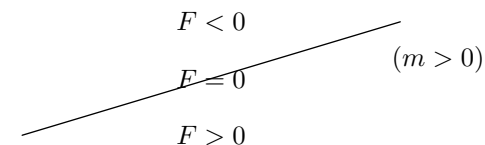


Fråga: Ska vi gå till NE eller E härnäst?

Linjens ekvation: $y = \frac{\Delta y}{\Delta x}x + B$.

Implicit form: $F(x, y) = \Delta y \cdot x - \Delta x \cdot y + B \cdot \Delta x = 0$.

$F(x, y) = 0$ så länge man befinner sig på linjen:



Titta på $F(M) = F(x_p + 1, y_p + 1/2)$

$$F(M) = \Delta y(x_p + 1) - \Delta x(y_p + 1/2) + B \cdot \Delta x$$

- Om $F(M) \leq 0$, gå till E :

$$F(x_p + 2, y_p + 1/2) = F(M) + \Delta y$$

- Om $F(M) > 0$, gå till NE :

$$F(x_p + 2, y_p + 3/2) = F(M) + \Delta y - \Delta x$$

Dvs, $F(M)$ kan uppdateras inkrementellt!

Första punkten är på linjen, dvs $F(x_0, y_0) = 0$.

Första värdet på $F(M)$ är därför

$$\begin{aligned} F(x_0 + 1, y_0 + 1/2) &= \Delta y(x_0 + 1) - \Delta x(y_0 + 1/2) + B\Delta x \\ &= F(x_0, y_0) + \Delta y - \Delta x/2 \\ &= \Delta y - \Delta x/2 \end{aligned}$$

Endast tecknet på $F(M)$ intressant, så skala allt med 2.

Egenskaper:

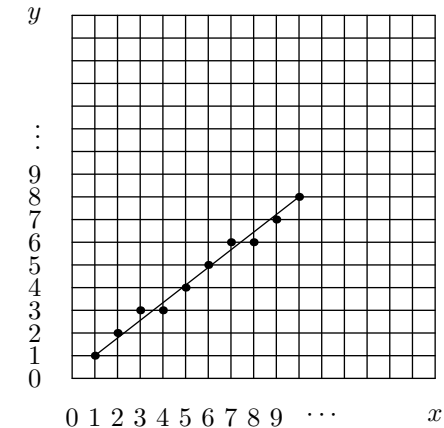
- Bara heltal.
- Bara addition-, skift-, och test-instruktioner.
- Exakt.

Bresenham's algorithm för linjedragning

```

 $x := x_0; y := y_0;$ 
 $F(M) := 2\Delta y - \Delta x;$ 
write( $x, y$ );
while ( $x < x_1$ )
    if  $F(M) \leq 0$  then    — go east
         $F(M) := F(M) + 2\Delta y;$ 
         $x := x + 1;$ 
    end else                — go north-east
         $F(M) := F(M) + 2(\Delta y - \Delta x);$ 
         $x := x + 1; y := y + 1;$ 
    end
    write( $x, y$ );
end

```



Körexempel: (1, 1)–(10, 8)

Vad använder riktiga arbetsstationer?

Silicon Graphics: DDA med fixpunktsaritmetik.

1024 sveplinjer ger minsta $m = \frac{1}{1024}$.

I princip är allting skalat med 1024:

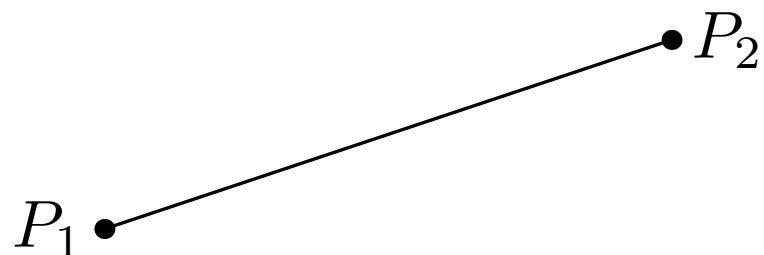
10 bits	.	10 bits
---------	---	---------

dvs $m = 1/1024$ kodas som 0000000000.0000000001.

(Jämför t ex en affär där priset 3.75 lagras som 375.)

Addition utförs på 20 bits heltal men **write**(x, y)
tittar bara på de första 10 bitarna.

Varför $y = mx + b$ inte används inom datorgrafik



- Kan inte representera vertikala linjer ($m = \infty$).
- Kan bara representera oändliga linjer, inte ändliga linjesegment.
- Kan bara representera 2D-linjer, inte 3D.

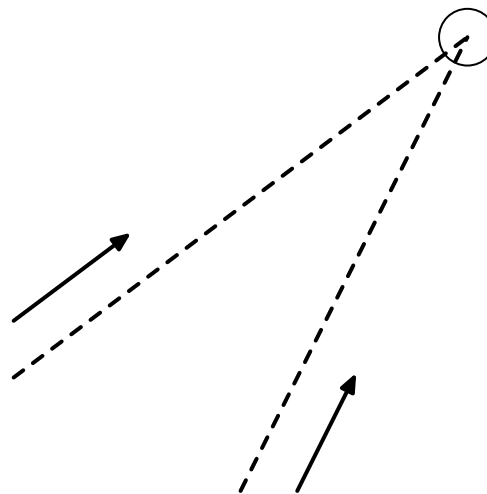
Vi kommer senare att använda *parametrisk form* för att undvika dessa problem.

Designbeslut för linjedragning

- **Ändpunktsordning:**

Samma pixlar ska tändas oavsett om linjen ritas från $(x_0, y_0) - (x_1, y_1)$ eller från $(x_1, y_1) - (x_0, y_0)$.

Byta ändpunkter ser inte bra ut om man ritar med mönster:



Designbeslut för linjedragning (forts)

- **Klippning:**

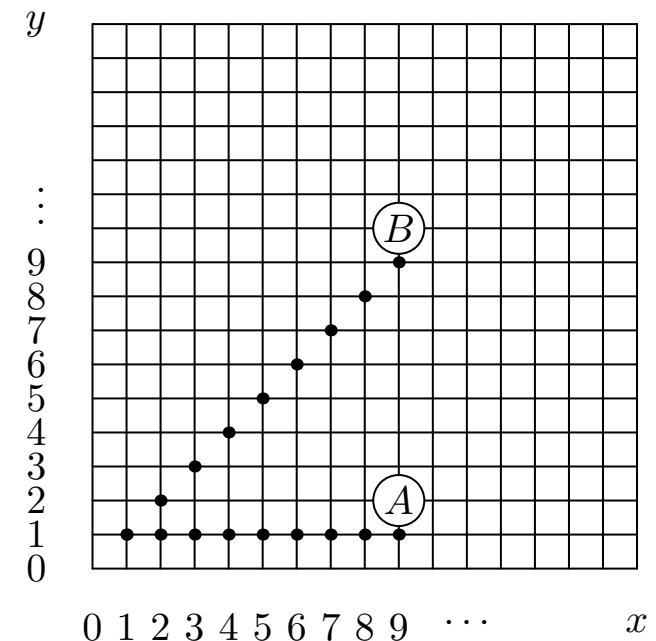
Om linjen måste klippas för att passa fönstret kan man få starta på icke-heltals-koordinater. Se boken.

- **Varierande intensitet:**

Linje A och B har samma antal pixlar.

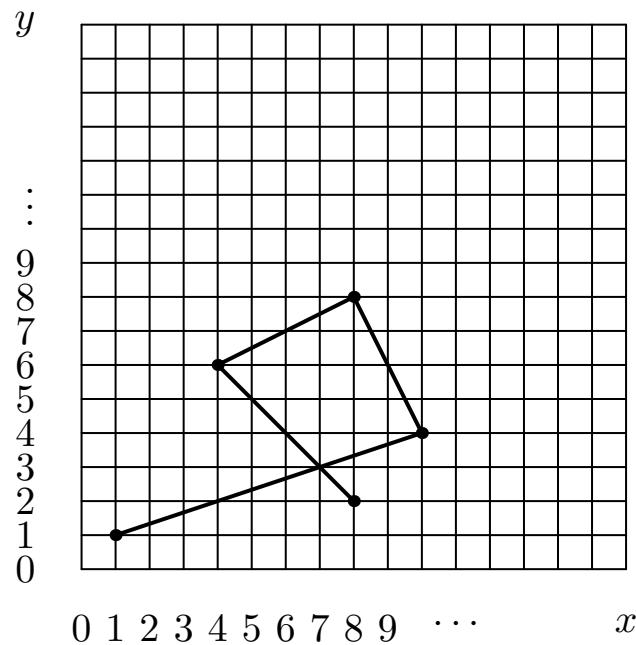
A har mer pixels per längdenhet.

Minska A 's intensitet med $\cos \theta$, $\theta \leq 45^\circ$.



Polylinjer

För polylinjer måste man undvika att besöka samma pixel flera gånger, speciellt om man ritar med XOR.



Svepkonvertering av cirklar, ellipser

Tas inte upp i kursen (används inte i 3D-grafik).

Boken innehåller information om Bresenham's algoritm.

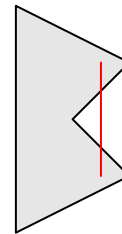
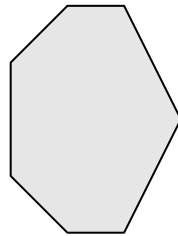
Se också

How many ways can you draw a circle?

Jim Blinn, IEEE Computer Graphics, aug 1987.

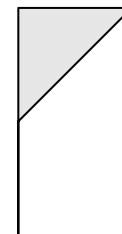
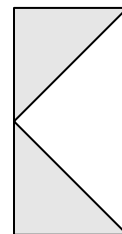
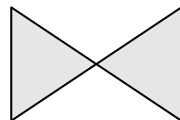
Svepkonvertering av polygoner

Enkla polygoner är *konvexa* eller *konkava*:

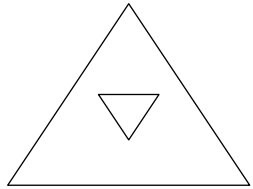


Konkav: Kan dra en linje mellan två invärtes punkter som passerar polygonens utsida.

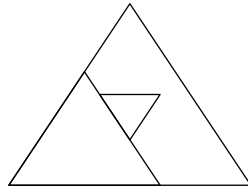
Enkla polygoner saknar hål, skärningar, etc.



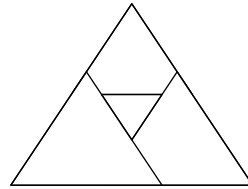
Godtyckliga polygoner kan delas upp i enkla



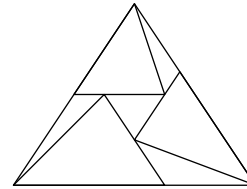
Med hål



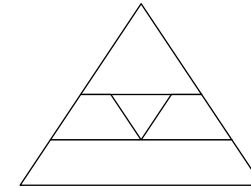
2 enkla



3 konvexa

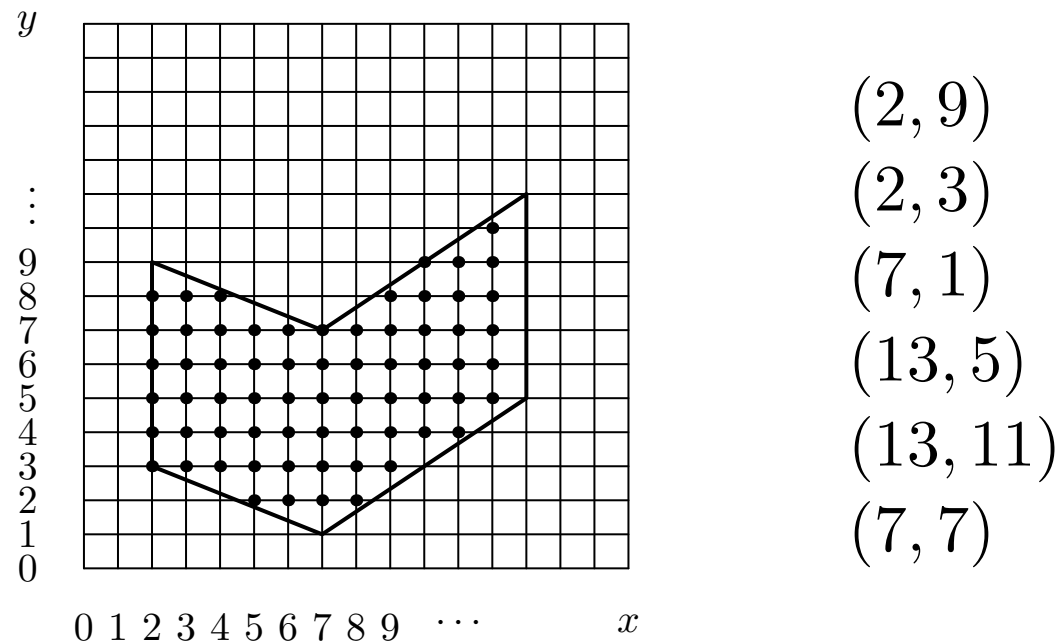


6 trianglar



4 parallelltrapetser

En polygon är en ordnad lista av hörnpunkter



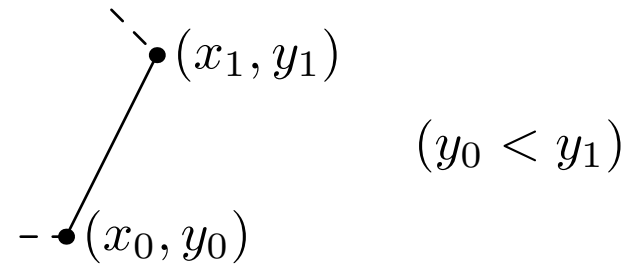
Algoritmen får *inte* sätta punkter utanför polygonen.

Duger alltså inte att rita kantlinjerna och fylla innanmätet eftersom linjealgoritmen kan sätta punkter utanför polygonen.

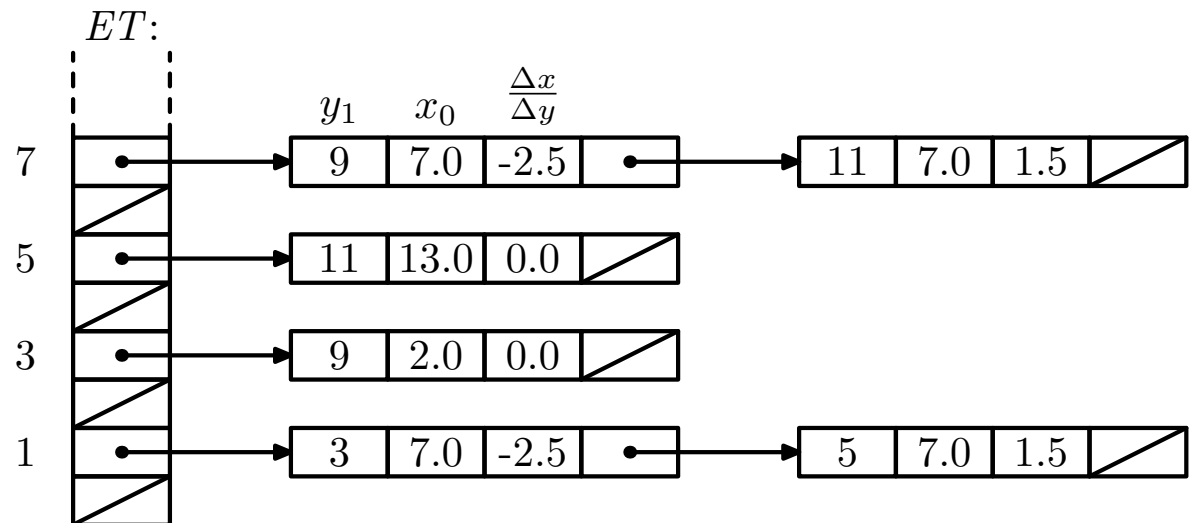
Kanttabell

Horisontella kanter ignoreras.

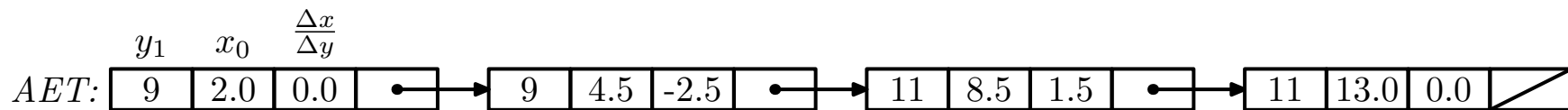
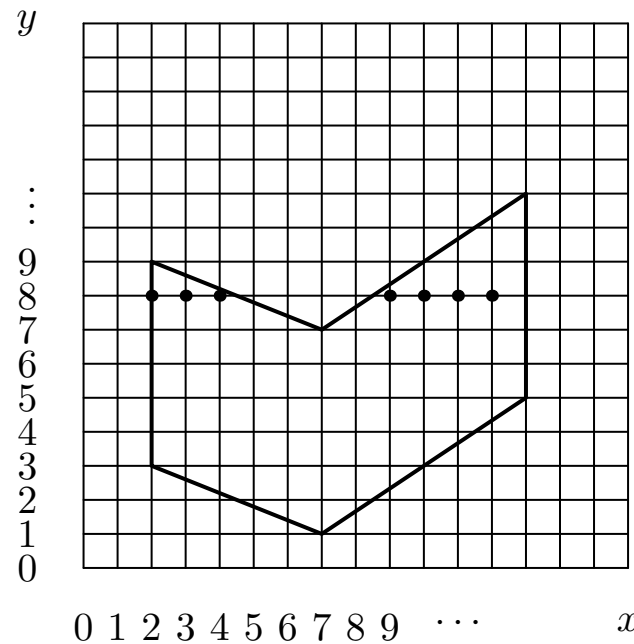
Varje annan kant representeras som en post $\langle y_1, x_0, \Delta x / \Delta y \rangle$.



Kanter med samma y_0 lagras i en länkad lista vars huvud är $ET[y_0]$:



För en given sveplinje upprätthålls en *aktiv kanttabell*:



Paritetsregeln: sveplinjen fylls genom att traversera *AET* och växelvis slå av/på pennan för varje passerad kant.

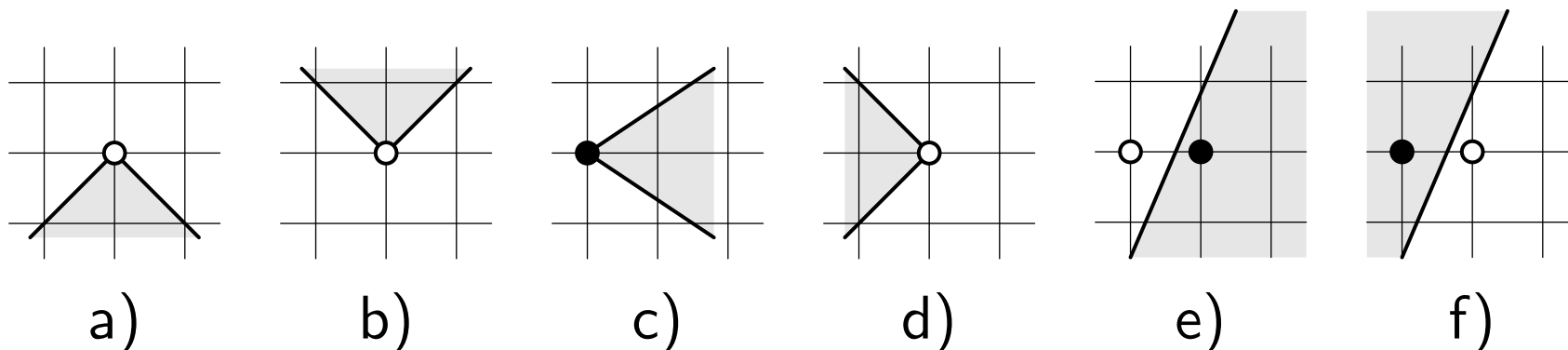
Algoritmen för svepkonvertering av polygoner

1. $y :=$ minsta icke-lediga index i ET .
2. $AET := null$.
3. Flytta alla kanter i $ET[y]$ till AET .
4. Ta bort kanter ur AET med $y_1 = y$.
5. Sortera AET med avseende på x .
6. Fyll sveplinjen med paritetsregeln.
7. $y := y + 1$.
8. Uppdatera kanterna i AET : $x := x + \Delta x / \Delta y$.
9. Upprepa (3–8) tills $AET = null$.

Designbeslut för svepkonvertering av polygoner

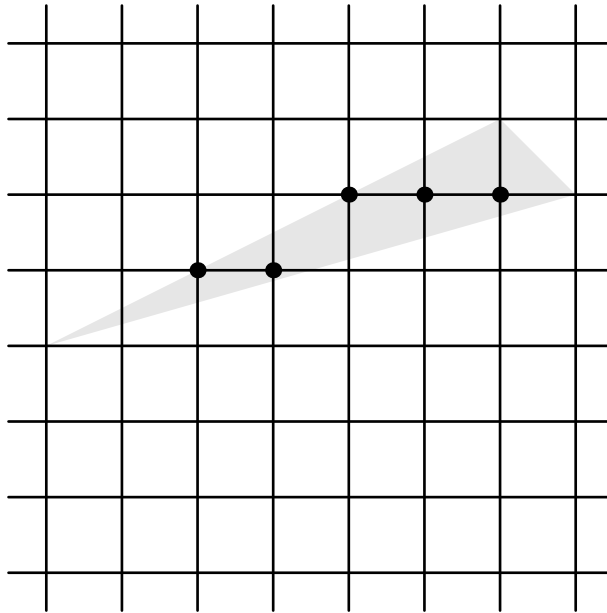
Angränsande polygoner: I ett "lapptäcke" av polygoner ska det helst inte existera glipor eller överlappning.

Det bör alltså vara möjligt att fylla en yta uppbyggd av mindre polygoner, även i XOR-mode. Viktigt med konsekventa regler!



Om flera fall är applicerbara: sätt inte pixeln!

Strimlor



Exempel på en digital representation av en analog signal.

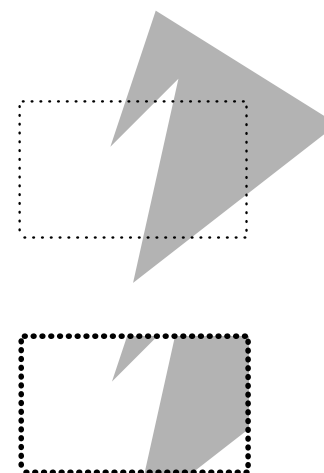
Kan lösas med anti-aliasing.

Klippning av polygoner

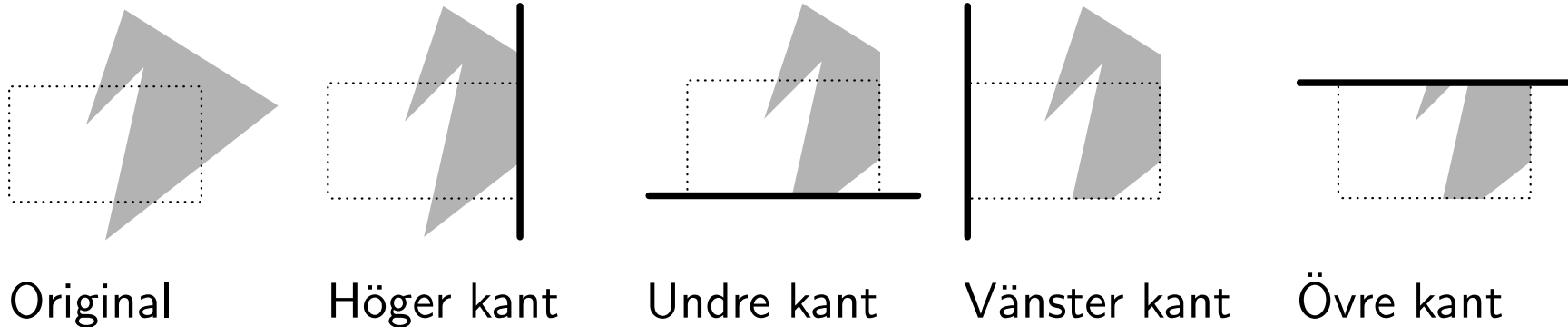
Vi hoppar över klippning av linjer. Man kan använda en polygonklippare istället, fast det är inte lika effektivt.

Sutherland-Hodgmans algoritm

- Klipper konvexa polygoner korrekt.
- Konkava polygoner kan ge extra linjer.
- Klipp-polygonen (fönstret) kan vara konvex.
- Kan generaliseras till 3D.



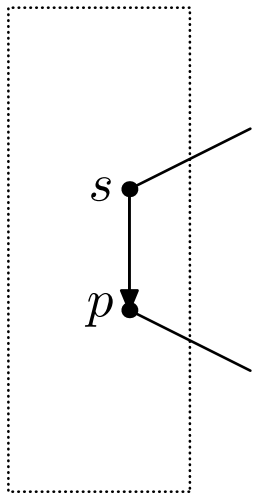
Klippning sker kant för kant



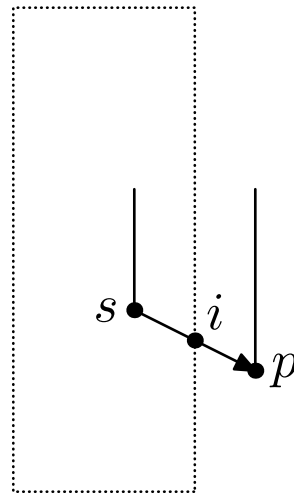
- Varje steg tar en sekvens av kanter och producerar en ny sekvens av kanter (oftast fler).
- Kan arrangeras som en pipeline.

De fyra fallen

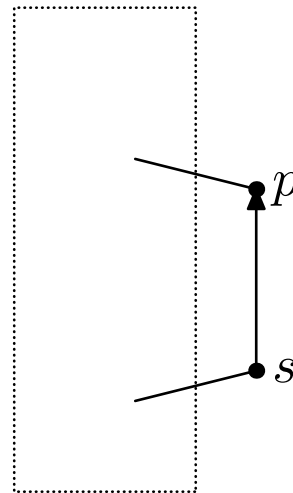
- I varje steg traverseras polygonen, kant för kant.
- För varje kant inträffar något av fyra fall:



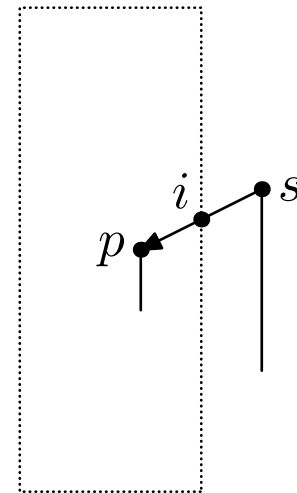
1. in-in:
mata ut p



2. in-ut:
mata ut i



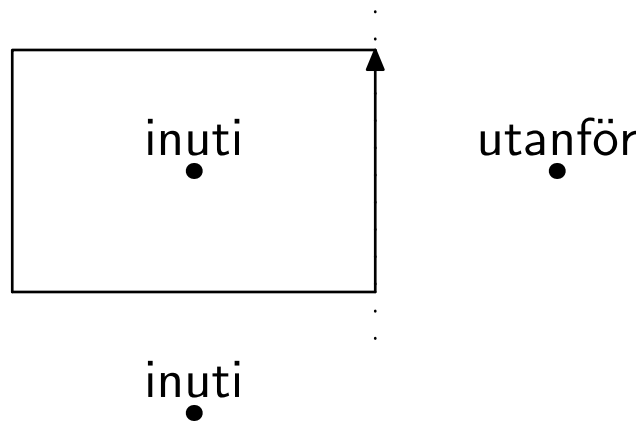
3. ut-ut:
ingen utmatning



4. ut-in:
mata ut i, p

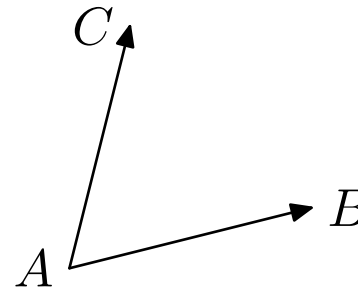
Verktyg

Inuti betyder *till vänster om* när man följer fönstrets kanter moturs.

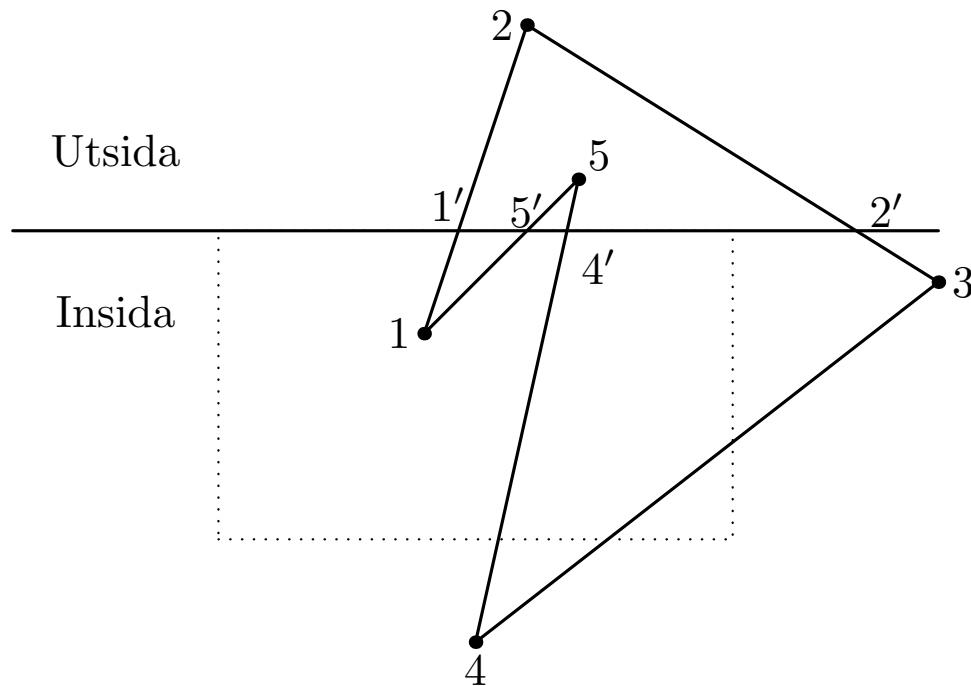


Hur kan man avgöra om en punkt är till vänster om en vektor?

Om z -komponenten i $\vec{AB} \times \vec{AC} > 0$
så är C till vänster om $\vec{AB}$.



Exempel: klipp polygonen längs övre kanten

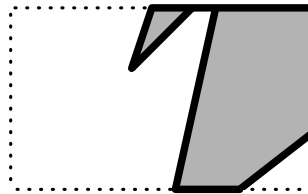


<i>Kant</i>	<i>Fall</i>	<i>Nya punkter</i>
1-2	in-ut	1'
2-3	ut-in	2', 3
3-4	in-in	4
4-5	in-ut	4'
5-1	ut-in	5', 1

Den nya polygonen representeras av 1'-2'-3-4-4'-5'-1-1'.

Klippning av konkava polygoner

Klippning av konkava polygoner kan leda till "extra" linjer.



Hantering:

- Dela upp i konvexa polygoner.
- Efterbehandling av grannpunkter på fönstrets kant.
- Använd en mer generell rutin, t ex Weiler-Athertons.

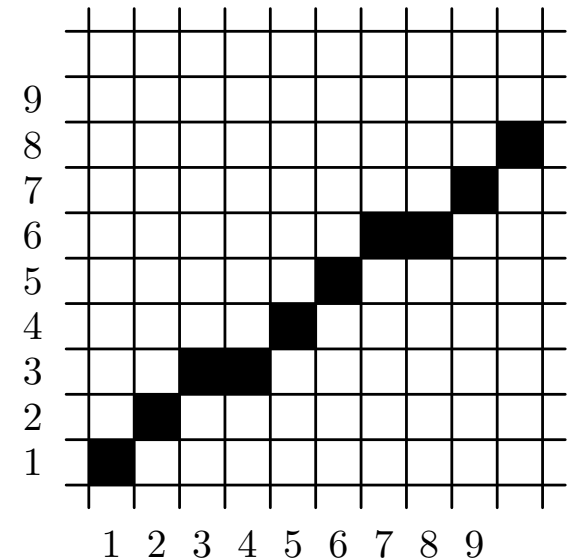
Anti-aliasing

Avrundning av koordinatvärden till heltal ger trappstegseffekt (*aliasing*) för vektorer och polygonkanter.

Hantering av aliasing kallas *anti-aliasing*

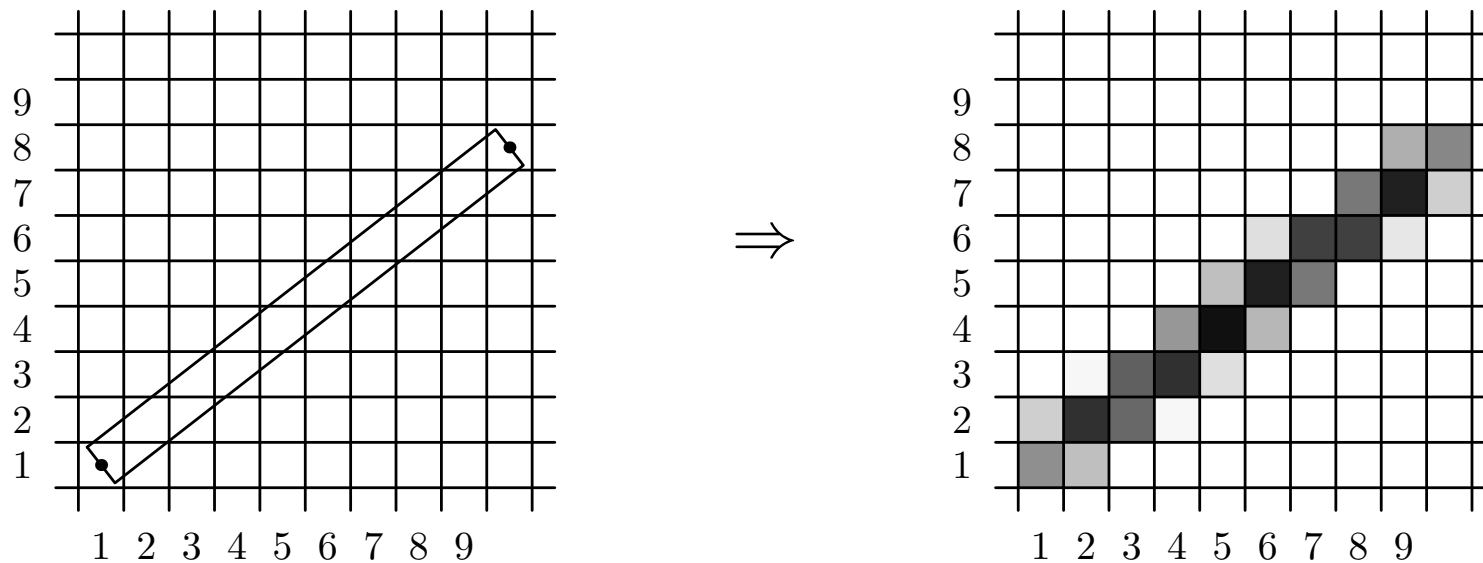
Hanteringen beror på typen av skärm:

- **Monokrom skärm:** högre upplösning.
- **Gråskale/färg:** area- eller super-sampling.



Area-sampling

- En pixel antas ha en viss utsträckning.
- Linjer antas vara en pixel breda.
- Intensiteten på en pixel är proportionell mot hur stor yta av den som täcks av linjen eller polygonen.



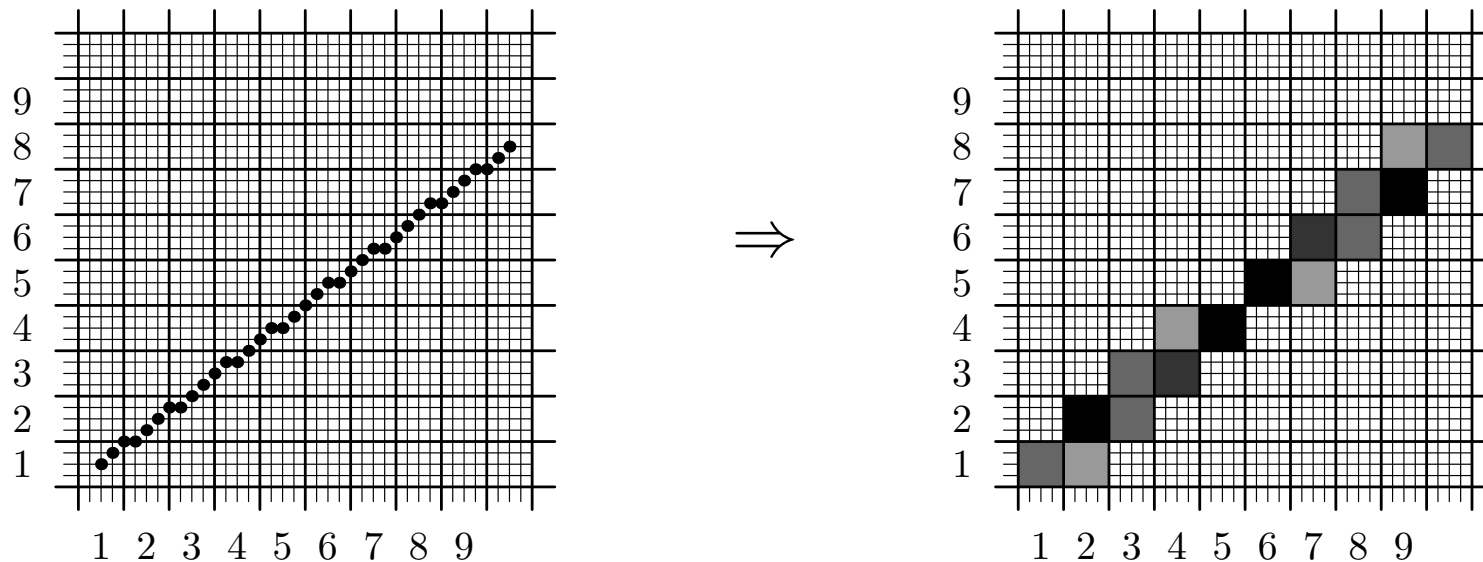
Interpolering mot bakgrund

- P = polygonens intensitet;
- B = bakgrundens intensitet;
- f = andel av pixelytan som täcks;
- $\Rightarrow$ pixelns intensitet $= fP + (1 - f)B$.

Supersampling

Dela upp varje pixel i subpixels. Pixelns intensitet är proportionell mot antalet subpixels som tänds.

För en oändligt tunn linje:



Supersampling

- Kan generaliseras till ändligt breda linjer och polygoner.
- Viktad supersampling:
Ge mer vikt åt subpixels nära pixelns centrum
- För en 3×3 -indelning, exempelvis:

1	2	1
2	4	2
1	2	1

OpenGL och svepkonvertering

```
glBegin(type);  
    glVertex2i( $x_1$ ,  $y_1$ );  
    :  
glEnd();
```

- **Punkter:** *type* = GL_POINTS.

Storleken sätts med `glPointSize(GLfloat size);`

- **Linjer:** *type* = GL_LINES.
- **Polylinjer:** *type* = GL_LINE_STRIP.

OpenGL och svepkonvertering (forts)

```
glBegin(type);  
    glVertex2i( $x_1$ ,  $y_1$ );  
    :  
glEnd();
```

- **Polygoner:** *type* = `GL_POLYGON`.

OpenGL klarar dock bara konvexa polygoner.

Konkava polygoner måste *tesseleras* (se GLU).

- **Anti-aliasing:**

Möjligt med hjälp av α -kanal, se guiden (`aargb.c`).

Lab 1 i OpenGL

- Ur defs.h:

```
typedef struct _point {  
    GLint x, y;  
} point;
```

- Ur scan3.c:

```
int scan3(point p[]) {  
    glBegin(GL_TRIANGLES);  
        glVertex2i(p[0].x, p[0].y);  
        glVertex2i(p[1].x, p[1].y);  
        glVertex2i(p[2].x, p[2].y);  
    glEnd();  
    glFlush();  
}
```

Föreläsning 5: Geometrisk transformationer

Vektorer

- Operationer på vektorer

- Skalarprodukt, Kryssprodukt

Matriser

- Matrismultiplikation, Transponering och Invers

2D-transformationer

- Homogena koordinater

- Sammansättning av 2D-transformationer

3D-transformationer

- Inverser till 2D och 3D-transformationer

- Affina avbildningar

- Stelkroppsavbildningar

3D-transformationer i OpenGL

- Mellanspel: språket Postscript

- Sammansatta transformationer i OpenGL

Vektorer

En n -dimensionell vektor har formen $\begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$.

Observera att en vektor inte är samma sak som en punkt!

Båda har samma form, men:

- En punkt är en *position*.
- En vektor är en *förflyttning*.

Vi använder $u, v, w, \dots$ för att beteckna vektorer.

Operationer på vektorer

- Längden på en vektor u är $|u| = \sqrt{x_1^2 + \dots + x_n^2}$.

Om $|u| = 1$ kallas u *enhetsvektor*.

- Addition: $u + v$

$$\begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} + \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} x_1 + y_1 \\ \vdots \\ x_n + y_n \end{pmatrix}$$

- Multiplikation med skalär: au

$$a \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} ax_1 \\ \vdots \\ ax_n \end{pmatrix}$$

Operationer på vektorer (forts)

- Linjärkombination av $v_1, \dots, v_n$:

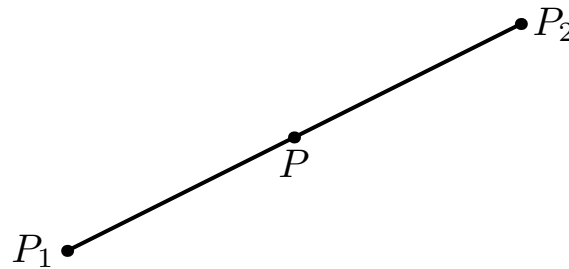
$$\sum_{i=1}^n \alpha_i v_i$$

- Konvexkombination av $v_1, \dots, v_n$:

$$\sum_{i=1}^n \alpha_i v_i$$

Om alla $\alpha_i \geq 0$ och $\sum_{i=1}^n \alpha_i = 1$.

Parametrisk form för linjens ekvation



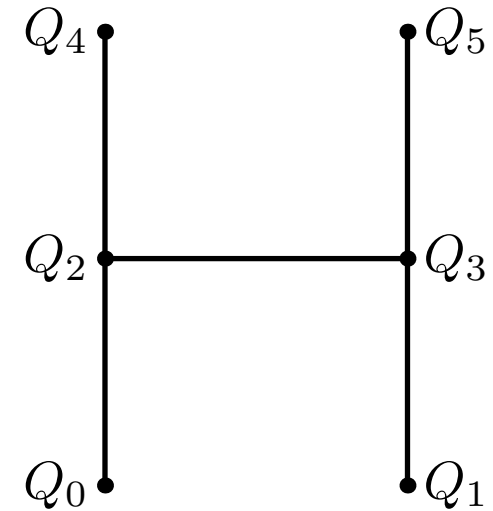
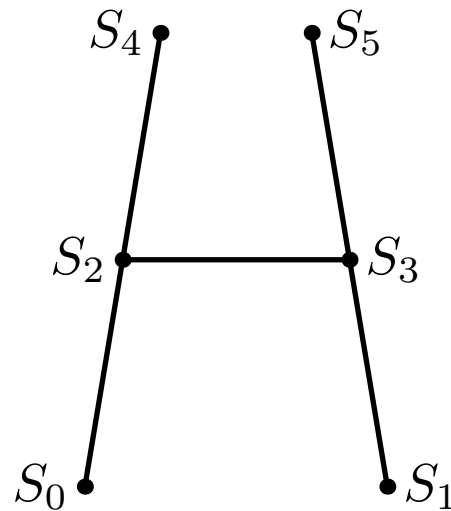
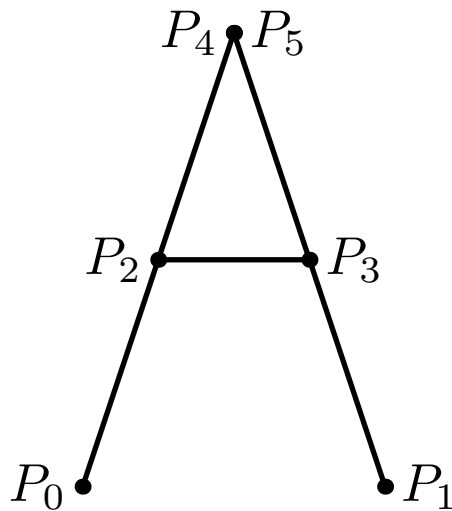
$$P = P_1 + t(P_2 - P_1) \quad t \in [0, 1]$$

Kan också framställas som en konvexkombination:

$$P = (1 - t)P_1 + tP_2 \quad t \in [0, 1]$$

Denna teknik kallas *blending* och kan användas på godtyckliga kvantiteter (färg, form, position, etc).

Exempel: interpolera mellan två figurer



$$S_i = (1 - t)P_i + tQ_i \quad t = 0.5$$

Operationer på vektorer (forts)

- Skalärprodukt: $u \cdot v$

$$\begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \sum_{i=1}^n x_i y_i$$

Observera att resultatet är en skalär.

Vinkeln θ mellan två vektorer ges av

$$\cos \theta = \frac{u \cdot v}{|u||v|}$$

Om u och v är enhetsvektorer så är $\cos \theta = u \cdot v$.

Skalärprodukt: applikationer för datorgrafik

- $u \cdot v = 0$ om u och v är ortogonala.
- Om $|v| = 1$ så gäller

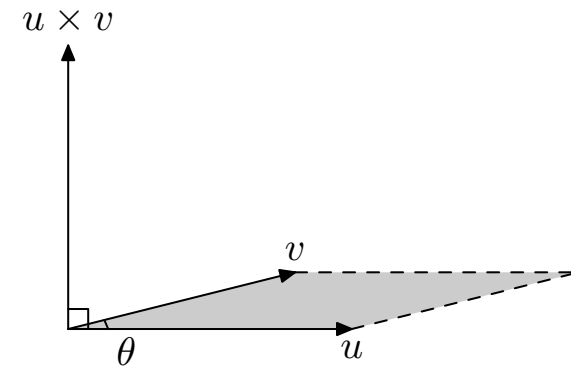
$$u \cdot v = |u||v| \cos \theta = |u| \cos \theta$$

dvs, $u \cdot v$ projicerar u på v .

Operationer på vektorer (forts)

- Kryssprodukt: $u \times v$: För 3D-vektorer:

$$\begin{pmatrix} x_1 \\ y_1 \\ z_1 \end{pmatrix} \times \begin{pmatrix} x_2 \\ y_2 \\ z_2 \end{pmatrix} = \begin{pmatrix} y_1 z_2 - z_1 y_2 \\ z_1 x_2 - x_1 z_2 \\ x_1 y_2 - y_1 x_2 \end{pmatrix}$$



Egenskaper:

- ▷ $|u \times v| = |u||v| \sin \theta$, dvs ytan av parallelogrammet.
- ▷ $u \times v$ är ortogonal mot både u och v .

Användbart för att generera normaler, testa konvexitet, etc.

Matriser

En $n \times m$ -matris har formen

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nm} \end{pmatrix}$$

Element a_{jk} står i rad j och kolonn k .

Vi kommer att intressera oss för $n, m \in \{2, 3, 4\}$.

Operationer på matriser

- Matrismultiplikation: $C = AB$

$$c_{ij} = \sum_{s=1}^m a_{is} b_{sj}$$

$$\begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{pmatrix}$$

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{pmatrix}$$

- m = antalet kolonner i A = antalet rader i B .

Några lagar för matrismultiplikation

- BA inte alltid lika med AB .

- $C(BA) = (CB)A$.

- $C(A + B) = CA + CB$

- Identitetsmatrisen,

$$I = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}$$

- För godtycklig matris A gäller $AI = IA = A$.

Operationer på matriser (forts)

- Transponering: A^T

$n \times m$ -matris $\Rightarrow m \times n$. Exempel:

$$\text{Om } A = \begin{pmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \end{pmatrix} \text{ så är } A^T = \begin{pmatrix} 3 & 1 \\ 1 & 5 \\ 4 & 9 \end{pmatrix}.$$

- Invers: A^{-1}

Om $AB = I$ så är B inversen till A och skrivs A^{-1} .

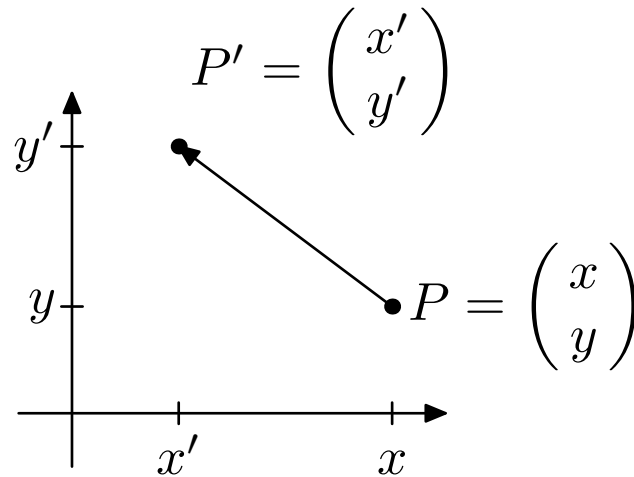
Endast icke-singulära $n \times n$ -matriser har invers.

Om $A^{-1} = A^T$ kallas A *ortogonal*

Effektivt att implementera: använd $A[j][i]$ istf $A[i][j]$.

2D-transformationer

För punkter:

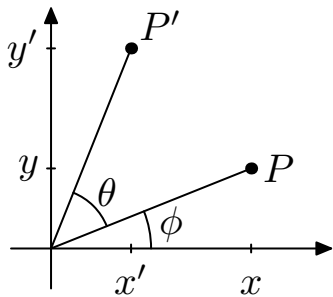


- Translation: $P' = P + T(d_x, d_y)$ $T(d_x, d_y) = \begin{pmatrix} d_x \\ d_y \end{pmatrix}$
- Skalning: $P' = S(s_x, s_y)P$ $S(s_x, s_y) = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$

2D-transformationer

- Rotation: $P' = R(\theta)P$ $R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$

▷ Vinkeln θ räknas moturs. Härledning av $R(\theta)$:



$$x = r \cos \phi \quad (1)$$

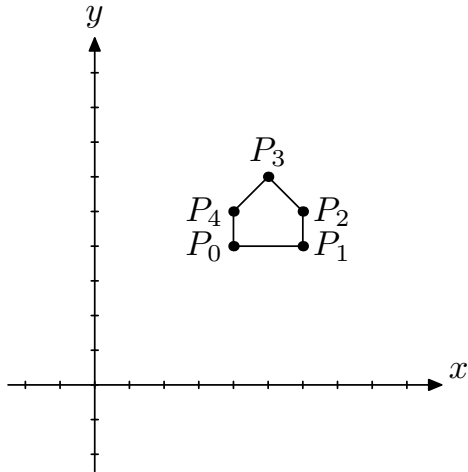
$$y = r \sin \phi \quad (2)$$

$$x' = r \cos(\phi + \theta) = r \cos \phi \cos \theta - r \sin \phi \sin \theta \quad (3)$$

$$y' = r \sin(\phi + \theta) = r \sin \phi \cos \theta + r \cos \phi \sin \theta \quad (4)$$

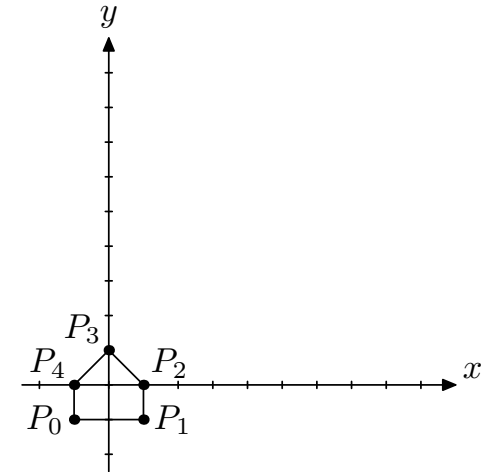
▷ Insättning av 1&2 i 3&4 ger $R(\theta)P$.

Exempel



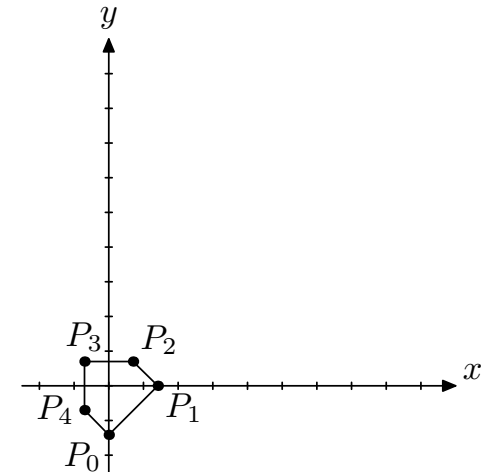
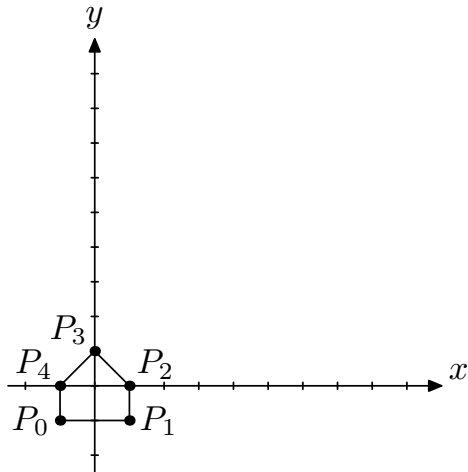
Translation:

$$P_i = P_i + \begin{pmatrix} -5 \\ -5 \end{pmatrix}$$



Rotation:

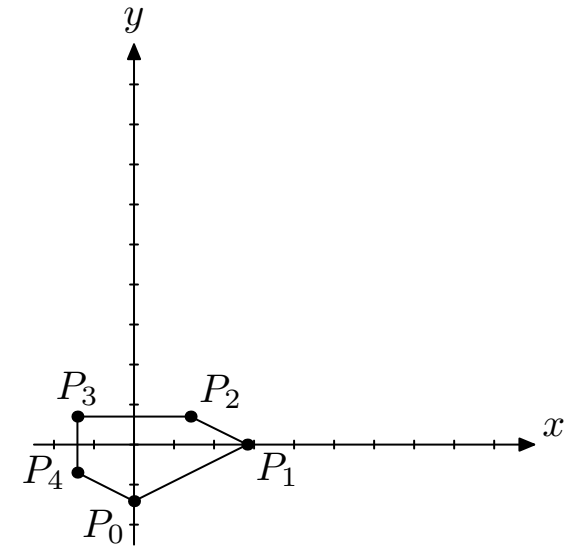
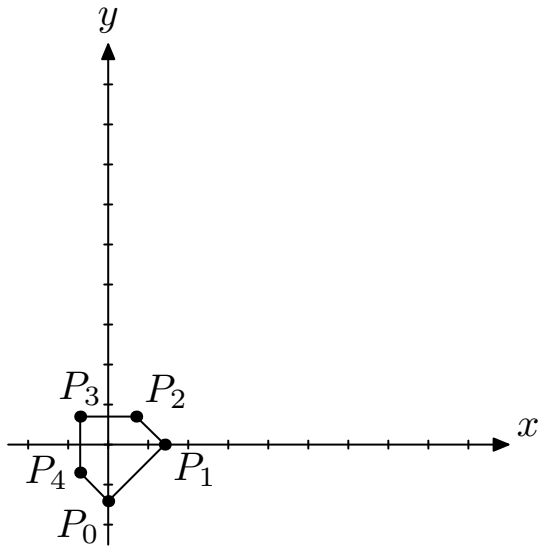
$$P_i = \begin{pmatrix} \cos 45^\circ & -\sin 45^\circ \\ \sin 45^\circ & \cos 45^\circ \end{pmatrix} P_i$$



Exempel (forts)

Skalning:

$$P_i = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} P_i$$



Homogena koordinater

Vi vill utföra *alla* transformationer med matrismultiplikation.

⇒ Förenklad implementation.

⇒ Enkel sammansättning av transformationer.

Lösning: Representera med homogena koordinater:

- Från 2D till homogen koordinat: $(x, y) \longrightarrow (x, y, 1)$
- Från homogen till 2D-koordinat: $(x, y, W) \longrightarrow (x/W, y/W)$
- Homogenisering: $(x, y, W) \longrightarrow (x/W, y/W, 1)$

Exempel: $(4, 0, 2)$ och $(2\pi, 0, \pi)$ representerar båda $(2, 0)$.

Homogen representation av 2D-transformationer

Givet $P = (x \ y \ 1)^T$

- Translation: $P' = T(d_x, d_y)P$

$$T(d_x, d_y) = \begin{pmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{pmatrix}$$

- Skalning: $P' = S(s_x, s_y)P$

$$S(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Rotation: $P' = R(\theta)P$

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Sammansättning av 2D-transformationer

Matrismultiplikation är associativ, så t ex

$$P' = T_2(T_1P) = (T_2T_1)P$$

- Translation är additiv:

$$T(d_{x2}, d_{y2})T(d_{x1}, d_{y1}) = \begin{pmatrix} 1 & 0 & d_{x1} + d_{x2} \\ 0 & 1 & d_{y1} + d_{y2} \\ 0 & 0 & 1 \end{pmatrix}$$

- Skalning är multiplikativ:

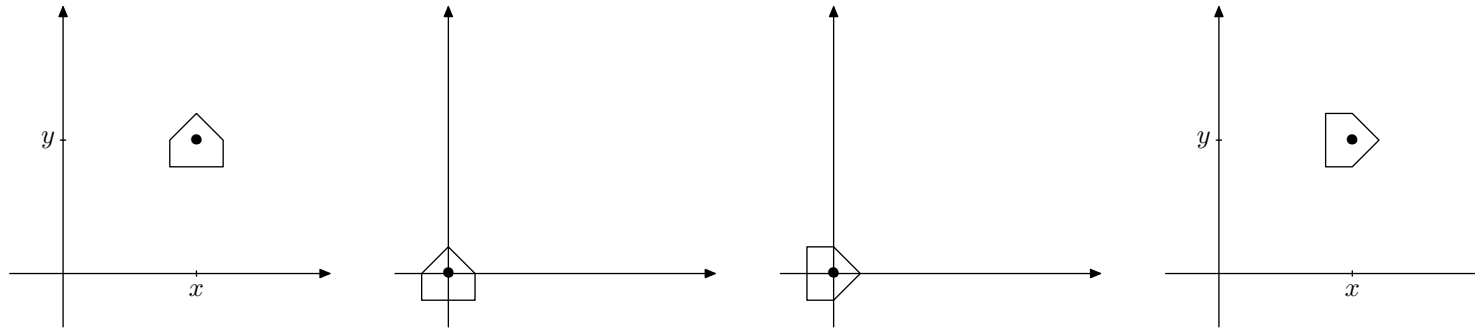
$$S(s_{x2}, s_{y2})S(s_{x1}, s_{y1}) = \begin{pmatrix} s_{x1}s_{x2} & 0 & 0 \\ 0 & s_{y1}s_{y2} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Sammansättning av 2D-transformationer (forts)

- Rotation är additiv:

$$R(\theta_2)R(\theta_1) = \begin{pmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Exempel: rotation kring en godtycklig punkt



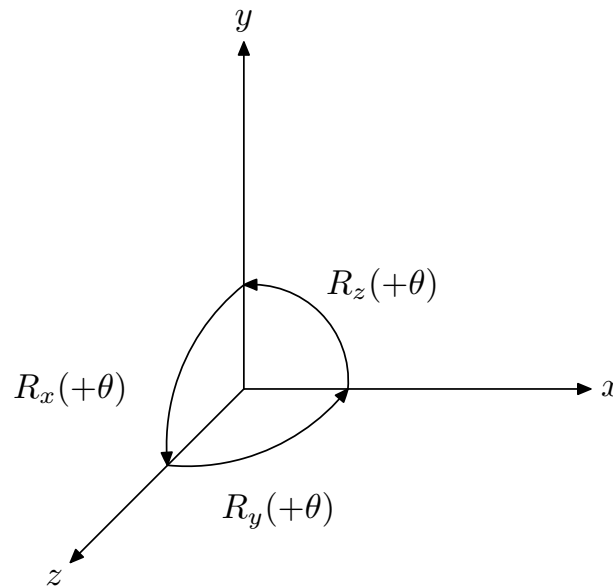
Sammanfatt transformation: $T(x, y) R(\theta) T(-x, -y) =$

$$\begin{pmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -x \\ 0 & 1 & -y \\ 0 & 0 & 1 \end{pmatrix} =$$

$$= \begin{pmatrix} \cos \theta & -\sin \theta & x(1 - \cos \theta) + y \sin \theta \\ \sin \theta & \cos \theta & y(1 - \cos \theta) - x \sin \theta \\ 0 & 0 & 1 \end{pmatrix}$$

3D-transformationer

Koordinatsystemet är ett högersystem:



Omvandling till homogena koordinater: $(x, y, z) \longrightarrow (x, y, z, 1)$

Transformationerna i 3D är utökningar av 2D-fallen

- Translation:
$$T(d_x, d_y, d_z) = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Skalning:
$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Rotation:
$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Inverser till (2D och) 3D-transformationer

- Translation $T^{-1}(d_x, d_y, d_z) = T(-d_x, -d_y, -d_z)$
- Skalning: $S^{-1}(s_x, s_y, s_z) = S(1/s_x, 1/s_y, 1/s_z)$
- Rotation: $R_{\{x,y,z\}}^{-1}(\theta) = R_{\{x,y,z\}}(-\theta) = R_{\{x,y,z\}}^T(\theta)$

eftersom R alltid är ortogonal.

Transformation av linjer, plan och dess normal

- **Linjer:** Transformera ändpunkterna.

- **Plan:** Transformera 3 punkter.

- **Normal:** $Ax + By + Cz + D = 0 \Rightarrow N = \begin{pmatrix} A \\ B \\ C \\ D \end{pmatrix}$

Planet är alla punkter $P = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$ så att $N \cdot P = 0$.

Efter $P' = MP$, vad blir N' ? Ansätt $N' = QN$.

$$0 = (QN) \cdot (MP) = (QN)^T (MP) = N^T (Q^T M) P$$

Gäller t ex om $Q^T M$ är en multipel av I .

Om $Q^T M \equiv I$: $Q^T = M^{-1} \iff Q = (M^{-1})^T$

Så vi transformerar N med $(M^{-1})^T$ (om den finns).

Exempel på inverser till 3D-transformationer

- Translation: $T(d_x, d_y, d_z) = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$

$$Q = (T^{-1})^T = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -d_x & -d_y & -d_z & 1 \end{pmatrix}$$

Exempel på inverser (forts)

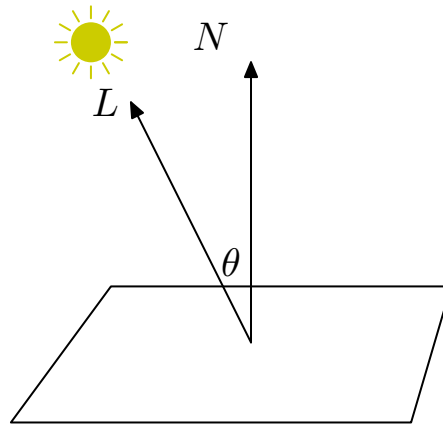
- Rotation: $Q = (R^{-1})^T = R$ eftersom R är ortogonal ($R^{-1} = R^T$).

- Skalning: $Q = (S^{-1})^T = \begin{pmatrix} 1/s_x & 0 & 0 & 0 \\ 0 & 1/s_y & 0 & 0 \\ 0 & 0 & 1/s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

Fungerar dock inte för perspektivprojektion (kap 6) då invers ibland saknas.

Exempel på användningsområde

Lamberts reflektionsmodell: reflekterad intensitet $\propto N \cdot L$.



Vid transformationer:

- Använd MP för svepkonvertering av polygonen.
- Använd QN för beräkning av reflekterat ljus.

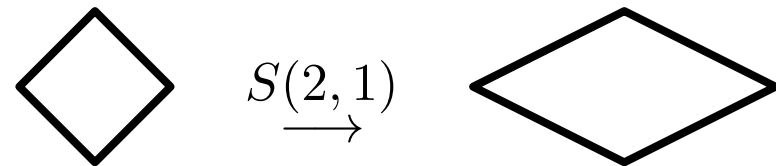
Affina avbildningar

För $2D \rightarrow 2D$:
$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Exempel: Translation, rotation, skalning, skjuvning.

Egenskaper:

- Bevarar *inte* längder.
- Bevarar *inte* vinklar.
- Bevarar parallella linjer och raka linjer.



Stelkroppsbildningar

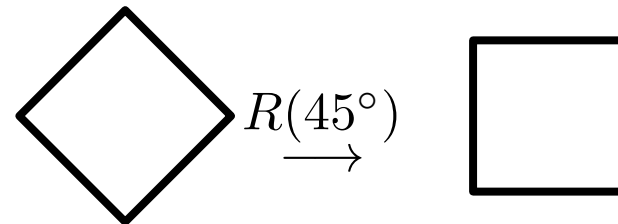
För 2D $\rightarrow$ 2D:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

där den skuggade delmatrisen är en ortogonal.

Exempel: Translation, rotation.

Egenskaper:



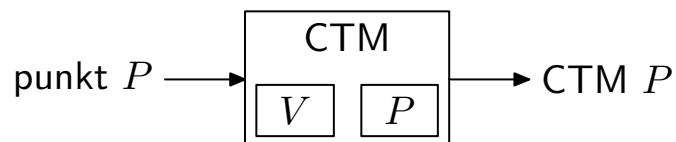
- Bevarar längder, vinklar, parallella linjer, raka linjer.

3D-transformationer i OpenGL

- Alla punkter transformeras med den så kallade *Current Transformation Matrix*, CTM.



- CTM är produkten av den så kallade *model-view*-matrisen och *projektions*-matrisen.



- Model-view-matrisen V beskriver förhållandet mellan kamerans vykoordinatsystem och världskoordinatsystemet.
- Vi låter projektions-matrisen $P = I$ tills vidare.

Operationer för model-view-matrisen V

- För att manipulera *model-view*-matrisen:

```
glMatrixMode(GL_MODELVIEW);
```

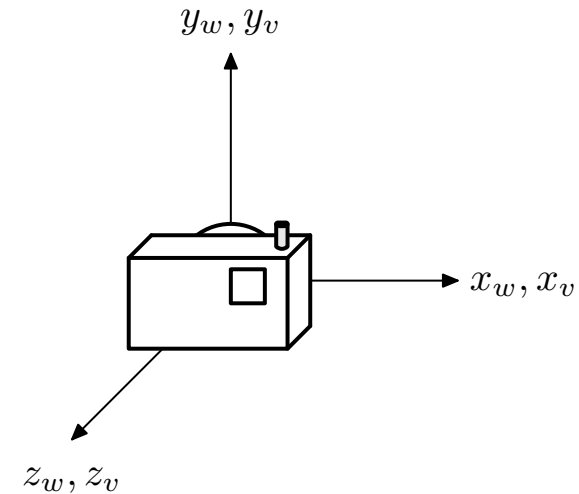
Model-view-matrisen är nu *aktuell matris*.

<code>glLoadIdentity()</code>	$V \leftarrow I$
<code>glLoadMatrixf(A)</code>	$V \leftarrow A$
<code>glMultMatrixf(A)</code>	$V \leftarrow VA$
<code>glTranslatef(x, y, z)</code>	$V \leftarrow V T(x, y, z)$
<code>glScalef(s_x, s_y, s_z)</code>	$V \leftarrow V S(s_x, s_y, s_z)$
<code>glRotatef(θ, v_x, v_y, v_z)</code>	$V \leftarrow V R(\theta, v_x, v_y, v_z)$

- A antas vara typdeklarerad som `GLfloat A[16]`.
- $R(\theta, v_x, v_y, v_z)$ roterar θ° runt axeln (v_x, v_y, v_z) .

OpenGL-exempel

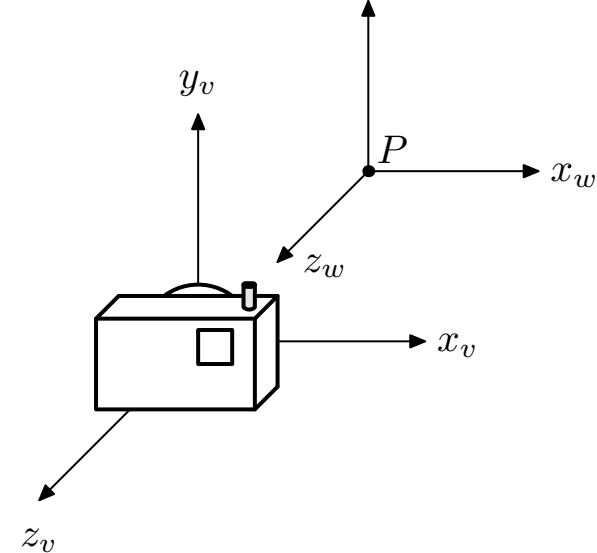
- Initialt sammanfaller kamerans vykoordinatsystem och världskoordinatsystemet.
- Flytta först världskoordinatsystemet 100 enheter längs $-z_w$:
- Placera sedan en punkt P i $(0, 0, 0)$:



```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0, 0, -100);
```

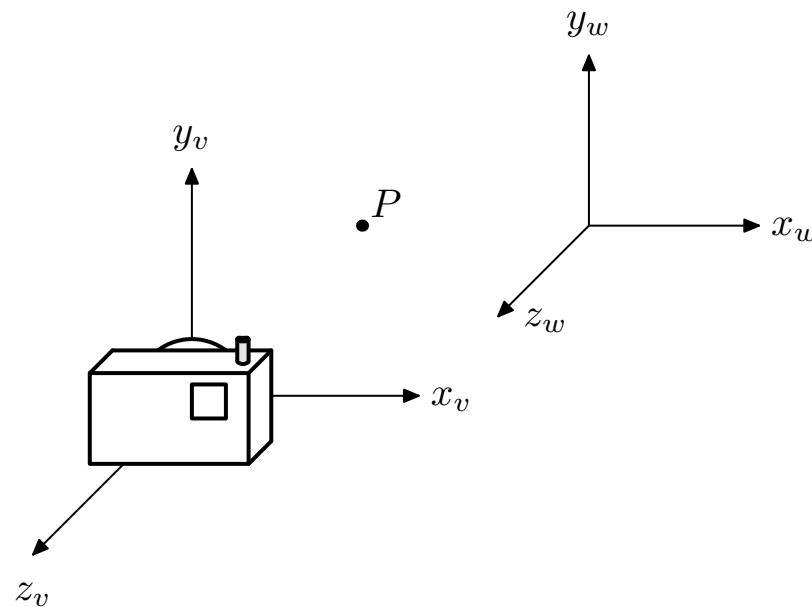
```
glBegin(GL_POINTS);
glVertex3i(0,0,0);
glEnd();
```

- I världskoordinater är $P = (0, 0, 0)_w$.
- I vykoordinater är $P = (0, 0, -100)_v$.



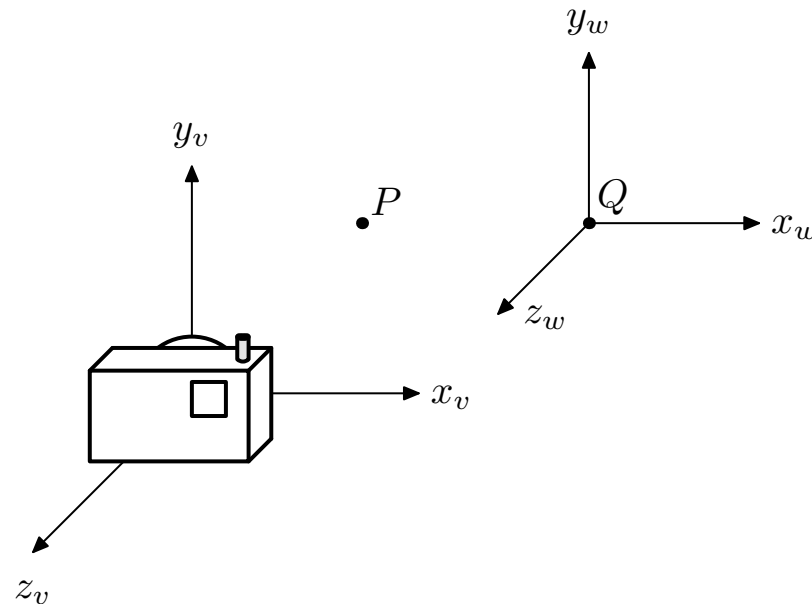
- Flytta världskoordinatsystemet igen:

`glTranslatef(50, 0, 0);`



- Placera en punkt Q i $(0, 0, 0)$:

```
glBegin(GL_POINTS);  
glVertex3i(0,0,0);  
glEnd();
```



Slutsats

1. Model-view-matrisen relaterar världskoordinatsystemet till kamerans vykoordinatsystem.
2. `glVertex()` använder världskoordinatsystemet. Eller, ekvivalent:
`glVertex()` använder kamerans vykoordinatsystem och transformerar punkten med model-view-matrisen.
3. Att förflytta/vrida modellen är ekvivalent med att förflytta/vrida kameran åt andra hållet.

Läs mer i OpenGL Programming Guide, kap 3.

Se också programmet `transformation` i *Interactive OpenGL*.

Mellanspel: språket Postscript

- Postscript är ett stackbaserat språk för skrivare och dokument.
- $2 + 3$ skrivs som `2 3 add`.
- PDF är en semikompilerad variant av Postscript.
- Postscript har intressanta 2D-transformationer.

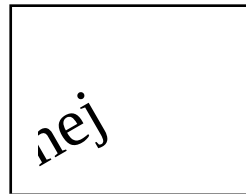
```
0 0 moveto
(hej) show
```



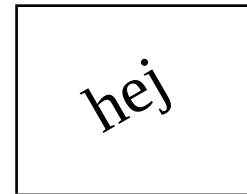
```
50 0 translate
0 0 moveto
(hej) show
```



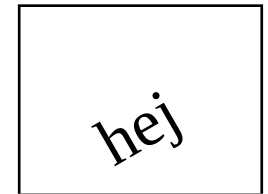
```
30 rotate
0 0 moveto
(hej) show
```



```
30 rotate
50 0 translate
0 0 moveto
(hej) show
```



```
50 0 translate
30 rotate
0 0 moveto
(hej) show
```



Korrespondens mellan PostScript och OpenGL

<i>Postscript</i>	<i>OpenGL</i>
30 rotate 50 0 translate <i>draw something</i>	<code>glRotatef(30, 0, 0, 1);</code> <code>glTranslatef(50, 0, 0);</code> <i>draw something</i>

Sammanfatta transformationer i OpenGL

- Kom ihåg: matricmult. är associativ: $ABC = A(BC) = (AB)C$

- Två sätt att räkna ut resultatet:

- OpenGL använder metod 2.

- GL-anropen sker i "bakvänd" ordning.

Metod 1	Metod 2
$M \leftarrow I$	$M \leftarrow I$
$M \leftarrow CM$	$M \leftarrow MA$
$M \leftarrow BM$	$M \leftarrow MB$
$M \leftarrow AM$	$M \leftarrow MC$

```
glLoadIdentity();           // initiera
glTranslatef(x, y);         // sist: flytta (x,y)
glRotatef(theta, 0, 0, 1);  // rotera runt z-axeln
glTranslatef(-x, -y, 0);     // först: flytta (-x,-y)
```

Föreläsning 6: 3D-Bilder

Den syntetiska kameran

Renderingskedjan

Från världs- till vykoordinater

Projektion

Taxonomi över projektioner

Specifikation av perspektivvy

Den stympade pyramiden

Parametrar till lab 2

Perspektivtransformationen P

Härledning av P

Klippning

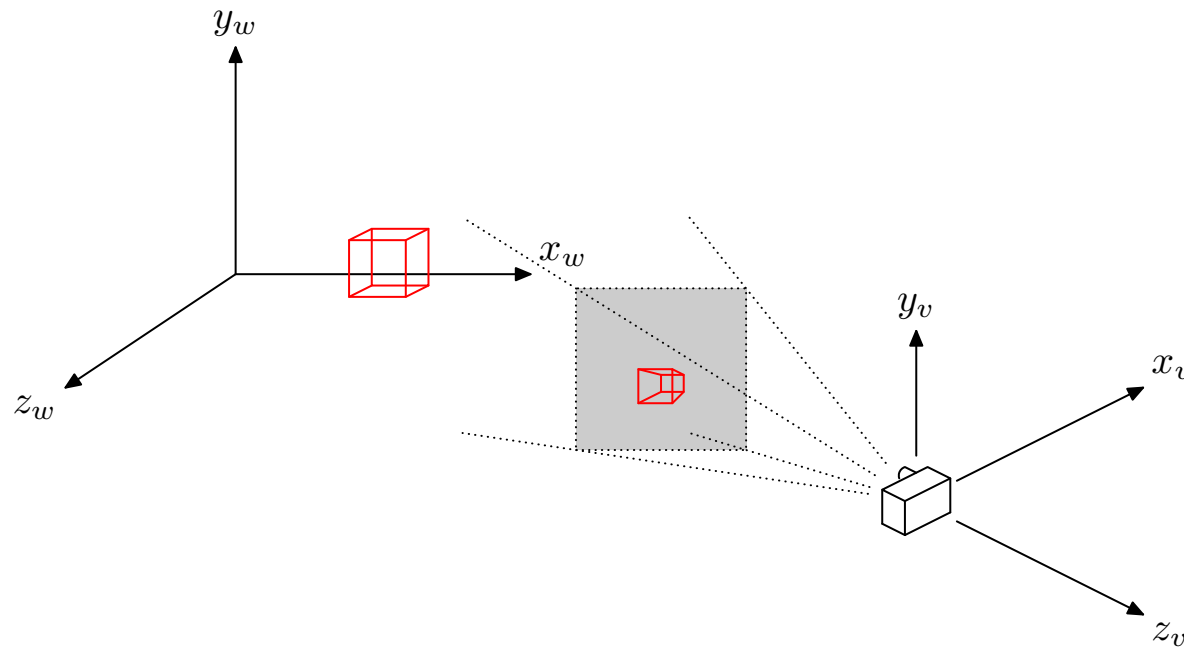
Transformation till skärmkoordinater

The rendering pipeline

Specifikation av vy i OpenGL

Lab 2 i OpenGL

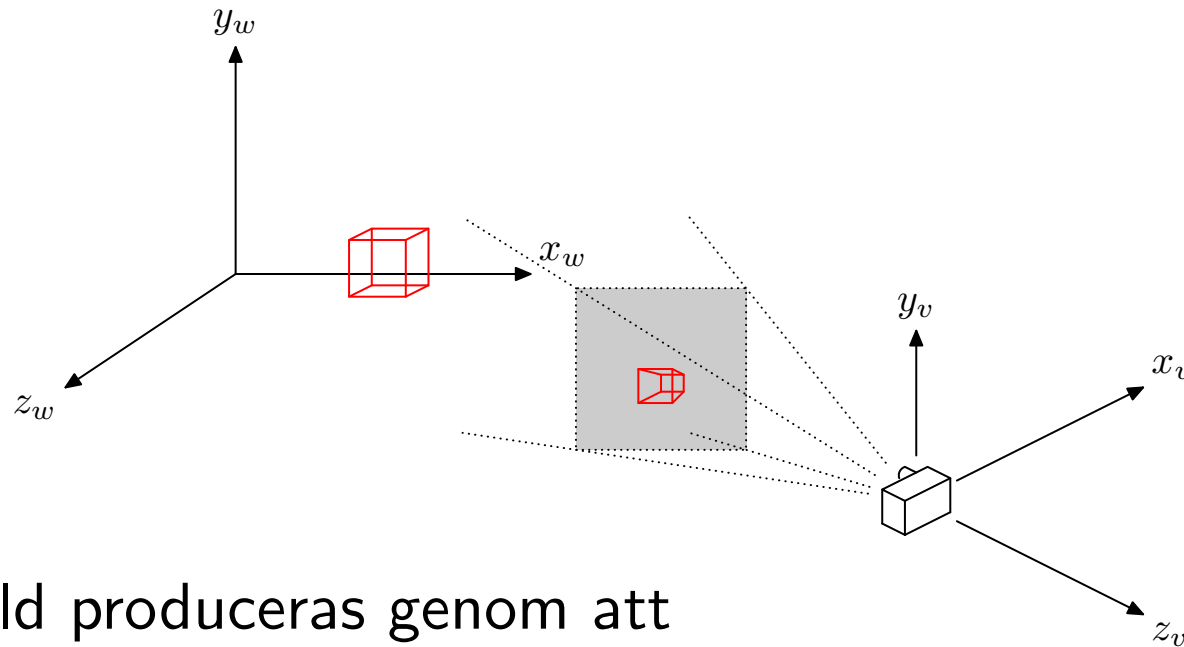
Specifikation av vyn – den syntetiska kameran



Alla objekt, inklusive kameran, har en position i världskoordinatsystemet (x_w, y_w, z_w) .

Kamerans sk vykoordinatsystem (x_v, y_v, z_v) kan flyttas och vridas.

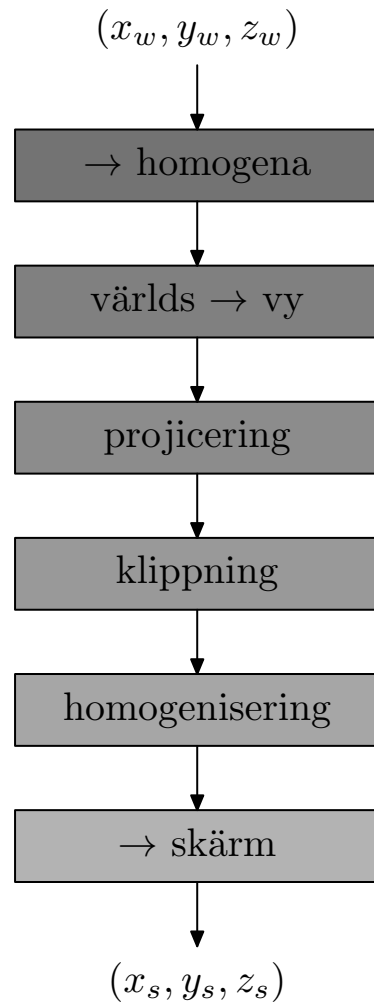
Den syntetiska kameran



Kamerans bild produceras genom att

1. Transformera världs- till vykoordinater;
2. Projicera från 3D till 2D;
3. Klippa bort objekt som inte syns (bakom, vid sidan).

Renderingskedjan — “the rendering pipeline”



Världskoordinater

$$p_w = \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix}$$

$$p_v = V \ p_w$$

$$p_p = P \ p_v$$

$$p_s = S \ p_p$$

(x_s, y_s) används för plottning
 z_s innehåller djupinformation

Från världs- till vykoordinater

- **Situation:** Kameran befinner sig i (x_w, y_w, z_w) och har vridit sitt vykoordinatsystem (ϕ_x, ϕ_y, ϕ_z) runt respektive axel.
- **Uppgift:** Uttryck alla objekt i kamerans vykoordinatsystem.
- **Lösning:** Transformera alla objekt med

$$V = R_z(-\phi_z) R_y(-\phi_y) R_x(-\phi_x) T(-x_w, -y_w, -z_w)$$

Andra namn för ϕ_x, ϕ_y och ϕ_z är *pitch*, *yaw* och *roll*, se boken avsnitt 5.2.

Projektion

Vi studerar *plangeometrisk*a projektioner.

- **Plan** = projicera till ett plan.

Motexempel: Planetarium.

- **Geometrisk**a = raka linjer $\rightarrow$ raka linjer.

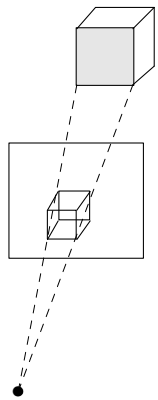
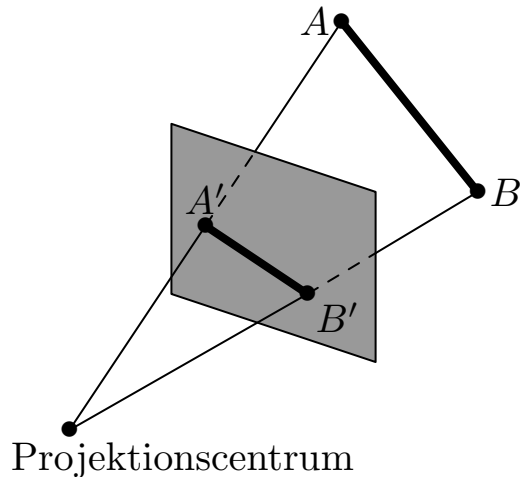
Motexempel: Mercator-projektion för kartor.

- **Projektioner** = 3D till 2D.

Motexempel: Tesseract ($4D \rightarrow 3D \rightarrow 2D$)

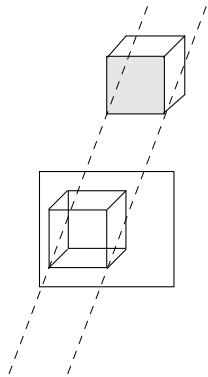
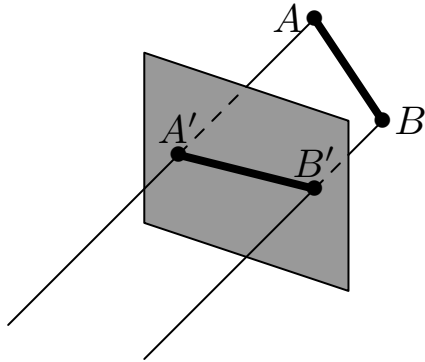
Två varianter: *Perspektivprojektion* och *Parallellprojektion*.

Perspektivprojektion



- Projektionen av en punkt A är skärningspunkten A' mellan projektionsstrålen och projektionsplanet.
- Strålen är en linje som förbinder punkten med projektionscentrum.
- Ger intryck av *djup*: ju längre bort ett objekt befinner sig, desto *mindre* blir dess projektion.
- Syftlinjen ej nödvändigtvis $\perp$ med planet.

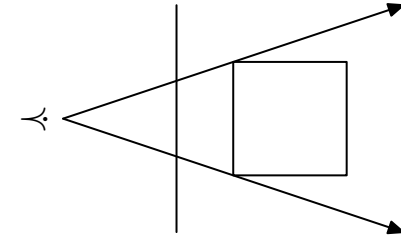
Parallellprojektion



- Samma som perspektivprojektion med projektionscentrum i ∞ .
- Innehåller ingen information om objektens avstånd till projektionsplanet.
- Användbart för ritningar: mäta vinklar och längder.

Taxonomi över projektioner

1. Perspektiv

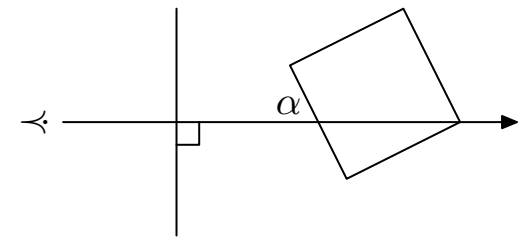


2. Parallell

(a) Ortografisk

i. Ortogonal: $\alpha = 90^\circ$

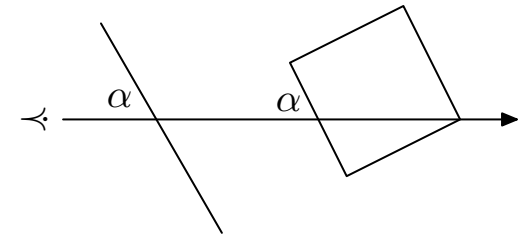
ii. Axonometrisk: $\alpha \neq 90^\circ$



(b) Sned

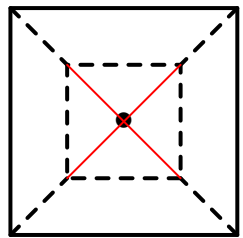
i. Kavaljer: $\alpha = 45^\circ$

ii. Kabinett: $\tan \alpha = 2$

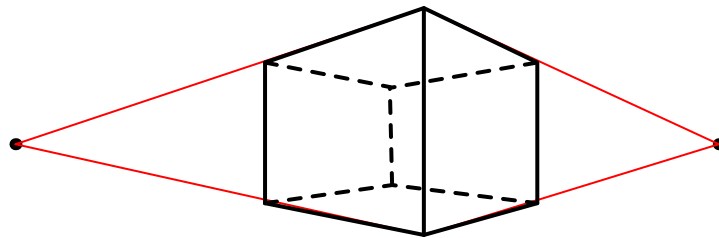


Perspektivprojektion

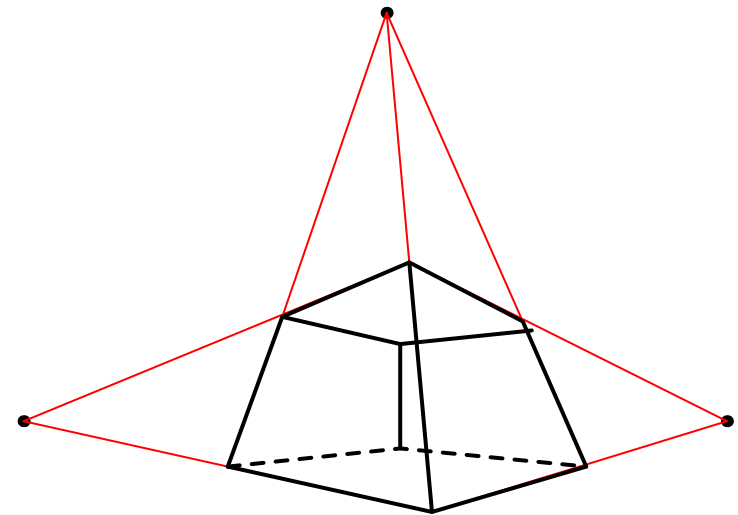
Olika typer beroende på hur *många axlar projektionsplanet skär*.



1-punkts

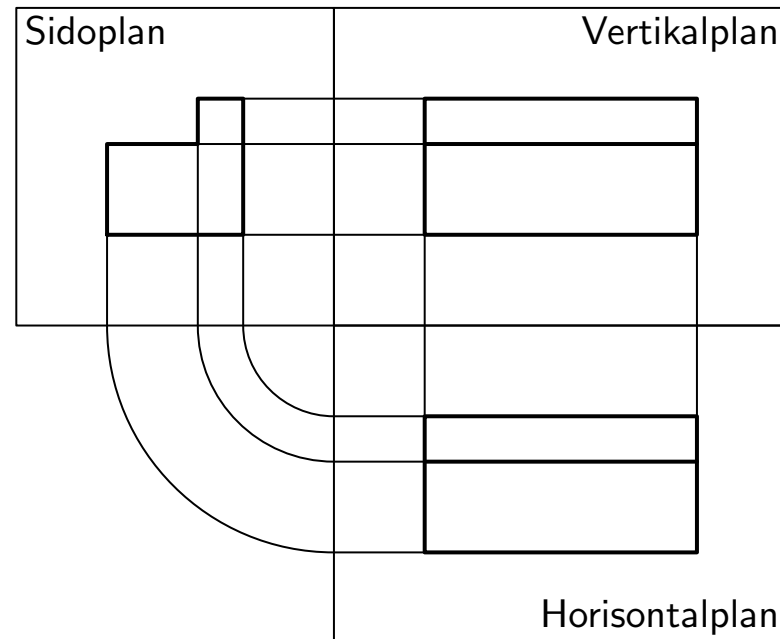


2-punkters



3-punkters

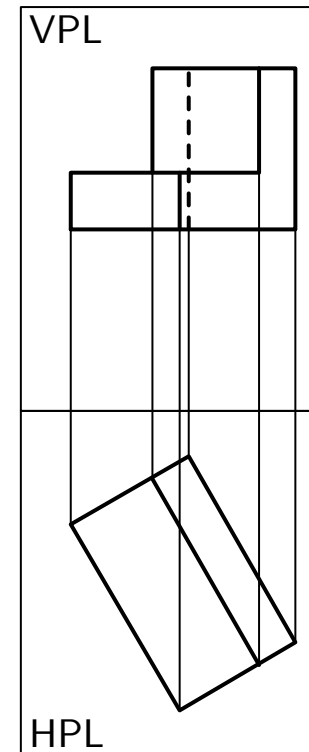
Ortogonal projektion



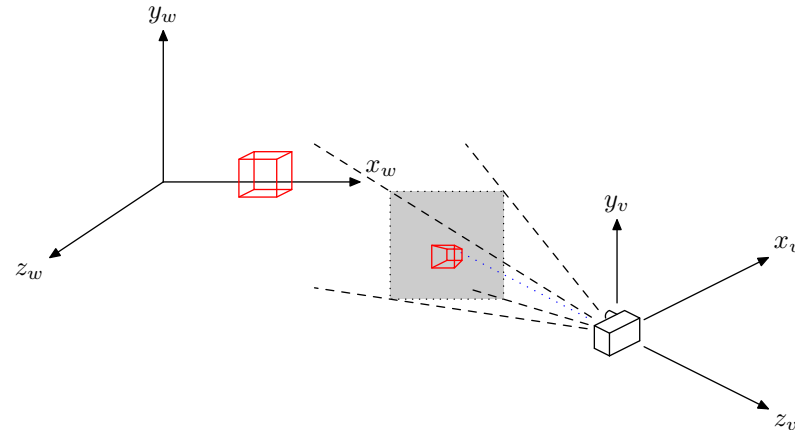
- Projektionsplanet vinkelrät mot strålarna.
- Används för vyer från olika sidor (fram, ovan, etc).
- *Längder och vinklar* bevaras.

Axonometrisk projektion

- Visar mer än en sida av objektet.
- *Parallella linjer* bevaras, liksom dess proportioner.
- Specialfall
 - ▷ *Isometrisk projektion*: planet skär alla axlar vid samma avstånd från origo.

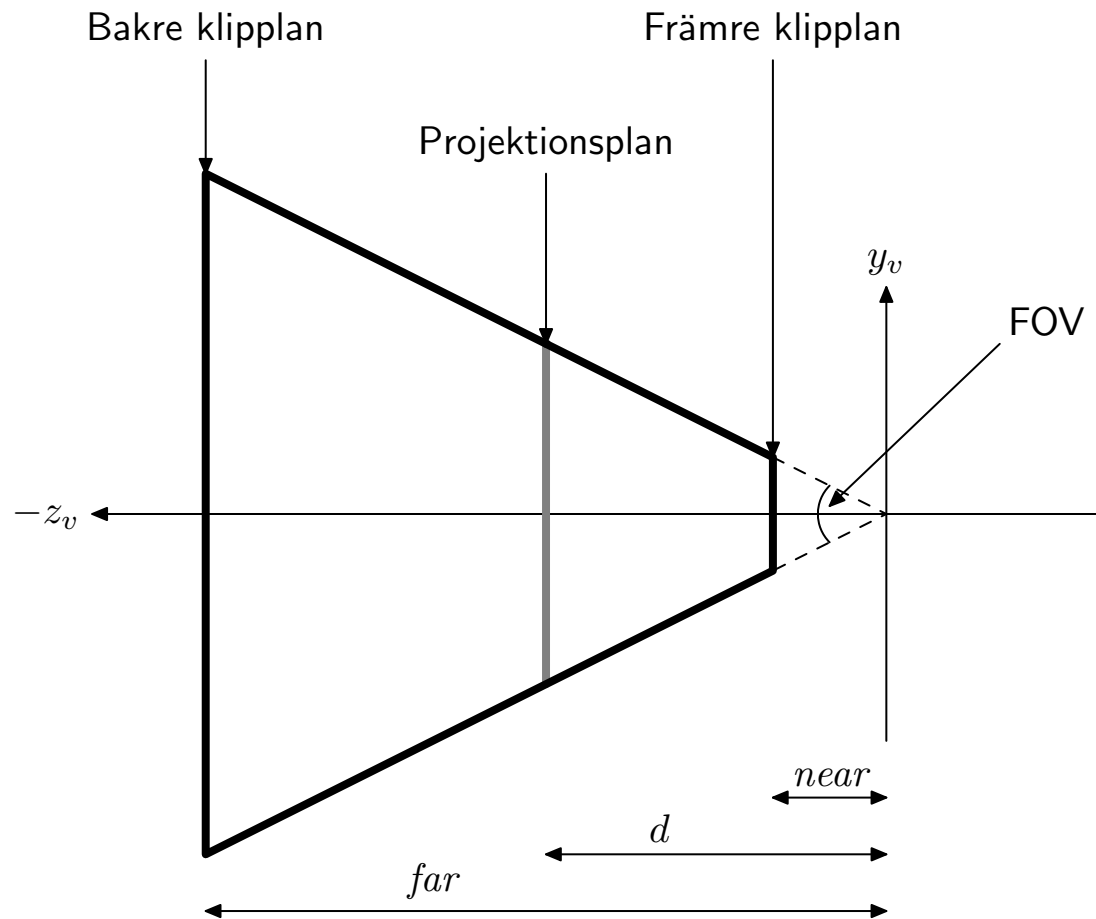


Specifikation av perspektivvy



- Kameran tittar längs $-z_v$.
- Avståndet till projektionsplanet och den rektangel i planet vars centrum z_v -axeln passerar bildar en halvoändlig pyramid.
- Alla objekt utanför pyramiden klipps bort.
- Vanligtvis klipper man även objekt som ligger för nära eller för långt bort.

Den stympade pyramiden (“viewing frustum”)



Parametrar till lab 2

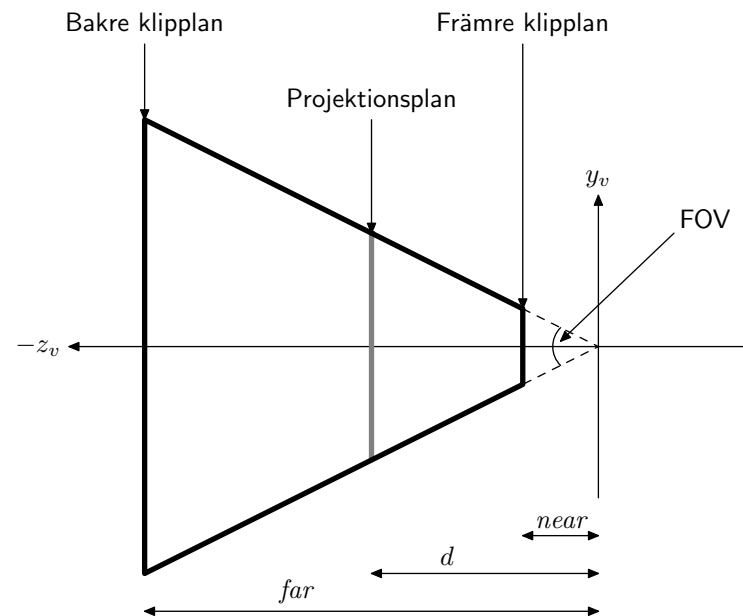
- Kamerans position**

Låt kameran befinna sig i
 $(x_v, y_v, z_v) = (0, 0, 0)$.

Vykoordinatsystemet ska kunna
 förflyttas och vridas i världen.

- Synfältets riktning**

Låt riktningen $= -z_v$ och vara
 en normal till projektionsplanet
 samt en centrumlinje till
 pyramiden.



Parametrar till lab 2 (forts)

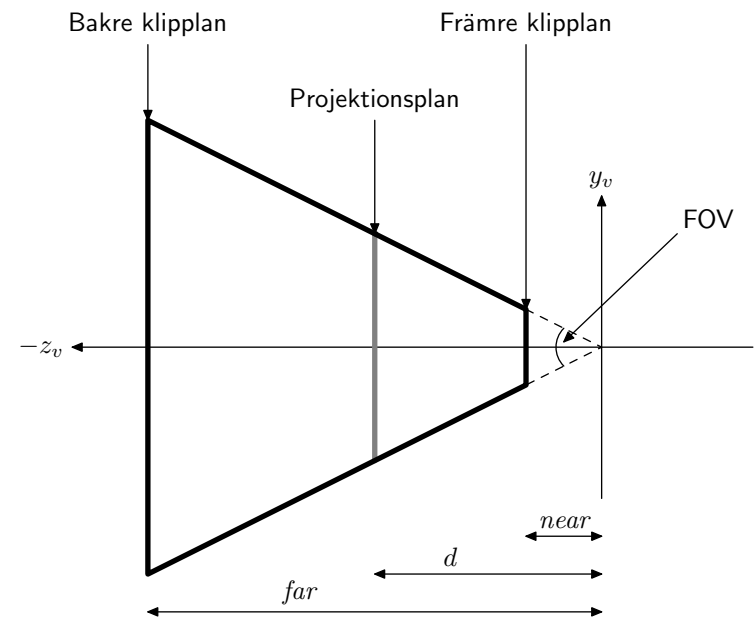
- **Storleken på den stympade pyramiden**

Definieras med *near* och *far*.

- **FOV = “field-of-view”**

Dvs synfältets vinkel.

Stort FOV ger effekten av vidvinkligt objektiv. Människa: 110–140 grader horisontellt, 80–110 vertikalt. Vid en datorskärm, kanske $\text{FOV} \approx 50^\circ$.

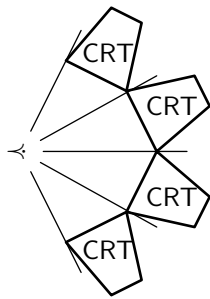


Konsekvenser av begränsningar

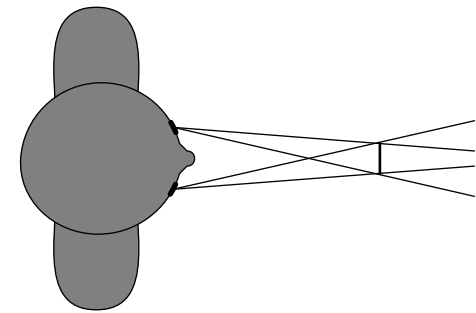
Några begränsningar som vi infört:

- $-z_v$ är normal till projektiionsplanet.
- $-z_v$ passerar mitten av den stympade pyramiden.

Vi klarar t ex alltså inte följande situationer:

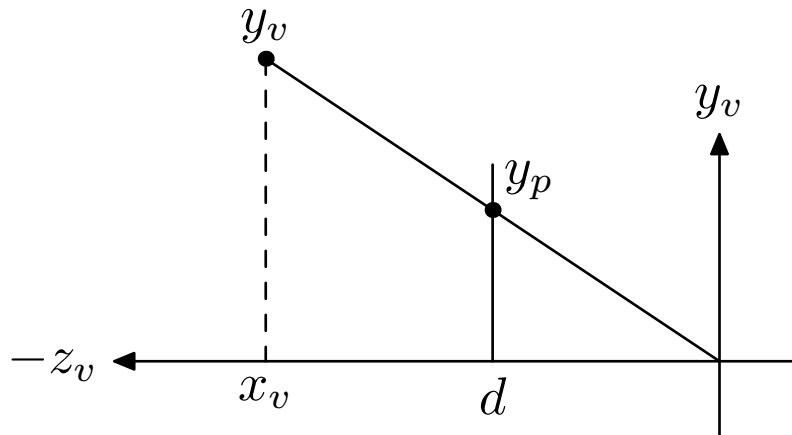


Multipla simulatorskärmar i en cockpit.
(Ingen av skärmarna ligger på $-z_v$ -axeln)



Stereo-display på vanlig skärm.

Perspektivprojicering



$$\frac{y_p}{d} = \frac{y_v}{z_v} \Rightarrow y_p = \frac{d \cdot y_v}{z_v}.$$

På samma sätt för x_p .

Matrisform

$$\begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix} \begin{pmatrix} x_v \\ y_v \\ z_v \\ 1 \end{pmatrix}$$

där

$$\begin{aligned} x_p &= X/W \\ y_p &= Y/W \\ z_p &= Z/W \end{aligned}$$

Härleds i boken men används sällan i praktiken eftersom *djupinformationen förstörs*: alla $z_p = d$.

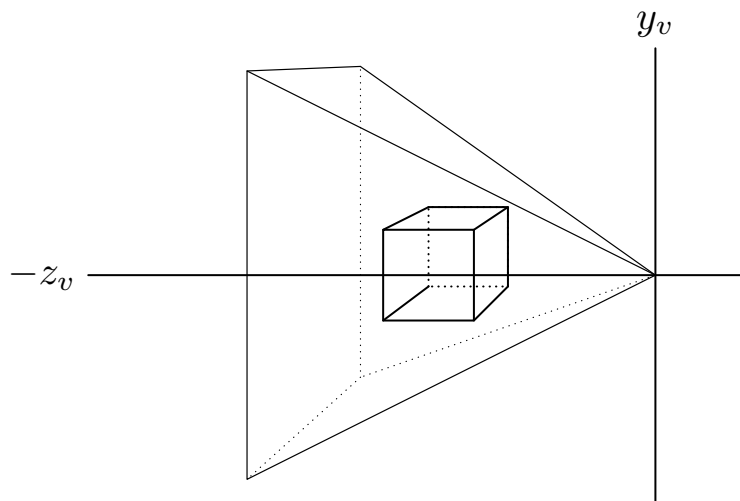
Perspektivtransformationen P

Idé: Transformera objekten så att

Perspektivprojicering
av de ursprungliga
objekten.

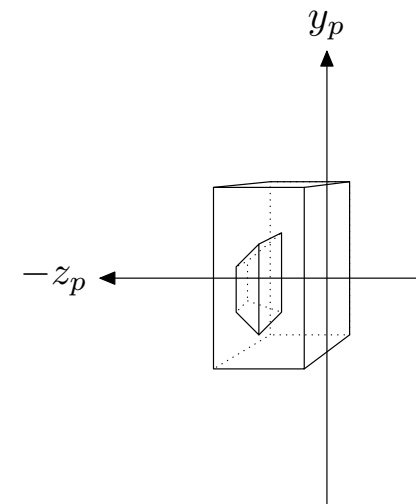
=

Parallellprojektion
av de transformerade
objekten.



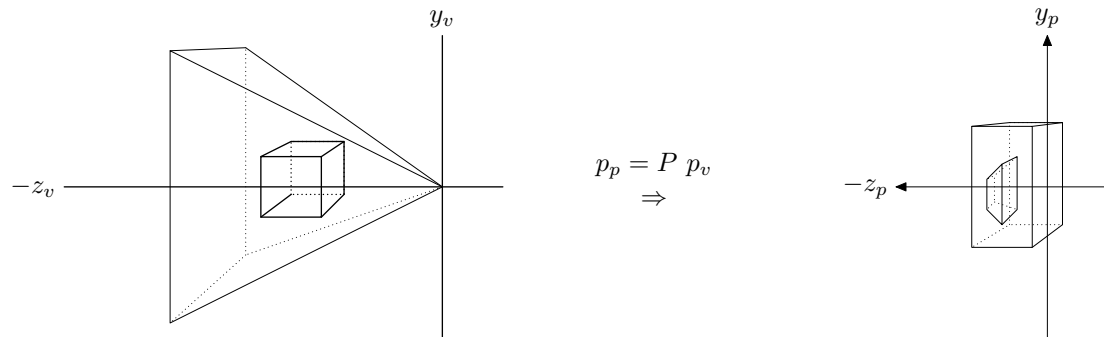
$$p_p = P p_v$$

$$\Rightarrow$$



Fördelar med perspektivtransformation

- Ger z -information för Z-buffer, intensitetsvariation, texturmappning, etc.
- Förenklar klippning (pyramiden blir en låda).
- Matrisen P är inverterbar (för texturmappning).



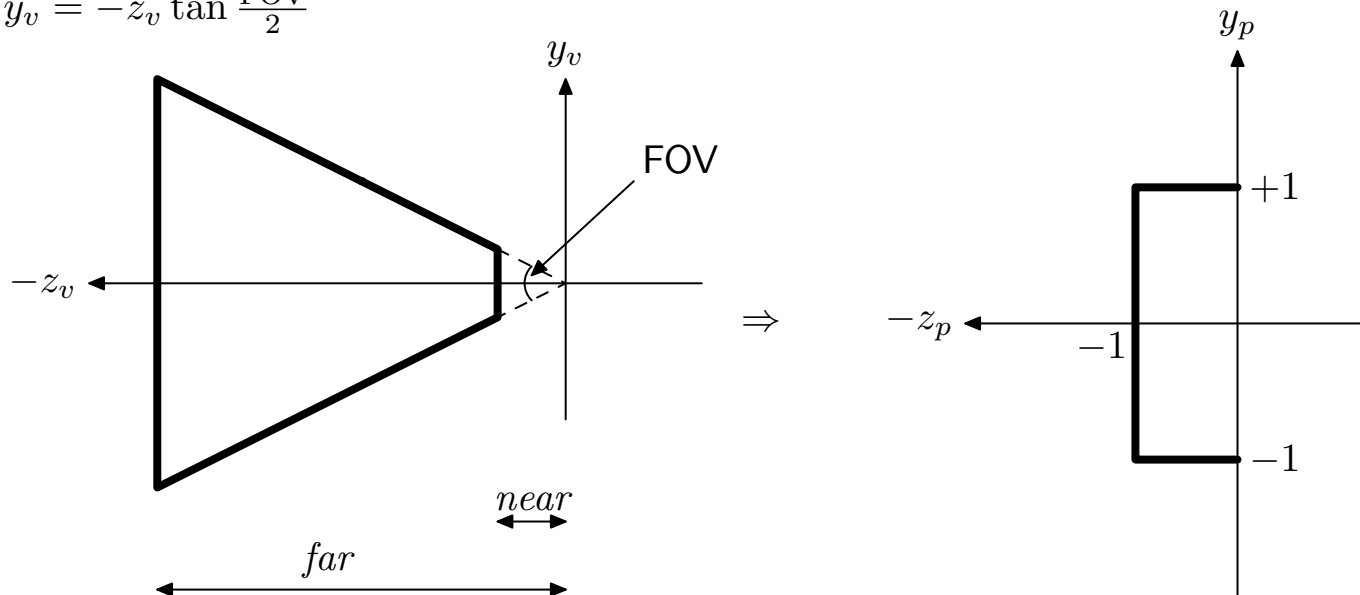
Men härleds inte i boken.

Härledning av P

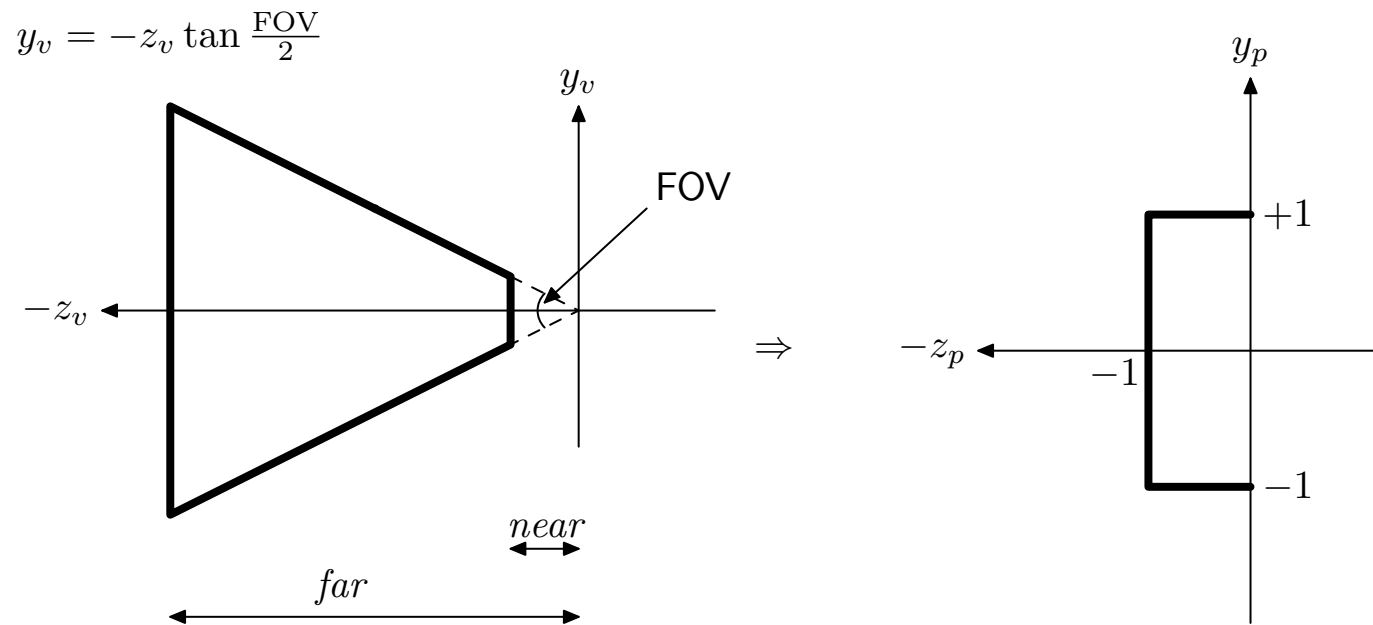
Vi passar samtidigt på att skala lådan så att

$$-1 \leq x_p \leq 1 \quad -1 \leq y_p \leq 1 \quad -1 \leq z_p \leq 0$$

$$y_v = -z_v \tan \frac{\text{FOV}}{2}$$



Härledning av P (forts)



Från figuren:

$$x_p = -\frac{x_v}{z_v \tan \frac{\text{FOV}}{2}}$$

$$y_p = -\frac{y_v}{z_v \tan \frac{\text{FOV}}{2}}$$

Hur göra med z_p ?

- P måste transformera linjer till linjer och plan till plan.
- Kan man sätta $z_p = z_v / far$?
- Nej, plan avbildas då inte på plan.
- Hemuppgift: undersök detta!
- Det måste finnas ett annat förhållande.

Hur göra med z_p (forts)?

Betrakta ett plan i *vykoordinatsystemet*:

$$Ax_v + By_v + Cz_v + D = 0$$

Detta ska via P avbildas på ett plan i *projektionkoordinatsystemet*:

$$A'x_p + B'y_p + C'z_p + D' = 0$$

Multipluera vykoordinatsystemets plan med $-\frac{1}{z_v \tan \frac{\text{FOV}}{2}}$:

$$-\frac{Ax_v}{z_v \tan \frac{\text{FOV}}{2}} - \frac{By_v}{z_v \tan \frac{\text{FOV}}{2}} - \frac{C}{\tan \frac{\text{FOV}}{2}} - \frac{D}{z_v \tan \frac{\text{FOV}}{2}} = 0$$

Stoppa in definitionen på x_p och y_p :

$$Ax_p + By_p - \frac{C}{\tan \frac{\text{FOV}}{2}} - \frac{D}{z_v \tan \frac{\text{FOV}}{2}} = 0$$

Identifiering av termer i projektionkoordinatsystemets plan:

$$A' = A \quad B' = B \quad C'z_p + D' = -\frac{C}{\tan \frac{\text{FOV}}{2}} - \frac{D}{z_v \tan \frac{\text{FOV}}{2}}$$

Tydligt råder sambandet $z_p = a + b/z_v$ för något a, b .

Bestäm a och b så att $-1 \leq z_p \leq 0$.

Två obekanta, men två ekvationer. . .

- Bakre klipplanet: $z_v = -far$, $z_p = -1$.

$$-1 = a - \frac{b}{far}$$

- Främre klipplanet: $z_v = -near$, $z_p = 0$.

$$0 = a - \frac{b}{near}$$

Lösning:

$$a = \frac{far}{near - far} \quad b = \frac{near \cdot far}{near - far}$$

Sammanställning i matrisform

$$\begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{1}{\tan \frac{\text{FOV}}{2}} & 0 & 0 & 0 \\ 0 & -\frac{1}{\tan \frac{\text{FOV}}{2}} & 0 & 0 \\ 0 & 0 & a & b \\ 0 & 0 & 1 & 0 \end{pmatrix}}_P \begin{pmatrix} x_v \\ y_v \\ z_v \\ 1 \end{pmatrix}$$

där

$$a = \frac{far}{near - far} \qquad b = \frac{near \cdot far}{near - far}$$

Kontroll

$$x_p = \frac{X}{W} = -\frac{x_v}{z_v \tan \frac{\text{FOV}}{2}} \quad \text{OK}$$

$$y_p = \frac{Y}{W} = -\frac{y_v}{z_v \tan \frac{\text{FOV}}{2}} \quad \text{OK}$$

$$z_p = \frac{Z}{W} = \frac{az_v + b}{z_v} = a + \frac{b}{z_v} \quad \text{OK}$$

Observera att det är divisionen med $W = z_v$ som ger perspektiveffekt! Detta sker i homogeniseringssteget.

Klippning

Klippa före eller efter homogenisering?

Helst före eftersom division kan undvikas, speciellt om $W = 0$!

Projektionssystemets klipplan bestäms av:

$$-1 \leq x_p \leq 1 \quad -1 \leq y_p \leq 1 \quad -1 \leq z_p \leq 0$$

Dvs, om $W > 0$:

$$-W \leq X \leq W \quad -W \leq Y \leq W \quad -W \leq Z \leq 0$$

och om $W < 0$:

$$-W \geq X \geq W \quad -W \geq Y \geq W \quad -W \geq Z \geq 0$$

Klippning i lab 2

- Kasta triangeln om en hörnpunkt ligger utanför.
- Bonusuppgift: Sutherland-Hodgmans för sex klipplan.
- Rekommenderad litteratur ur artikelsamlingen:
 - ▷ James F. Blinn, *Clipping using Homogenous Coordinates*.
 - ▷ James F. Blinn, *Line Clipping*.

Transformation till skärmkoordinater

Efter klippning och homogenisering gäller

$$-1 \leq x_p \leq 1 \quad -1 \leq y_p \leq 1 \quad -1 \leq z_p \leq 0$$

för alla $p_p = (x_p, y_p, z_p)$.

Återstår nu att transformera till skärmkoordinater

$$0 \leq x_s \leq XSIZE \quad 0 \leq y_s \leq YSIZE \quad 0 \leq z_s \leq ZSIZE$$

med en matrismultiplikation $p_s = S p_p$.

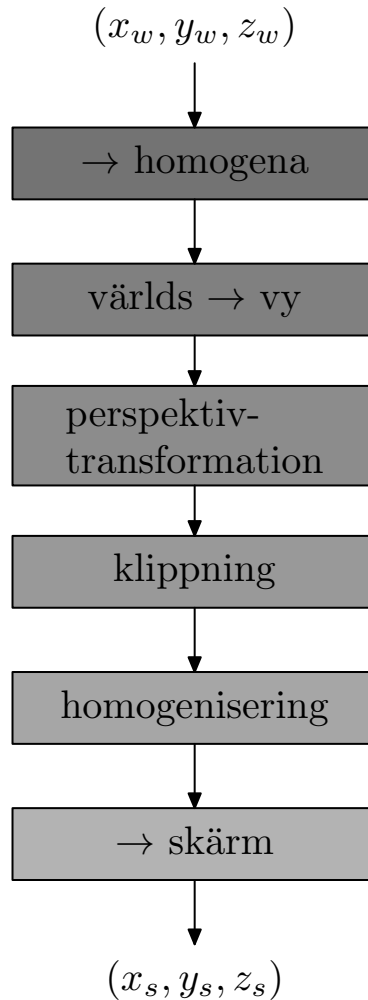
Transformation till skärmkoordinater (forts)

- Utför först en translation $T(1, 1, 1)$ och skala sedan till skärmens storlek:

$$S = S(XSIZE/2, YSIZE/2, ZSIZE) T(1, 1, 1)$$

- x_s, y_s kan används för plottning.
- z_s innehåller djupinformation.

The rendering pipeline



Världskoordinater

$$p_w = \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix}$$

$$p_v = V p_w$$

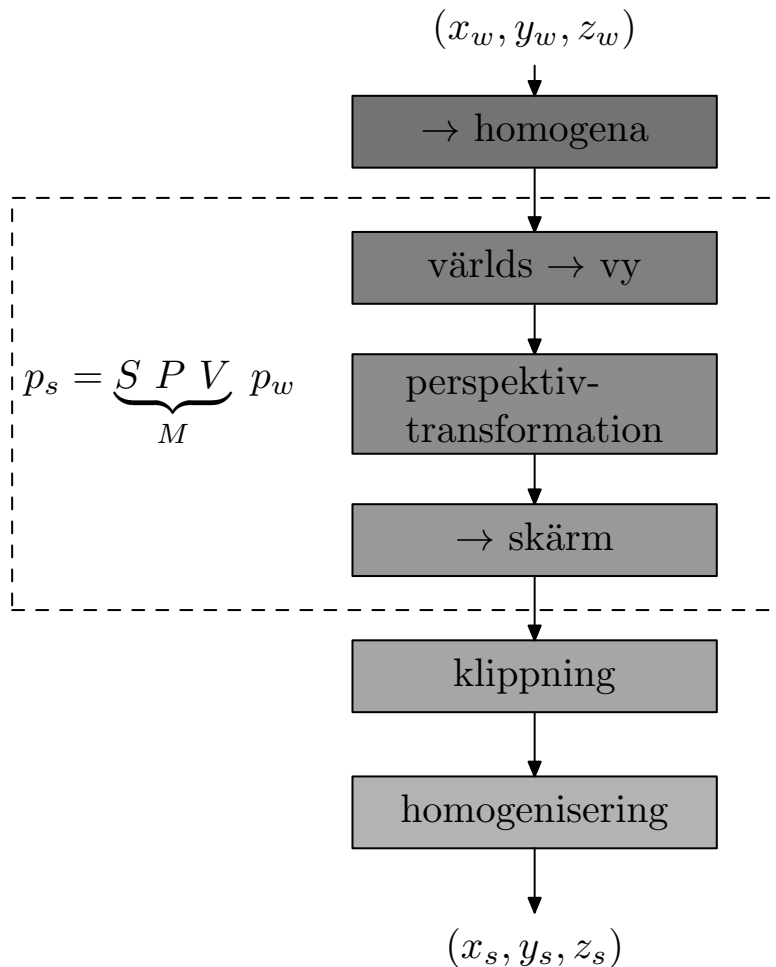
$$p_p = P p_v$$

$$\begin{aligned} x_p &\in [-1, 1] \\ y_p &\in [-1, 1] \\ z_p &\in [-1, 0] \end{aligned} \Leftrightarrow$$

$$\begin{aligned} &W > 0 \wedge -W \leq X \leq W \\ &\quad \wedge -W \leq Y \leq W \\ &\quad \wedge -W \leq Z \leq 0 \\ &\quad \vee \\ &W < 0 \wedge -W \geq X \geq W \\ &\quad \wedge -W \geq Y \geq W \\ &\quad \wedge -W \geq Z \geq 0 \end{aligned}$$

$$p_s = S p_p$$

Ekvivalent (men mer effektiv) formulering



$$p_w = \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix}$$

$$p_v = V p_w$$

$$p_p = P p_v$$

$$p_s = S p_p$$

$$x_s \in [0, XSIZE]$$

$$y_s \in [0, YSIZE]$$

$$z_s \in [0, ZSIZE]$$

 $\Leftrightarrow$

$$\begin{aligned} & W > 0 \wedge X \geq 0 \wedge X \leq W \cdot XSIZE \\ & \quad \wedge Y \geq 0 \wedge Y \leq W \cdot YSIZE \\ & \quad \wedge Z \geq 0 \wedge Z \leq W \cdot ZSIZE \\ & \quad \vee \\ & W < 0 \wedge X \leq 0 \wedge X \geq W \cdot XSIZE \\ & \quad \wedge Y \leq 0 \wedge Y \geq W \cdot YSIZE \\ & \quad \wedge Z \leq 0 \wedge Z \geq W \cdot ZSIZE \end{aligned}$$

Beräkna M i förväg $\Rightarrow$ en enda matrismultiplikation per nod!

Pseudokod för lab 2

1. För varje vy: beräkna transformationsmatrisen M .
2. För varje triangel:
 - (a) För varje hörnpunkt p_w i triangeln:
 - i. Transformera från världs- till skärmkoordinater:

$$p_s = M p_w$$

- ii. Klipp efter $0 \leq x_s, y_s, z_s \leq SIZE$.
Om utanför, fortsätt med nästa triangel.
 - iii. Homogenisera p_s : dividera med w_s .
 - (b) Rita triangeln.

Specifikation av vy i OpenGL

Vår matris V heter *modelview*-matrisen i OpenGL.

- Placera kameran i (e_x, e_y, e_z) vriden $(\theta_x, \theta_y, \theta_z)$:

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
glRotated(- $\theta_z$ , 0.0, 0.0, 1.0);  
glRotated(- $\theta_y$ , 0.0, 1.0, 0.0);  
glRotated(- $\theta_x$ , 1.0, 0.0, 0.0);  
glTranslated(- $e_x$ , - $e_y$ , - $e_z$ );
```

Specifikation av vy i OpenGL (forts)

- Placera kameran horisontellt i (e_x, e_y, e_z) , tittandes på (x, y, z) med "uppåt" = y-axeln:

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
gluLookAt( $e_x, e_y, e_z, x, y, z, 0, 1, 0$ );
```

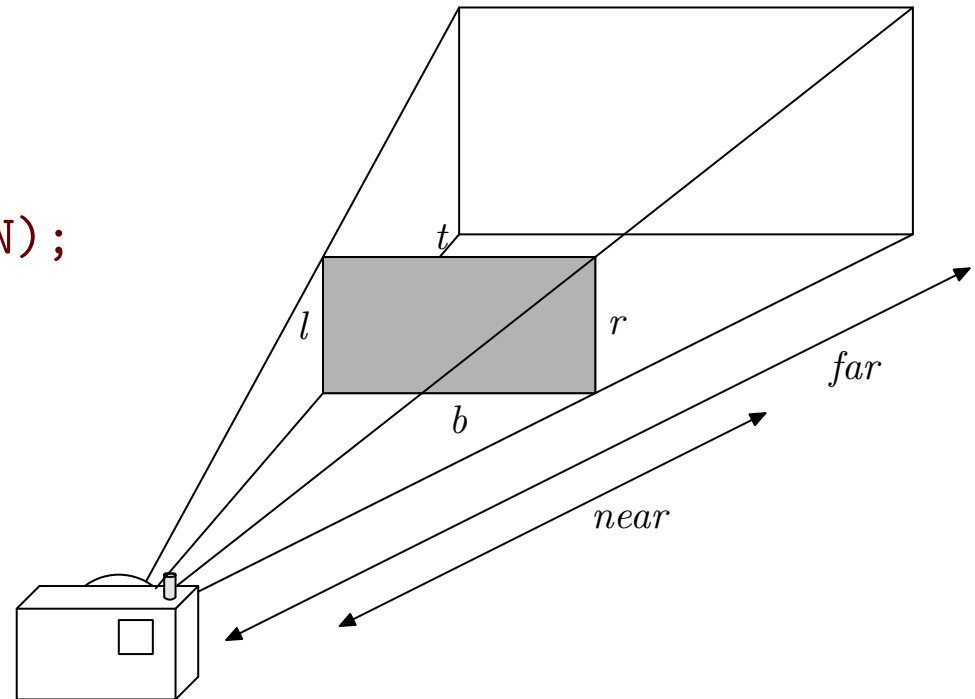
Default: $(e_x, e_y, e_z) = (0, 0, 0)$, $(x, y, z) = (0, 0, -100)$.

Specifikation av projektion i OpenGL

Vår matris P heter *projektionsmatrisen* i OpenGL.

- Perspektivprojektion:

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
glFrustum( $l, r, b, t, near, far$ );
```



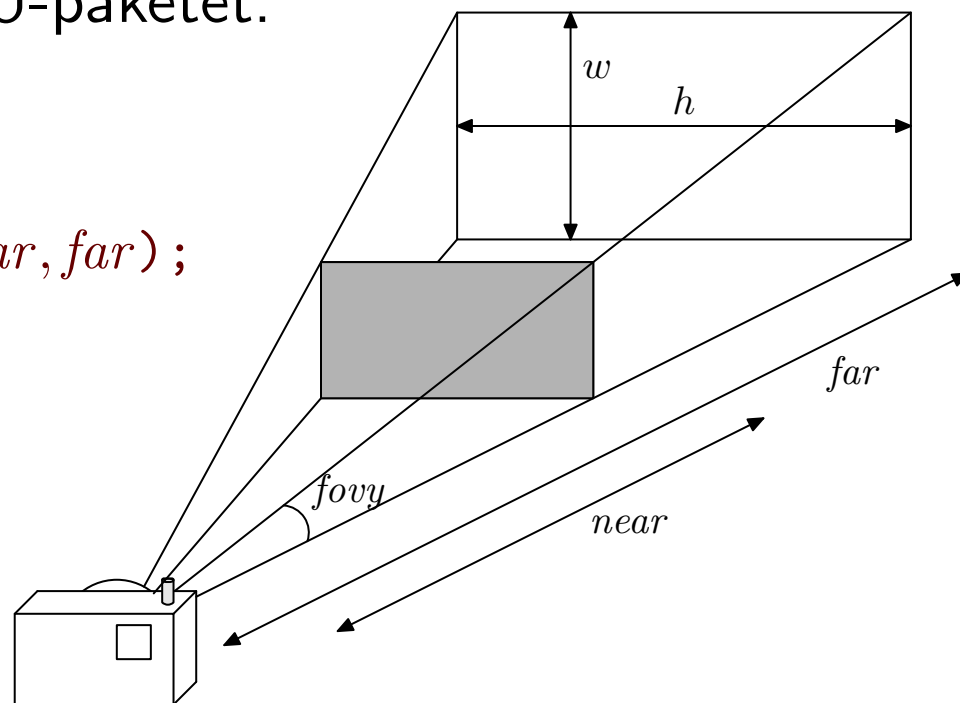
Specifikation av projektion i OpenGL (forts)

`glFrustum()` är väldigt generell, vypyramiden behöver inte vara symmetrisk kring syftlinjen.

Ett alternativ är att använda GLU-paketet:

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
gluPerspective(fovy, aspect, near, far);
```

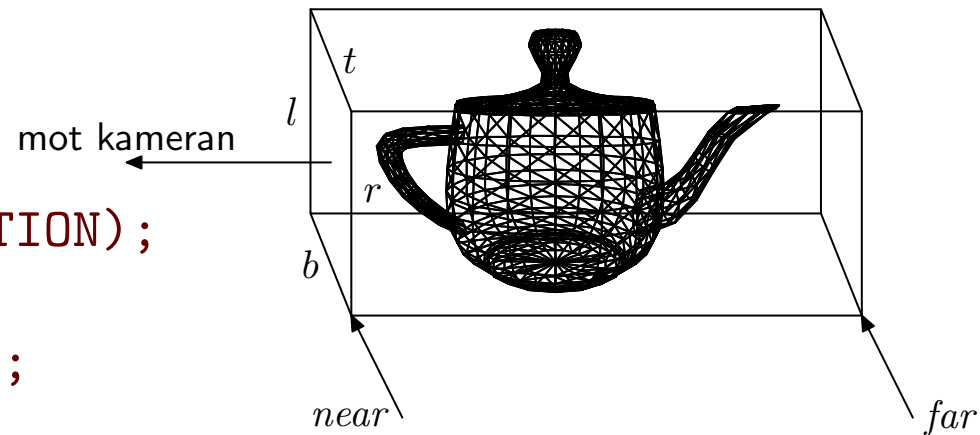
där $aspect = w/h$.



Specifikation av projektion i OpenGL (forts)

- Parallellprojektion:

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
glOrtho(l, r, b, t, near, far);
```



Notera: tillåtet för *near* och *far* att vara < 0 .

- Specialfall för 2D-grafik:

```
gluOrtho2D(l, r, b, t); /* near = -1 och far = 1 */
```

Klippning i OpenGL

- Alla primitiver som skickas genom modelview- och projektionsmatrisen och hamnar utanför något av de sex klipplanen som definieras av den stympade vypyramiden klipps ned.
- Ytterligare klipplan kan specificeras, `glClipPlane()`.

Till skärmkoordinater

- Matrisen S bestäms av den s k *viewport*-arean.

`glViewport( $x, y, width, height$ );`

- Default är att den täcker hela skärmen.
- Kan ställas om ifall man önskar rita i endast en del av fönstret.
- `glViewport` utför ingen klippning.

Lab 2 i OpenGL

```
int main(int argc, char *argv[]) {
    int i;
    Obj *obj;

    progname = argv[0];
    glutInit(&argc, argv);
    if (argc == 1) {
        fprintf(stderr, "usage: %s filename\n", argv[0]);
        exit(1);
    }
    for (i = 1; i < argc; i++) {
        obj = load(argv[i]); /* load PLG file */
        obj->next = objlist;
        objlist = obj;
    }
    myinit();
    glutMainLoop();
}
```

Initieringsrutiner

```
double fov = 50.0;      /* field of vision */
double near = 5.0;      /* near clip plane */
double far = 10000.0;   /* far clip plane */

void myinit(void) {
    /* single buffering and true color */
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    /* create the window */
    glutInitWindowSize(SIZE, SIZE);
    glutCreateWindow(progname);
    /* call display() when the window needs to be drawn */
    glutDisplayFunc(display);
    /* call keys(key, x, y) when a key is pressed */
    glutKeyboardFunc(keys);
    /* black background color (clear window later) */
    glClearColor(0.0, 0.0, 0.0, 0.0);
    /* set up perspective projection */
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(fov, 1.0, near, far);
    glMatrixMode(GL_MODELVIEW);
}
```

När skärmen behöver ritas om

```
void display(void) {
    Obj *obj;

    /* clear the window */
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0);

    /* specify the view */
    glLoadIdentity();
    glRotated(-viewer_orientation.z, 0.0, 0.0, 1.0);
    glRotated(-viewer_orientation.y, 0.0, 1.0, 0.0);
    glRotated(-viewer_orientation.x, 1.0, 0.0, 0.0);
    glTranslated(-viewer_position.x,
                 -viewer_position.y,
                 -viewer_position.z);

    for (obj = objlist; obj; obj = obj->next)
        wire(obj);

    glFlush();
}
```


Visa ett objekt som en trådmodell

```
/* display obj as a wireframe model */
void wire(Obj *obj) {
    int i, j;

    for (i = 0; i < obj->nt; i++) {
        glBegin(GL_LINE_LOOP);
        for (j = 0; j < 3; j++) {
            glVertex3d(VERTEX(obj, i, j).world.x,
                      VERTEX(obj, i, j).world.y,
                      VERTEX(obj, i, j).world.z);
        }
        glEnd();
    }
}
```

/* for every triangle */

/* for each triangle vertex */

Föreläsning 11: Färg

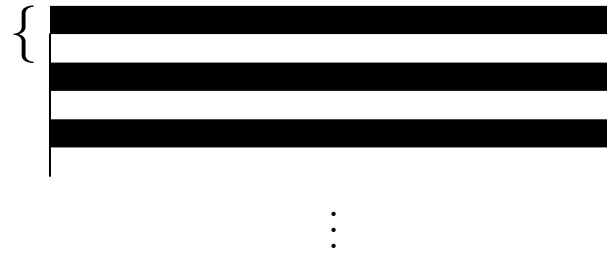
Upplösning
Katodstråleröret (CRT)
LCD-skärmar
Monokromt ljus
Gamma-korrigerings
Färg
Lite om ögats fysiologi
Hur beskriva en färg?
Hue, Saturation och Value
Systematisering av färger
Spektralfördelningar
Terminologi från kolorimetri
RGB-systemet
CIE-standard
Färgmodeller
Att använda färg



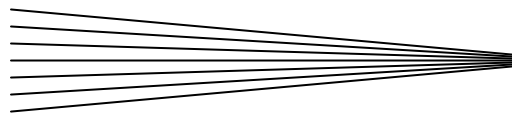
Upplösning

Fysisk upplösning:

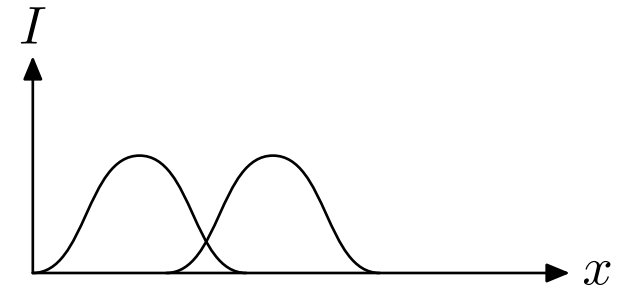
- Räknas i maximala antal separerbara *linjepar per tum*:



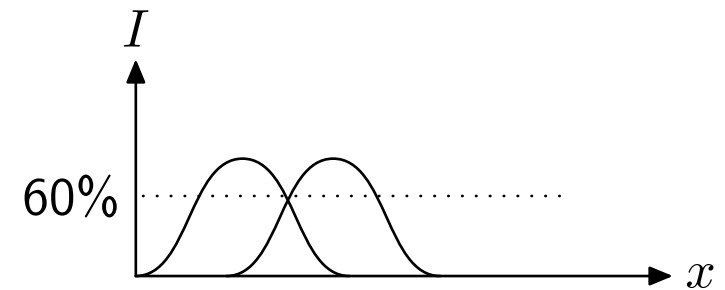
- Bestäms med ett krympande raster:



- Begränsande faktorn för upplösning är punktstorleken (eng. *dot size*). Punktprofilen anses Gauss-distribuerad.

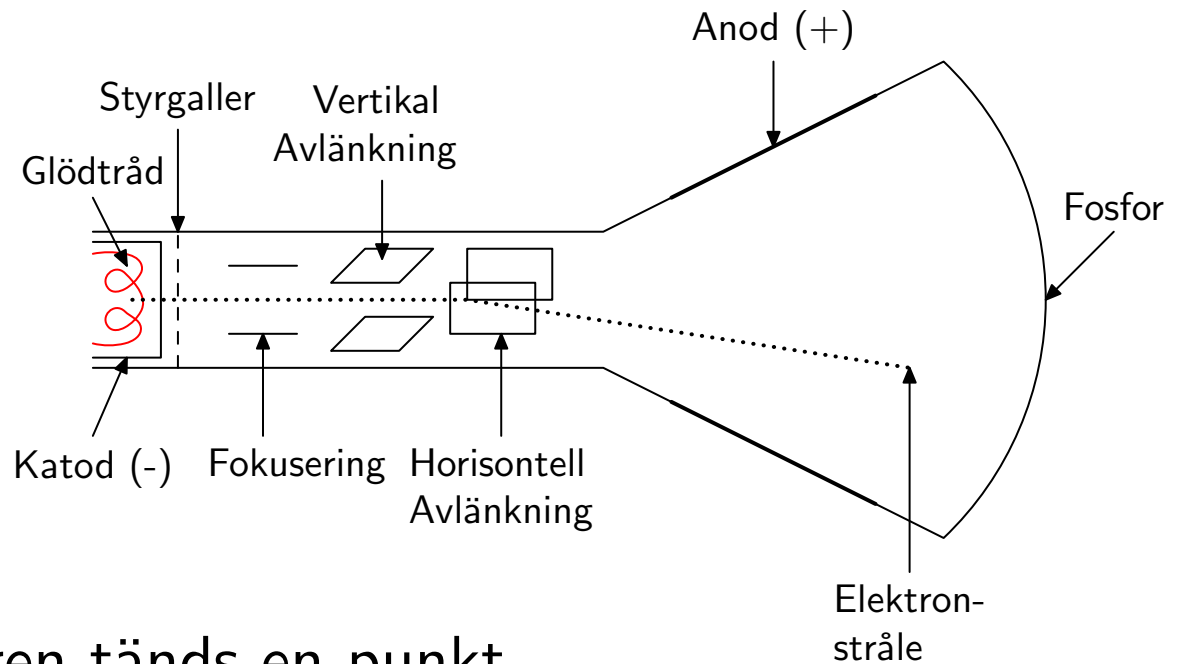


- När överlappningen är vid 60% av maxintensiteten har man nått gränsen för urskillning:



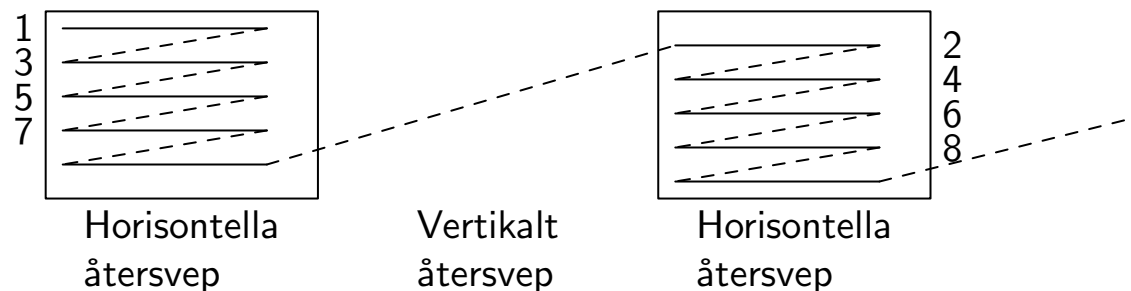
- Antal *adresserbara* punkter mäts i *dpi* (dots/inch).
- Begränsande faktor är bl a storleken på bildminnet.
- Geometrin måste svepkonverteras.
- För t ex laserskrivare räcker en sveplinje.
- För t ex en skärm krävs ett helt bildminne.

Katodstråleröret (CRT)



- Då strålen träffar fosforen tänds en punkt.
- Strålen avböjs med lämpliga signaler på avläkningsplattorna.
- Rastergrafik: strålen sveper över hela skärmen, bilden genereras genom att variera strålens intensitet.

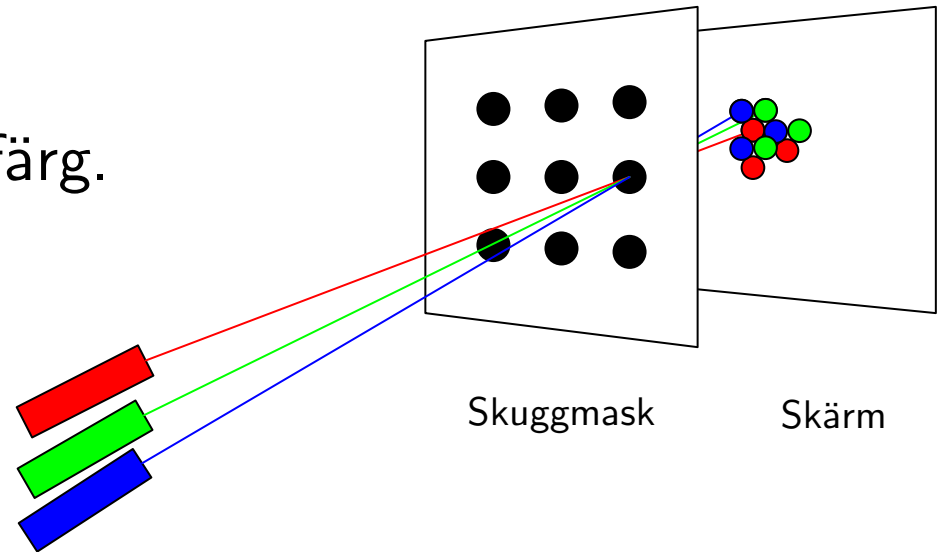
- Uppritningsfrekvens, ca 60 Hz; beror lite på . . .
- Fosfors efterlysningstid, vanligtvis 10–60 μs .
- "Interlaced scanning" medför lägre frekvens.



- Bandbredd: hur snabbt strålen kan slås av/på.
 1000×1000 punkter med 60 Hz uppdatering
 $\Rightarrow 1$ punkt var $1/(60 \cdot 10^6) = 16$ ns.
 Overhead med återsvep $\Rightarrow 11$ ns per punkt
 $\Rightarrow$ en av/på-cykel på 22 ns $\Rightarrow$ bandbredd 45 Mhz.

Katodstrålerör med färg

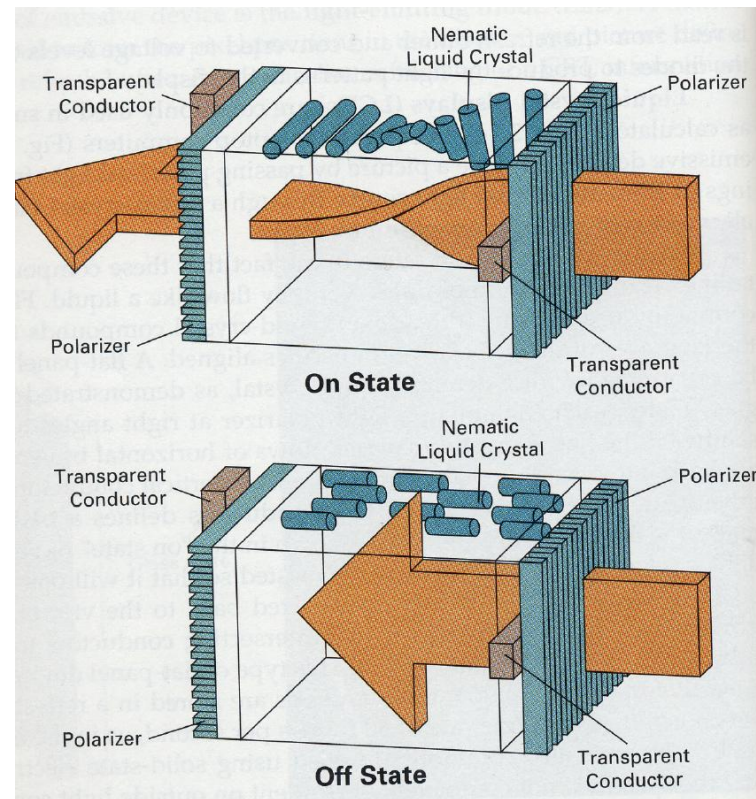
- Tre elektronstrålar, en för varje färg.



- Fosforpunkter ordnade i triangelmönster (triads), avger rött, grönt eller blått ljus.
- *Pitch* $\equiv$ avståndet mellan fosfor av samma färg.
- Typisk färgskärm: pitch = 0.3 mm, 72 dpi.

LCD-skärmar

Består av flytande kristaller som bestämmer om ljus passerar eller ej.



LCD-skärmar

- Två typer:
 1. Passiva skärmar: långsamma, kräver refresh.
 2. Aktiva skärmar (TFT).
- Kristallvätskan vrider normalt ljuset 90° så att ljuset passerar/reflekteras.
- En spänning på en kristall förhindrar vridningen $\Rightarrow$ inget ljus passerar/reflekteras där.
- Ofta kombinerat med bakgrundsbelysning.
- Kristallerna kan färgas i triader som för CRT.

Egenskaper hos LCD-skärmar

- + Låg effektförbrukning.
- + Inget magnetfält.
- + Platt.
- Polarisationsfiltret begränsar betraktningssvinkeln.
- Sämre färgåtergivning.
- Långsammare svarstider.

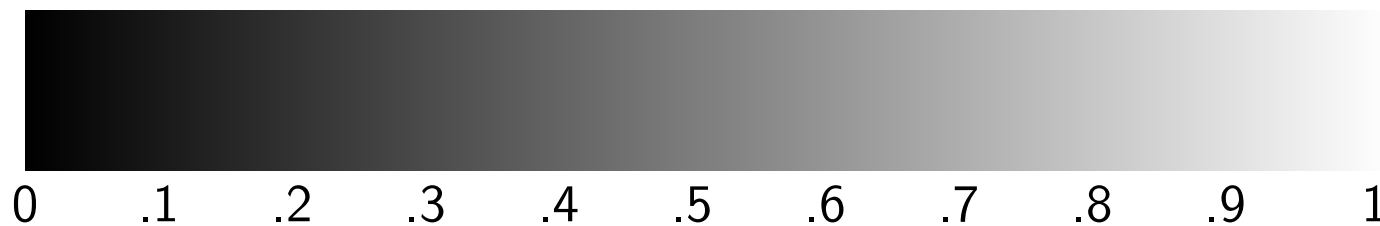
DLP-teknik (Texas Instruments)

- DLP = Digital Light Processing
- En kiselbricka med tusentals små speglar ($4 \times 4\mu m$)
- Varje spegel kan vridas upp till 10°
- Reflekterar infallande ljus till en lins om rätt vriden.
- Infallande ljus kan delas upp i färg med prisma; reflekteras sedan med tre speglar (jfr triad).
- DLP-teknik finns för projektorer, inte ännu för skärmar.
- Mer information: <http://www.ti.com/dlp>.

Monokromt ljus

Ingen färg, bara *intensitet* (mängd ljus), mäts i W/m^2 .

Behöver mellan 64 och 256 gråskalenivåer för jämn övergång. Hur ska dessa fördelas? Linjärt: $I_0 = 0; I_1 = 1/255; \dots; I_{255} = 1$.



Ögat uppfattar dock steget mellan en 50W och 100W glödlampa som lika stor som mellan 100W och 200W:

$$\frac{100}{50} = \frac{200}{100} = 2$$

Vi borde alltså fördela intensiteterna *logaritmiskt*.

Logaritmisk fördelning av 256 nivåer

- $\frac{I_{255}}{I_{254}} = \frac{I_{254}}{I_{253}} = \dots = \frac{I_2}{I_1} = \frac{I_1}{I_0} = r$
- $I_0 = I_0; I_1 = rI_0; I_2 = r^2I_0; \dots; I_{255} = r^{255}I_0 = 1$
- $r = (1/I_0)^{1/255}$
- För en CRT: $I_0 \in [1/40, 1/200]$ av maxintensiteten.
- r bör helst vara < 1.01 ty ögat kan inte uppfatta mindre än 1% skillnad i intensitet.
- $I_j = r^j I_0 = I_0^{\frac{255-j}{255}}$

Logaritmisk fördelning av 256 nivåer (forts)

- För $I_0 = 0.02$: $r = 1.0154595 \dots$

$$I_0 = 0.02; I_1 = 0.0203; I_2 = 0.0206; \dots; I_{254} = 0.9848; I_{255} = 1.00$$



Gamma-korrigerig för CRT-skärmar

Pixelns intensitet I beror av spänningen V på kanonen.

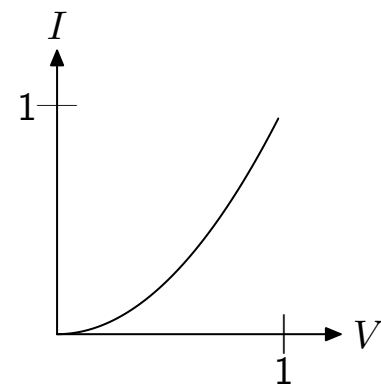
V är proportionellt mot innehållet i färgtabellen, men hur förhåller sig I till V ?

I praktiken *icke-linjärt*: $I = KV^\gamma$, där

V : spänningen på kanonen

K : konstant, beror på skärmen

γ : konstant, vanligtvis 2 – 2.5



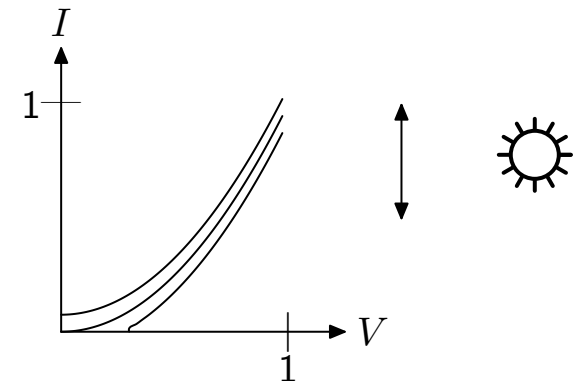
Alltså, använd $V = (I/K)^{1/\gamma}$ för att visa I . Färgtabellen kan laddas för att kompensera för denna effekt, s k *gamma-korrigerig*.

Inställning av ljusstyrka ☀ och kontrast 🌑

Ställ in ljusstyrkan först, sedan kontrasten!

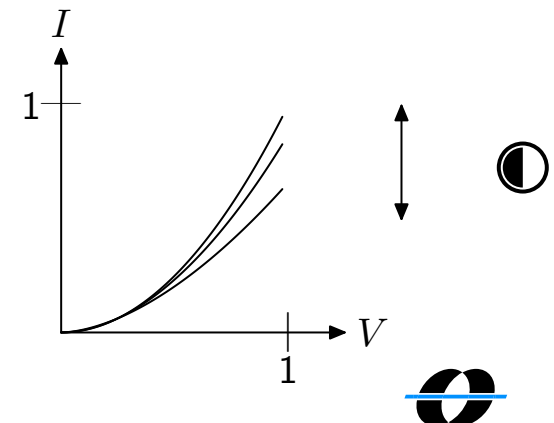
- **Ljusstyrka:**

Ställs in så svart signal visas som svart.



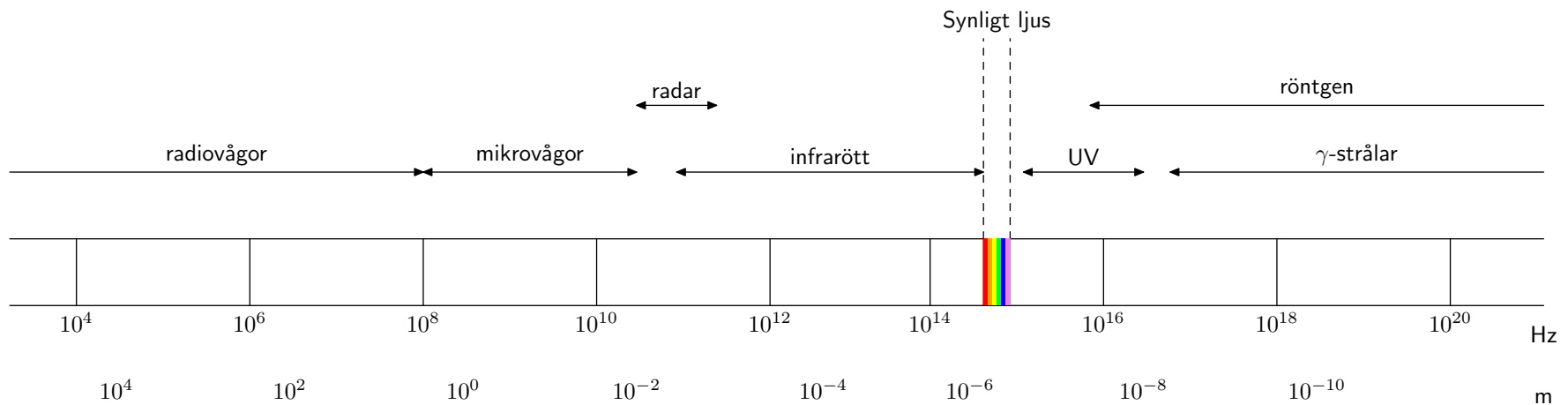
- **Kontrast:**

Kontrollerar intensiteten för vitt ljus.



Färg

Synligt ljus = elektromagnetiska vågor.



Rött: 610 – 750 nm

Orange: 590 – 610 nm

Gult: 570 – 590 nm

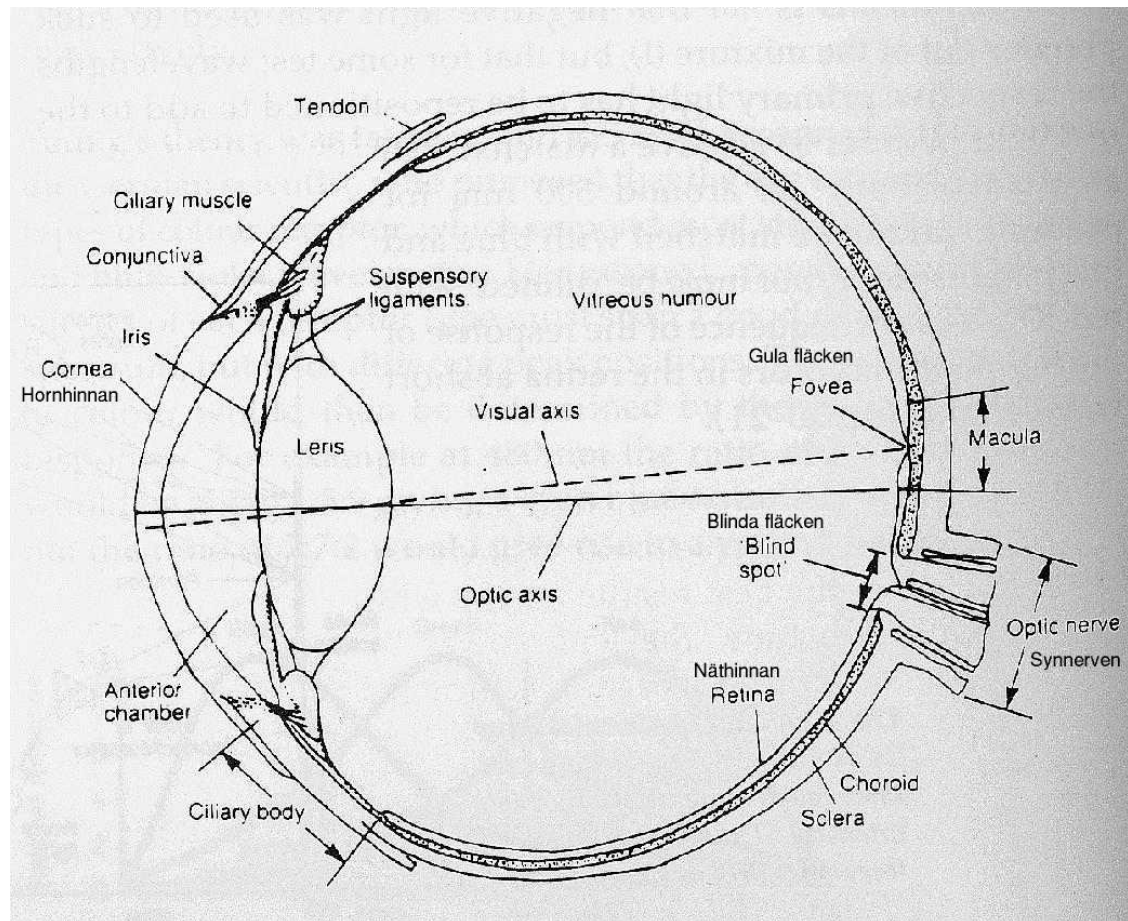
Grönt: 500 – 570 nm

Blått: 450 – 500 nm

Violett: 400 – 450 nm

Lite om ögats fysiologi

Elektromagnetiska frekvenser stimulerar receptorer i ögat som sänder signaler till hjärnan vilket uppfattas som färg.



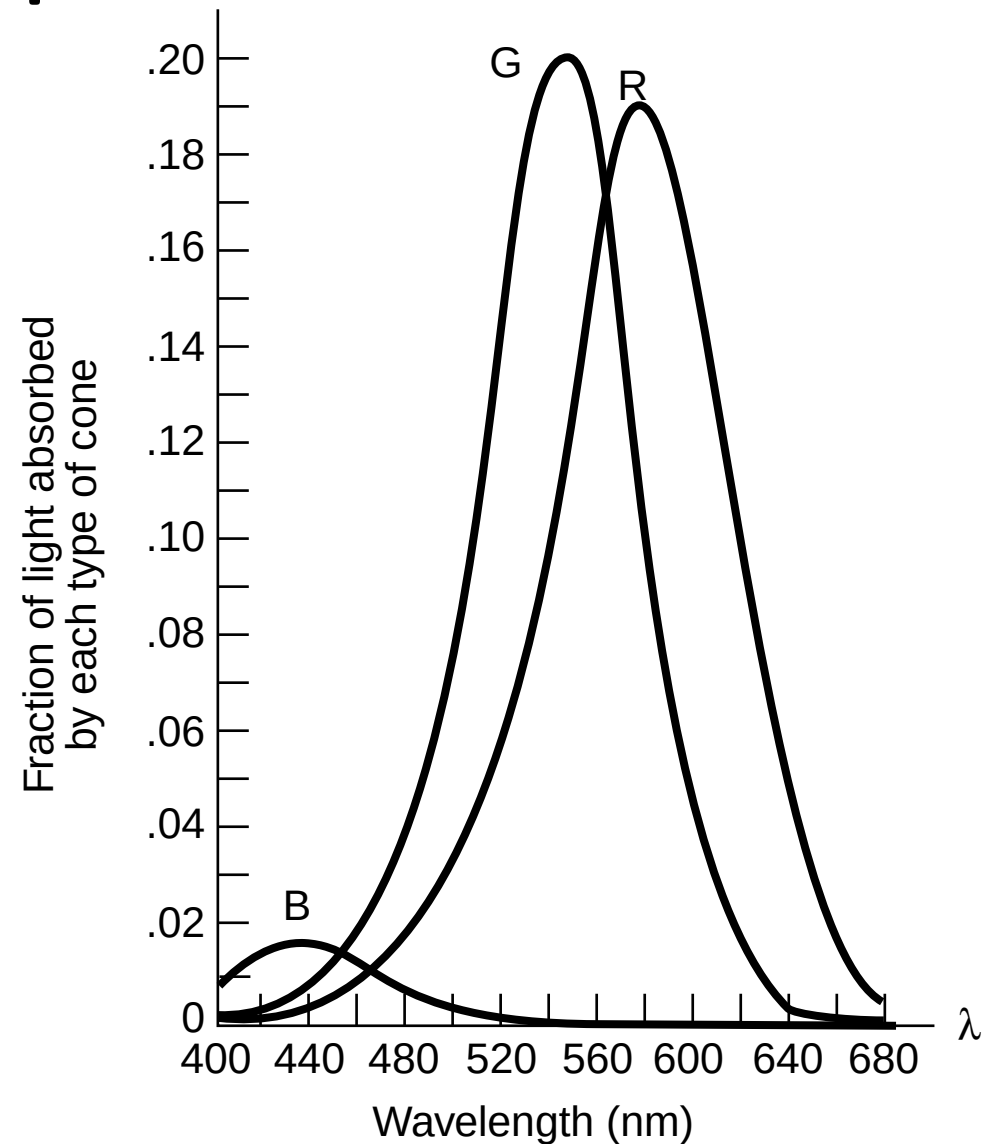
Ögats fotoreceptorer

Fotoreceptorn är en ljuskänslig cell. Två typer:

1. **Stavar:** gråskala; mycket ljuskänsliga.
2. **Tappar:** färgseende; mindre ljuskänsliga.
 - Koncentrerad till gula fläcken $\Rightarrow$ bara färgseende "rakt fram".
 - Tristimulus-teorin: tapparna finns i tre varianter; reagerar på olika färgområden.
 - Kan tillsammans uppfatta ca en miljon färgnyanser.
 - Saknas någon typ av tapp är man färgblind (ca 10% av män, 1% av kvinnor).

Spektral-responsfunktion

Diagrammet visar den spektral-responsfunktionen för de tre olika tapparna. Varje tapp har maximal respons i ett visst område, ungefär rött, grönt och blått.



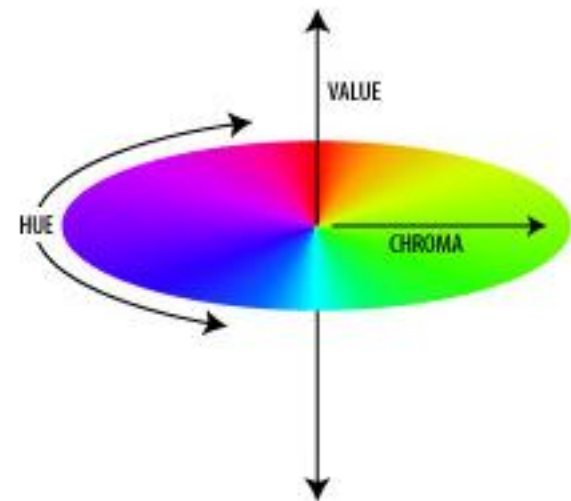
Hur beskriva en färg?

- **Intuitivt:** Intuitiva beskrivningar är subjektiva.
 - ▷ Bengt Grive: "den mörkvita klänningen . . ."
 - ▷ Olika katalogsystem.
- **Fysikaliskt:** Fysikaliska beskrivningar är kvantitativa.
 - ▷ Spektralfördelning.
 - ▷ CIE, RGB, CMY, YIQ, HSV, . . .

Hue, Saturation och Value

Hur en färg uppfattas kan beskrivas med tre egenskaper:

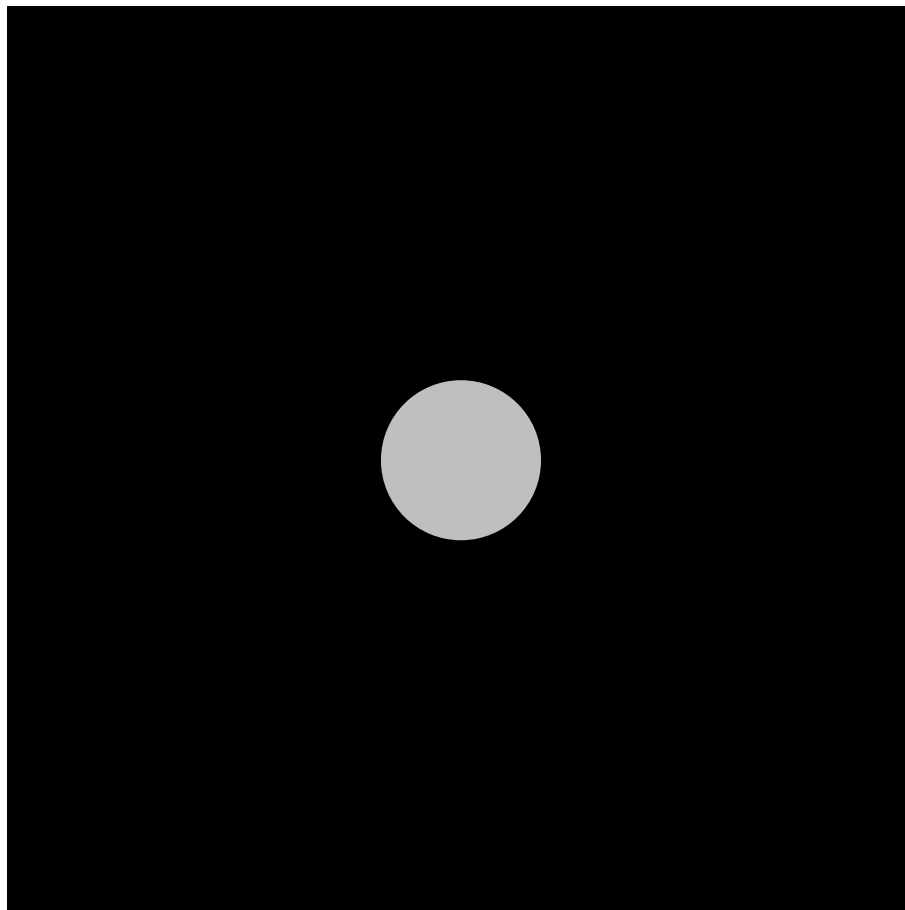
- **Hue** (färgton): Skiljer mellan olika färger, t ex röd, grön, gul. Ögat kan skilja på ca 128 olika färgtoner.
- **Saturation** (mättnad): Anger hur "klar" färgen är, dvs hur mycket vitt som (inte) blandats med färgen.
 - ▷ Omättade färger har högre andel av vitt.
 - ▷ Röd: mättad; rosa: omättad.
 - ▷ Pastellfärger är relativt omättade.
- **Value** (även "Brightness" eller "Lightness") $\approx$ intensitet. Brunt är en röd/orange färgton med låg intensitet.

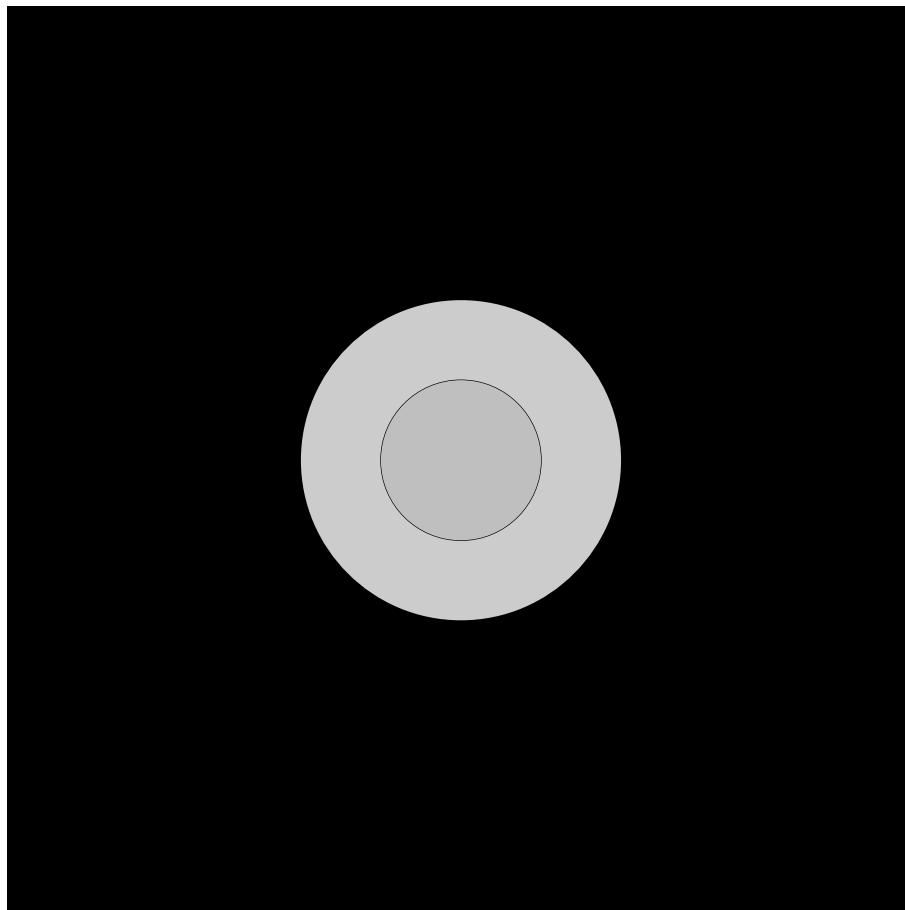


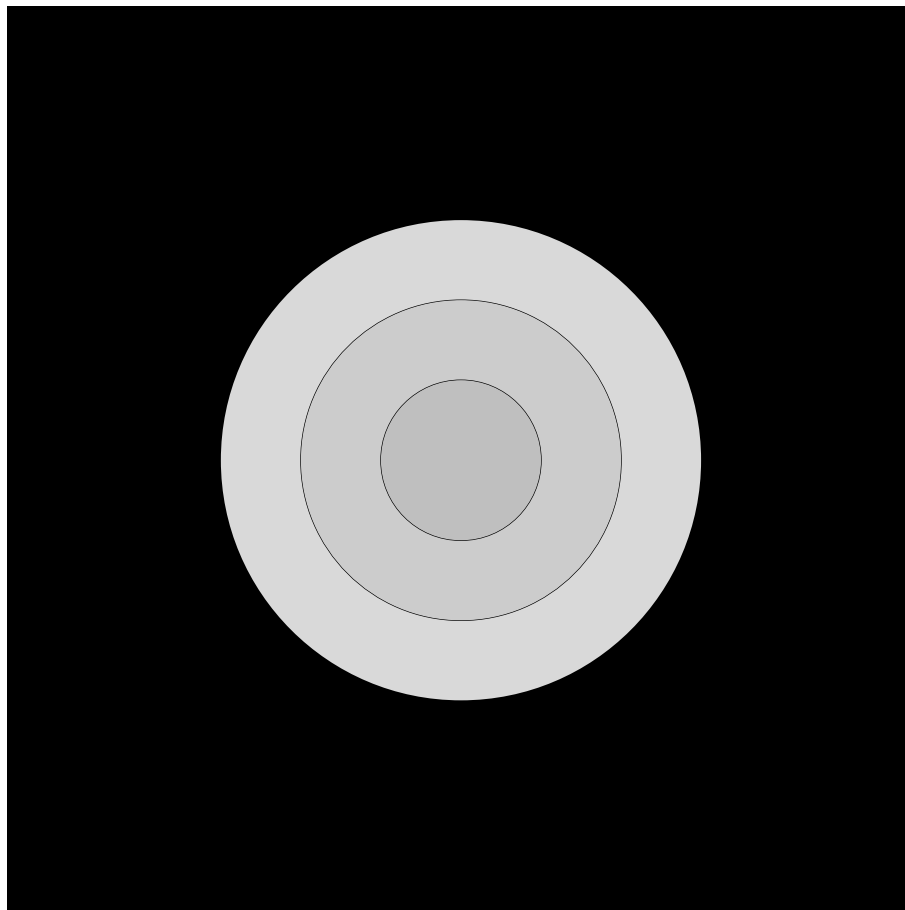
Intensitet uppfattas relativt

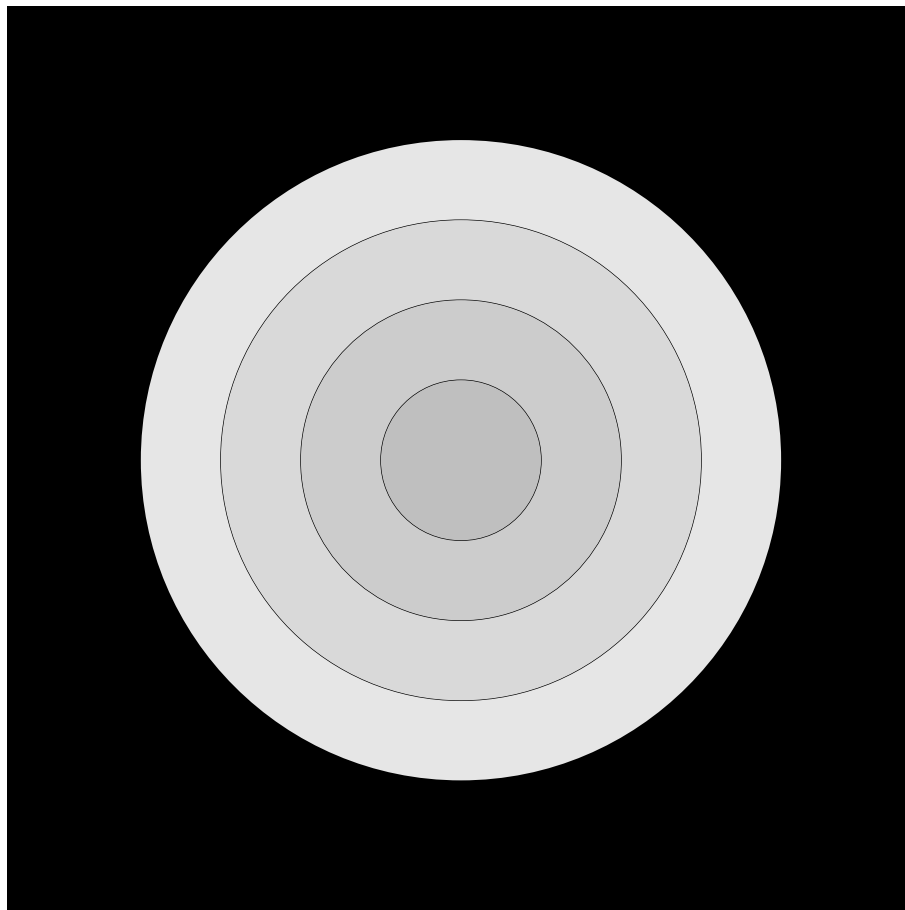
- Intensitet uppfattas *inte* absolut.
- Följande exempel visar varför:

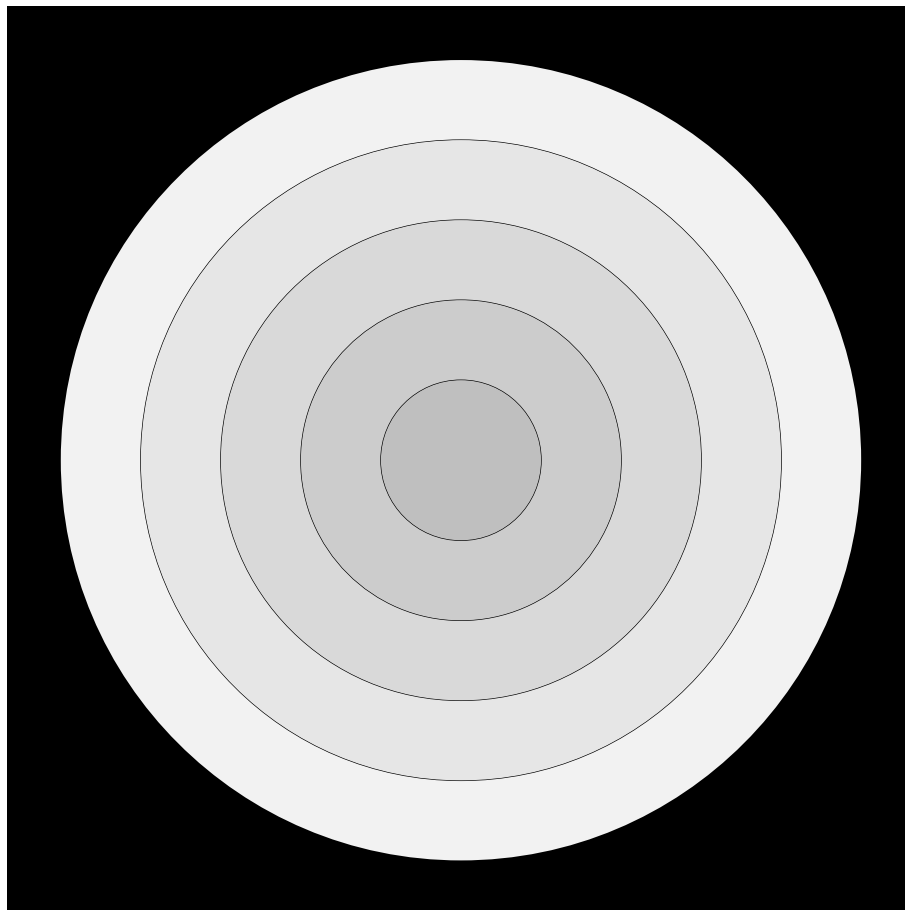
Vad är intensiteten på den yttersta ringen?

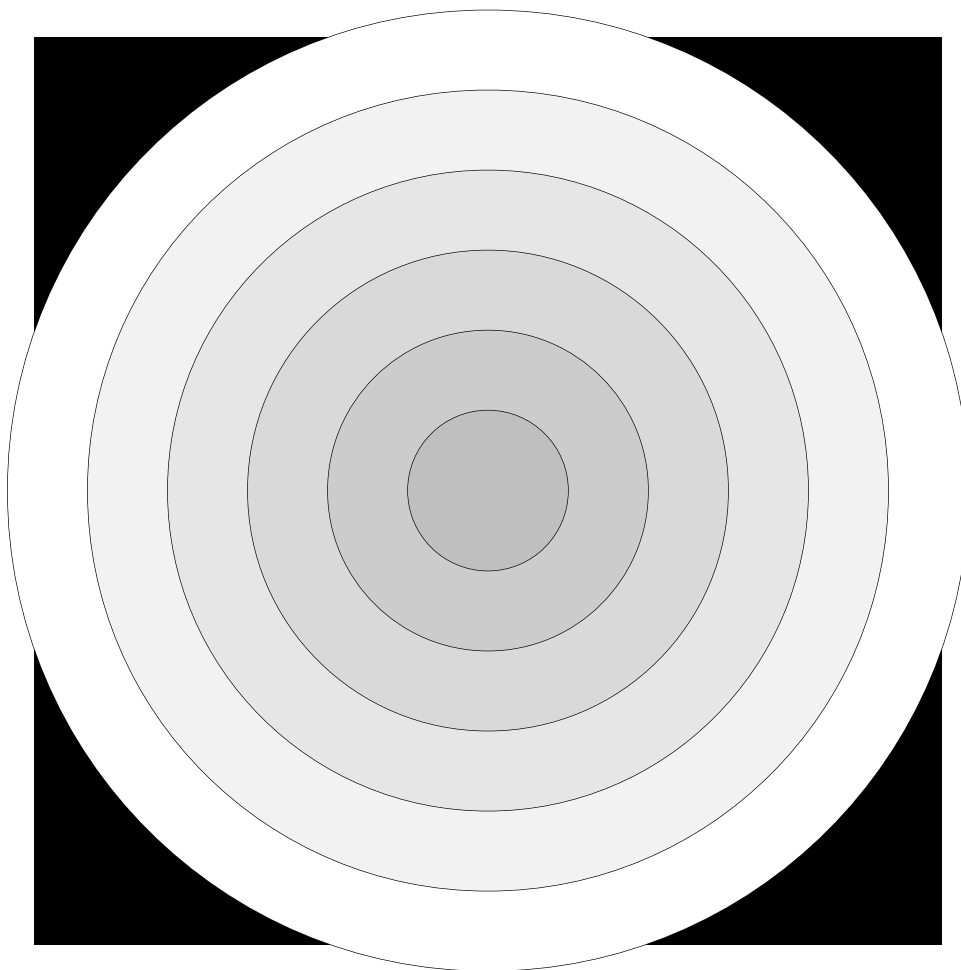












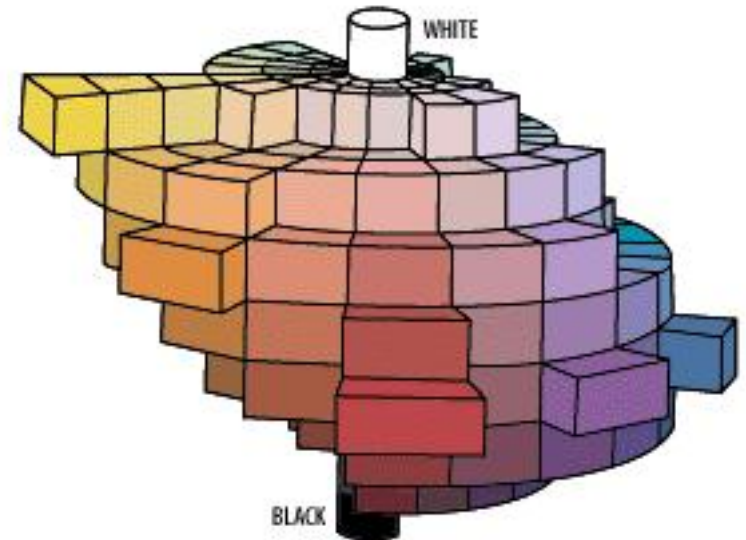
Systematisering av färger

Man kan leta efter en okänd färg i en katalog.

- PANTONE Matching System, för tryckindustrin.

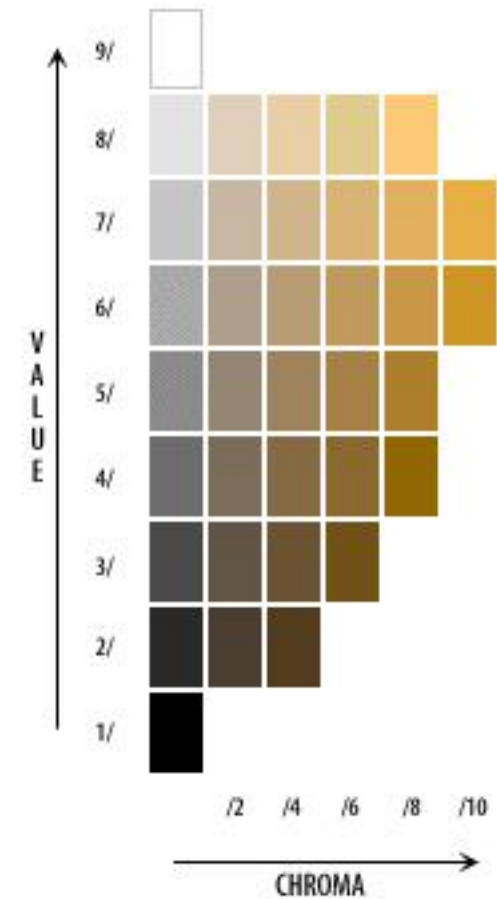
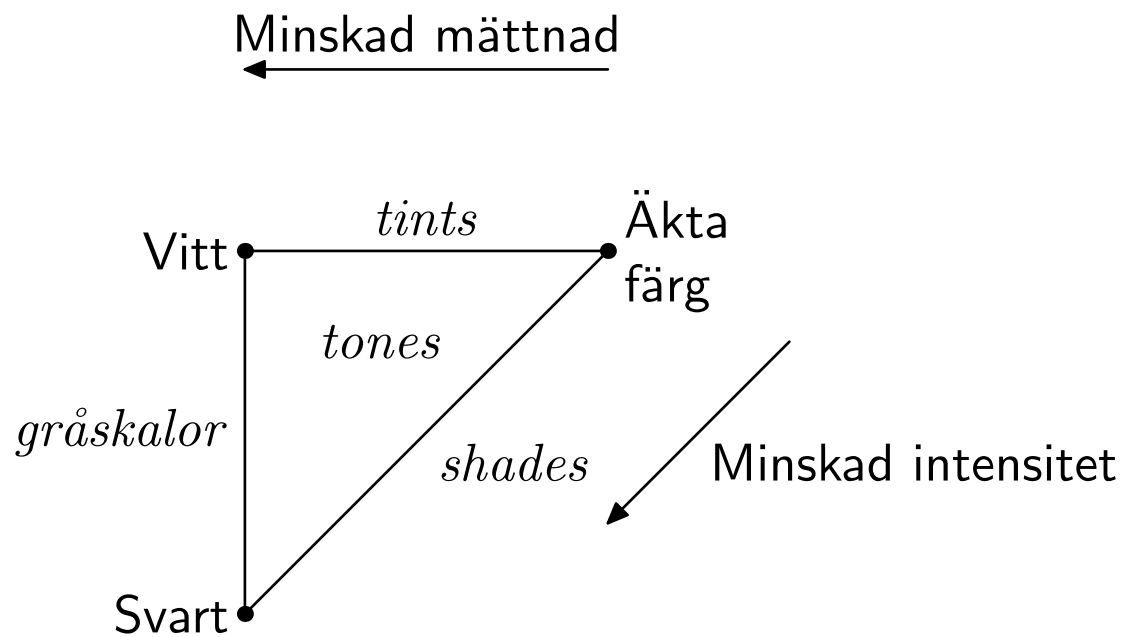


- Albert Munsells skapade en 3D-rymd av HSV-värden med namn.



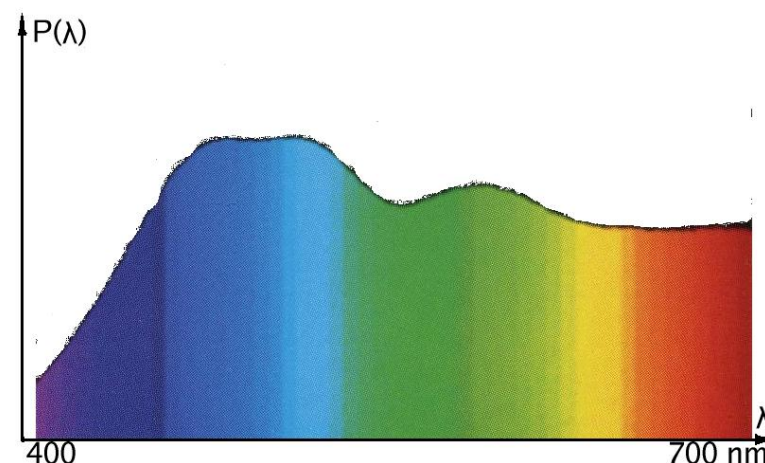
Systematisering av färger (forts)

- Ostwalds system: Baserat på *tints*, *tones* och *shades*.

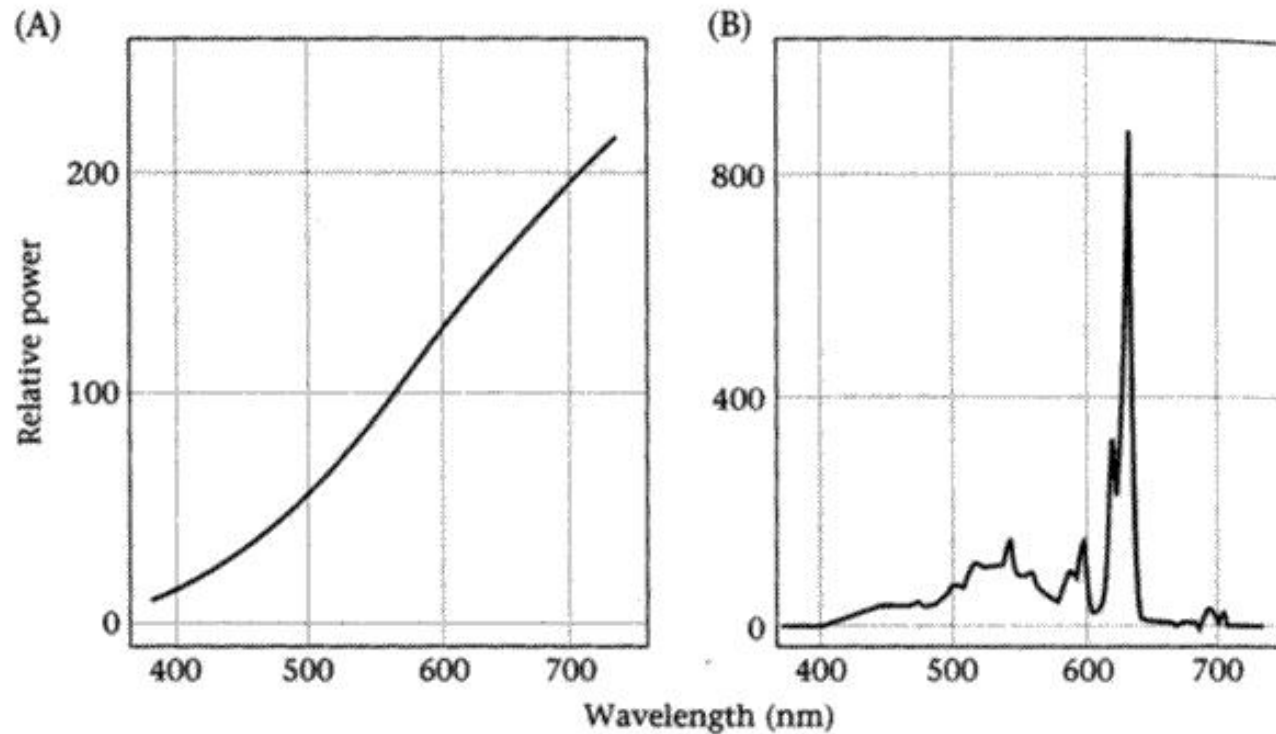


Spektralfördelningar

- Spektralfärg: Enstaka frekvens (från t ex en laser).
- Vanligast är dock en blandning av olika frekvenser. Kan åskådliggöras med en *spektralfördelning*:
- y -axeln visar effekt per våglängd.
- Om ögat uppfattar flera olika spektralfördelningar som samma färg kallas dessa *metamerer*.



Metamerer



Spektralfördelning från en "vit" tungstenslampa.

Spektralfördelning från en skärm, inställd för att matcha lampan.

Terminologi från kolorimetri

Den visuella effekten av en spektralfördelning kan beskrivas med tre kvantiteter:

- **Dominant våglängd:** ($\approx$ Hue)

Där energispiken e_2 sker.

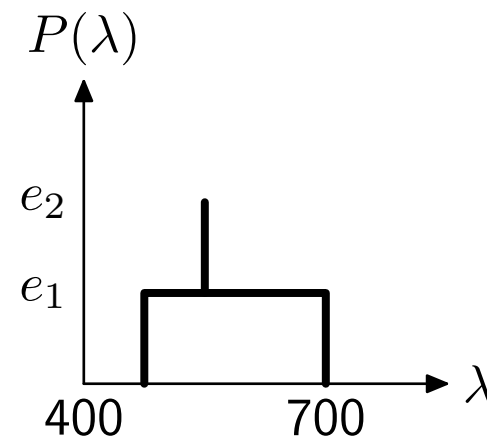
- **Exciteringsgrad:** ($\approx$ Saturation)

Om $e_1 = e_2$ är exciteringsgraden 0% (omättad).

Om $e_1 = 0$ är exciteringsgraden 100%

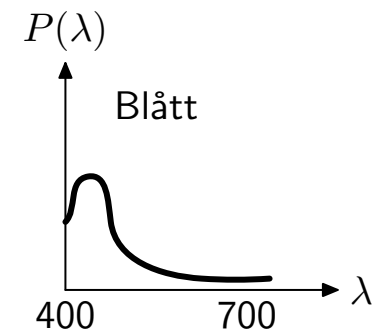
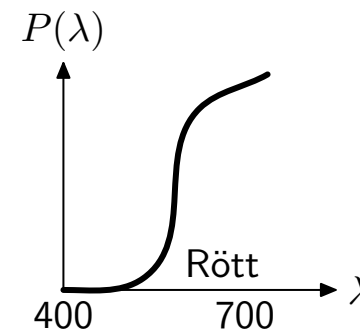
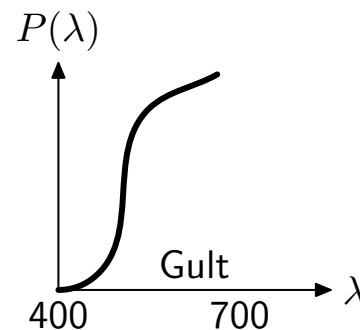
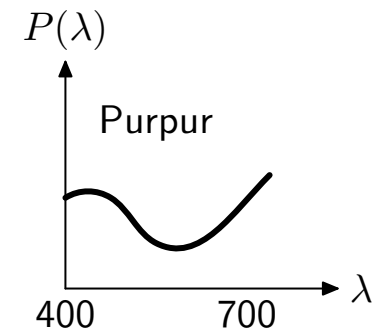
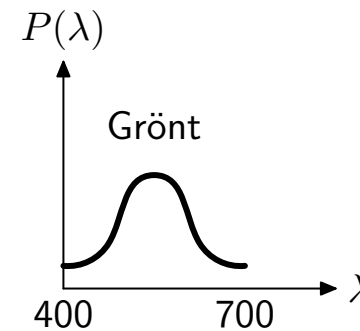
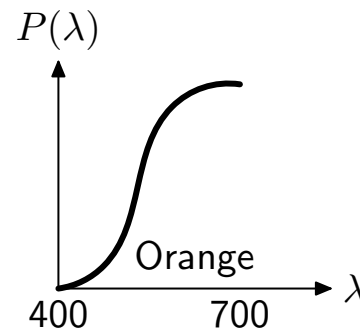
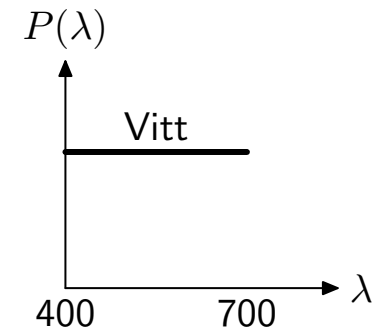
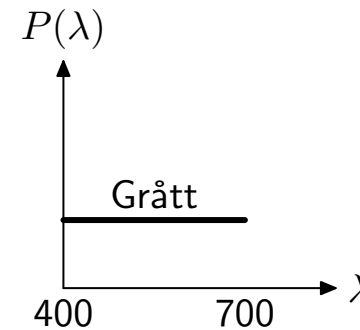
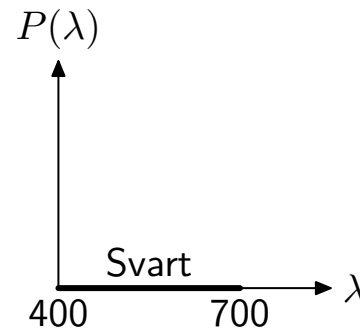
- **Luminans:** ($\approx$ Value)

Integralen av produkten mellan $P(\lambda)$ och ögats spektral-responsfunktion. Ett mått på den totala energin.



Några olika spektralfördelningar

- Hur ska vi kunna beskriva en färg numeriskt?
- Måste vi ange effekten för hundratals våglängder?



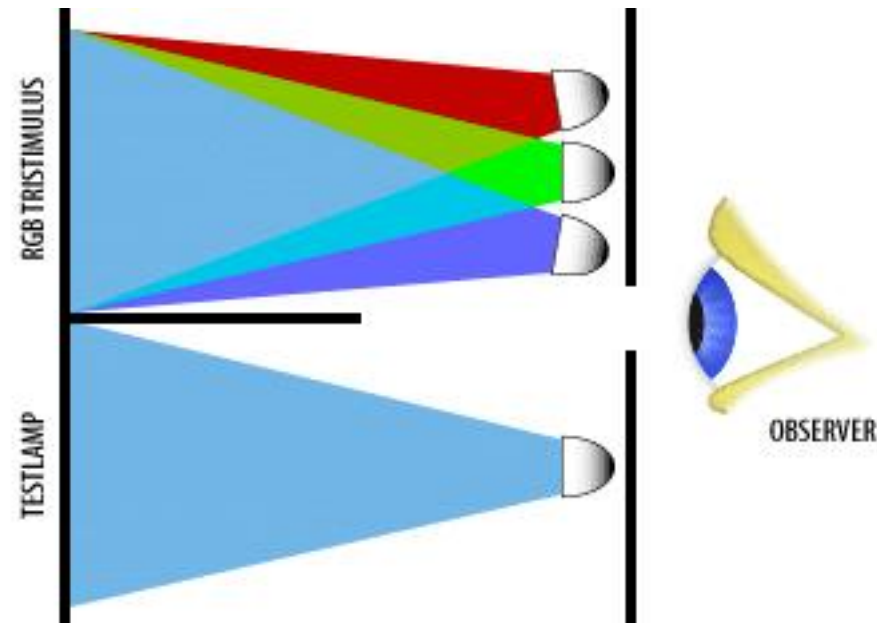
RGB-systemet

Lyckligtvis räcker det med tre halter av t ex **R**, **G**, **B**.

För varje färg C kan en människa hitta R, G, B så att $C = R\mathbf{R} + G\mathbf{G} + B\mathbf{B}$.

Varje annan linjärt oberoende uppsättning av tre primära färger duger också.

Exempel: $(R, G, B) = (1.0, 1.0, 0.0)$ ger gult.



RGB-systemet (forts)

Nackdelen med RGB-systemet är att vissa färger kräver *negativa* vikter av R .

$R < 0$ innebär t ex att man kan matcha en färg med grönt och blått om man först blandar färgen med rött.

Detta innebär att en CRT inte kan visa en del färger, t ex cyan.

RGB-systemet (forts)

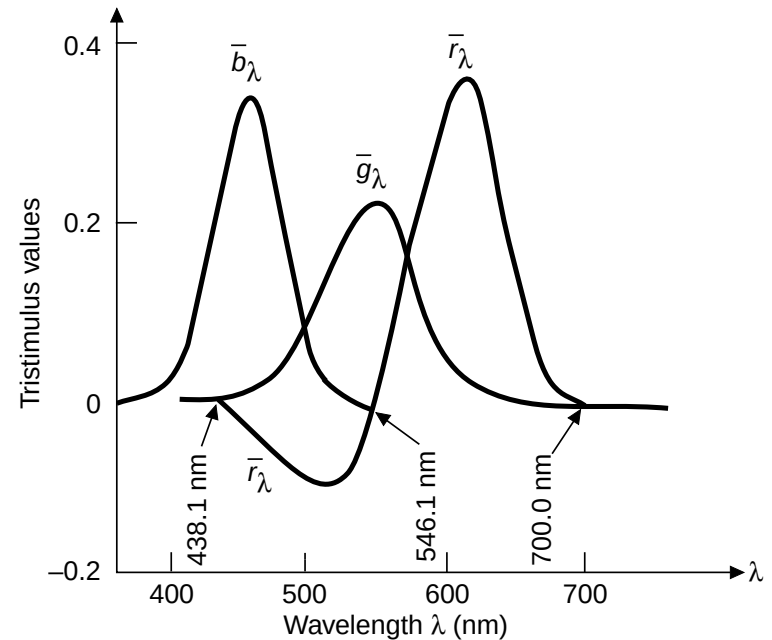
Diagrammet visar hur mycket R, G, B som behövs för att matcha en spektralfärg (våglängd).

För en godtycklig spektralfördelning $P(\lambda)$:

$$R = k \int P(\lambda) \bar{r}_\lambda d\lambda$$

$$G = k \int P(\lambda) \bar{g}_\lambda d\lambda$$

$$B = k \int P(\lambda) \bar{b}_\lambda d\lambda$$

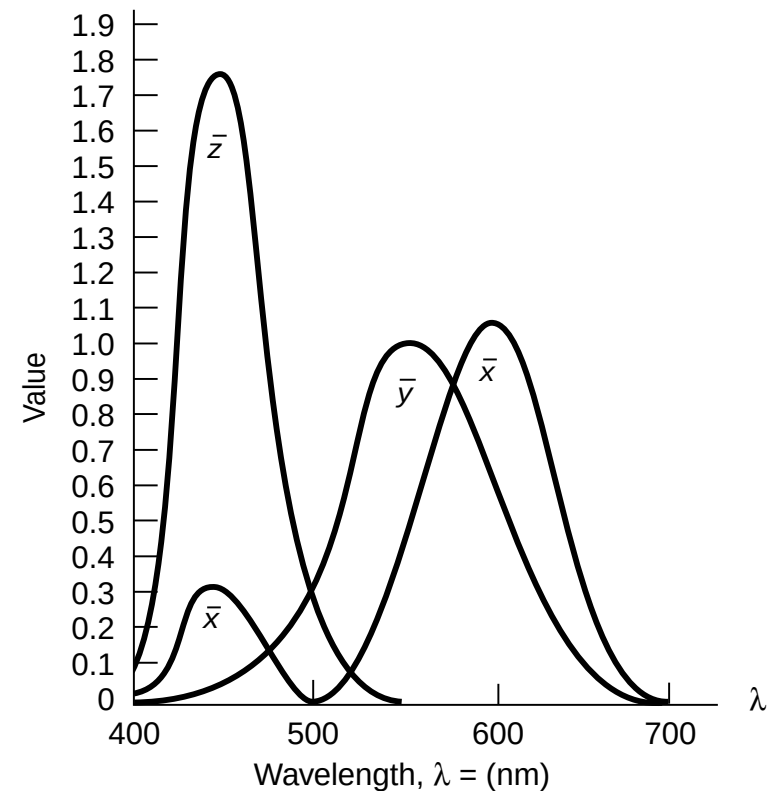


CIE-standarden

1931: Commission Internationale de l'Éclairage (CIE) definierade tre primärer **X**, **Y** och **Z**.

X, **Y** och **Z** motsvarar inga riktiga färger men alla färger kan uttryckas som en *positiv* kombination av dessa.

Diagrammet visar hur mycket X , Y , Z som behövs för att matcha en spektralfärg med $X\mathbf{X} + Y\mathbf{Y} + Z\mathbf{Z}$.

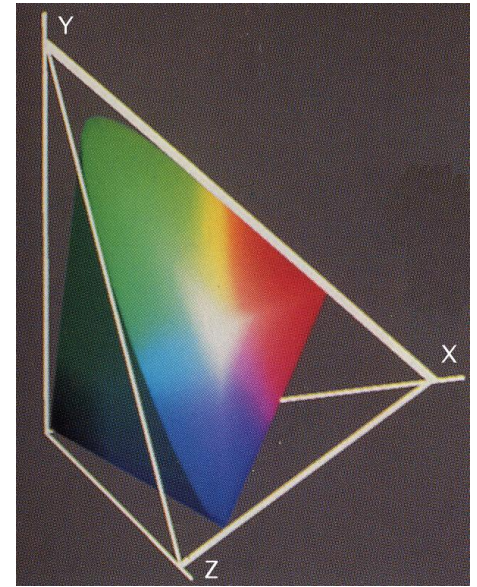


CIE-standaren (forts)

Alla färger $X\mathbf{X} + Y\mathbf{Y} + Z\mathbf{Z}$ bildar
den s k *CIE XYZ-rymden*:

(Se även färgbild 14 i boken.)

Rymden kan projiceras på planet i figuren mha



$$x = \frac{X}{X + Y + Z} \quad y = \frac{Y}{X + Y + Z} \quad z = \frac{Z}{X + Y + Z}$$

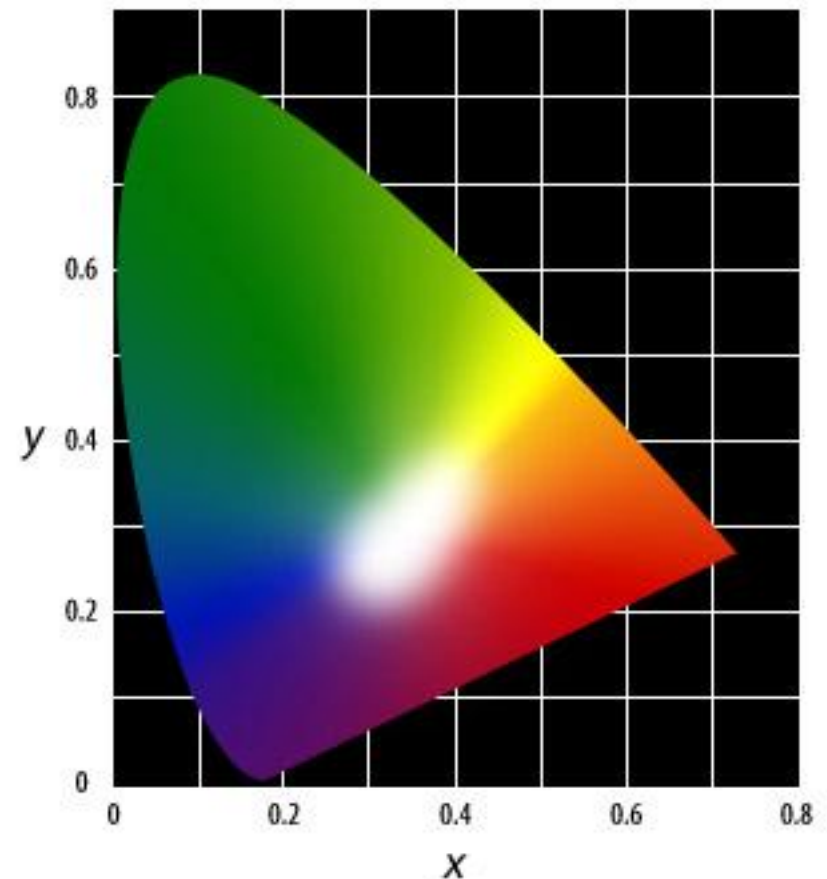
Observera att $x + y + z = 1$ (triangelns plan i figuren).

CIE-standarden (forts)

En ortografisk avbildning av detta plan på (X, Y) -planet definierar CIEs *kromatiska diagram*:

Från x, y får vi $z = 1 - x - y$.

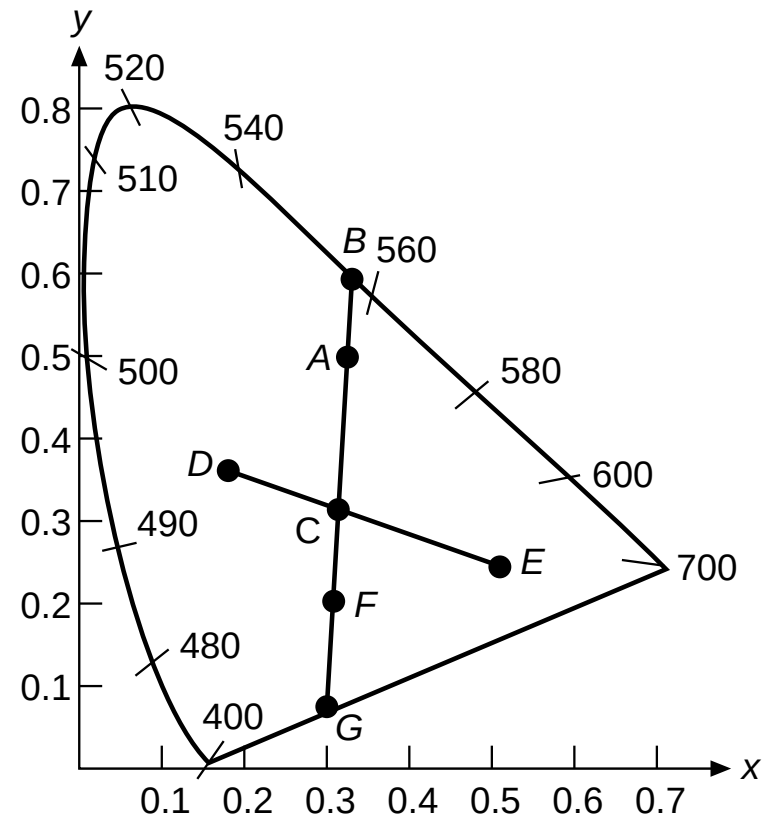
För att återfå X, Y, Z krävs normalt även Y vars viktfordelning är vald så att den är lika med summan av spektral-responsfunktionerna i ögats tappar, dvs Y är ett mått på intensiteten.



CIEs kromatiska diagram

- Alla 100% mättade spektralfärger ligger längs periferin på kurvan.
- Kombinationerna av dessa ligger innanför.
- Alla färger med olika intensiteter avbildas på samma punkt i diagrammet.

Dvs, diagrammet är ingen palett!



- Punkten C ($x, y, z \approx \frac{1}{3}$) definierar dagsljus.

Användningsområden

Ur CIEs kromatiska diagram kan man utläsa följande:

- **Dominant våglängd:**

A är en blandning av vitt ljus (C) och spektralfärgen B , dvs B är den dominanta våglängden i A :s färg.

En del färger (F) kan inte definieras med en dominant våglängd (t ex purpur) och kallas *icke-spektrala*.

- **Exciteringsgrad:**

Förhållandet mellan sträckorna AC och BC anger A :s exciteringsgrad.

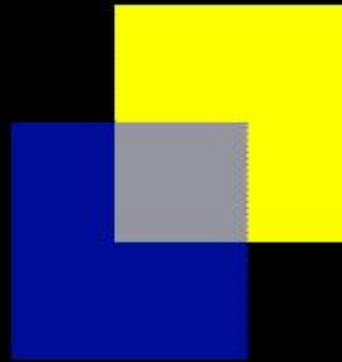
- **Komplementfärger:**

De färger som kan blandas för att få vitt ljus, t ex D och E .

Exempel på komplementfärger

Complimentary colours

pairs of monochromatic lights that are white when mixed together



When mixed properly, blue & yellow lights can yield white, and therefore are complimentary. N.B. there are many complimentary pairs of lights

Färgregister ("Gamut")

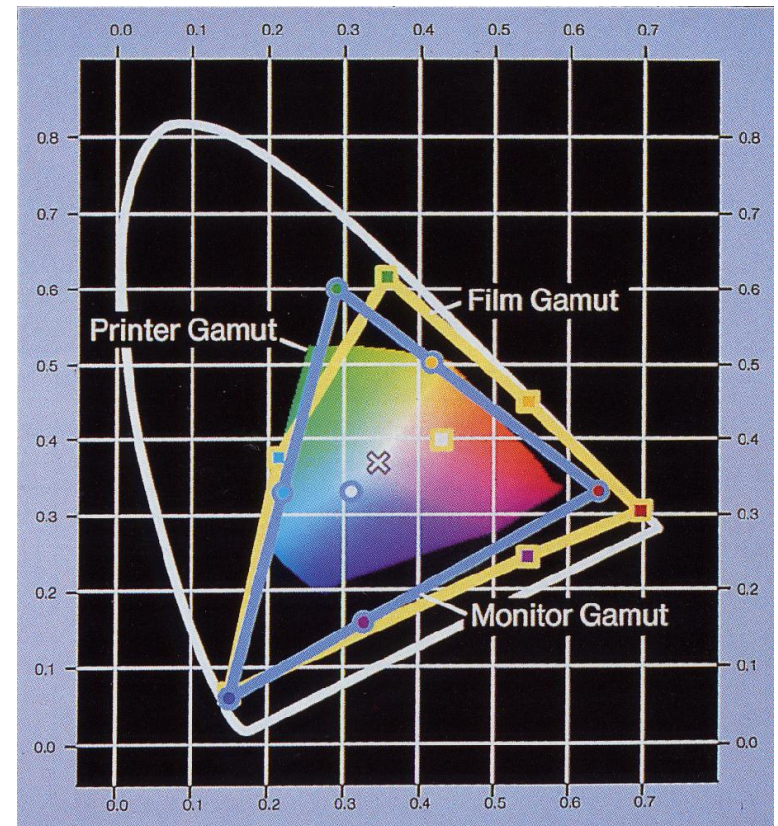
- Med tre färger I, J, K kan man erhålla *färgregistret* av alla färger i triangeln IJK .

Det är nu lätt att se att RGB-modellen inte kan visa alla färger, speciellt inte CIE:s kromatiska diagram!

Typiska värden för en CRT:

$$R = (0.6, 0.34); G = (0.25, 0.6); B = (0.15, 0.06).$$

- För en skrivare brukar registret vara mindre.



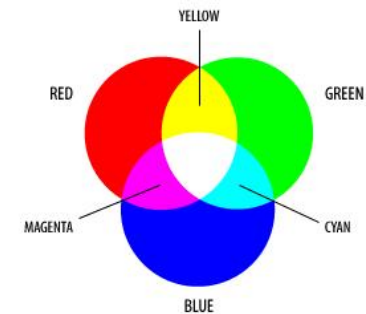
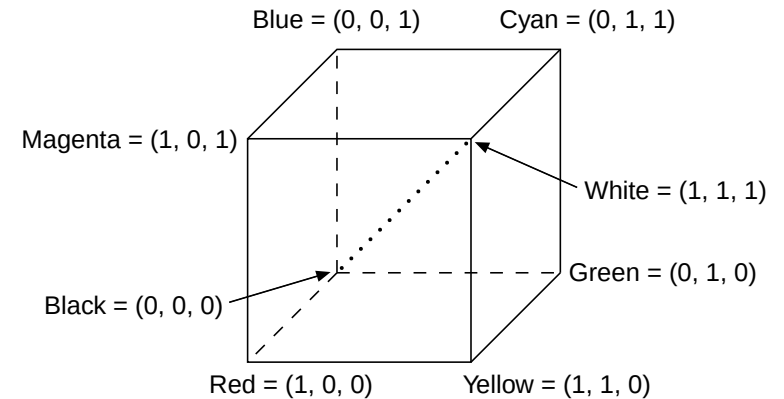
Färgmodeller

En *färgmodell* beskriver alla färger i ett visst register.

RGB-modellen:

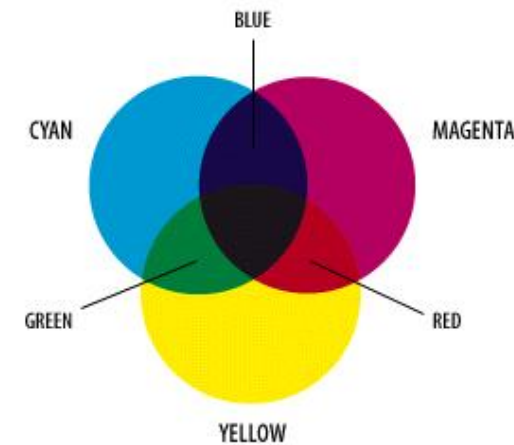
- Additivt färgsystem.
- Ger upphov till en färgkub.

$$\bullet \begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 3.24 & -1.54 & -0.50 \\ -0.97 & 1.88 & 0.04 \\ 0.06 & -0.20 & 1.06 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}$$



CMY-modellen

Färg	Absorberar	Reflekterar
Cyan	Rött	Blått och grönt
Magenta	Grönt	Blått och rött
Gult	Blått	Rött och grönt
Svart	Allt	Inget



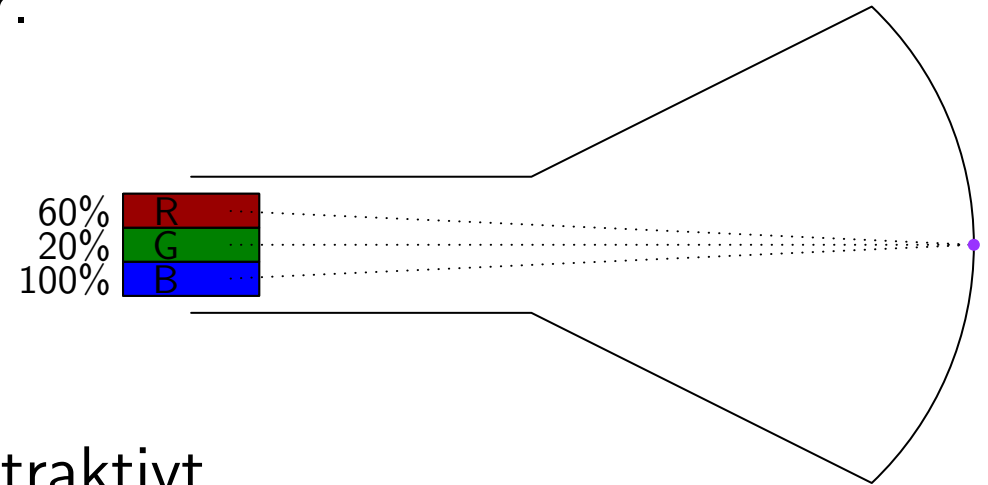
Exempel: blanda cyan och magenta $\Rightarrow$ blått.

Används tillsammans med svart för skrivare, s k *subtraktiv färgblandning* eftersom t ex ett grönt papper absorberar alla färger utom grönt.

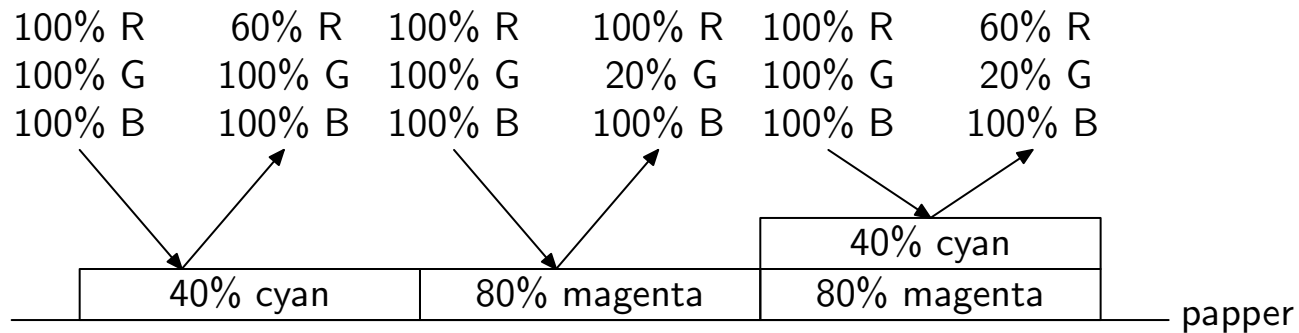
Additiv och subtraktiv färgblandning

Antag att vi vill framställa purpur.

- Med CRT: Additivt.



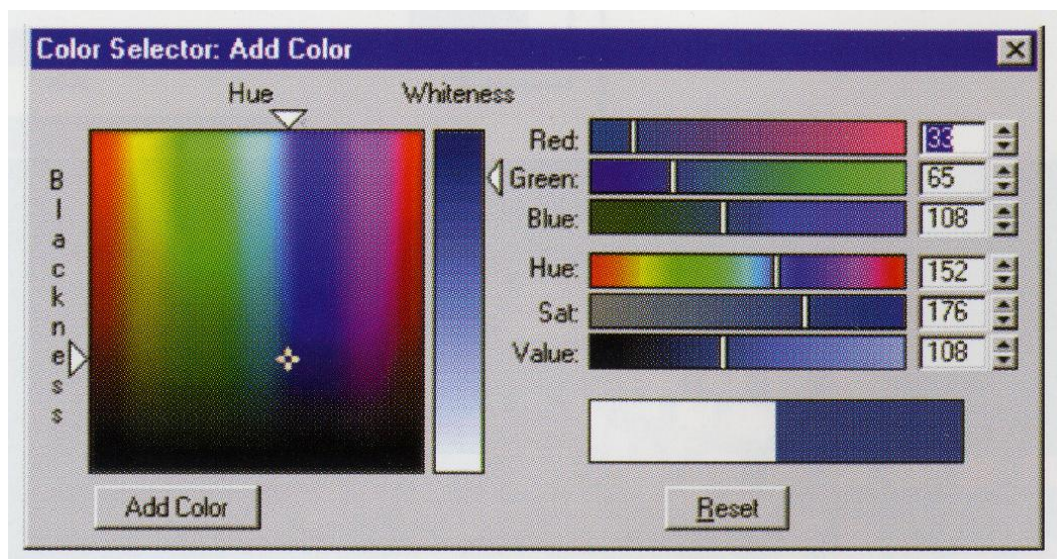
- Med bläckstråleskrivare: Subtraktivt.



Några andra färgmodeller

- **YIQ:** Används för TV-apparater (NTSC) och MPEG-kodning.
- **HSV:** Baserat på Hue, Saturation och Value enligt tidigare.

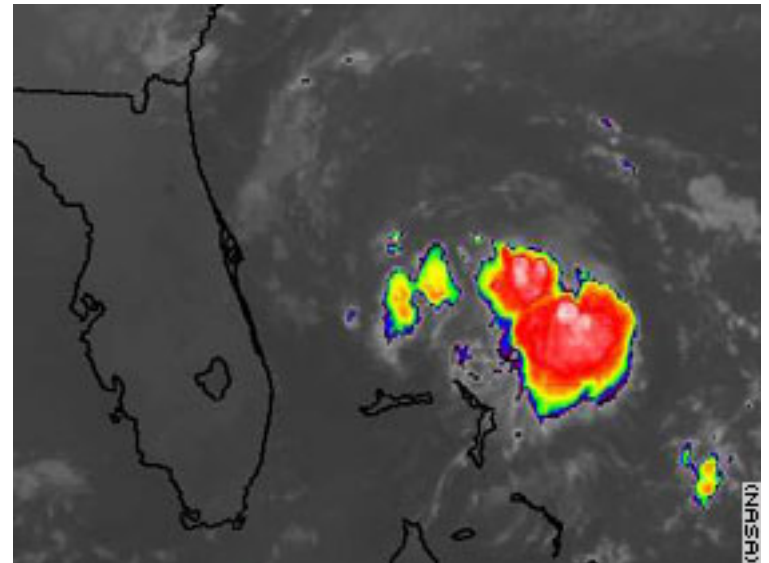
Ett intuitivt sätt att specificera färg i t ex ritprogram.



Att använda färg

Vad används färg till?

- Höja realismen eller förmedla en stämning.
- Markera något **viktigt**.
- Förmedla en numerisk kvantitet, t ex temperatur.



Att använda färg i användargränssnitt

- Lätt att använda massor av olika färger.
- Experiment måste utföras för att se vad som fungerar.
- Försök först göra en monokrom design.
- Om inte det går, använd så få färger som möjligt.
Fast samma färg på olika objekt ger felaktiga associationer.
- Använd variationer i *intensitet*, inte mellan färger.
- Mättade (starka) färgområden distraherar användaren.
De lämnar dessutom en efterbild på näthinnan.

Att använda färg i användargränssnitt (forts)

- Få färger: använd omättad komplementfärg som bakgrund.
- Många färger: använd neutral färg (t ex grå) som bakgrund.
- Blå text fungerar dåligt, likaså kontrastskillnader $< 1 : 3$.

Blå text på svart bakgrund

Vit text på gul bakgrund

- Kombinera speciellt inte rött och blått.

Falsk stereoskopisk effekt

Falsk stereoskopisk effekt

- Tänk på att färgblinda också ska kunna använda programmet.

Föreläsning 13: Skymda ytor

Problemformulering

Placering i renderingskedjan

Eliminering av baksidor

Eliminering av skymda ytor

- Beräkning av normalen

- Z-buffertalgoritmen

- Sveplinjealgoritmen

- BSP-träd

- Strålkastning

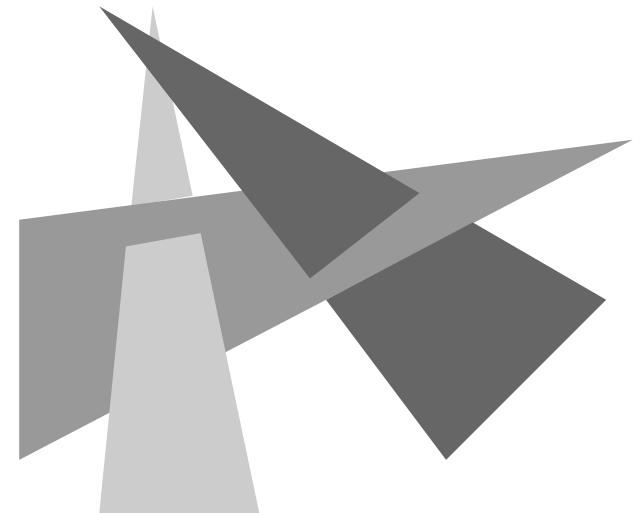
 - Beräkning av skärningspunkter

 - Optimeringar för skärningsberäkningar

- Andra algoritmer för skymda ytor

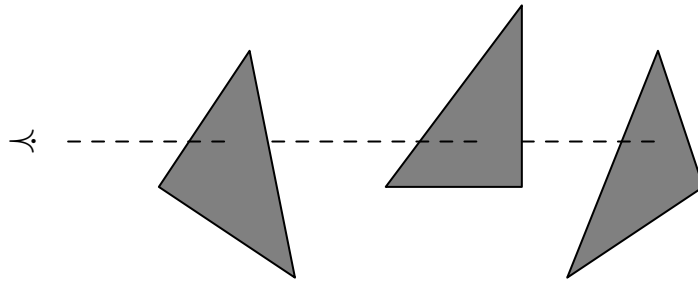
Baksidor och skymda ytor i OpenGL

Lab 3 i OpenGL



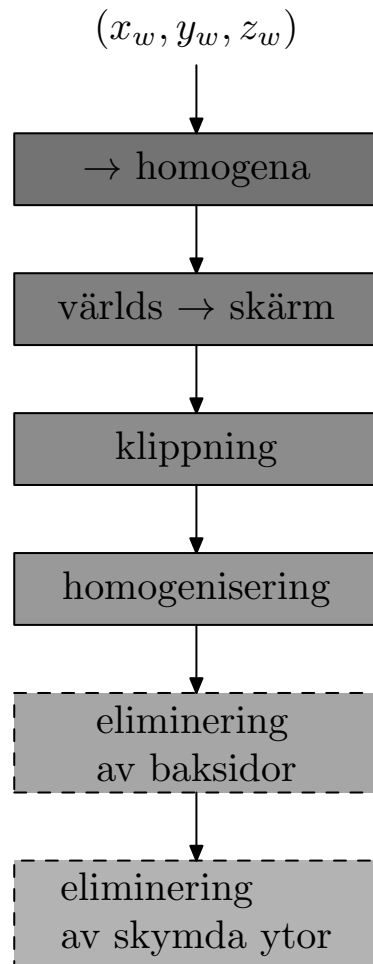
Problemformulering

- Världen består av *ytor i vakuum*. Ingen dimma, moln, hägringar,...
- För varje punkt i bilden söker vi en *djupordning bland ytorna*.



- Alla ytor approximeras med *polygoner*.
 - ▷ Krökta ytor kan *tesseleras* till ett lapptäcke av polygoner.
 - ▷ I strålkastning tillåter vi även sfärer och andra enkla algebraiska strukturer.

Placering i renderingskedjan



$$p_w = \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix}$$

$$p_s = M p_w$$

$$x_s \in [0, XSIZE]$$

$$y_s \in [0, YSIZE]$$

$$z_s \in [0, ZSIZE]$$

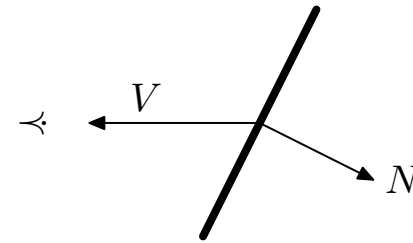
Om $N_z < 0$ så ...

Z-buffert eller sveplinjealgoritmen

Eliminering av baksidor (“back-face culling”)

Sker efter klippning och homogenisering (division med w).

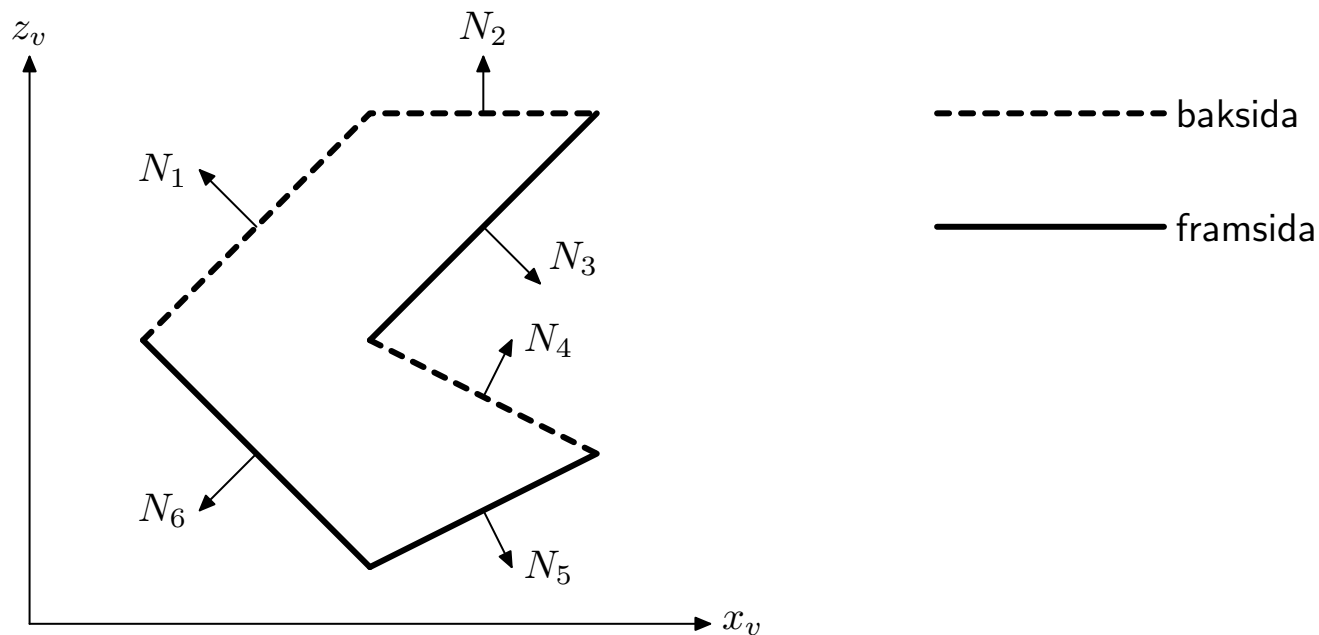
Allmänt: För en vyvektor V och en polygonyta med normal N så är polygonen en baksida om $V \cdot N < 0$.



För parallelprojektion: V är konstant $= (0 \ 0 \ 1)^T$ så eliminera baksidan om $N_z < 0$.

- Elimineras i regel hälften av alla polygoner.
- För enstaka solida konvexa polyedrar är detta allt som behövs!
- För konkava polyedrar krävs ytterligare undersökningar.

Eliminering av baksidor, konkav polyeder

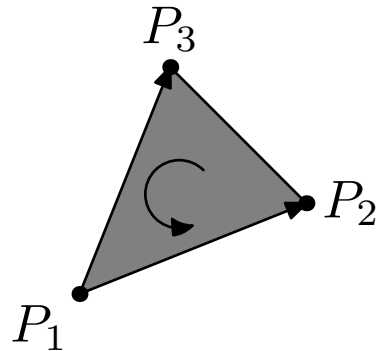


Observera att polygonen med normalen N_3 är skymd.

Lab 3: Ni använder skärmkoordinater, inte vykoordinater, men vyvektorn är ändå $(0 \ 0 \ 1)^T$.

Beräkning av normalen

Normalen beräknas från hörnpunkterna om dessa listas konsekvent.



$$N = \frac{P_1\vec{P}_2 \times P_1\vec{P}_3}{|P_1\vec{P}_2 \times P_1\vec{P}_3|}$$

- **Konvention:** lista punkterna moturs, sett från utsidan.
- **OBS 1:** Moturs i högersystem = medurs i vänstersystem.
- **OBS 2:** Endast tecknet på N_z är intressant!

Alternativ till beräkning av normalen

- **Alternativ 1:** Räkna ut N initialt för varje polygon.

Efter varje transformation M , applicera $Q = (M^{-1})^T$ på N (se kap. 5). OpenGL gör detta.

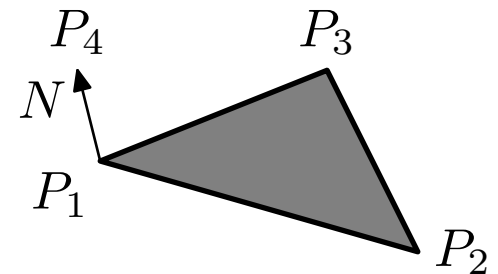
- **Alternativ 2:** Beräkna en extra punkt för varje polygon.

P_4 är konstruerad så att $P_4 = P_1 + N$.

P_4 transformeras med de andra punkterna.

N finns alltid tillgänglig som $\vec{P_1P_4}$.

N måste dock justeras vid skalning.



Eliminering av skymda ytor

- **Objektbaserade algoritmer** (eng. “object-precision”):

för varje objekt i världen:

bestäm de ytor på objektet som inte skymms av andra ytor;
markera dessa ytor för svepkonvertering;

- ▷ Geometri in, geometri ut.
- ▷ Vissa ytor markeras som skymda.
- ▷ Oberoende av upplösningen på skärmen.
- ▷ Svepkonvertering sker vanligtvis efteråt.

Eliminering av skymda ytor (forts)

- **Pixelbaserade algoritmer** (“image-precision”):

för varje pixel i bilden:

 bilda en stråle från observatören till pixeln;

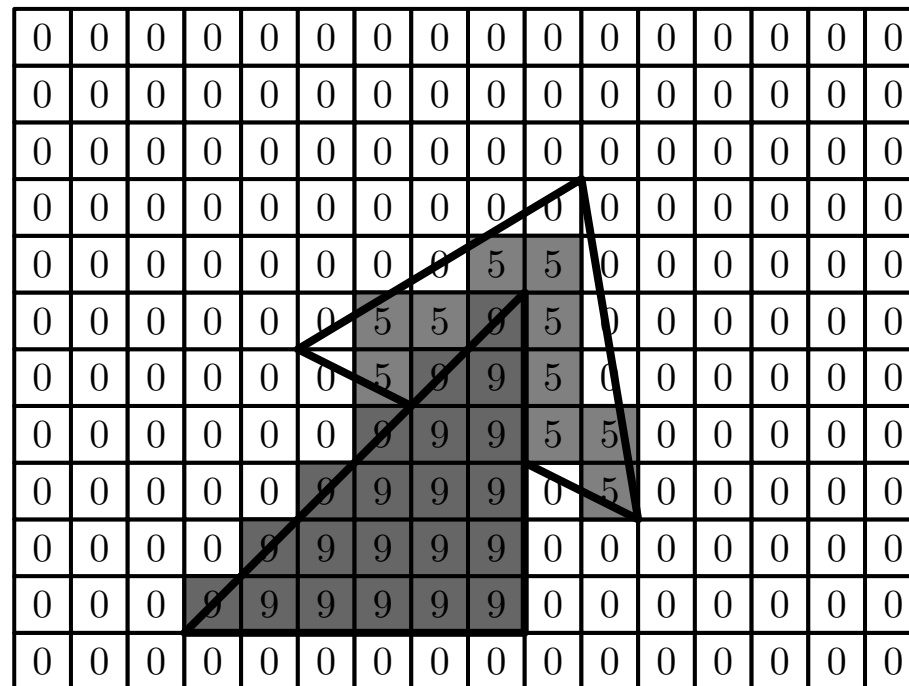
 bestäm det objekt som ligger närmast observatören utefter strålen;

 rita pixeln med lämplig färg;

- ▷ Geometri in, pixels ut.
- ▷ Bra/dålig beroende på tillämpning och upplösning.
- ▷ Synlighet beräknas bara i diskreta steg.
- ▷ Sker vanligtvis som en del av svepkonverteringen.

Z-buffertalgoritmen (depth buffer)

Använder en 2D-buffert $Z(x, y)$, lika stor som bildminnet.



$Z(x, y)$ innehåller z -avståndet till närmaste polygon (uttryckt i skärmkoordinater) för en pixel (x, y) .

Z-buffertalgoritmen (forts)

- I samband med att skärmen rensas för en ny bild:

for $y \in 0 \dots YSIZE$

for $x \in 0 \dots XSIZE$

$Z(x, y) := 0;$ — bakre klipplan

- **Observera:** M transformerar

▷ främre klipplan till $z_s = ZSIZE$

▷ bakre klipplan till $z_s = 0$.

Z-buffertalgoritmen (forts)

- När en polygon svepkonverteras:

for varje pixel (x, y)

$z :=$ polygonens z -värde för (x, y)

if $z > Z(x, y)$ **then** — närmare

$Z(x, y) := z;$

write $(x, y);$

Hur beräknas z ?

1. Först för hörnpunkterna.

- **Perspektivprojicering:**

Planets ekvation: $Ax + By + Cz + D = 0$, dvs

$$z(x, y) = (-Ax - By - D)/C$$

- **Perspektivtransformation:** Använd $z = z_s$!

En av anledningarna till att vi använder perspektivtransformation.

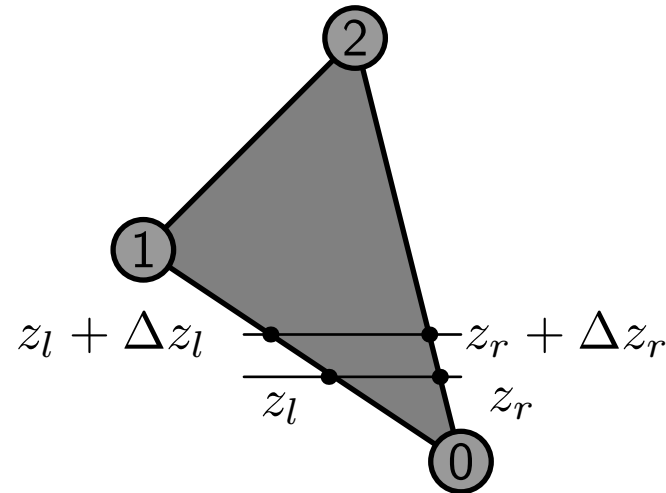
Hur beräknas z (forts)?

2. Sedan linjär interpolering av z längs triangelns sidor:

z -inkrement längs vänster kant:

$$\Delta z_l = (z_1 - z_0)/(y_1 - y_0)$$

$$\Delta z_r = (z_2 - z_0)/(y_2 - y_0)$$



Inkrementen är konstanter, kan räknas ut i förväg.

3. Sedan linjär interpolering längs sveplinjen:

$$\Delta z = \frac{z_r - z_l}{x_r - x_l}$$

Egenskaper för Z-buffertalgoritmen

- Pixelbaserad algoritm.
 - Enkel algoritm, finns implementerad i hårdvara.
 - Ingen förbehandling av objekten krävs.
 - Belysningsaspekter:
 - ▷ Om inte pixeln syns behöver man inte utföra beräkningen.
 - ▷ Men i värsta fallet ritas polygonerna bakifrån.
- ⇒ djupsortera eller använd enkel belysningsmodell.

Egenskaper för Z-buffertalgoritmen (forts)

- Parallellism?
 - ▷ I transformationskedjan (Silicon Graphics).
 - ▷ Pixelvis (men inga inkrementella egenskaper).
 - ▷ I horisontella remsor (kräver mindre Z-buffert).
- Oberoende av objektform ($z = f(x, y)$ räcker).

Nackdelar med Z-buffertalgoritmen

- Minneskrävande
 - ▷ Inte lika stort problem idag, $700 \times 700 \text{ double} < 4\text{MB}$.
 - ▷ Man kan arbeta i remsor för att spara minne.
- Upplösning i z -led
 - ▷ 16 bits räcker inte för mm-noggrannhet när objekten är utspridda över 1 km. I allmänhet krävs 20–32 bits.
 - ▷ Perspektivtransformationen gör det värre eftersom z -värden trycks ihop mot bakplanet.

Sveplinjealgoritmen

En utökning av svepkonverteraren från kap 3 med följande tillägg:

- Arbetar med flera polygoner, inte bara en.
- En kant i kanttabellen innehåller också polygon-id.
- En *polygontabell* innehåller *för varje polygon*:
 1. Dess id-nummer.
 2. De fyra koefficienterna i planets ekvation.
 3. Intensitets- eller färginformation.
 4. En boolesk flagga *in*, initialt falsk.

in är sann då sveplinjen passerar polygonen.
in sätts till falsk när en ny sveplinje startar.

Aktiva kanttabellen sorterad efter stigande x , som förut.

Exempel: DEF framför ABC framför GHIJ

1. $in(GHIJ) := \text{sann.}$

Fyll med GHIJ:s färg.

2. $in(ABC) := \text{sann.}$

$z(ABC) < z(GHIJ) \text{ i } (x, y) \Rightarrow ABC \text{ främst.}$

Fyll med ABC:s färg.

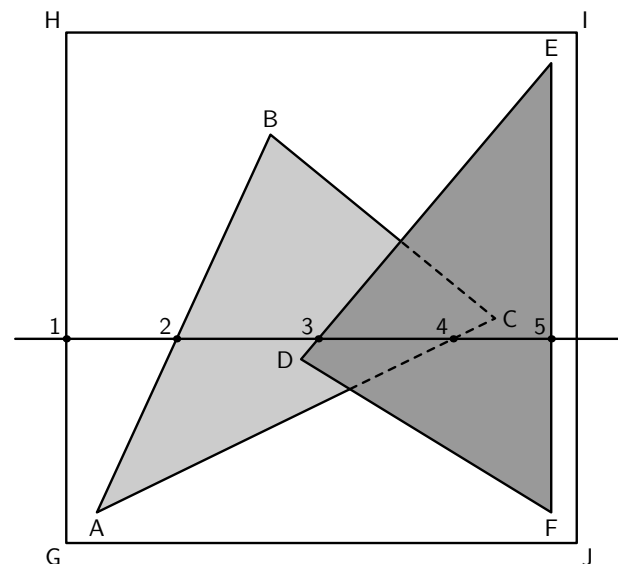
3. $in(DEF) := \text{sann.}$

$z(ABC) > z(DEF) \text{ i } (x, y) \Rightarrow DEF \text{ främst.}$

Fyll med DEF:s färg.

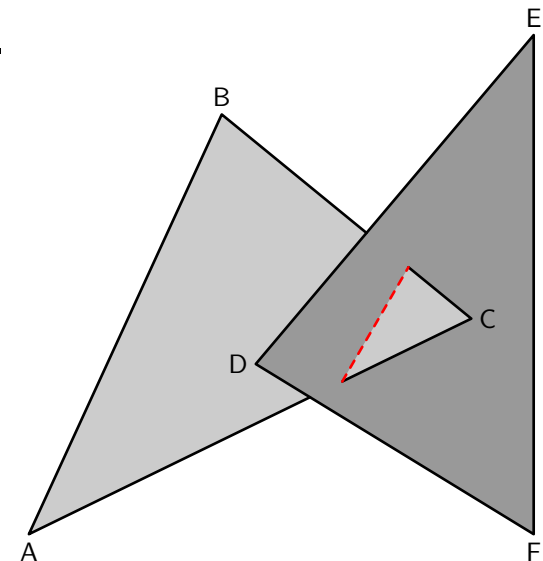
4. $in(ABC) := \text{falsk.}$

Om inte polygonerna skär varandra behövs ingen djupjämförelse;
fortsätt fyll med DEF:s färg.



Skärande polygoner

- Räkna ut skärningslinjen.
- Lägg till linjen i kanttabellen.
- Vidtag "lämpliga åtgärder" när den korsas.



Egenskaper hos sveplinjealgoritmen

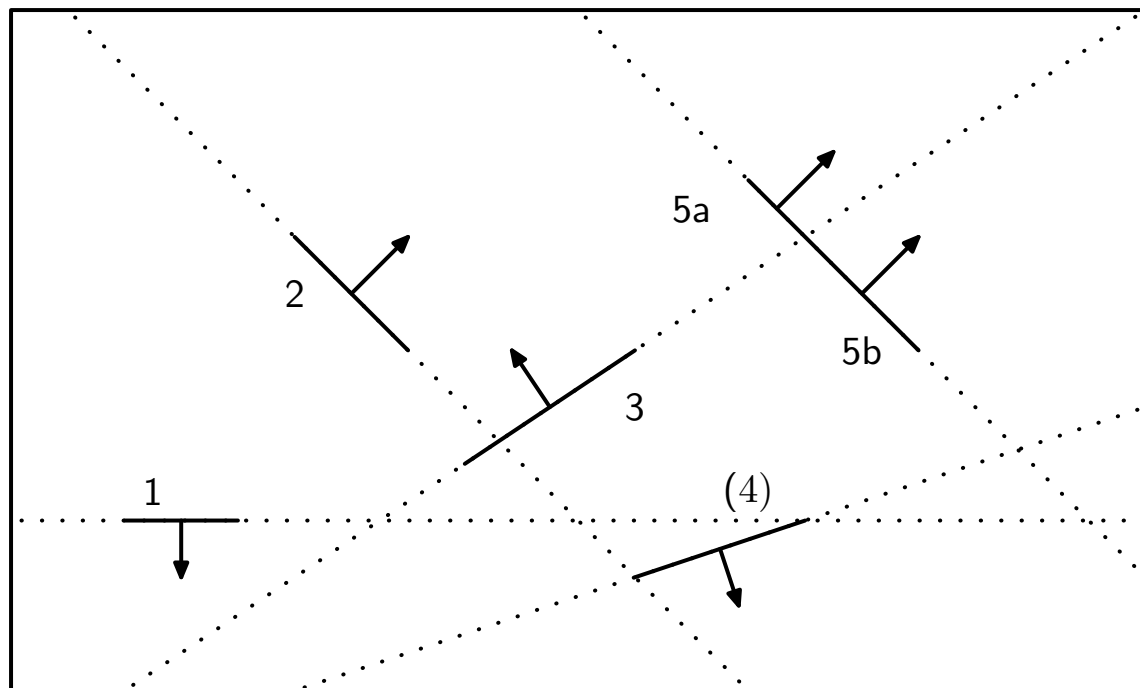
- En objektbaserad algoritm.

- Belysningsaspekter:

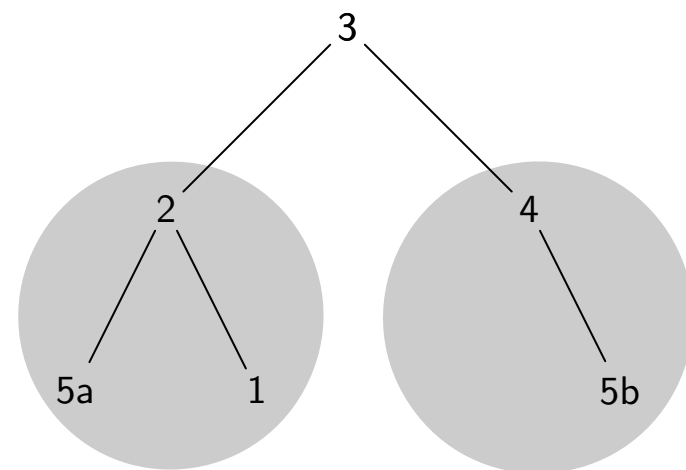
Varje pixel ritas bara en gång så komplicerade ljusberäkningar OK.

- Ingen förbehandling av objekten krävs.
- Hårdvarustöd? Finns för flygsimulatorer – extremt komplicerat.

BSP-träd



Scen



BSP-träd

BSP-träd: konstruktion

Polygonerna i scenen utgör noderna i BSP-trädet.

- Välj en godtycklig polygon (t ex 3 i figuren).
- Vänster delträd = de polygoner som befinner sig framför 3:s plan.
- Höger delträd = de polygoner som befinner sig bakom 3:s plan.
- Polygoner som skär planet måste delas (5a och 5b).
- Upprepa rekursivt.

Observera att konstruktionen av BSP-trädet är vyoberoende.

BSP-träd: traversering

Traversering görs för en given vyvektor V .

procedure *traverse*(p)

if $V \cdot N_p > 0$ — observatören framför p

traverse($p.back$);

display(p);

traverse($p.front$);

else

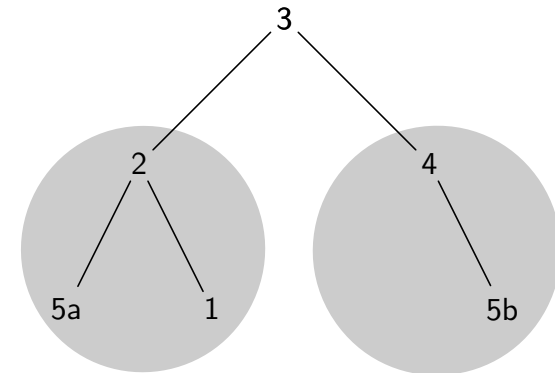
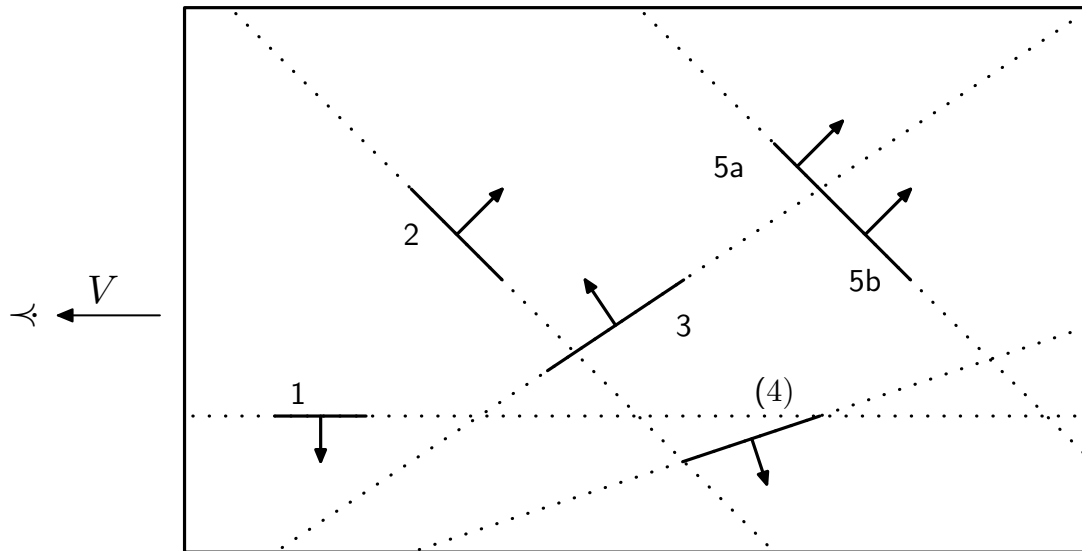
traverse($p.front$);

display(p); — om baksidan syns

traverse($p.back$);

end

Exempel på traversering av BSP-träd



För ögat i figuren $\Rightarrow$ 4, 5b, 3, 5a, 2, 1.

BSP-träd: egenskaper

- + Vyoberoende
- + Snabb traversering
- Kräver förbehandling
- Fungerar ej i en dynamisk värld
- Hur många polygoner som måste delas beror på vilken polygon som väljs härnäst.

Optimalt val är (förmodligen?) NP-hårt.

Strålkastning

Pixelbaserad algoritm.

- Första strålen: strålkastning (“ray casting”)
- Rekursivt: strålföljning (“ray tracing”)

Strålkastningsalgoritm

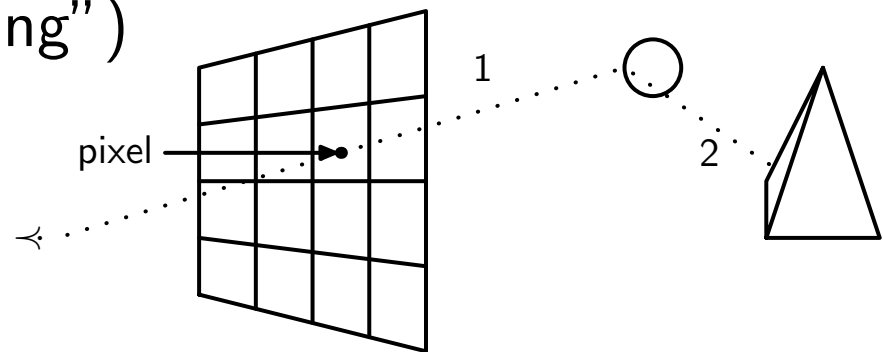
för varje pixel

 bilda en stråle från observatören till pixeln;

 för varje objekt

 räkna ut var strålen skär objektet;

 välj närmsta skärningspunkt och fyll pixeln;



Beräkning av skärningspunkter

(Teori för både strålkastning och strålföljning.)

- Görs vanligtvis i *världskoordinater*, ty
 - ▷ Världs- till vytransformation förenklar bara första strålen.
 - ▷ Perspektivtransformationen P bevarar inte vinklar, så reflektion av strålar blir besvärliga att analysera.
 - ▷ Sfärer blir ellipsoider i perspektiv, osv.

Beräkning av skärningspunkter (forts)

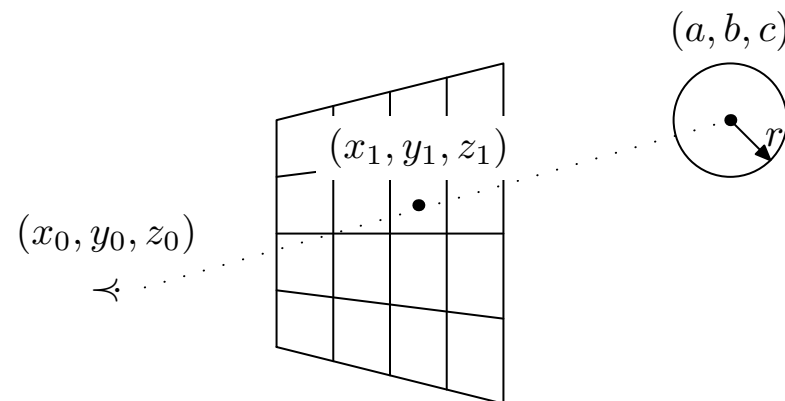
Använd *parametrisk form* för att beskriva strålen:

$$x = x_0 + t(x_1 - x_0) = x_0 + t\Delta x$$

$$y = y_0 + t(y_1 - y_0) = y_0 + t\Delta y$$

$$z = z_0 + t(z_1 - z_0) = z_0 + t\Delta z$$

Enklarest att räkna ut skärning (och spegling) för en *sfär*:



Beräkning av skärningspunkt för sfär

Sfär, implicit form: $(x - a)^2 + (y - b)^2 + (z - c)^2 - r^2 = 0$.

Ersätt x med $x_0 + t\Delta x$, y med $y_0 + t\Delta y$, z med $z_0 + t\Delta z$:

Efter lite algebra fås $At^2 + Bt + C = 0$ där

$$A = \Delta x^2 + \Delta y^2 + \Delta z^2$$

$$B = 2(\Delta x(x_0 - a) + \Delta y(y_0 - b) + \Delta z(z_0 - c))$$

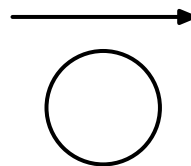
$$C = (x_0 - a)^2 + (y_0 - b)^2 + (z_0 - c)^2 - r^2$$

Lösning till andragradsekvationen:

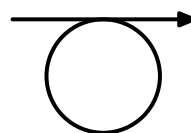
$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

Tolkning av lösningen för skärningspunkten

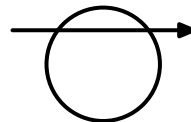
• Om $B^2 - 4AC < 0 \Rightarrow$ inga (reella) lösningar:



• Om $B^2 - 4AC = 0 \Rightarrow$ en lösning:



• Om $B^2 - 4AC > 0 \Rightarrow$ två lösningar:



Den lösning med minsta t -värdet > 0 är närmast observatören.

Skärningspunkten fås med

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix} + t \begin{pmatrix} \Delta x \\ \Delta y \\ \Delta z \end{pmatrix}$$

Skärningsberäkning för polygoner

Undersök först om strålen passerar polygonens plan.

Plan, implicit form: $Ax + By + Cz + D = 0$.

Ersätt x med $x_0 + t\Delta x$, y med $y_0 + t\Delta y$, z med $z_0 + t\Delta z$.

Efter lite algebra fås $t = -\frac{Ax_0 + By_0 + Cz_0 + D}{A\Delta x + B\Delta y + C\Delta z}$

Om nämnaren $= 0$ är strålen parallell med planet.

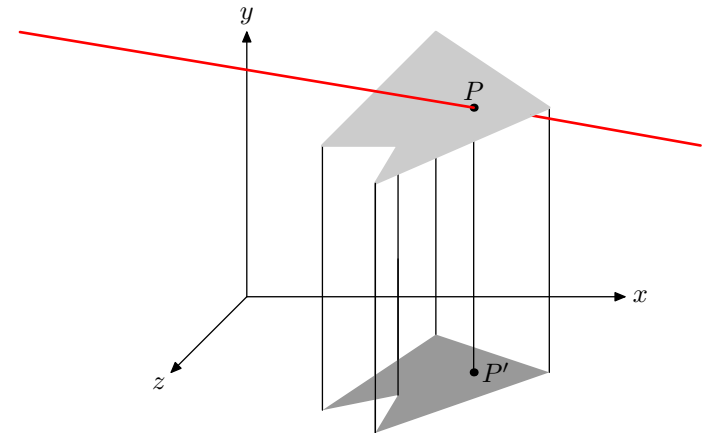


Om $t > 0$ passeras planet – fortsätt undersökandet.

Skärningsberäkning för polygoner (forts)

Undersök sedan om strålen går igenom polygonen:

- Projicera mot det plan som ger störst avbildning: yz -planet om $|A|$ är störst, xz -planet om $|B|$ är störst, xy -planet om $|C|$ är störst.
- För att ta reda på om P är inuti polygonen, undersök om P' är inuti den projicerade polygonen genom att dra en linje från P' och räkna antalet korsningar: udda antal $\Rightarrow$ inuti.



(Men se upp om linjen går igenom hörnpunkter!)

Strålkastning: egenskaper

- **Tidskomplexitet**

Ingen möjlighet till inkrementella beräkningar (“coherence”) betyder att strålkastning är mycket värre än Z-buffert:

- ▷ Z-buffert: $\mathcal{O}(pd)$, där p = antal pixels, d = djup.
- ▷ Strålkastning: $\mathcal{O}(pn)$, där n = antal polygoner som skär.

$n \gg d$, så strålkastning är *mycket värre*.

Strålkastning: egenskaper (forts)

- **Några vanliga siffror**

1024 × 1024-skärm med 40 000 polygoner.

En Silicon Graphics Indigo ritar dessa på ca 1 sek med Z-buffert.

Med strålkastning: 40 000 · 1024² skärningstest.

Ett skärningstest tar 100 instruktioner i bästa fall.

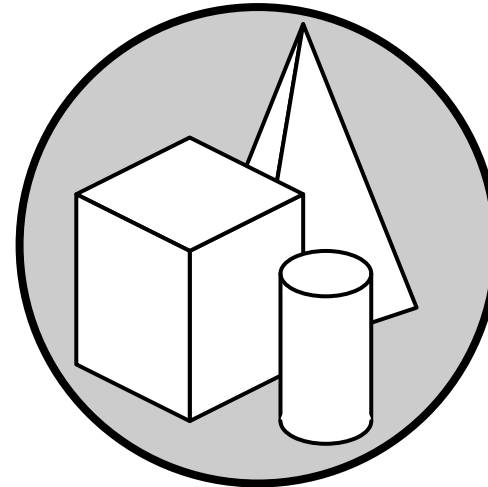
Processorn kör med 100 MIPS $\Rightarrow$ 11 timmar!!

Turner Whitted fann att skärningstesten tog 75–95% av tiden.

Optimeringar för skärningsberäkningar

- **Omslutande volymer**

- ▷ Vanligtvis en sfär eller en parallelepiped.
- ▷ Kan flyttas tillsammans med objekten.
- ▷ Om strålen inte skär omslutande volymen kan den omöjligt skära de inkapslade objekten.

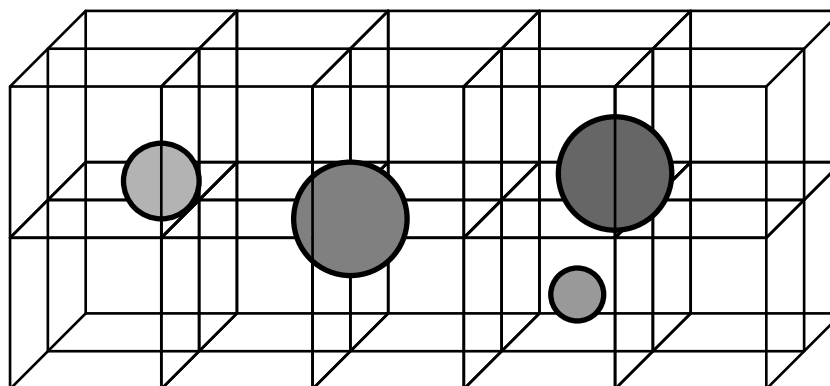


Ännu bättre är hierarkier av omslutande volymer, men dessa är svåra att generera automatiskt.

Optimeringar för skärningsberäkningar (forts)

- **Spatial indelning**

- ▷ Strålen traverserar cellerna i Bresenham-stil.
- ▷ Varje cell innehåller ett antal objekt som måste undersökas närmare (gärna omslutande volymer).
- ▷ Indelningen ej optimal för de flesta scener.
- ▷ Sk *octrees* användbart för hierarkisk indelning ner till viss nivå.



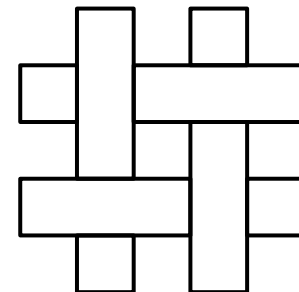
Strålkastning: ytterligare egenskaper

- **Anti-aliasing?** Kasta fler strålar, räkna ut medelvärdet. Eftersom strålkastning inte är inkrementell kan detta göras adaptivt.
- **Förbehandling?** Ja, för vissa optimeringar.
- **Hårdvara?** Finns faktiskt, <http://www.art.co.uk>.
- **Parallellism?** Ja, en process per stråle. Vanligt att dela upp ett jobb på flera maskiner.
- **Enkel?** Ganska enkel, t o m med någon av optimeringarna.

Andra algoritmer för skymda ytor

Djupsorteringsalgoritmen

- Polygonerna sorteras efter z så att bilden blir korrekt om de svepkonverteras i omvänd ordning.
- I varje sorteringssteg undersöks om två polygoner överlappar varandra i x -, y - eller z -led.
- I vissa situationer kan man ej upprätta en ordning:
Algoritmen hamnar då i en oändlig loop.
- Åtgärd: dela upp överlappande polygoner i mindre polygoner.



Sammanfattning

Eliminera baksidorna först. Sedan beror det på om du har. . .

- 10 ytor: BSP-träd eller djupsortering.
- 1000 ytor: BSP-träd, djupsortering eller sveplinje.
- 10000 ytor: Z-buffert.

Z-buffertalgoritmen har nästan konstant tidskomplexitet ty ju fler polygoner desto mindre brukar dess ytor vara.

- Buktiga ytor: strålkastning
- Olika vyer: BSP-träd

Andra faktorer som spelar in: belysningsmodell, hårdvarustöd, utspridning i $x/y/z$ -led, osv.

Baksidor och skymda ytor i OpenGL

OpenGL har både eliminering av baksidor och Z-buffert.

- Eliminering av baksidor (*backface culling*):

```
glEnable(GL_CULL_FACE);  
glCullFace(GL_BACK);
```

- Z-buffert (*depth buffer*)

```
glutInitDisplayMode(GLUT_DEPTH | ... );  
glEnable(GL_DEPTH_TEST);
```

För att rensa Z-bufferten:

```
glClear(GL_DEPTH_BUFFER_BIT | ... );
```

Lab 3 i OpenGL

```
void myinit(void) {  
    /* depth buffer, single buffering, and true color */  
    glutInitDisplayMode(GLUT_DEPTH|GLUT_SINGLE|GLUT_RGB);  
  
    glutInitWindowSize(SIZE, SIZE);  
    glutCreateWindow(progname);  
  
    /* cull back-facing polygons */  
    glEnable(GL_CULL_FACE);  
    glCullFace(GL_BACK);  
  
    /* turn on the depth buffer */  
    glEnable(GL_DEPTH_TEST);  
  
    /* callback functions for redrawing and keyboard */  
    glutDisplayFunc(display);  
    glutKeyboardFunc(keys);  
    /* black background color */
```

```
glClearColor(0.0, 0.0, 0.0, 0.0);

/* set up perspective projection */
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(fov, 1.0, near, far);
glMatrixMode(GL_MODELVIEW);
}

void display(void) {
    Obj *obj;
    /* clear the window and the depth buffer */
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    :
    for (obj = objlist; obj; obj = obj->next)
        wire(obj);
    glFlush();
}
```

Föreläsning 14A: Lokala belysningsmodeller

Belysning

Definition av lokal belysning

Ljuskällor

Reflektion

- Diffus reflektion

- Speglande reflektion

Bakgrundsbelysning

Dämpningsfaktor

Färg

Flera ljuskällor

Approximation av buktiga ytor

Implementation

- Flat shading

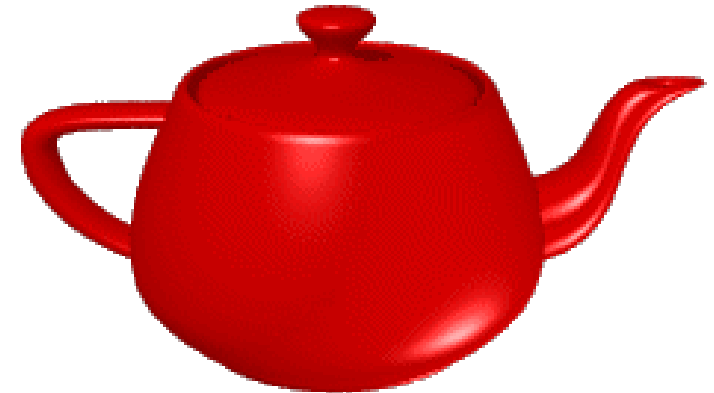
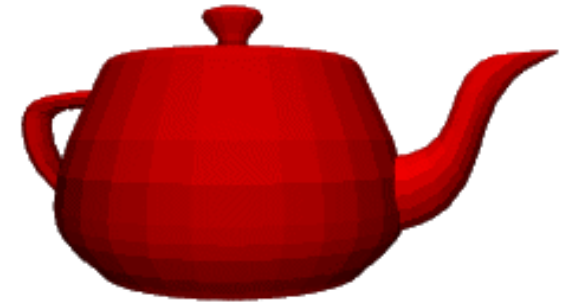
- Mach-band

- Gouraud shading

- Phong shading

Belysning i OpenGL

Lab 4 i OpenGL



Belysning

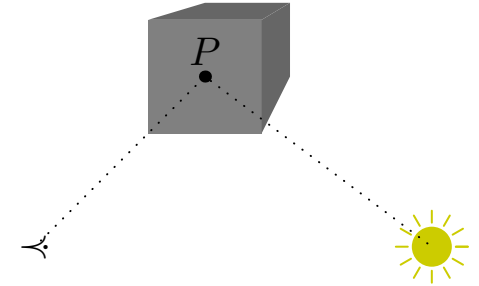
Nu när vi vet vad som syns, vad har det för färg?

Huvudproblem: Beräkna intensiteten för P .

- **Fysikaliska principer:** komplicerade.

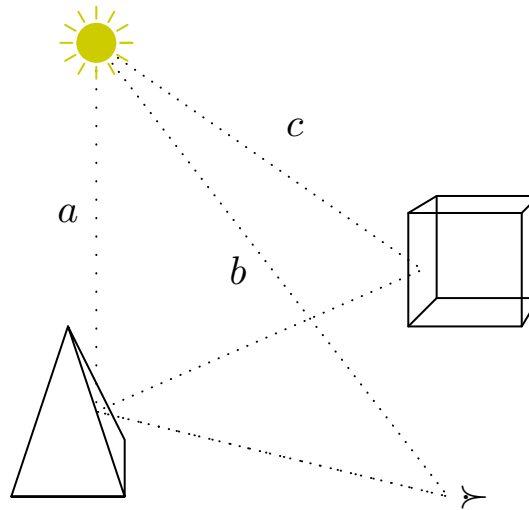
Reflektion, brytning, skuggor, materialegenskaper, belysningskällor, etc.

- **Vår belysningsmodell:** empirisk, förenklad.
- **Mål:** Snabba beräkningar, realistiska resultat.



Definition av lokal belysning

Endast 1:a ordningens reflektioner (a) räknas.



Bidrag från andra strålar – direkt från ljuskällan (b) eller via två eller fler ytor (c) – approximeras med en konstant term, s k bakgrundsljus (*ambient light*).

Ljuskällor

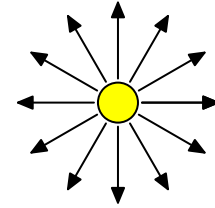
Två typer av ljuskällor:

- **Punktformat ljus:** Ljuskällan nära scenen.

Exempel: skrivbordslampa.

Karaktäriseras av ljuskällans placering i världen.

Fördel: ger skarpa reflexer på blanka ytor.

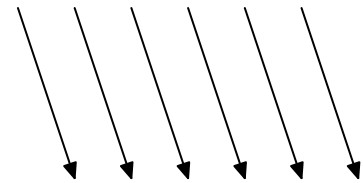


- **Parallellt ljus:** Ljuskällan oändligt långt bort.

Exempel: solljus.

Karaktäriseras av ljusets riktning (en vektor).

Fördel: förenklar belysningsberäkningarna.



Reflektion

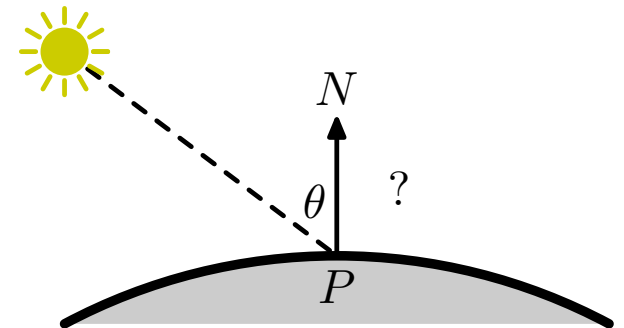
Antagande: alla objektytor är ogenomskinliga.

Vad händer när ljuset träffar ytan i punkt P ? Det beror på:

- **Materialegenskaper**

Matta ytor $\Rightarrow$ *diffus* reflektion.

Blanka ytor $\Rightarrow$ *speglande* reflektion.



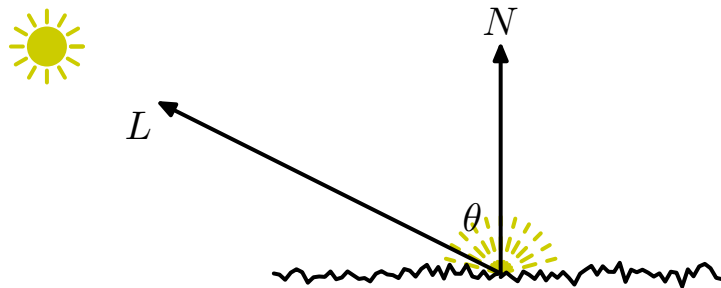
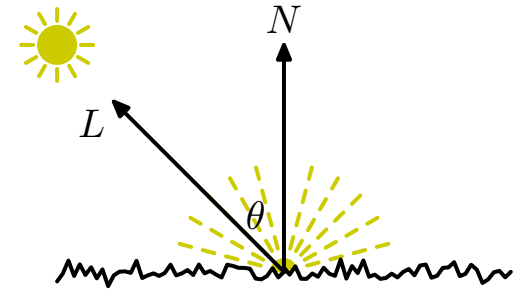
- **Infallsvinkeln θ**

Reflekterad intensitet varierar med infallsvinkeln.

- **Ljusets våglängd** (försummas i vår modell)

Diffus reflektion

- Förekommer på "grova" ytor (tyg, damm, etc).
- Ljuset sprids *likformigt i alla riktningar*.
- Reflekterad intensitet $\propto \cos \theta$ (prova själv!)



- Reflekterad intensitet är *vyoberoende*!

Empirisk modell för diffus reflektion

Lamberts cosinuslag: $I = I_p k_d \cos \theta$ där

I_p = ljuskällans intensitet $\in [0, 1]$

k_d = materialets diffusa reflektionskoefficient $\in [0, 1]$

$\theta \in [-90^\circ, 90^\circ]$

Om L och N är enhetsvektorer: $I = I_p k_d (L \cdot N)$ där

L = vektor som pekar mot ljuskällan.

N = ytans normal.

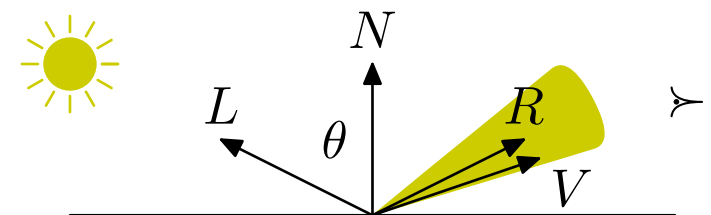
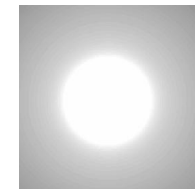
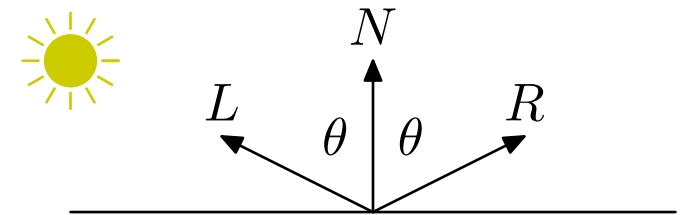
OBS: om $L \cdot N < 0$ så ligger ljuskällan "under" ytan och $I = 0$.

Speglande reflektion

- Förekommer på blanka ytor (glas, plast).
- Infallsvinkel = Reflektionsvinkel
- Åstadkommer ett "blänk" på ytan:
- Reflekterad intensitet är *vyberoende*.

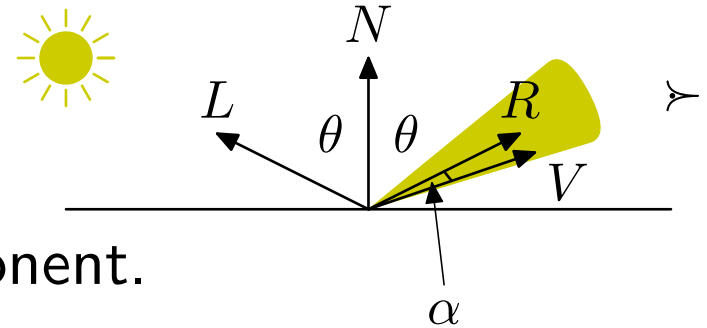
V : vektor *mot* observatören.

Spridningen beror på materialet.



Empirisk modell för speglande reflektion

- Maximal reflektion observeras när $\alpha = 0$.
- Intensiteten avtar sedan med $\cos^n \alpha$.
- n är materialets speglande reflektionsexponent.
 - ▷ Kontrollerar spridningen på reflektionen.
 - ▷ Exempel: $n_{\text{krita}} = 1, n_{\text{spegel}} = 100$.
- Intensitetens belopp beror på θ men brukar approximeras med en konstant $k_s \in [0, 1]$.

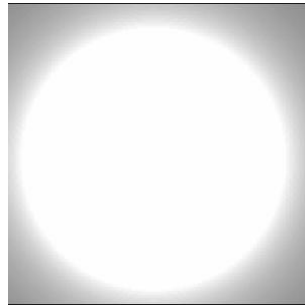


Phongs belysningsmodell: $I = I_p k_s \cos^n \alpha$,

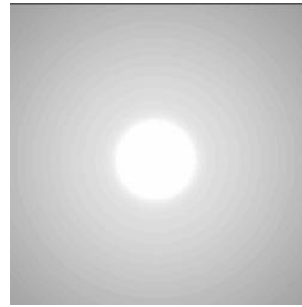
där k_s är materialets speglande reflektionskoefficient.

- Om R och V är enhetsvektorer: $I = I_p k_s (R \cdot V)^n$.

Exempel på olika reflektionsexponenter



$$n = 4$$

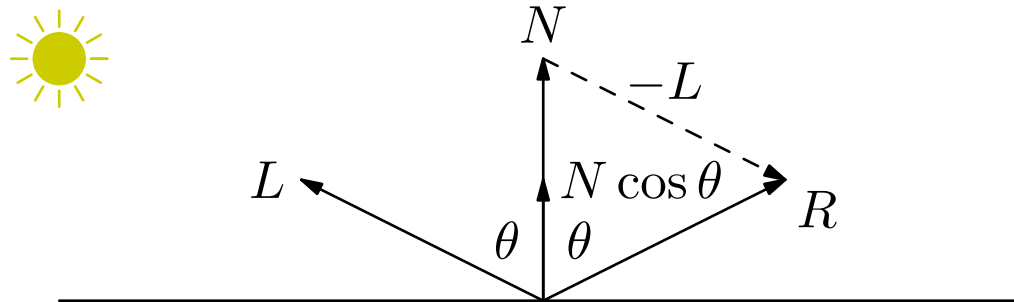


$$n = 100$$

Se också bild 14.9 och 14.10 i kursboken.

Beräkning av R

- Hur beräkna R ?



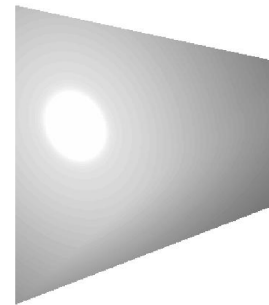
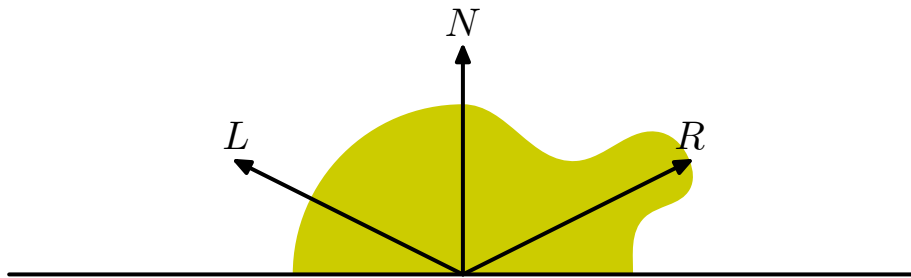
- Ur figuren: $R = 2N \cos \theta - L$
- Om L och N är enhetsvektorer: $\cos \theta = L \cdot N$, så

$$R = 2N(L \cdot N) - L$$

Kombinerad diffus och speglande reflektion

De flesta ytor uppvisar båda typer av reflektion, så

$$I = \underbrace{I_p k_d (L \cdot N)}_{\text{diffus}} + \underbrace{I_p k_s (R \cdot V)^n}_{\text{speglande}}$$



Bakgrundsbelysning (“ambient light”)

Reflektion från andra ytor och ljus direkt från ljuskällan sammanfattas med en konstant bakgrundsterm:

$$I = \underbrace{I_a k_a}_{\text{bakgrund}} + \underbrace{I_p k_d (L \cdot N)}_{\text{diffus}} + \underbrace{I_p k_s (R \cdot V)^n}_{\text{speglade}}$$

där

I_a = Bakgrundsbelysningens intensitet.

k_a = Reflektionskoefficient för bakgrundsljus $\in [0, 1]$.

Varför k_a vs k_d ? Inget bra skäl.

Sammanfattning

$$I = I_a k_a + I_p k_d (L \cdot N) + I_p k_s (R \cdot V)^n$$

I_a = bakgrundsbelysningens intensitet $\in [0, 1]$.

I_p = ljuskällans intensitet $\in [0, 1]$.

L = enhetsvektor som pekar mot ljuskällan.

N = ytans normal, enhetsvektor.

$R = 2N(L \cdot N) - L$

k_a = reflektionskoefficient för bakgrundsljus $\in [0, 1]$.

k_d = materialets diffusa reflektionskoefficient $\in [0, 1]$.

k_s = materialets speglade reflektionskoefficient $\in [0, 1]$.

n = materialets speglade reflektionsexponent > 0 .

OBS: Endast bakgrundsljus om $L \cdot N < 0$; inget speglade ljus om $R \cdot V < 0$.

Dämpningsfaktor $f(d)$

- Energin avtar med avståndet². (Pluto är kallare än Jorden)
- Vi borde därför multiplicera I_p med $f(d) = 1/d^2$.
- I praktiken används $f(d) = \min(\frac{1}{c_1 + c_2d + c_3d^2}, 1)$ där c_1 används för att förhindra alltför starkt ljus nära observatören och $\min(\dots, 1)$ garanterar dämpning. OpenGL stöder detta.
- Varför denna formel? Ser bättre ut.
- Ny formel: $I = I_a k_a + f(d) I_p [k_d (L \cdot N) + k_s (R \cdot V)^n]$.
- Observera att ingen hänsyn tas till avståndet observatör/objekt.

Färg

- Behandla varje färg (våglängd λ) för sig.

$$I_{\lambda} = I_{a\lambda}k_aO_{d\lambda} + f(d)I_{p\lambda}[k_dO_{d\lambda}(N \cdot L) + k_s(R \cdot V)^n]$$

där $O_{d\lambda} \in [0, 1]$ indikerar objektets diffusa färg.

Exempel: rött äpple har $O_{d\lambda} = 1$ för $\lambda = \text{röd}$, men $O_{d\lambda} = 0$ för alla andra färger.

- Vanliga λ är rött, grönt, blått.
- Varför ingår inte $O_{d\lambda}$ i speglande reflektion?

Lys på en röd julgranskula med blått ljus $\Rightarrow$ speglande reflektionen är blå.

Flera ljuskällor

Summera bidraget från alla ljuskällor i :

$$I_{\lambda} = I_{a\lambda} k_a O_{d\lambda} + \sum_i f(d_i) I_{pi\lambda} [k_d O_{d\lambda} (N \cdot L_i) + k_s (R_i \cdot V)^n]$$

Vid summeringen fås problemet med att I_{λ} kan överskrida maxgränsen. Åtgärder:

- **Clamping:** Om $I_{\lambda} > \text{MAX}$ så $I_{\lambda} := \text{MAX}$.

Hela bilden kan bli vit.

- **Skalning:** Skala alla I_{λ} efter högsta intensiteten i bilden.

Kräver i regel två pass. Ej bra för animering.

Exempel

Exempel: $I = I_a k_a + f(d) I_p [k_d (L \cdot N) + k_s (R \cdot V)^n]$.

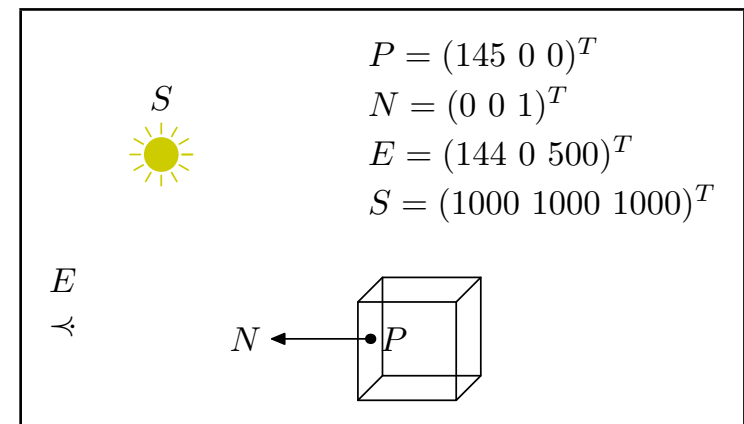
Antag: $k_a = k_d = k_s = 1; n = 4; f(d) = 1; I_a = 0.2; I_p = 0.4$.

$$L = \frac{S - P}{|S - P|} = \frac{(855 \ 1000 \ 1000)^T}{|\dots|} = \begin{pmatrix} 0.52 \\ 0.60 \\ 0.60 \end{pmatrix}$$

$$V = \frac{E - P}{|E - P|} = \frac{(-1 \ 0 \ 500)^T}{|\dots|} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

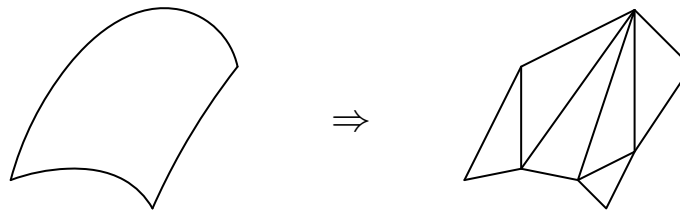
$$R = 2N(N \cdot L) - L = 2 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} 0.6 - \begin{pmatrix} 0.52 \\ 0.60 \\ 0.60 \end{pmatrix} = \begin{pmatrix} -0.52 \\ -0.60 \\ 0.60 \end{pmatrix}$$

$$I = 0.2 + 0.4(0.60 + 0.60^4) = 0.2 + 0.4 \cdot 0.73 = 0.492$$



Approximation av buktiga ytor

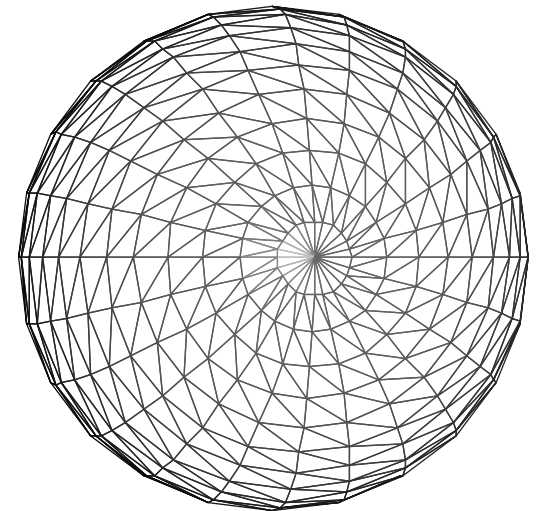
Buktiga ytor kan tesseleras till polygoner:



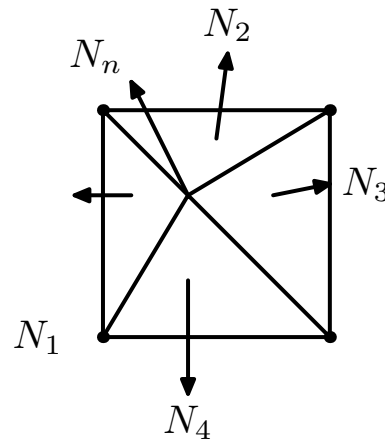
Trianglar garanterar plana polygonytor.

Problem: Kantig silhuett.

Lösning: Finare uppdelning.

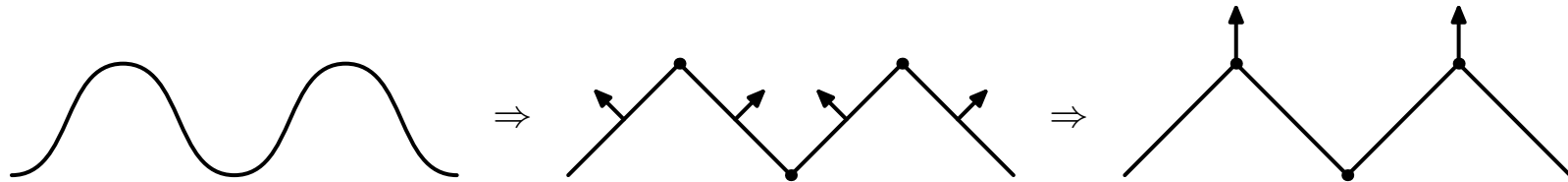


Beräkning av normaler för buktiga ytor



$$N_n = \frac{\sum_{i=1}^n N_i}{\left| \sum_{i=1}^n N_i \right|}$$

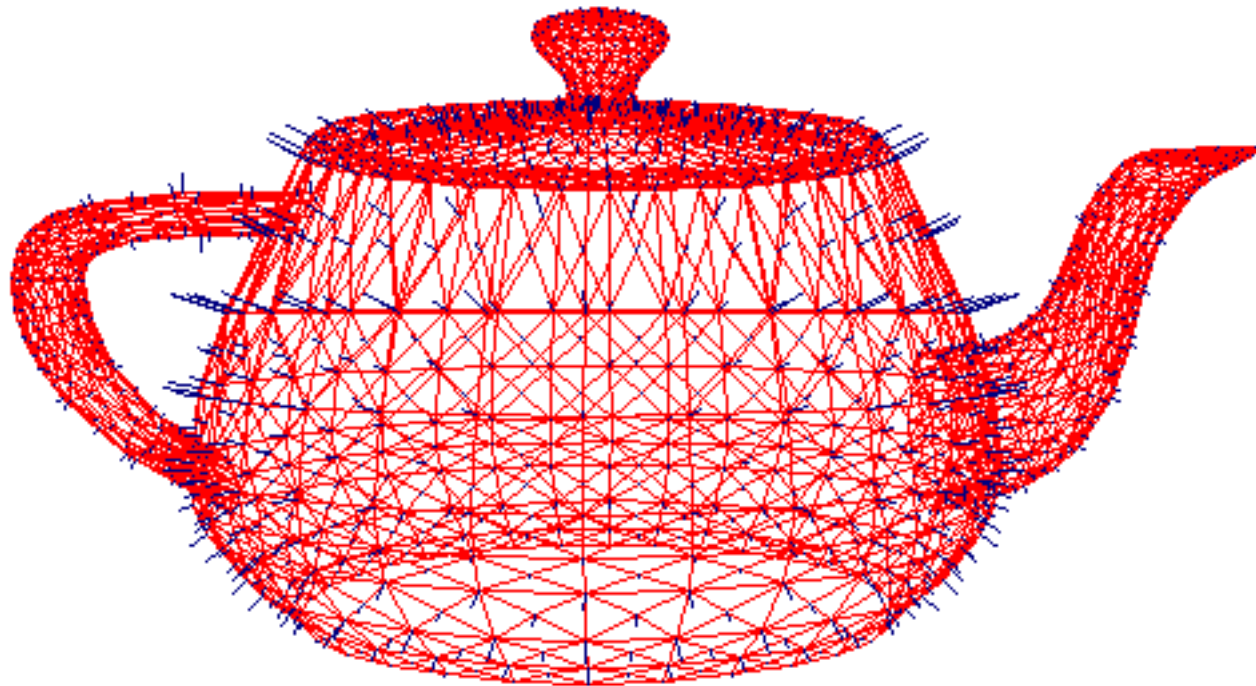
Problem: Icke-representativa normaler.



Ett samplingsproblem. Lösning: Finare uppdelning.

Observera att denna beräkning endast görs för buktiga ytor, ej för "flata" objekt, t ex rätblock.

Exempel på beräknade normaler för en buktig yta



Implementation

Illumination = belysningsberäkningarna.

Shading = svepkonvertering med intensitetsvariation.

Tre varianter:

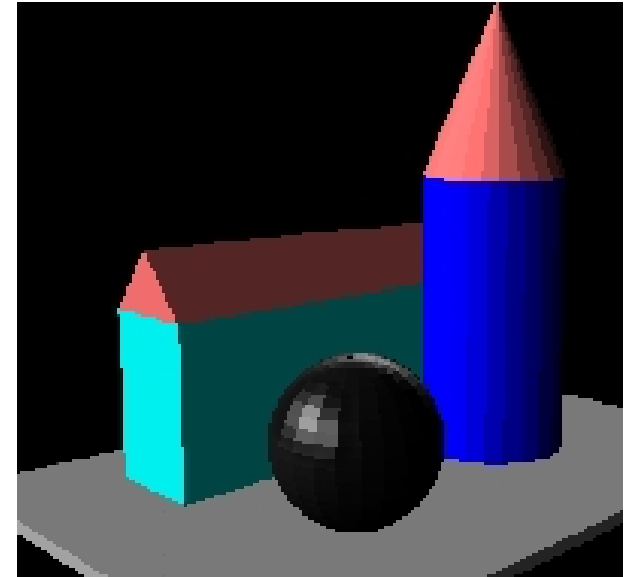
- **Flat shading:** Varje polygon har konstant intensitet.
- **Gouraud shading:** Räkna ut I i varje hörnpunkt, interpolera intensiteten över polygonen.
- **Phong shading:** Interpolera normalen och beräkna intensiteten i varje punkt.

Teorin visas för trianglar; kan generaliseras till godtyckliga polygoner.

Flat shading

- Välj en godtycklig punkt P i triangeln.
- Beräkna I för P enligt tidigare.

Oftast struntar man i speglande reflektion och använder bara Lamberts cosinuslag.



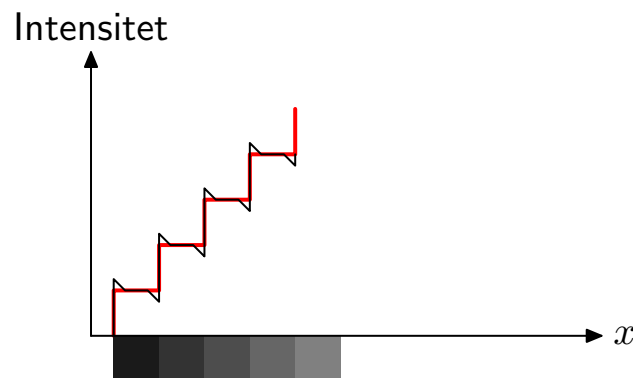
- Svepkonvertera hela triangeln med intensitet I .
- Ger upphov till så kallade *Mach-band* på buktiga ytor:

Mach-band

Ögat är en *kantdetektor* och är känsligt för

- Variationer i magnituden på intensiteten.
- Variationer i derivatan på intensiteten.

En fotoreceptor som "överstimuleras" blockerar sina grannar.



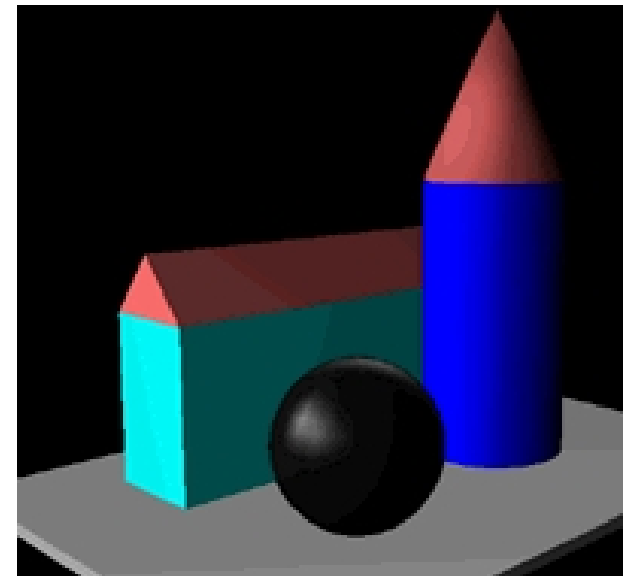
Röda linjen = verklig intensitet.
Svarta linjen = upplevd intensitet.

Gouraud shading

- Beräkna I_0 i P_0 , I_1 i P_1 , I_2 i P_2 .

Vanligtvis försummas speglande termen (dock inte i OpenGL).

- Interpolera I under svepkonverteringen, p s s som vi gör med z .



Egenskaper hos Gouraud-shading

- Elimineras mesta Mach-band-effekterna, men inte alla. Svårt att få kontinuitet i derivatan.

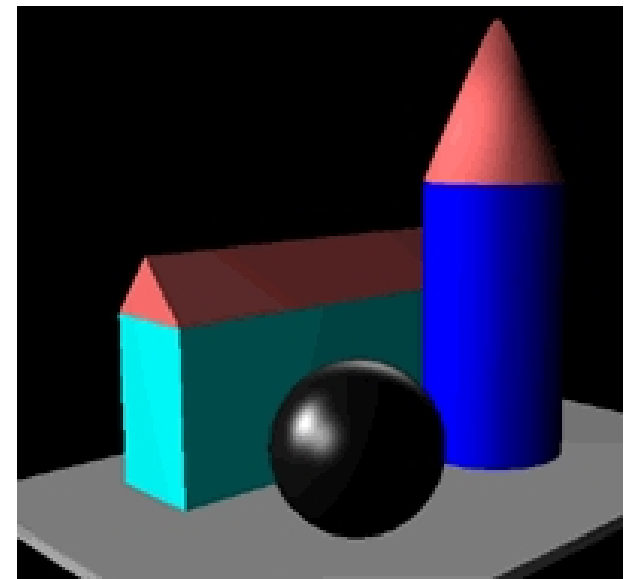
- Missar "blänket" om polygonerna är för stora.

I kan aldrig bli större än $\max(I_0, I_1, I_2)$.

Lösning: finare uppdelning (eller Phong shading).

Phong shading

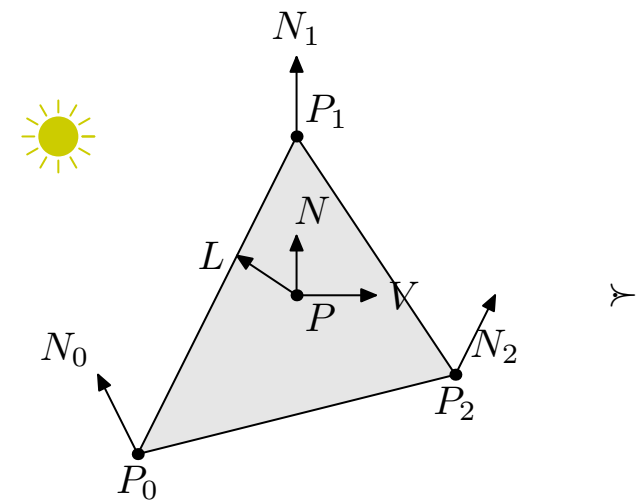
- Interpolera normalen under svepkonverteringen, p s s som för z , fast nu med en vektor!
- Beräkna I , nu per pixel.



Phong shading (forts)

För att räkna ut L och V behövs P ,
den världskoordinat som projiceras på pixeln.

1. Invertera projektionsmatrisen och mappa
tillbaks skärmkoordinaten, eller. . .
2. Interpolera P från P_0, P_1, P_2 .



Metod (2) enklast eftersom ni kan interpolera P på samma sätt som ni interpolerar normalen.

Pseudokod för en Phong-shader

S : lampans position, E : observatörens position.

- För varje vy:
 1. Beräkna transformationsmatrisen M .
 2. Rensa bildminnet och Z-bufferten.
- För varje triangel:
 - ▷ För varje hörnpunkt p_w i triangeln:
 1. Transformera från världs- till skärmkoordinater: $p_s = Mp_w$.
 2. Klipp efter $0 \leq x_s, y_s, z_s \leq SIZE$;
Om utanför: fortsätt med nästa triangel.
 3. Homogenisera p_s : dividera med w_s .
 - ▷ Eliminera baksidor.

- För varje pixel i triangeln (svepkonverteraren):

1. Interpolera N från hörnpunkternas normaler och konvertera till enhetsvektor.
2. Interpolera P på samma sätt från p_w -koordinaterna i hörnpunkterna.
3. Om Z-testet uppfylls:
 - (a) $I_d := 0; I_s := 0;$
 - (b) Beräkna enhetsvektor mot ljuskällan, $L := (S - P)/|S - P|$
 - (c) Om $L \cdot N > 0$,
 - i. $I_d := k_d(L \cdot N).$
 - ii. Beräkna reflektionsvektorn, $R := 2N(L \cdot N) - L$
 - iii. Beräkna enhetsvektor mot observatören, $V := (E - P)/|E - P|$
 - iv. Om $R \cdot V > 0$, $I_s := k_s(R \cdot V)^n.$
 - (d) $I := I_a k_a + f(d)I_p[I_d + I_s].$

OBS: Beräkningar av I_d, R, V, I_s förhållas i det längsta.

Phong shading, egenskaper

- Kostsamt att beräkna I per pixel.

För parallellt ljus och parallell observatör:

- ▷ Parallellt ljus: L konstant.
- ▷ Parallell observatör: V konstant.
- ▷ Behöver inte interpolera P .
- ▷ L och V behöver inte räknas ut för varje pixel.

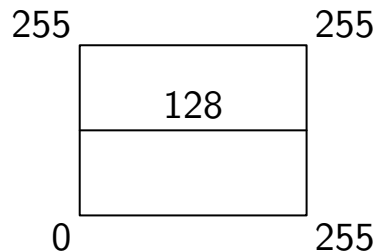
Dessutom:

- ▷ $(R \cdot V)^n$ kan approximeras med $(N \cdot H)^n$.
- ▷ H är den så kallade *halfway vector*, ett medelvärde av L och V .
- ▷ t^n kan approximeras med $t/(n - nt + t)$.

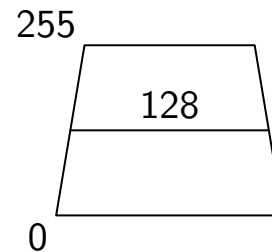
- Elimineras så gott som alla Mach-band.

Phong shading, problem

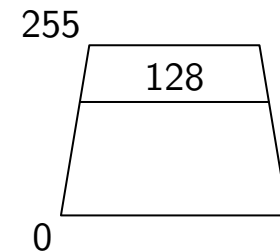
- **Perspektivdistortion:**



Sett rakt framifrån



Gouraud eller Phong



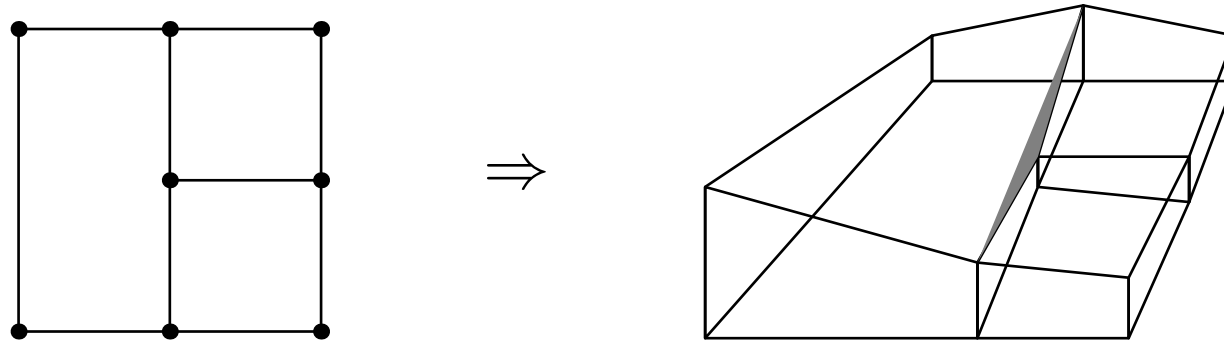
Korrekt

I praktiken svårt att upptäcka (dröjde många år).

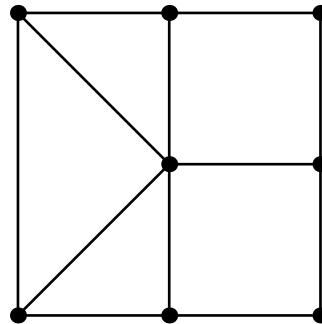
Lösning: Finare uppdelning eller interpolera världskoordinater i perspektiv.

Samma problem dyker upp i texturmappning, men där syns det mycket tydligare.

- **T-korsningar:** Sprickor uppstår ibland då noder delas.



Lösning: Undvik T-korsningar:



Belysning i OpenGL

- OpenGL stöder lokal belysning, enligt Phong.
- Programmeraren måste definiera normalerna.
- Möjlighet att definiera upp till 8 ljuskällor.
- Varje ljuskälla har följande inställningar:
 - ▷ Punktformad eller parallell.
 - ▷ Position/riktning
 - ▷ Färg: bakgrund, diffus och speglade.
 - ▷ Dämpningsfaktorer.
 - ▷ Spotlight-egenskaper.

- Material kan också definieras, per primitiv:
 - ▷ Reflektionskoefficienter: k_a, k_d, k_s .
 - ▷ Reflektionsexponent n .
 - ▷ Självlysande färg.

Definiering av normaler

- Definieras i samband med modellen:

```
glBegin(GL_POLYGON);
    glNormal3d( $N_x[0]$ ,  $N_y[0]$ ,  $N_z[0]$ );
    glVertex3d( $V_x[0]$ ,  $V_y[0]$ ,  $V_z[0]$ );
    glNormal3d( $N_x[1]$ ,  $N_y[1]$ ,  $N_z[1]$ );
    glVertex3d( $V_x[1]$ ,  $V_y[1]$ ,  $V_z[1]$ );
    :
glEnd();
```

- `glNormal` definierar normalen för nästa `glVertex`.
- Normalerna $N[0 \dots]$ får man räkna ut själv, helst enhetsvektorer.
- GLU-funktionerna `gluSphere()` etc, definierar egna normaler.

Definiering av en ljuskälla

- `glEnable(GL_LIGHTING)`; slår på belysning.
- `glShadeModel(GL_FLAT)`; slår på flat-shading.
- `glShadeModel(GL_SMOOTH)`; slår på interpolering.
- Endast intensiteten interpoleras, inte normalerna.
- `glEnable(GL_LIGHTi)`; slår på ljuskälla *i*.
- `glDisable(GL_LIGHTi)`; slår av ljuskälla *i*.
- Det finns `GL_LIGHT0 . . . GL_LIGHT7`.

Definiering av ljuskälla (forts)

- Inställning av ljuskällans position:

```
GLfloat light_pos[] = { x,y,z,w };  
glLightfv(GL_LIGHTi, GL_POSITION, light_pos);
```

- $w = 0 \Rightarrow$ parallell ljuskälla med riktning (x, y, z) .
- $w = 1 \Rightarrow$ punktformad ljuskälla i (x, y, z) .

Ljuskällans färg

- Ljuskälla i :s bakgrundsbidrag, $I_{a\lambda}$:

```
GLfloat light_ambient[] = {  $I_{aR}$ ,  $I_{aG}$ ,  $I_{aB}$ ,  $I_{aA}$  };  
glLightfv(GL_LIGHTi, GL_AMBIENT, light_ambient);
```

- $I_{aR} = I_{aG} = I_{aB} = 0$ ger inget bakgrundsbidrag.
- $I_{aR} = 1, I_{aG} = I_{aB} = 0$ ger endast rött bakgrundsbidrag.
- I_{aA} endast intressant för blending-operationer.

Ljuskällans färg (forts)

- Ljuskälla i :s diffusa ljusbidrag, $I_{d\lambda}$:

```
GLfloat light_diffuse[] = {  $I_{dR}$ ,  $I_{dG}$ ,  $I_{dB}$ ,  $I_{dA}$  };  
glLightfv(GL_LIGHTi, GL_DIFFUSE, light_diffuse);
```

- Ljuskällas i :s speglade ljusbidrag, $I_{s\lambda}$:

```
GLfloat light_specular[] = {  $I_{sR}$ ,  $I_{sG}$ ,  $I_{sB}$ ,  $I_{sA}$  };  
glLightfv(GL_LIGHTi, GL_SPECULAR, light_specular);
```

- (Vi antog att $I_{d\lambda} = I_{s\lambda}$ och kallade det $I_{p\lambda}$.)

Allmänt bakgrundsljus

Allmänt bakgrundsljus, oberoende av några ljuskällor:

```
GLfloat global_amb[] = { R, G, B, A };  
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_amb);
```


Parallell observatör

Parallell observatör betyder att vektorn V är konstant per pixel.

Detta är default i OpenGL men kan slås av med:

```
glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE);
```

Definiering av materialegenskaper

- Materialets reflektion av bakgrundslys, $k_a O_{d\lambda}$.

```
GLfloat ambient[] = {  $k_a O_{dR}$ ,  $k_a O_{dG}$ ,  $k_a O_{dB}$ ,  $k_a O_{dA}$  };  
glMaterialfv(GL_FRONT, GL_AMBIENT, ambient);
```

- Materialets reflektion av diffust ljus, $k_d O_{d\lambda}$:

```
GLfloat diffuse[] = {  $k_d O_{dR}$ ,  $k_d O_{dG}$ ,  $k_d O_{dB}$ ,  $k_d O_{dA}$  };  
glMaterialfv(GL_FRONT, GL_DIFFUSE, diffuse);
```

Definiering av materialegenskaper (forts)

- Materialets reflektion av speglande ljus, $k_s O_{s\lambda}$:

```
GLfloat specular[] = {  $k_s O_{sR}$ ,  $k_s O_{sG}$ ,  $k_s O_{sB}$ ,  $k_s O_{sA}$  };  
glMaterialfv(GL_FRONT, GL_SPECULAR, specular);
```

- OpenGL använder här istället bokens ekv. (14.15).
- Materialets reflektionsexponent, $n \in [0, 128]$:

```
GLfloat shine[] = {  $n$  };  
glMaterialfv(GL_FRONT, GL_SHININESS, shine);
```

Default-inställningar för ljuskälla

```
GLfloat light_position[] = { 0.0, 0.0, 1.0, 0.0 };  
GLfloat light_ambient[]  = { 0.0, 0.0, 0.0, 1.0 };  
GLfloat light_diffuse[]   = { 1.0, 1.0, 1.0, 1.0 };  
GLfloat light_specular[]  = { 1.0, 1.0, 1.0, 1.0 };  
glLightfv(GL_LIGHT0, GL_POSITION, light_position);  
glLightfv(GL_LIGHT0, GL_AMBIENT,  light_ambient);  
glLightfv(GL_LIGHT0, GL_DIFFUSE,  light_diffuse);  
glLightfv(GL_LIGHT0, GL_SPECULAR, light_specular);
```

Default-inställningar för material

```
GLfloat ambient[] = { 0.2, 0.2, 0.2, 1.0 };  
GLfloat diffuse[] = { 0.8, 0.8, 0.8, 1.0 };  
GLfloat specular[] = { 0.0, 0.0, 0.0, 1.0 };  
GLfloat shine[] = { 0.0 };  
glMaterialfv(GL_FRONT, GL_AMBIENT, ambient);  
glMaterialfv(GL_FRONT, GL_DIFFUSE, diffuse);  
glMaterialfv(GL_FRONT, GL_SPECULAR, specular);  
glMaterialfv(GL_FRONT, GL_SHININESS, shine);
```

Utplacering av ljuskällor

- Placering/riktning av ljuskällor behandlas som övriga primitiver.
- Dvs, modelview-matrisen påverkar ljuskällans position/riktning.
- Det går med enkla medel att åstadkomma:
 1. En stationär ljuskälla.
 2. En ljuskälla som följer med kameran.
 3. En ljuskälla som roterar runt ett objekt.

Stationär ljuskälla

Initiera modelview-matrisen, placera sen ut ljuskällan.

```
void display(void) {  
    :  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity();  
    :  
    /* Anrop till glTranslate/glRotate/gluLookAt, etc */  
    :  
    glLightfv(GL_LIGHT0, GL_POSITION, light_position);  
}
```

Roterande ljuskälla

Se `movelight.c` i OpenGL-guiden/Mesa-distributionen.

Kamerafixerad ljuskälla

Placera ut ljuskällan, initiera sen modelview-matrisen.

```
void display(void) {  
    :  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity();  
    glLightfv(GL_LIGHT0, GL_POSITION, light_position);  
    :  
    /* Anrop till glTranslate/glRotate/gluLookAt, etc */  
}
```


OpenGL-demonstrationer på hemsidan

- `fixed` – visar en stationär ljuskälla.
- `camera` – visar en kamerafixerad ljuskälla.
- `spin` – visar en roterande ljuskälla.

Lab 4 i OpenGL

Nedanstående program är hårdkodat för tekannan och laddar alltså inte in några PLG-filer.



Huvudprogrammet

```
#include <stdio.h>
#include <GL/glut.h>

char *programe;

/* position and orientation of the viewer */
typedef struct { GLdouble x, y, z; } Vector;
Vector viewer_pos = { 0, 0, 500.0 };
Vector viewer_orientation;

/* screen parameters */
#define SIZE 700
double fov = 50.0; /* field of vision */
double near = 5.0; /* near clip plane */
double far = 10000.0; /* far clip plane */

int main(int argc, char *argv[])
```

```
{  
    progname = argv[0];  
    glutInit(&argc, argv);  
    myinit();  
    glutMainLoop();  
}
```

Initieringsrutiner

```
void myinit(void) {  
    GLfloat global_ambient[] = { 0.0, 0.0, 0.0, 1.0 };  
  
    glutInitDisplayMode(GLUT_DEPTH | GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowSize(SIZE, SIZE);  
    glutCreateWindow(progname);  
    glEnable(GL_CULL_FACE);  
    glCullFace(GL_BACK);  
    glEnable(GL_DEPTH_TEST);  
  
    /* create a single light source */  
    glEnable(GL_LIGHTING);  
    glEnable(GL_LIGHT0);  
  
    /* turn off parallell viewing */  
    glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE);  
}
```

```
/* set global ambient lighting to 0.0 */  
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient);  
  
glutDisplayFunc(display);  
glutKeyboardFunc(keys);  
glClearColor(0.0, 0.0, 0.0, 0.0);  
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
gluPerspective(fov, 1.0, near, far);  
glMatrixMode(GL_MODELVIEW);  
}
```

När skärmen behöver ritas om

```
void display(void) {  
    GLfloat light_position[] = { 0.0, 0.0, 1000.0, 1.0 };  
    GLfloat light_ambient[] = { 0.2, 0.2, 0.2, 1.0 };  
    GLfloat light_diffuse[] = { 0.4, 0.4, 0.4, 1.0 };  
    GLfloat light_specular[] = { 0.4, 0.4, 0.4, 1.0 };  
  
    GLfloat ambient[] = { 1.0, 1.0, 1.0, 1.0 };  
    GLfloat diffuse[] = { 1.0, 1.0, 1.0, 1.0 };  
    GLfloat specular[] = { 1.0, 1.0, 1.0, 1.0 };  
    GLfloat shine[] = { 100.0 };  
  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
  
    glLoadIdentity();  
    glRotated(-viewer_orientation.z, 0.0, 0.0, 1.0);  
    glRotated(-viewer_orientation.y, 0.0, 1.0, 0.0);  
    glRotated(-viewer_orientation.x, 1.0, 0.0, 0.0);  
}
```

```
glTranslated(-viewer_pos.x, -viewer_pos.y, -viewer_pos.z);

glLightfv(GL_LIGHT0, GL_POSITION, light_position);
glLightfv(GL_LIGHT0, GL_AMBIENT, light_ambient);
glLightfv(GL_LIGHT0, GL_DIFFUSE, light_diffuse);
glLightfv(GL_LIGHT0, GL_SPECULAR, light_specular);

glMaterialfv(GL_FRONT, GL_AMBIENT, ambient);
glMaterialfv(GL_FRONT, GL_DIFFUSE, diffuse);
glMaterialfv(GL_FRONT, GL_SPECULAR, specular);
glMaterialfv(GL_FRONT, GL_SHININESS, shine);

glFrontFace(GL_CW);          /* teapot defined using CW-rule */
glutSolidTeapot(100.0);
glFlush();
}
```


Hantering av kommandon

```
void keys(unsigned char key, int x, int y) {  
    switch(key) {  
        case 'x': viewer_pos.x -= 10.0; break;  
        case 'X': viewer_pos.x += 10.0; break;  
        case 'y': viewer_pos.y -= 10.0; break;  
        case 'Y': viewer_pos.y += 10.0; break;  
        case 'z': viewer_pos.z -= 10.0; break;  
        case 'Z': viewer_pos.z += 10.0; break;  
        case 'k':  
            if ((viewer_orientation.x += 2.0) > 360.0)  
                viewer_orientation.x -= 360.0;  
            break;  
        case 'i':  
            if ((viewer_orientation.x -= 2.0) < 0.0)  
                viewer_orientation.x += 360.0;  
            break;  
        case 'l':
```

```
    if ((viewer_orientation.y += 2.0) > 360.0)
        viewer_orientation.y -= 360.0;
    break;
case 'j':
    if ((viewer_orientation.y -= 2.0) < 0.0)
        viewer_orientation.y += 360.0;
    break;
case 'q':
case 27:
    exit(0);
default:
}
glutPostRedisplay();
}
```

Föreläsning 14B: Texturmappling

2D texturmappling

- Definitioner

- Texturbeskrivningar

- Från texels till pixels?

- Från pixels till texels

- Linjärinterpolering ger perspektivdistortion

- Designproblem

- Algoritm för texturmappling

- Hur ska texturinformationen tolkas?

3D texturmappling

- Optimeringar för spelprogram

- Texturmappling i OpenGL

- Filtrering

- Lab 5 i OpenGL



2D texturmappling

”Klistra” en 2D-bild på en eller flera polygonytor.

Motivation:

- **Öka realismen.**

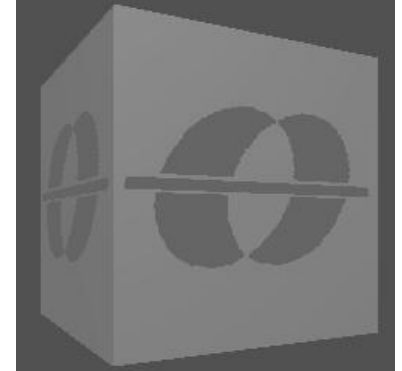
Med enbart belysning ser objekten ”plastiga” ut.

Vi vill kunna ge intryck av trä, marmor, etc.

- **Förenkla modelleringen.**

Modellera en komplicerad husvägg: svårt.

Klistra en inscannad bild på plan yta: enkelt.



Definitioner

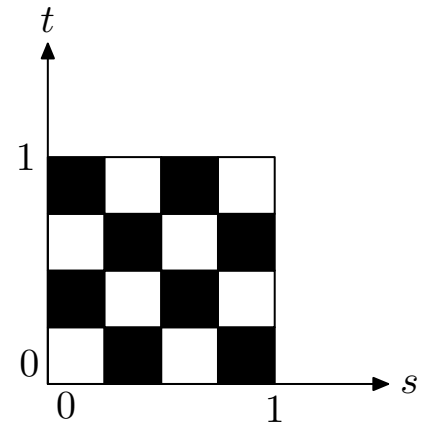
- En *textur* är en 2D-array av (intensitets-)värden.
- Ett element kallas *texel* (jfr. pixel).

Men: pixels är diskreta, texels kontinuerliga.

- Använd (s, t) för texturkoordinater.

Default: $s, t \in [0, 1]$.

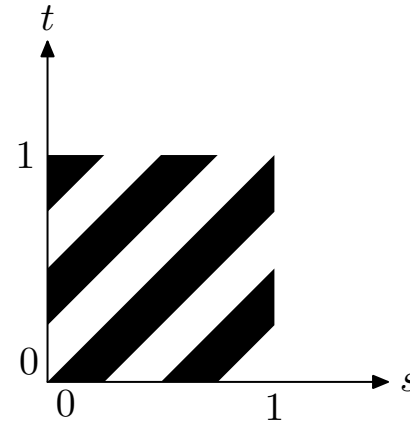
- Viss litteratur använder (u, v) .



Texturbeskrivningar

Procedurellt:

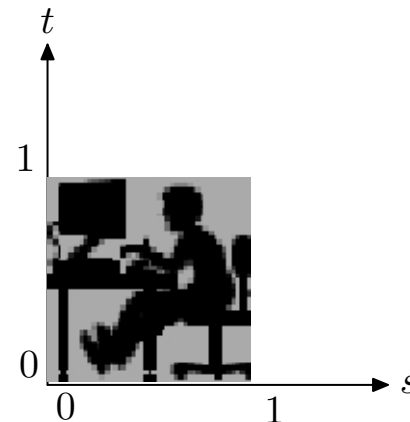
$$\text{texel}(s, t) = \begin{cases} 0 & \text{om } (4s + 4t) \bmod 2 < 1 \\ 1 & \text{annars} \end{cases}$$



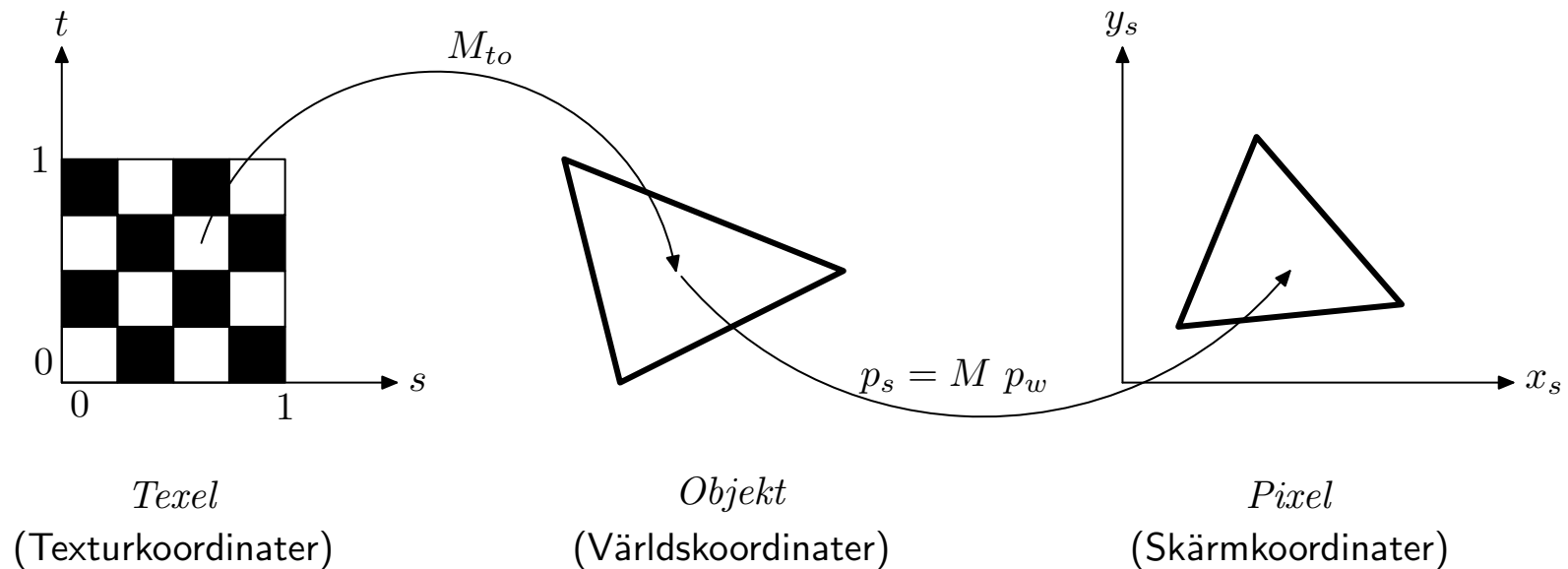
Bitmaps:

Antag en bitmap $bm(0 \dots 63, 0 \dots 63)$

$$\text{texel}(s, t) = bm(\lfloor 64 \cdot s \rfloor, \lfloor 64 \cdot t \rfloor)$$



Från texels till pixels?



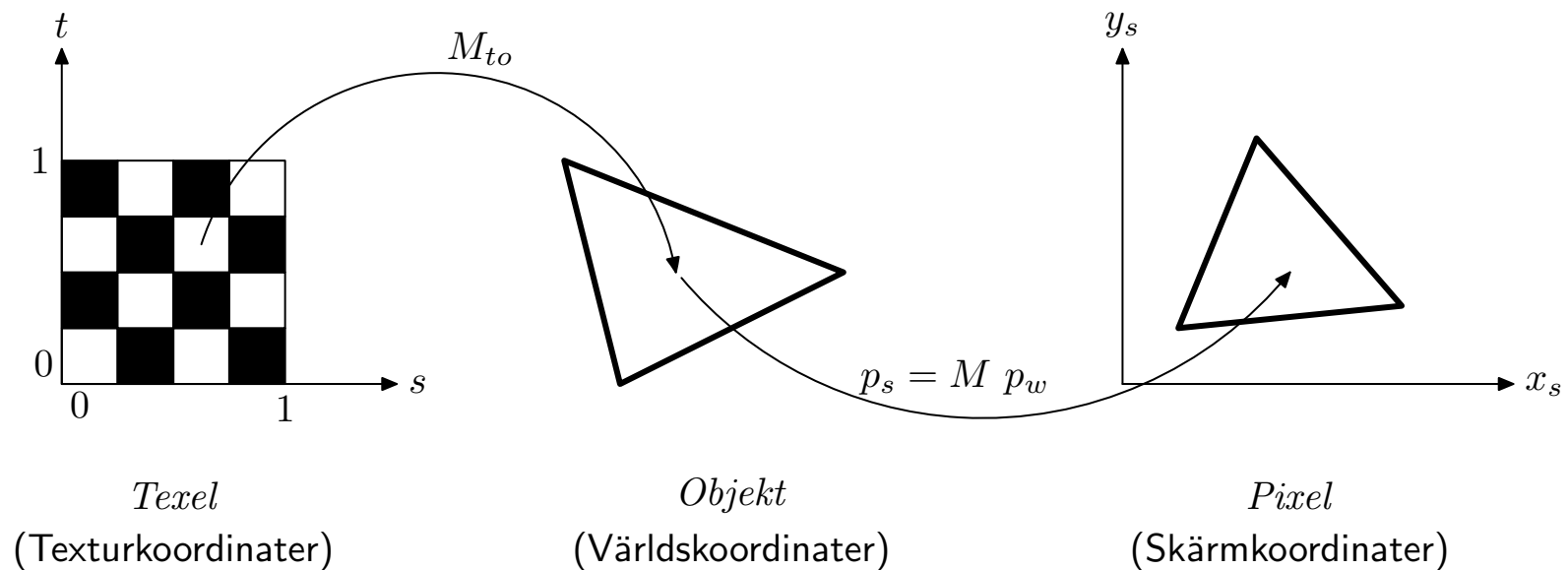
for t

for s

beräkna $x_s(s, t)$ och $y_s(s, t)$;

$screen(x_s, y_s) := texel(s, t)$;

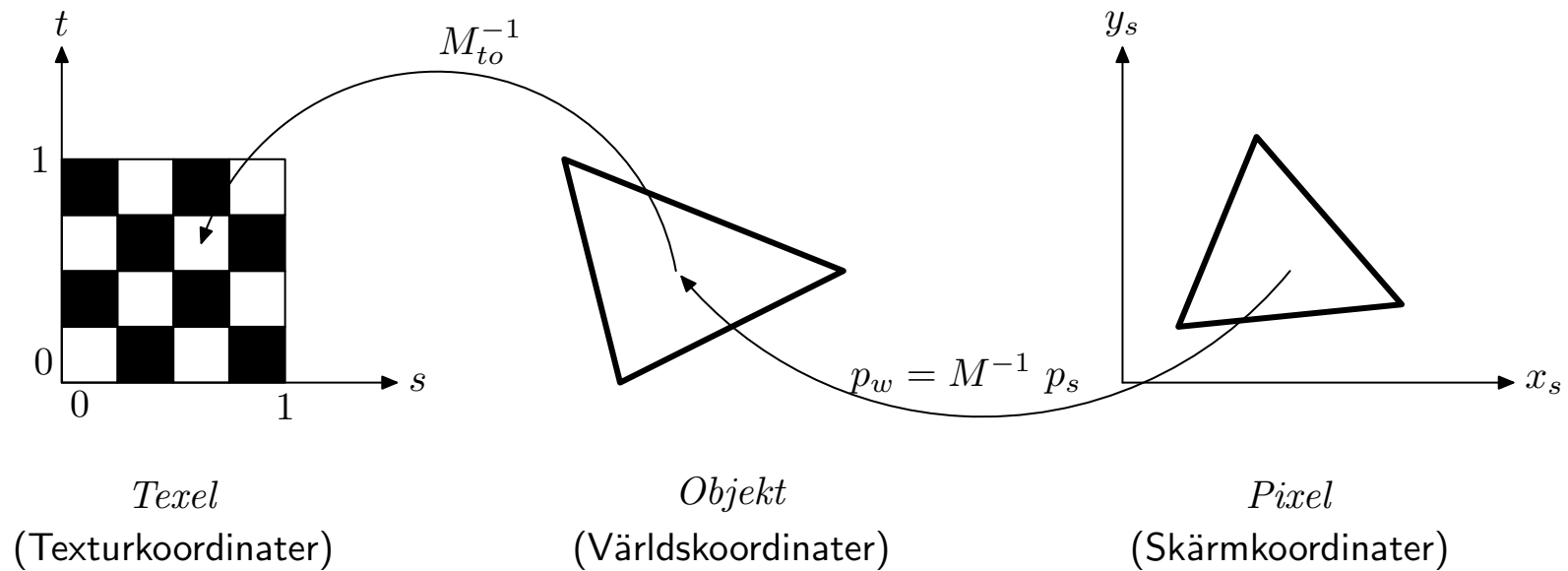
Från texels till pixels (forts)?



Problem:

- Flera texels kan avbildas på en pixel.
- Noll texels kan avbildas på en pixel $\Rightarrow$ hål!
- Bättre att gå åt andra hållet.

Från pixels till texels



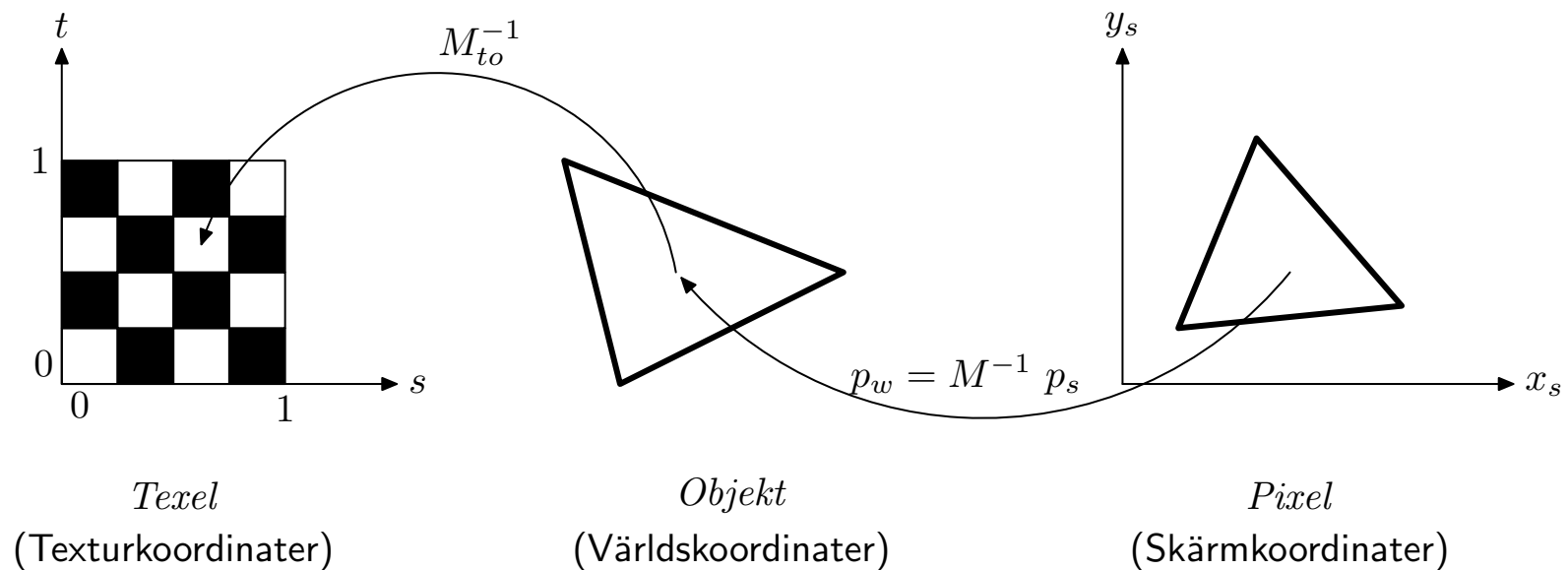
for y_s

for x_s

Beräkna $s(x_s, y_s)$ och $t(x_s, y_s)$;

$screen(x_s, y_s) := texel(s, t)$;

Från pixels till texels (forts)



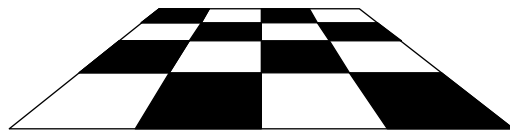
Egenskaper:

- Om en pixel avbildas på mer än en texel: supersampla.
- Det vanligaste sättet att ordna texturmappling.

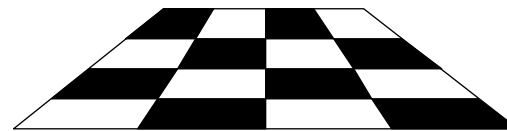
Linjärinterpolering ger perspektivdistortion

Många tror att man kan linjärinterpolera (s, t) -värden över triangelytan, ungefär som vi gör med z och I .

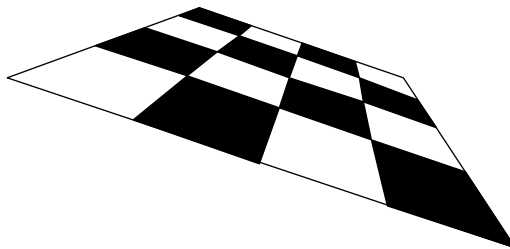
Detta tar dock inte hänsyn till *perspektiveffekten*.



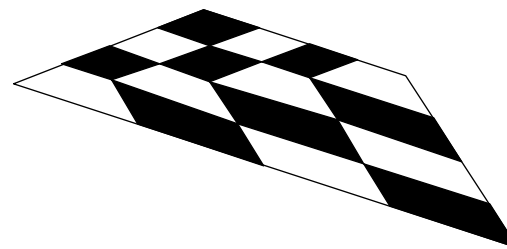
Korrekt perspektiv



Felaktigt perspektiv



Korrekt perspektiv



Felaktigt perspektiv

Designproblem

- Tilldelning av (s, t) -värden till objektets noder.

Problem att få kontinuitet i sömmarna.

- Vad händer om $s, t > 1$?

Repetition eller "clamping" ?

- Hur beräknas M^{-1} och M_{to}^{-1} ?

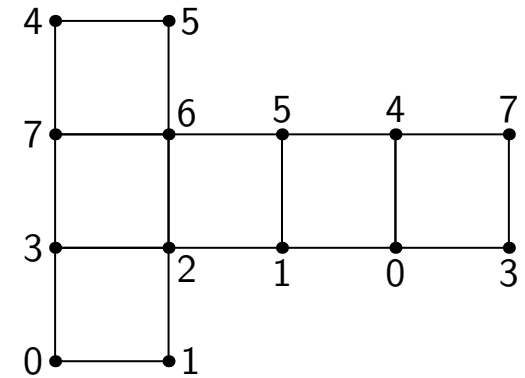
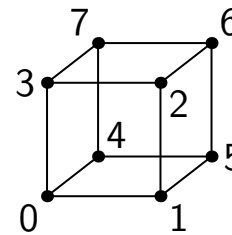
Detta kan undvikas.

- Hur ska texturinformationen tolkas?

Flera möjligheter!

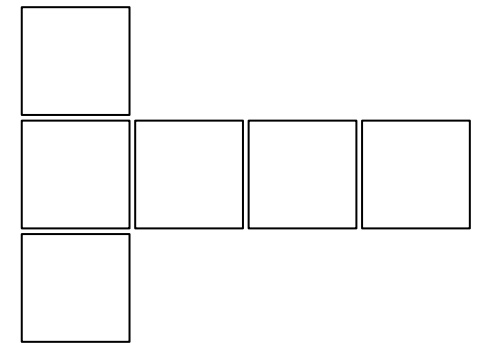
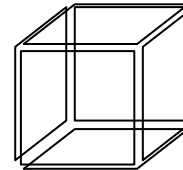
Tilldelning av texturkoordinater till noder

- **För flata objekt:**



Problem: Varje nod "delas" av flera polygonytor.

Lösning: Definiera objekten så att varje nod bara används en gång.

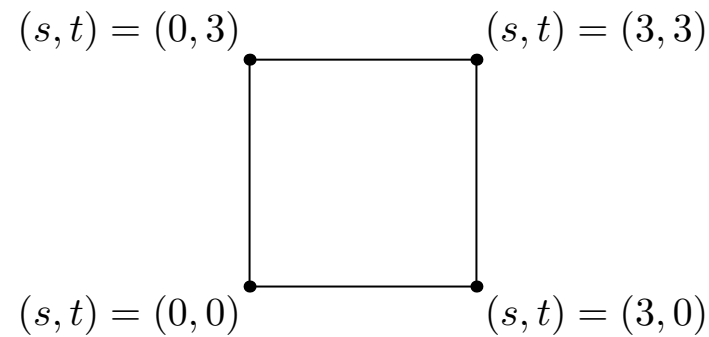


- **För buktiga ytor:**

Dekal på buktig yta $\Rightarrow$ veck.

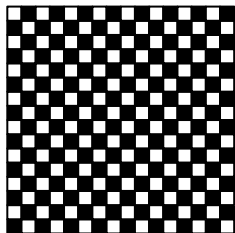
Avbildningar kan göras för cylindrar, sfärer . . .

Vad händer om $s, t > 1$?



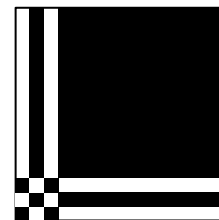
Två alternativ:

Repetition:



Indexera med
 $\text{texel}(s \bmod 1, t \bmod 1)$

Clamping:



Indexera med
 $\text{texel}(\min(s, 1), \min(t, 1))$

Algoritm för texturmappling

”För varje pixel i triangeln, räkna ut motsvarande texel.”

1. För varje texturmappad triangel:

- (a) Beräkna M^{-1} och M_{to}^{-1}
- (b) För varje pixel $p_s = (x_s \ y_s \ z_s \ 1)$ i triangeln:
 - i. $p_w = M^{-1} p_s$
 - ii. $p_t = (s \ t \ q) = M_{to}^{-1} p_w$
 - iii. Referera $texel(s, t)$

Kan göras mer inkrementellt, men kräver ändå 170 aritmetiska operationer i (a) och 5 aritmetiska operationer per pixel.

Förenkling av algoritmen

Några iakttagelser från lab 2 förenklar algoritmen:

- Nyckeln till perspektiveffekt är att dividera x och y i p_w med dess "djup". Detta görs av P .
- Efter $p_s = M p_w$ innehåller w -komponenten i p_s djupet. Divisionen sker vid homogeniseringen.
- Om man använder homogena texturkoordinater $(s \ t \ q)$ borde det finnas ett enkelt förhållande mellan w och q . Det gör det också, nämligen:

$$q = 1/w \quad (!)$$

- Härledningen presenteras i "Interpolation for Polygon Texture Mapping and Shading" av Heckbert och Moreton.

Förenklad algoritm för texturmapning (lab 5)

Varje hörnpunkt har tilldelats (s, t) -koordinater (vilket ersätter behovet av M_{to}) och kan dessutom lagra en (s', t', q') -trippel.

För varje triangel:

- För varje hörnpunkt p_w i triangeln:
 1. Värld till skärm: $p_s = (x_s \ y_s \ z_s \ w_s) := M \ p_w$.
 2. Klipp efter $0 \leq x_s, y_s, z_s \leq SIZE$.
 3. Homogenisera:

$$x_s := x_s/w_s; \quad y_s := y_s/w_s; \quad z_s := z_s/w_s$$

Steg 1–3 känner ni redan till; de upprepas bara för fullständighetens skull.

4. Dividera också hörnpunktens texturkoordinater med w_s :

$$s' := s/w_s; \quad t' := t/w_s$$

Denna divisionen gör att texturkoordinaterna utsätts för perspektiveffekt. Vi kan då interpolera dessa "i perspektiv".

5. Skapa en homogen texturkoordinat

$$q' := 1/w_s$$

- För varje pixel i triangeln (svepkonverteraren):

1. Interpolera (s'', t'', q'') från hörnpunkternas (s', t', q') :

$$(s'', t'', q'') := (s'', t'', q'') + (\Delta s', \Delta t', \Delta q')$$

2. Homogenisera:

$$s''' := s''/q''; \quad t''' := t''/q''$$

Detta "inverterar" perspektiveffekten och ger texturkoordinater.

3. Referera $\text{texel}(s''', t''')$ med repetition eller "clamping".

Hur ska texturinformationen tolkas?

- **Som intensitet:**

- ▷ **Dekalmode:** $I = \text{texel}(s, t)$.

Ignorerar diffus och speglande reflektion.

- ▷ **Modulerad:** $I = I_d \cdot \text{texel}(s, t) / 256 + \dots$

Modulerar texelns intensitet med diffusa komponenten.

Rekommenderas för lab 5.

- **Som transparentvärde:** Kräver α -kanal.

Kan användas för rök, moln, etc.

- **Som inkrement till ytnormalen:**

Bump-mapping. Ger intryck av ojämnheter i ytan.

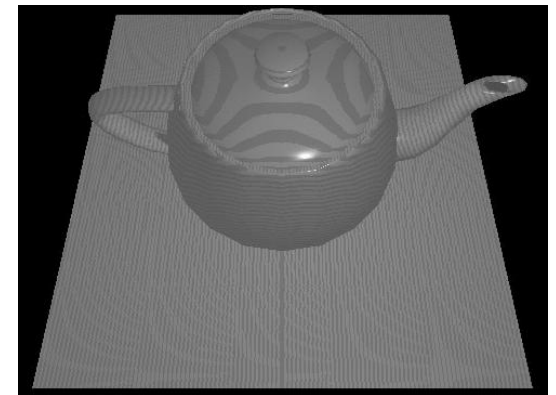
3D texturmappling

- Använd en 3D texturymd, $\text{texel}(x, y, z)$.
- Under svepkonverteringen: $\text{screen}(x_s, y_s) := \text{texel}(x_w, y_w, z_w)$
där (x_w, y_w, z_w) är världskoordinaterna som avbildades på (x_s, y_s) .
- Mycket enklare än 2D texturmappling, men . . .
- Kräver att man interpolerar världskoordinater under svepkonverteringen, helst med perspektiveffekt.
- En 3D textur måste beskrivas procedurellt.
- Ger intryck av att objektet täljts eller hackats fram ur ett block.

Procedurell beskrivning av trä

```
double wood(double xw, double yw, double zw)
{
    double radius, angle;
    int grain;

    radius = sqrt(xw*xw + zw*zw);
    angle = (zw == 0.0 ? M_PI/2.0 : atan2(xw,zw));
    radius += 2*sin(20*angle + yw/150.0);
    grain = (int) (radius+0.5) % 60;
    return grain < 40 ? 1.0 : 0.0;
}
```



Optimeringar för spelprogram

Olika knep för att optimera “the rendering pipeline”.

Indelat med avseende på de olika stegen:

1. Traversering av scenen
2. Geometrisk transformationer
3. Belysning
4. Svepkonvertering och texturmappling.

Största flaskhalsen är (4).

Traversering av scenen

- “Viewing frustum”:

Använd den stympade vykonen för att eliminera så många polygoner så tidigt som möjligt.

- “Portal culling”:

Utnyttja att dörrar och väggar skymmer de flesta polygoner för att minska djupkomplexiteten.

Implementation: i varje rum r lagras de rum som kan ses från r . Rum kan representeras med bitar $\Rightarrow$ listan blir ett bitmönster.

Användbart i labyrintspel typ Doom, men inte i t ex en flygsimulator.

Geometrisk transformationer

- Använd få polygoner:

Använd istället texturer för detaljrikedom.

Texturmappning med perspektivkorrektion kostar en division/pixel, men

- ▷ För små polygoner behövs det inte.
- ▷ Quake utför divisionen endast var 8:e pixel.

En *billboard* är en rektangel som alltid är vänd mot observatören. Ett foto påklistrat på en billboard kan användas för träd, lampor, etc.

Geometrisk transformationer (forts)

- Level of detail (LOD):

Använd en liten modell när objektet är långt borta. Byt till en större modell när observatören kommer nära.

Problem: “popping” (ögat reagerar på små förändringar).

Lösning: morphing eller blending.

Belysning

- Gouraud-shading i bästa fall.
- Quake använder s k “lighting maps”, en polygon med lägre upplösning (färre pixels att tända) och modulerar in intensitetsvärden i texturen.

Svepkonvertering och texturmappling

- Lägre upplösning:

Största flaskhalsen är att sätta pixels.

- Undvik att sätta pixels > 1 gång:

Inträffar ofta då Z-buffert används.

Använd “portal culling” för att minska djupet.

- Rita utan Z-buffert om du kan.

Quake använder BSP-träd för att ta reda på vilken polygon som är främst $\Rightarrow$ behöver bara sätta pixeln en gång.

(2D) Texturmappling i OpenGL

- En 2D-textur deklarerats t ex med

```
GLubyte mono_texels[512][512];    /* monokrom textur */  
GLubyte rgb_texels[128][64][3];    /* textur med färg */
```

- Bredd och höjd måste vara jämna tvåpotenser.
- OpenGL gör normalt perspektivkorrektion, men det kan begäras

```
glHint(GL_PERSPECTIVE_CORRECTION, GL_NICEST);
```

- Texturkoordinater anges med `glTexCoord2f(s, t)`;

```
glNormal3f(1, 0, 0);  
glTexCoord2f(1, 0);  
glVertex3f(100, 100, 100);
```

- Texturen laddas först med anropet

```
glTexImage2D(GL_TEXTURE_2D, level, components, mytex→width,  
             mytex→height, border, format, type, mytex→image);
```

- *level* = 0 om du inte använder mip-mapping.
- *components* = GL_RGB för våra ändamål.
⇒ alla färgerna (R,G,B) påverkas av texturen.
- *border* = 0 för våra ändamål.
- *format* = $\begin{cases} \text{GL_LUMINANCE} & \text{för 8-bitars texels.} \\ \text{GL_RGB} & \text{för 24-bitars texels.} \end{cases}$
- *type* = GL_UNSIGNED_BYTE.

- Hjälprutin i lab 5: `Texture *readppm(char *fname);`
Läser 24-bits PPM-fil, men sparar som 8-bits texels (gråskala).
- `glEnable(GL_TEXTURE_2D);` slår på texturer.
- `glDisable(GL_TEXTURE_2D);` slår av texturer.
- För att modulera ljuset med texturen:

```
glTexEnvf(GL_TEXTURE_ENV,  
          GL_TEXTURE_ENV_MODE,  
          GL_MODULATE);
```

- Använd `GL_DECAL` istället för dekal-mode.

- För repetition av texturer i både s – och t -led:

```
glTexParameteri(GL_TEXTURE_2D,  
                GL_TEXTURE_WRAP_S,  
                GL_REPEAT);  
glTexParameteri(GL_TEXTURE_2D,  
                GL_TEXTURE_WRAP_T,  
                GL_REPEAT);
```

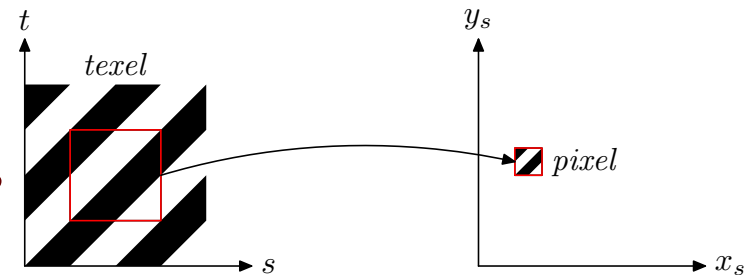
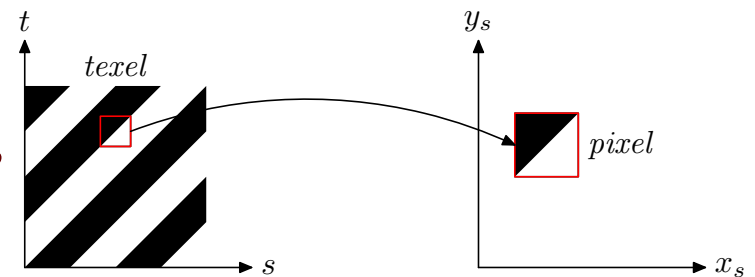
- Använd **GL_CLAMP** istället för clamping.

Filtrering

- För att slå på filtrering:

```
/* Magnification */
glTexParameterf(GL_TEXTURE_2D,
                GL_TEXTURE_MAG_FILTER,
                GL_LINEAR);
```

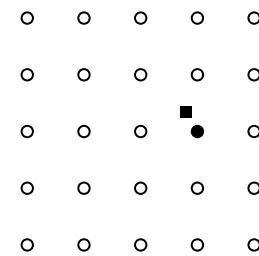
```
/* Minification */
glTexParameterf(GL_TEXTURE_2D,
                GL_TEXTURE_MIN_FILTER,
                GL_LINEAR);
```



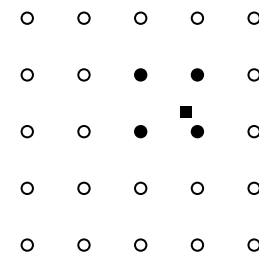
- Använd **GL_NEAREST** för att slå av filtrering.

Hur fungerar filtrering?

- Med **GL_NEAREST** använder OpenGL värdet på närmast liggande texel.



- Med **GL_LINEAR** använder OpenGL ett medelvärde på de fyra närmaste texlarna, s k *bilinjär filtrering*



- För minifiering finns också *mipmaps*.

Mipmaps

- *Multim in parvo* $\approx$ "många ting i litet utrymme".
- Givet ett original, skapa nedskalade kopior av successivt halva storleken, ända ner till 1 pixel.



Original



1/2



1/16



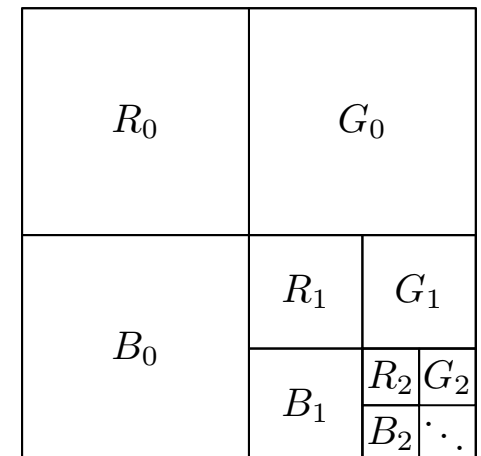
1/64

...



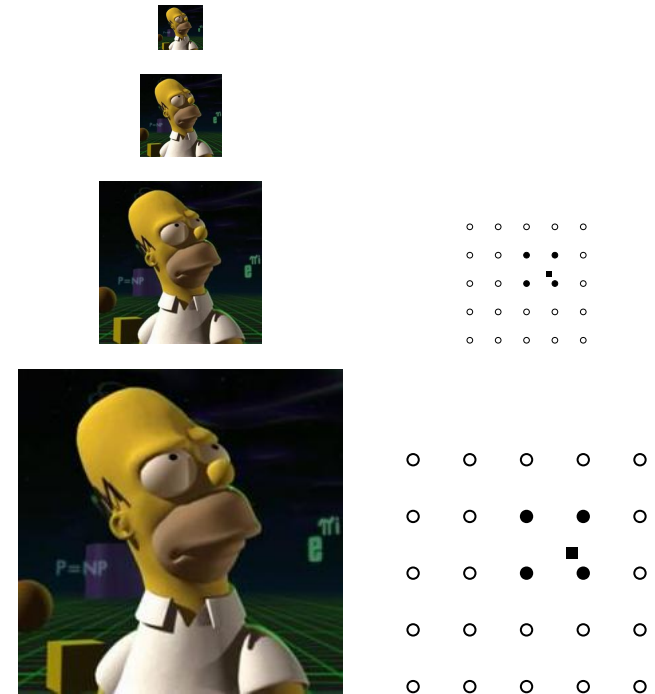
1 pixel

- Anropa `glTexImage2D()` n gånger med successivt mindre texturer och $level = 0, \dots, n - 1$.
- OpenGL renderar med den storlek som passar bäst.
- Reducerar "popping"-effekt när objekt ändrar storlek.
- `gluBuild2DMipmaps()` konstruerar förminskningar.
- Mipmaps kan lagras på ett effektivt sätt:
- Prova programmet `texture` på *Interactive OpenGL*.



Filtrering för mipmaps

- Speciellt för mipmaps: *trilinjär filtrering*.
- Gör bilinjär filtrering på de *två* storlekar som passar bäst.
- Aktiveras med flaggan `GL_LINEAR_MIPMAP_LINEAR`.
- Ger bäst resultat men kostar också mest.



Lab 5 i OpenGL, hårdkodat för tbox.plg

```
void myinit(void) {  
    GLfloat global_ambient[] = { 0.0, 0.0, 0.0, 1.0 };  
    glutInitDisplayMode(GLUT_DEPTH | GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowSize(SIZE, SIZE);  
    glutCreateWindow(progname);  
    glEnable(GL_CULL_FACE);  
    glCullFace(GL_BACK);  
    glEnable(GL_DEPTH_TEST);  
    glEnable(GL_LIGHTING);  
    glEnable(GL_LIGHT0);  
    glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_TRUE);  
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient);  
}
```

```
mytex = readppm("mdh.ppm");
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, mytex->width, mytex->height,
             0, GL_LUMINANCE, GL_UNSIGNED_BYTE, mytex->image);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
glEnable(GL_TEXTURE_2D);

glutDisplayFunc(display); glutKeyboardFunc(keys);
glClearColor(0.0, 0.0, 0.0, 0.0);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(fov, 1.0, near, far);
glMatrixMode(GL_MODELVIEW);
}
```

```
void display(void) {  
    :  
    /* hårdkodad variant av tbox.plg */  
    glBegin(GL_QUADS);  
  
    glNormal3f(0, 1, 0);  
    glTexCoord2f(1, 1); glVertex3f( 100, 100, -100);  
    glTexCoord2f(0, 1); glVertex3f(-100, 100, -100);  
    glTexCoord2f(0, 0); glVertex3f(-100, 100, 100);  
    glTexCoord2f(1, 0); glVertex3f( 100, 100, 100);  
  
    glNormal3f(0, 0, -1);  
    glTexCoord2f(1, 1); glVertex3f( 100, 100, -100);  
    glTexCoord2f(1, 0); glVertex3f( 100, -100, -100);  
    glTexCoord2f(0, 0); glVertex3f(-100, -100, -100);  
    glTexCoord2f(0, 1); glVertex3f(-100, 100, -100);  
  
    glNormal3f(1, 0, 0);  
    glTexCoord2f(1, 1); glVertex3f(100, 100, 100);
```



```
glTexCoord2f(1, 0); glVertex3f(100, -100, 100);  
glTexCoord2f(0, 0); glVertex3f(100, -100, -100);  
glTexCoord2f(0, 1); glVertex3f(100, 100, -100);  
  
:  
  
glEnd();  
glFlush();  
}
```

Föreläsning 14C: Globala belysningsmodeller

Globala belysningsmodeller

Strålföljning

Strålföljningens huvudidé

Pseudokod för strålföljning

Resultatet

Radiosity

Vad är radiosity?

Radiosity-ekvationen

Formfaktorer

Hemikuben

Radiosity-bilder

Progressiv radiosity-algoritm

Jämförelse: strålföljning vs. radiosity

Ett par minimala strålföljare

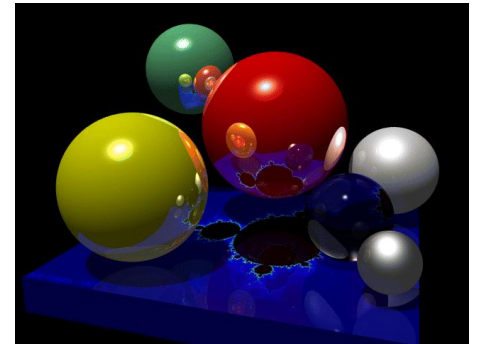
Globala belysningsmodeller

Tar även hänsyn till ljus som reflekterats via andra objekt.

Två vanligaste metoderna:

- **Strålföljning (1969):**

- ▷ Vyberoende.
- ▷ För speglande reflektion.



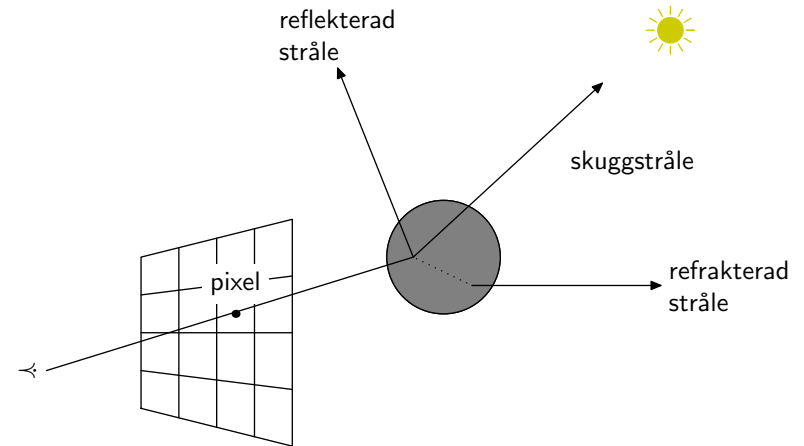
- **Radiosity (1984):**

- ▷ Vyberoende.
- ▷ För diffus reflektion.



Strålföljning

- Utökning av strålkastning (kap. 13).
- Vanligaste metoden för att skapa fotorealistiska bilder.
- Baserad på optik: Följ ljusstrålarna baklänges från ögat.
- Enkel algoritm, stor tidskomplexitet.
- Hanterar eliminering av skymda ytor, skuggor, genomskinliga ytor (refraktion/transmission), speglande reflektion och texturmappning i ett slag.



Medverkande i programmet . . .

Två typer av föremål:

1. **Objekt:** Kännetecknas av

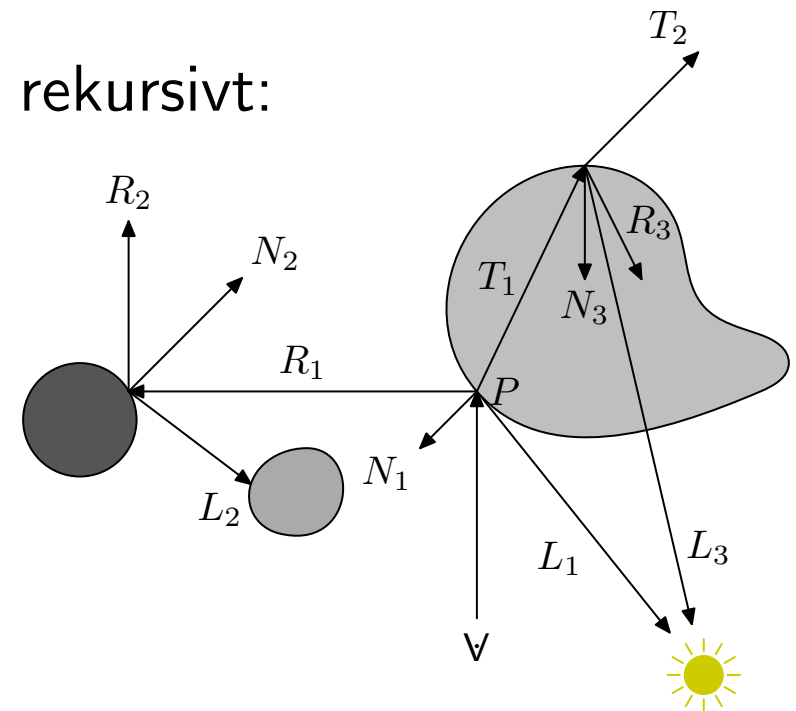
- Kontur. Implicit form: $f(x, y, z) = 0$.
Sfär: $(x - a)^2 + (y - b)^2 + (z - c)^2 - r^2 = 0$.
- Reflektionskoefficient för bakgrundsljus, k_a ;
- Diffus reflektionskoefficient, k_d ;
- Speglande reflektionskoefficient, k_s ;
- Reflektionsexponent, n ;
- Transmissionskoefficient, k_t ($= 1$ om genomskinlig);
- Brytningsindex, η .

2. **Ljuskällor:** Punktformade med intensitet I_p .

Strålföljningens huvudidé

Intensiteten i den pixel som motsvarar skärningspunkten P mellan strålen och det objekt som ligger närmast fås genom bidrag från

- de ljuskällor som “syns” från P , samt rekursivt:
- ljus som når P från andra objekt via reflektion;
- ljus som når P från andra objekt via transmission.

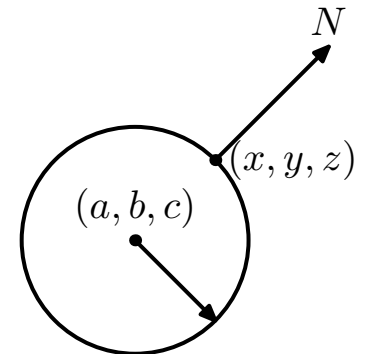


Processen avbryts antingen när en stråle missar alla objekt eller rekursionen överskrider maxgränsen.

Beräkning av normalen

- Ytan definieras av $f(x, y, z) = 0$.
- Normalen i en punkt (x, y, z) ges av *gradienten*:

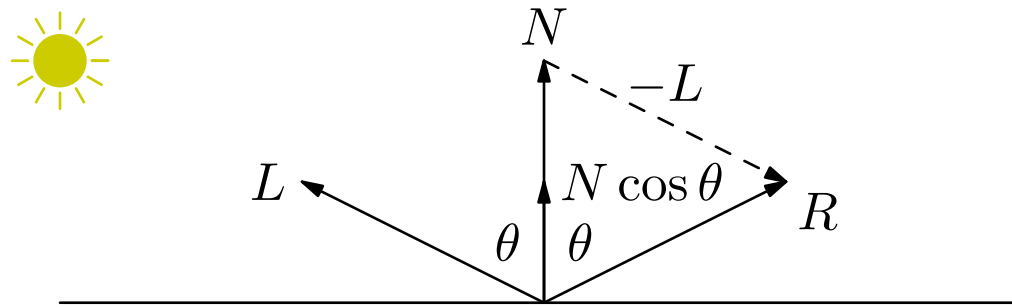
$$N = \nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$$



För en sfär: $f(x, y, z) = (x - a)^2 + (y - b)^2 + (z - c)^2 - r^2$

$$N = \nabla f(x, y, z) = (2x - 2a, 2y - 2b, 2z - 2c)$$

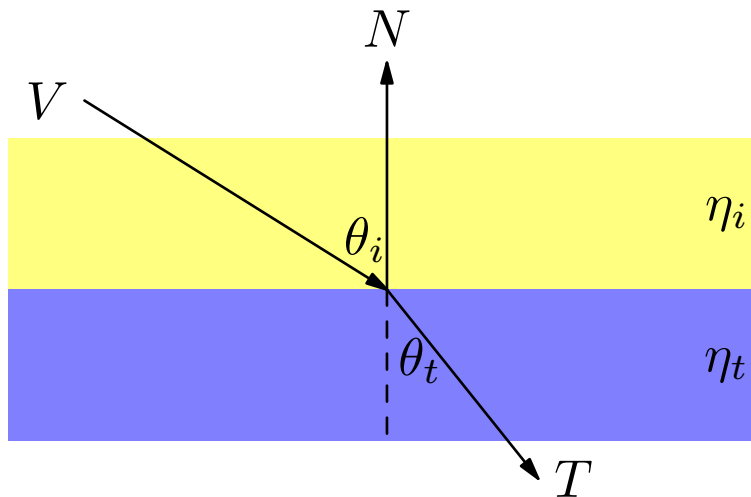
Beräkning av reflektion



$$R = V - 2N(V \cdot N)$$

(Från föreläsningen om lokal belysning.)

Beräkning av transmission



Ex. vakuum, $\eta = 1$

Ex. vatten, $\eta = 1.3$

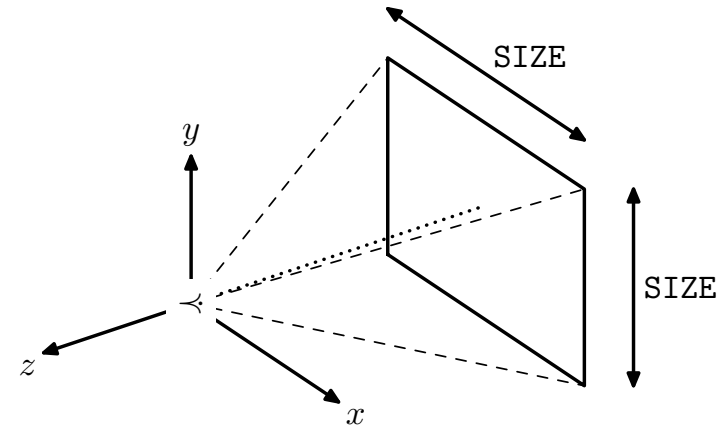
Snellius lag: $\frac{\sin \theta_i}{\sin \theta_t} = \frac{\eta_t}{\eta_i}$.

Om $\eta_i > \eta_t$ och $\theta_i > \sin^{-1}\left(\frac{\eta_t}{\eta_i}\right)$ uppstår *totalreflektion*.

Brytningen egentligen våglängdsberoende (jfr prisma).

Pseudokod för strålföljning

- Observatören i origo.
- Projektionsplanet = $\text{SIZE} \times \text{SIZE}$.
- Avståndet till planet bestäms av FOV.



För varje pixel, skjut iväg en stråle från ögat i riktning mot pixeln:

```

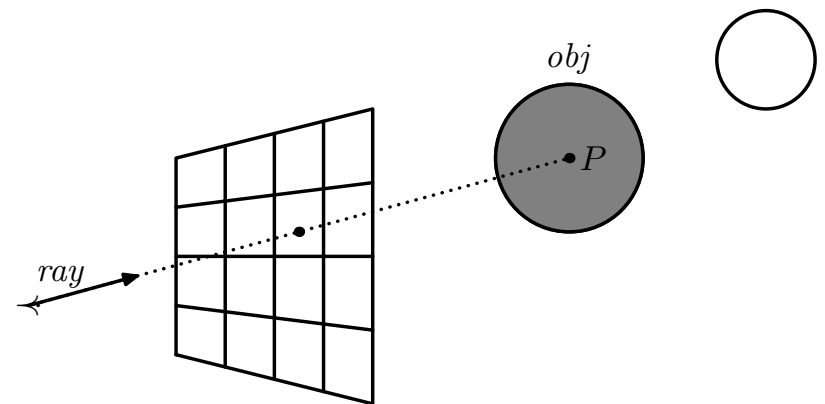
ray.start := (0, 0, 0);
ray.dz := -SIZE / (2 * tan FOV);
for ray.dx := -SIZE/2 to SIZE/2 do
    for ray.dy := -SIZE/2 to SIZE/2 do
        pixel := trace(ray, 1);
  
```

- Om strålen träffar ett objekt i punkt P , räkna ut intensiteten.
- Om strålen missar alla objekt används bakgrundsintensiteten.

```

function trace(ray, depth)
     $\langle obj, P \rangle := first\_hit(ray);$ 
    if obj  $\neq$  NULL then begin
         $N := normal(obj, P);$ 
        return shade(obj, ray, P, N, depth);
    end else
        return  $k_a \cdot I_a;$ 

```



Intensiteten i P bestäms dels av de ljuskällor som syns:

Kasta en “shadow ray” L mot varje ljuskälla ℓ . Om L träffar ett annat objekt ligger P i skugga. Annars beräknas kombinerad diffus och speglande reflektion från ℓ som i en lokal belysningsmodell.

```
function shade(obj, ray,  $P$ ,  $N$ , depth)
    color :=  $k_a \cdot I_a$ ;
    for alla ljuskällor  $\ell$  do begin
         $L := \ell - P$ ;
        if  $L \cdot N > 0$  and  $\neg first\_hit(L)$  then
            color := color +  $I_p \cdot [k_d(L \cdot N) + k_s(R \cdot V)^n]$ ;
        end
    :
```

Sedan tillkommer (rekursivt) bidrag från reflektion och transmission.

- Om rekursionen överskridit maxdjupet, avbryt.

```

      ⋮
    if  $depth > \text{MAXDEPTH}$  then return  $color$ ;
      ⋮
  
```

- Om objektet är reflekterande, kasta (rekursivt) en ny stråle i reflektionsriktningen och skala med reflektionskoefficienten.

```

      ⋮
    if  $k_s > 0$  then begin
       $R := \text{reflect}(ray, N);$ 
       $color := color + k_s \cdot \text{trace}(R, depth + 1);$ 
    end
      ⋮
  
```

- Om objektet är genomskinligt, kasta (rekursivt) en ny stråle i brytningsriktningen och skala med transmissionkoefficienten.

Vid totalreflektion ($T = \text{NULL}$): inget bidrag. Exempel: fiberoptik.

```

      :
if  $k_t > 0$  then begin
     $T := \text{refract}(\text{ray}, N, \text{obj});$ 
    if  $T \neq \text{NULL}$  then
         $\text{color} := \text{color} + k_t \cdot \text{trace}(T, \text{depth} + 1);$ 
    end
    return  $\text{color};$ 
end

```

Belysningsekvation för strålföljning

$$I = k_a I_a + \sum_{1 \leq i \leq m} S_i I_p [k_d (N \cdot L) + k_s (R \cdot V)^n] + k_s I_r + k_t I_t$$

m = Antalet ljuskällor;

I_r = Intensiteten på den reflekterade strålen;

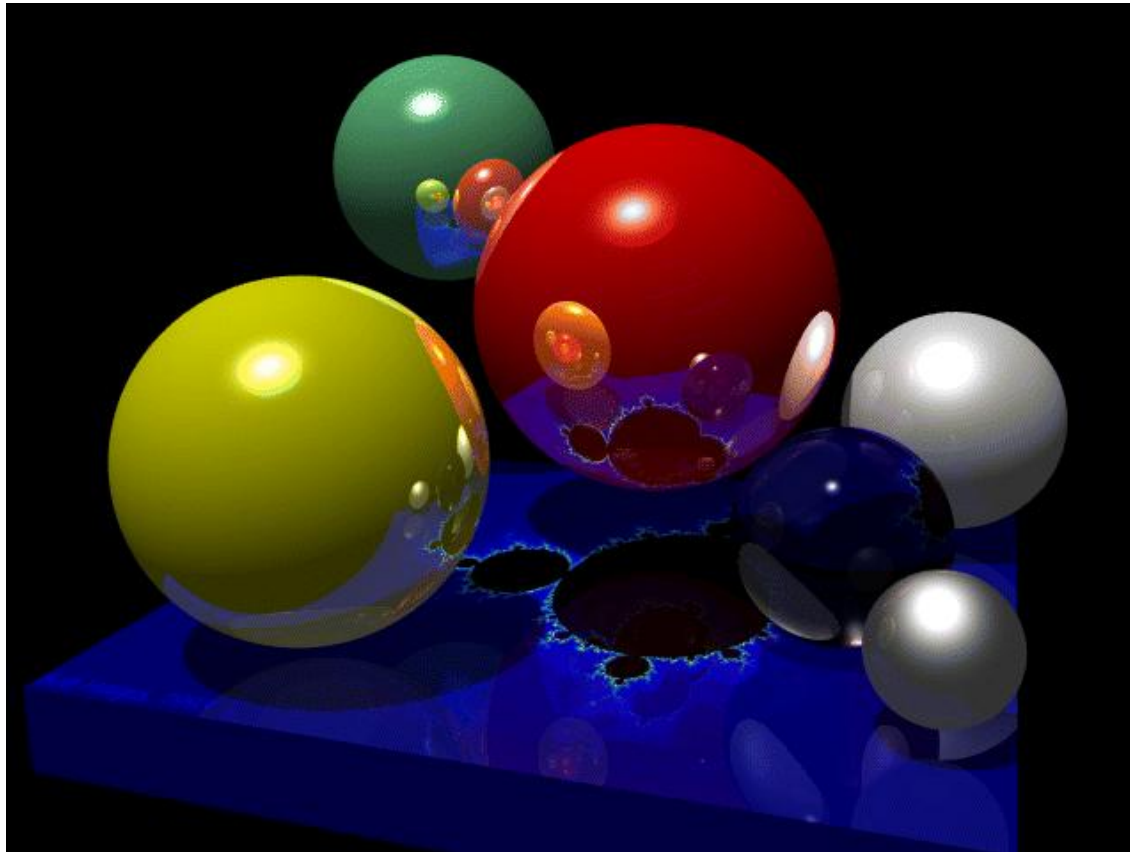
I_t = Intensiteten på den transmitterade strålen;

I_p = Intensiteten hos ljuskälla i ;

S_i = 1 om ljuskälla i syns, annars 0;

Värden för I_r och I_t fås genom att rekursivt applicera formeln.

Resultatet



Se även Foley, van Dam: färgbild 41.

Vad som utelämnades

- **Färg.** Upprepa beräkningarna för olika λ .

- **Optimeringar.**

De optimeringar som diskuterades i samband med strålkastning är ännu viktigare nu då strålföljning kan generera exponentiellt fler strålar.

- **Anti-aliasing.**

Supersampling hanterar de flesta “jaggies”. Kan göras adaptivt.

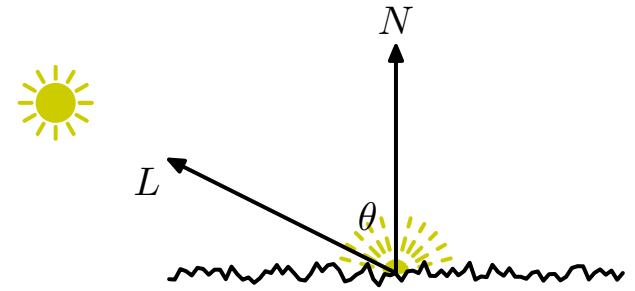
- **Massor med andra variationer.** Se litteraturen!

Vad strålföljning normalt inte klarar

- Diffust ljus som sprids via reflektion/transmission.
- Skuggstrålar L som blockeras av t ex glas.
 L borde brytas men hamnar då inte i ljuskällan.

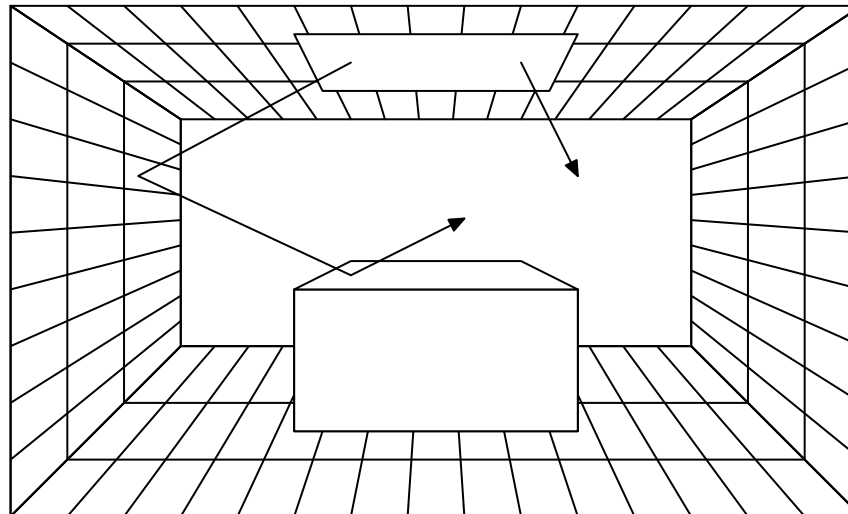
Radiosity

- Sluten värld uppbyggd av n st flata ytor.
- Alla ytor uppvisar diffus reflektion.
- Vissa ytor utstrålar även ljus ($\approx$ lysrör).
- Ytornas intensitet beräknas en gång för alla, oberoende av observatörens position.
Intensiteten är ju densamma, oberoende av vyn.
- Beräkningen baseras på energiöverföring mellan ytorna. För ett slutet rum råder jämvikt.
- Vid rendering behövs bara vad lab 3 klarar, dvs eliminera skymda ytor. Intensiteten för varje yta tas från databasen.



Ytor

- Alla ytor är icke genomskinliga diffusa reflektorer.
- Ljuskällor utstrålar diffust ljus.
- Stora ytor indelas i lapptäcken (eng. *patches*).



Ett rum indelat i ytor.

Vad är radiosity?

Radiosity B för en yta är den totala mängd energi som lämnar en enhet av ytan varje sekund. Detta energiflöde beskriver ljuset.

Radiosity mäts därför i $J/(m^2 \cdot s) = W/m^2$.

- För varje yta:

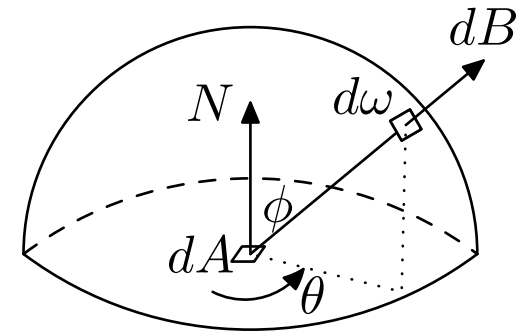
Utgående ljus = k_d · inkommande ljus + eget ljus.

- Leder till ett ekvationssystem för hela rummet.
- Vi vill egentligen veta varje ytas intensitet, I .
- Det finns ett enkelt förhållande, nämligen $I = B/\pi$.

Härledning

Intensiteten I i riktning (θ, ϕ) är det ljusflöde som utstrålas per projicerad ytenhet och rymdvinkel $d\omega$:

$$I = \frac{dB}{d\omega \cos \phi}$$



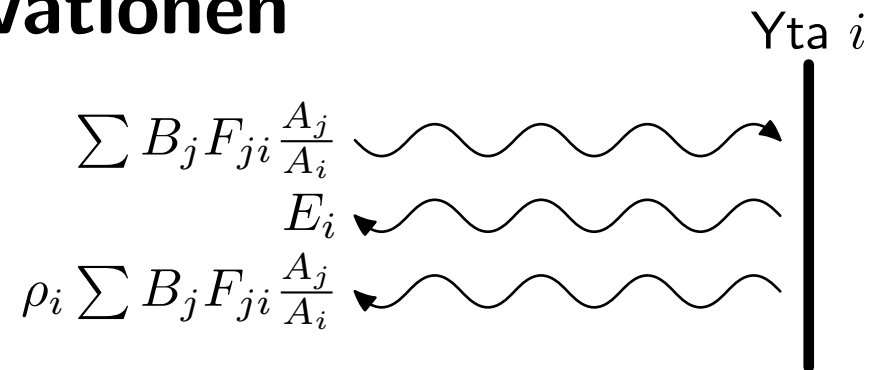
Rymdvinkeln $d\omega = \sin \phi \, d\phi \, d\theta$ mäts i steradianer.

$$B = \int_{\text{hemi}} dB = \int I \, d\omega \cos \phi = \int I \cos \phi \sin \phi \, d\phi \, d\theta$$

$$= I \int_{\theta=0}^{2\pi} \int_{\phi=0}^{\pi/2} \cos \phi \sin \phi \, d\phi \, d\theta = I\pi$$

Radiosity-ekvationen

$$B_i = E_i + \rho_i \sum_{1 \leq j \leq n} B_j F_{j-i} \frac{A_j}{A_i}$$



- E_i — ljus som utstrålas från i (W/m^2).
- B_j — ljus från j (W/m^2).
- F_{j-i} — formfaktorn, $0 \leq F_{j-i} \leq 1$; andelen ljus från j som når i .
- A_i, A_j — arean för i, j respektive.
- ρ_i — i :s reflektionskoefficient, $0 \leq \rho_i \leq 1$.

Kommentarer

- $E_i > 0$ om i är en ljuskälla, annars 0.
- $B_j F_{j-i}$ är den mängd ljus som når en ytenhet av i från en ytenhet av j .

Därför måste vi multiplicera med A_j för att få fram *allt ljus* från j och dividera med A_i eftersom B_i är per ytenhet.

- Ett enkelt förhållande råder mellan formfaktorer:

$$A_i F_{i-j} = A_j F_{j-i}$$

Förenklad radiosity-ekvation

$$B_i = E_i + \rho_i \sum_{1 \leq j \leq n} B_j F_{i-j}$$

Efter omskrivning:

$$B_i - \rho_i \sum_{1 \leq j \leq n} B_j F_{i-j} = E_i$$

Ekvationssystem

Ljusflödet bland alla ytor kan formuleras som ett ekvationssystem:

$$\begin{pmatrix} 1 - \rho_1 F_{1-1} & -\rho_1 F_{1-2} & \cdots & -\rho_1 F_{1-n} \\ -\rho_2 F_{2-1} & 1 - \rho_2 F_{2-2} & \cdots & -\rho_2 F_{2-n} \\ \vdots & \vdots & \cdots & \vdots \\ -\rho_n F_{n-1} & \rho_n F_{n-2} & \cdots & 1 - \rho_n F_{n-n} \end{pmatrix} \begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_n \end{pmatrix} = \begin{pmatrix} E_1 \\ E_2 \\ \vdots \\ E_n \end{pmatrix}$$

Måste lösas för t ex $\lambda = \text{röd, grön, blå}$ eftersom ρ_i, E_i beror på λ .

Huvuddelen av tiden spenderas dock inte i lösningen av ekvationssystemet utan i beräkningen av formfaktorerna.

Formfaktorerna beror dock endast på geometrin och behöver bara beräknas en gång.

Formfaktorer F_{i-j}

- Om i och j är parallella och nära:

$$F_{i-j} \approx 1$$

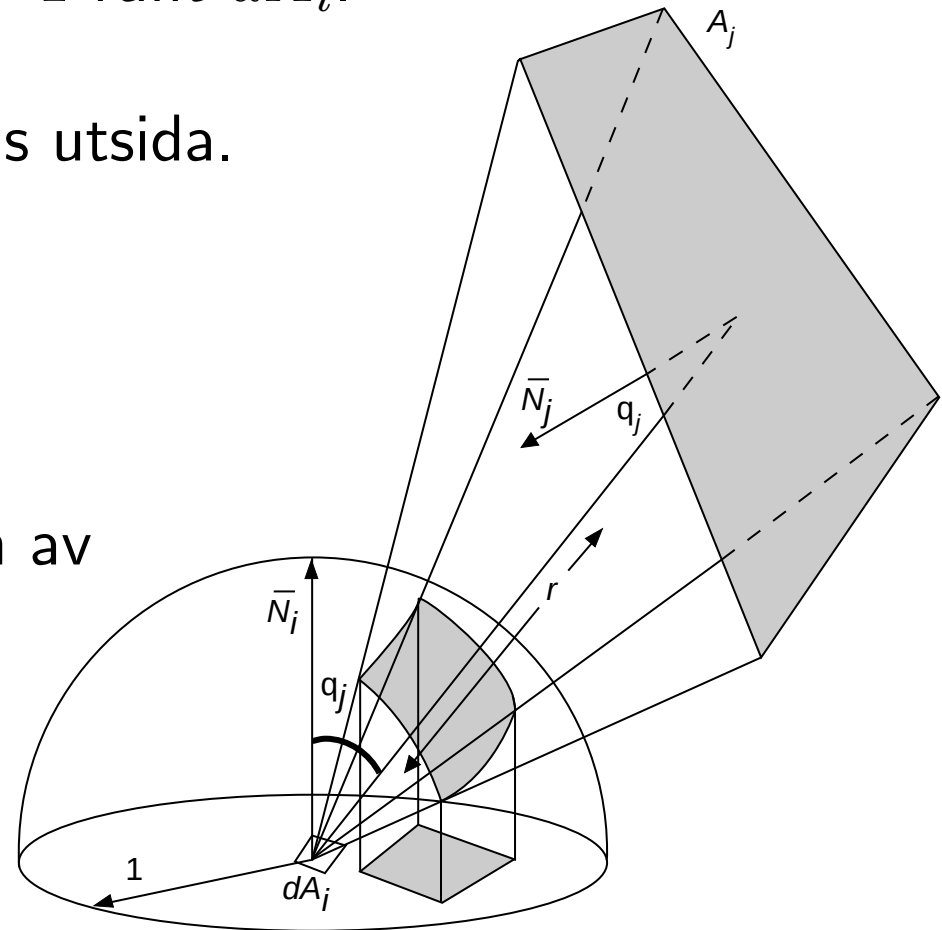
- Om i och j är bredvid/långt ifrån varandra:

$$F_{i-j} \approx 0$$

- Oftast besvärlig att lösa analytiskt.
- I praktiken används numeriska metoder.
- Ger tillräcklig noggrannhet om varje *patch* är liten (dA).

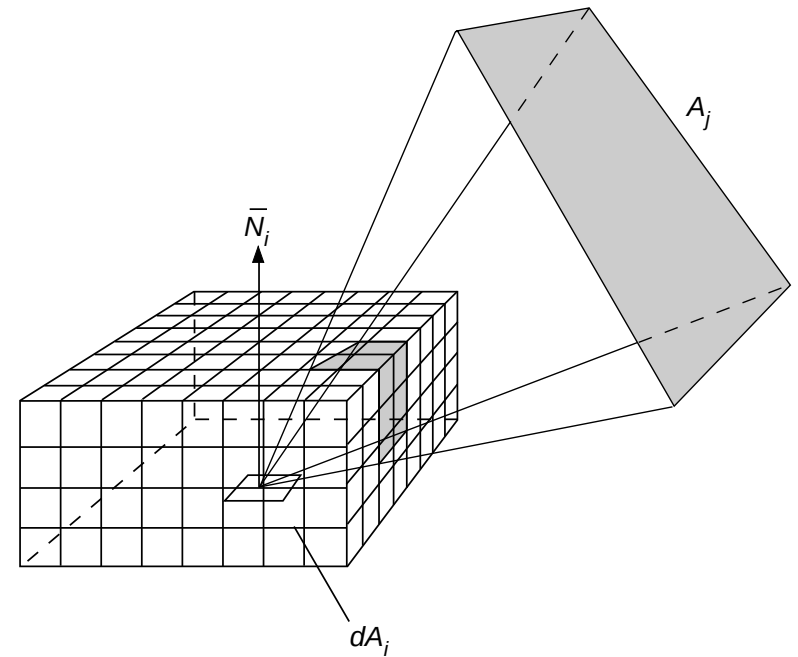
Nusselts analogi

- Placera en hemisfär med radie = 1 runt dA_i .
- Projicera först A_j på hemisfärens utsida.
- Projicera sedan ortografiskt ner på hemisfärens bas.
- Formfaktorn $F_{dA_i-A_j}$ är andelen av hemisfärens basyta som täcks.



Hemikuben

- Omslut dA_i med en hemikub istället.
- Varje cell på hemikuben lagrar den yta j som syns från dA_i och ligger närmast (jfr. Z-buffert). Hemikuben för dA_i lagrar alltså samtliga $F_{dA_i-A_j}$.
- Bidraget till $F_{dA_i-A_j}$ från en cell räknas ut i förväg.
- $F_{dA_i-A_j} = \sum$ alla celler med utsikt över A_j .
- Om A_i tillräckligt liten: $F_{ij} \approx F_{dA_i-A_j}$.



Radiosity-bilder uppvisar ofta en enastående realism



Fotografi



Radiosity



Skillnad

Delar av detta rum visas i Foley, van Dam: färgbild 43.

Observera där effekten av “color bleeding”, dvs hur en yta kan färgas blå av det ljus som reflekterats från en blå yta, precis som i verkligheten.

Progressiv radiosity-algoritm

Rakt-på-implementationen måste lagra *alla* formfaktorer och lösa *hela* ekvationssystemet innan bilden kan visas.

Radiosity-ekvationen: B_i pga $B_j = \rho_i B_j F_{i-j}$, för alla j .

Denna formel samlar in ljus från omgivningen till yta i .

Alternativ approach: B_j pga $B_i = \rho_j B_i F_{j-i}$, för alla j .

Här sänds ljuset från i ut till alla omgivande ytor j . Dock måste F_{j-i} beräknas för varje j , så alla formfaktorer måste fortfarande lagras. Ingen vinst.

Progressiv radiosity-algoritm (forts)

Men vi visste ju också att $A_i F_{i-j} = A_j F_{j-i}$:

$$B_j \text{ pga } B_i = \rho_j B_i F_{i-j} \frac{A_i}{A_j}, \text{ för alla } j$$

Denna metod kräver bara att hemikuben för i beräknas.

Istället för att sända B_i 's bidrag till j , skicka skillnaden sen förra gången, ΔB_i .

Följande algoritm simulerar hur ljuset verkligen sprids:

(Se färgbild 44 i Foley, van Dam.)

Progressiv radiosity-algoritm (forts)

1. Sätt $B_i = \Delta B_i = E_i$ för alla ytor i .
2. Välj den yta i vars $\Delta B_i A_i$ är störst; om $\Delta B_i A_i < \epsilon$, avsluta.
3. Beräkna hemikuben (F_{i-j}) för i .
4. För varje yta j , sänd "nya ljuset" ΔB från i :
 - (a) $\Delta B := \rho_j \Delta B_i F_{i-j} A_i / A_j$
 - (b) $\Delta B_j := \Delta B_j + \Delta B$
 - (c) $B_j := B_j + \Delta B$
5. $\Delta B_i := 0$
6. Gå till 2.

Jämförelse: strålföljning vs. radiosity

- Strålföljning: vyberoende; Radiosity: vyoberoende.
- Radiosity: ingen skillnad på objekt och lampor.
- Strålföljning: speglade ytor, punktformade ljus.
Favoritexempel: genomskinlig glasboll.
Strålföljning klarar inte diffus sekundär reflektion.
- Radiosity: matta ytor, "lysrör".
Favoritexempel: kontorslandskap.
Radiosity klarar inte refraktion eller speglade reflektion.
- Strålföljning: enkel algoritm; Radiosity: komplicerad algoritm
- Hybridalgoritmer finns.

En minimal strålföljare, skriven i C

```
typedef struct{double x,y,z}vec;vec U,black,amb={.02,.02,.02};struct sphere{
vec cen,color;double rad,kd,ks,kt,kl,ir}*s,*best,sph[]={0.,6.,.5,1.,1.,1.,.9,
.05,.2,.85,0.,1.7,-1.,8.,-.5,1.,.5,.2,1.,.7,.3,0.,.05,1.2,1.,8.,-.5,.1,.8,.8,
1.,.3,.7,0.,0.,1.2,3.,-6.,15.,1.,.8,1.,7.,0.,0.,0.,.6,1.5,-3.,-3.,12.,.8,1.,
1.,5.,0.,0.,0.,.5,1.5,};yx;double u,b,tmin,sqrt(),tan();double vdot(A,B)vec A
,B;{return A.x*B.x+A.y*B.y+A.z*B.z;}vec vcomb(a,A,B)double a;vec A,B;{B.x+=a*
A.x;B.y+=a*A.y;B.z+=a*A.z;return B;}vec vunit(A)vec A;{return vcomb(1./sqrt(
vdot(A,A)),A,black);}struct sphere*intersect(P,D)vec P,D;{best=0;tmin=1e30;s=
sph+5;while(s-->sph)b=vdot(D,U=vcomb(-1.,P,s->cen)),u=b*b-vdot(U,U)+s->rad*s
->rad,u=u>0?sqrt(u):1e31,u=b-u>1e-7?b-u:b+u,tmin=u>1e-7&&u<tmin?best=s,u:
tmin;return best;}vec trace(level,P,D)vec P,D;{double d,eta,e;vec N,color;
struct sphere*s,*l;if(!level--)return black;if(s=intersect(P,D));else return
amb;color=amb;eta=s->ir;d= -vdot(D,N=vunit(vcomb(-1.,P=vcomb(tmin,D,P),s->cen
)));if(d<0)N=vcomb(-1.,N,black),eta=1/eta,d= -d;l=sph+5;while(l-->sph)if((e=l
->kl*vdot(N,U=vunit(vcomb(-1.,P,l->cen))))>0&&intersect(P,U)==l)color=vcomb(e
,l->color,color);U=s->color;color.x*=U.x;color.y*=U.y;color.z*=U.z;e=1-eta*
eta*(1-d*d);return vcomb(s->kt,e>0?trace(level,P,vcomb(eta,D,vcomb(eta*d-sqrt
(e),N,black))):black,vcomb(s->ks,trace(level,P,vcomb(2*d,N,D)),vcomb(s->kd,
color,vcomb(s->kl,U,black))));}main(){printf(' '%d %d\n',32,32);while(yx<32*32)
U.x=yx%32-32/2,U.z=32/2-yx++/32,U.y=32/2/tan(25/114.5915590261),U=vcomb(255.,
trace(3,black,vunit(U)),black),printf('%.0f %.0f %.0f\n',U);}/*minray!*/
```

En annan strålföljare i PostScript

```
%! Tiny RayTracing by HAYAKAWA,Takashi(h-takasi@isea.is.titech.ac.jp)
/p/floor/S/add/A/copy/n/exch/i/index/J/ifndef/r/roll/e/sqrt/H{count 2 idiv exch
repeat}def/q/gt/h/exp/t/and/C/neg/T/dup/Y/pop/d/mul/w/div/s/cvi/R/rlineto{load
def}H/c(j1idj2id42rd)/G(140N7)/Q(31C85d4)/B(V0R0VRVCOR)/K(WCVW)/U(4C577d7)300
T translate/I(3STinTinTinY)/l(993dC99Cc96raN)/k(X&E9!&1!J)/Z(blxC1SdC9n5dh)/j
(43r)/O(Y43d9rE3IaN96r63rvx2dcaN)/z(&93r6IQ02Z4o3AQYaNlxS2w!)/N(3A3Axe1nwc)/W
270 def/L(1i2A00053r45hNvQXz&vUX&U0vQXzFJ!FJ!J)/D(cjS5o32rS4oS3o)/v(6A)/b(7o)
/F(&vGYx4oGbxSd0nq&3IGbxSGY4Iwxca3AlvvUkbQkdbGYx4ofwnw!&v1x2w13wSb8Z4wS!J!)/X
(4I3Ax52r8Ia3A3Ax65rTdCS4iw5o5IxnwTTd32rCST0q&eCST0q&D1!&EYE0!J!&EYEO!J0q)/V
3 def/x(jd5o32rd4odSS)/a(1CD)/E(YYY)/o(1r)/f(nY9wn7wpSps1t1S){[n{( )T 0 4 3 r
put T(/)q{T(9)q{cvn}{s}J}{($)q{[]{[]}}J}J cvx}forall]cvx def}H K{K{L setgray
moveto B fill}for Y}for showpage
```

